CMOSインバータ設計・半導体チップ製作ハンズオン (GF180mcu)

ISHI会

森　瑞紀 (Mizuki Mori) , https://github.com/3zki

今村　謙之（Noritsuna Imamura）

Rev.1

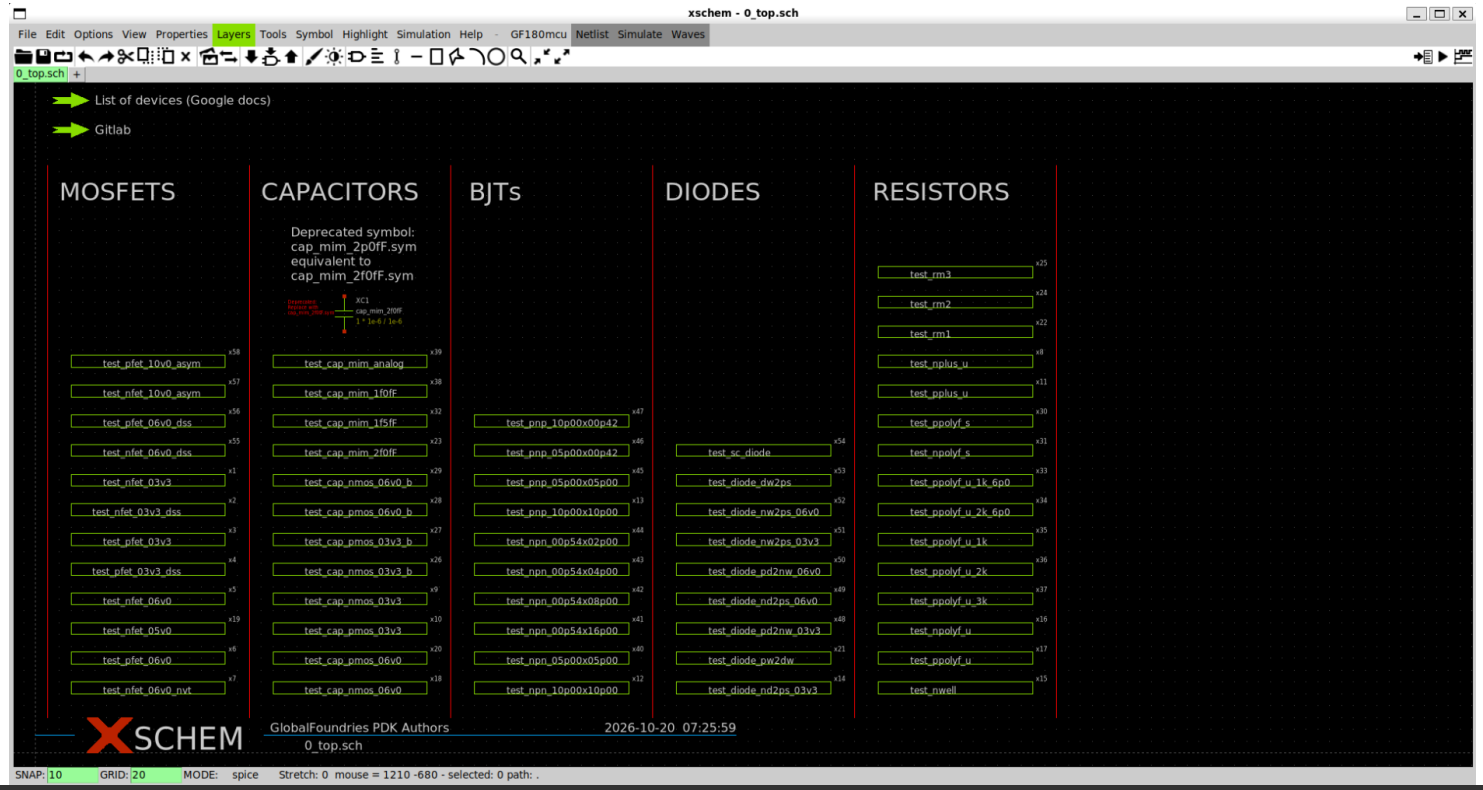
最初に 1/3

- 運営から下記のものがメールにて届いていることを確認してください。届いていない場合は、運営に問い合わせてください。
  - 運営連絡先： shumpei.kawasaki@riscv.or.jp

- ハンズオンに必要な情報（運営から通知されます）
  1. DUT番号
  2. 割り当てられたインバータ回路のスペック
     1. P-FETのモジュール名、W値、L値
     2. N-FETのモジュール名、W値、L値
  3. シークレットキー
  4. 登録先サイトのURL

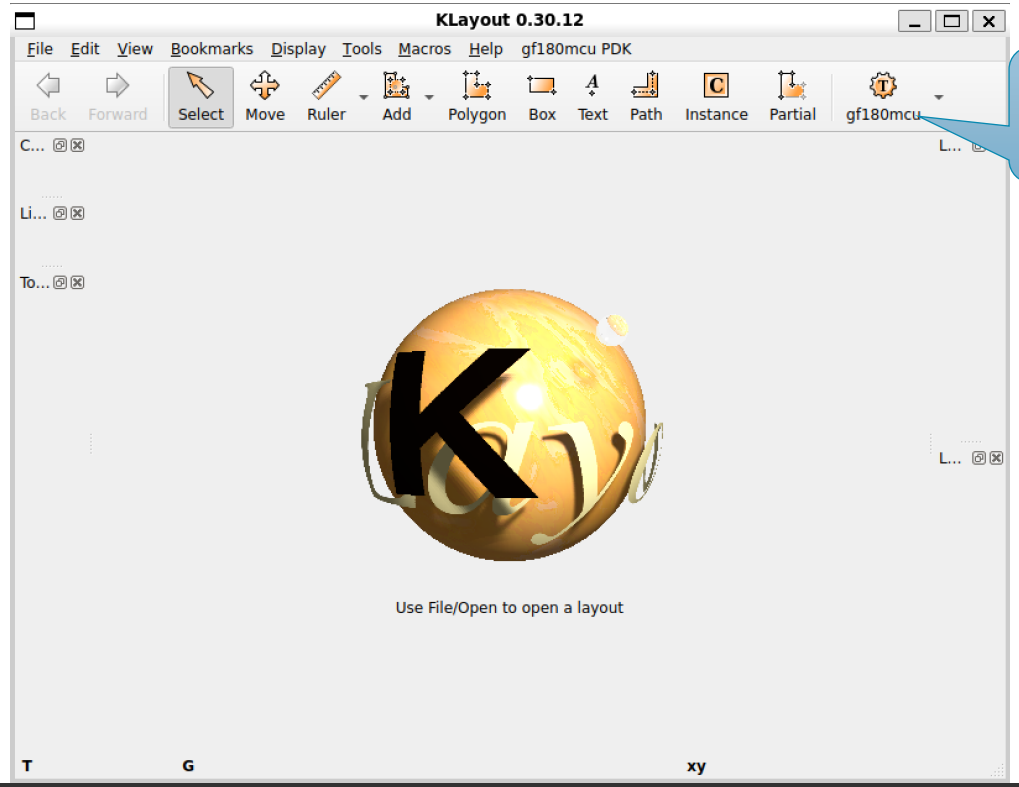
最初に 2/3

- ISHI会のWSLを起動後、「xschem」とコマンドを打って、下記の画面が出ることを確認してください。
  - 出ない場合は、インストールに失敗しています。再インストールをお願いします。
    - 上記内容で意味が分からない方は、勉強しておいてください。WSLやコマンドラインの使い方は知っている前提となります。



最初に 3/3

- ISHI会のWSLを起動後、「klayout」とコマンドを打って、下記の画面が出ることを確認してください。
  - 出ない場合は、インストールに失敗しています。再インストールをお願いします。
    - 上記内容で意味が分からない方は、勉強しておいてください。WSLやコマンドラインの使い方は知っている前提となります。



gf180mcuとなっている

アジェンダ

- なぜ、半導体を学ぶのか？
- 半導体の基礎知識
- 開発ツールの紹介、PDKの中身
- アナログ半導体の設計フロー
- アナログ半導体設計デモンストレーション　（CMOSインバータ）
- フレーム・パッドとテープアウト（製造依頼）

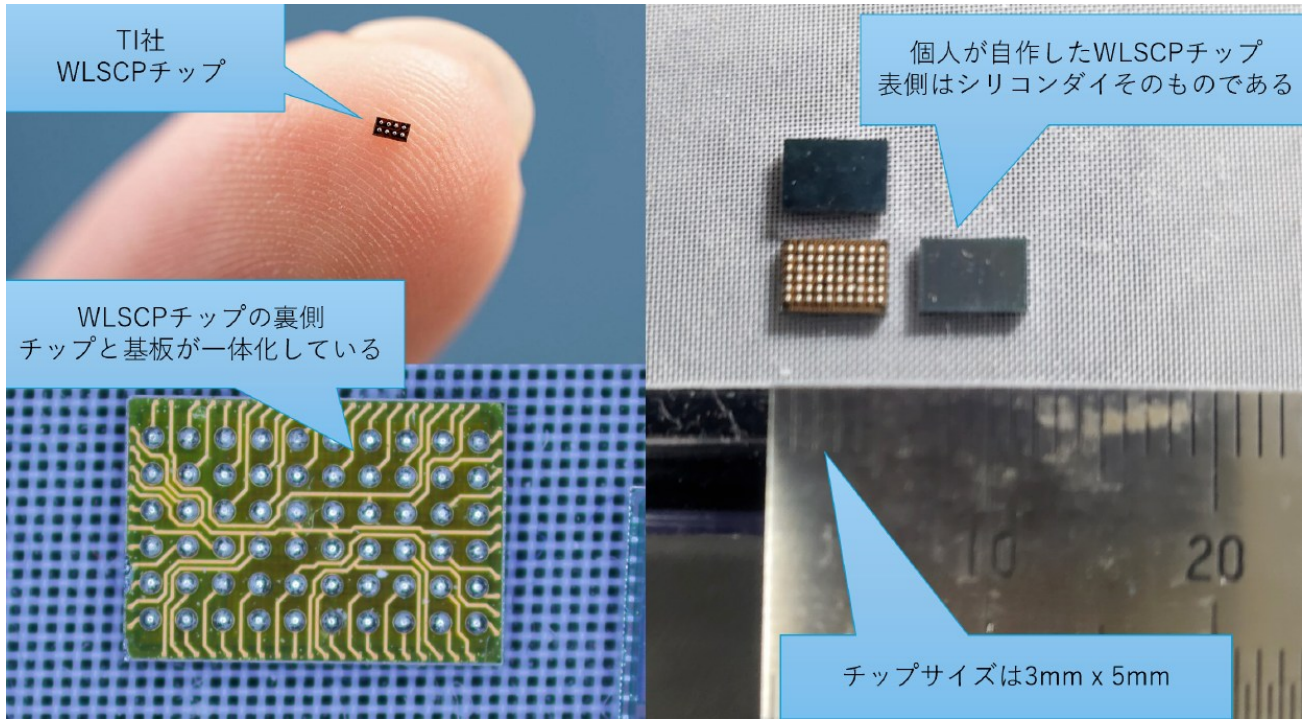
なぜ、半導体を学ぶのか？

次世代電子工作の世界へ

現在は、専用チップ化が進む世界

超小型パッケージがもたらす大きな可能性～TI 社の世界最小マイコン MSPM0C1104 のプレスリリースより～

消費者は、電動歯ブラシやスタイラス ペンなど日常的に使用する電子機器に対し、より小型で低コストかつ多機能な製品を常に求めています。こうした製品の小型化が進む中で、エンジニアは限られた基板面積で新たな機能を追加できる、コンパクトな統合された部品を求めています。MSPM0C1104 マイコンは、搭載する機能を厳選し、TI のコスト最適化戦略を取り入れるとともに、WCSP のパッケージング技術の利点を活かしています。8 つの接点を持つ WCSP のサイズはわずか 1.38mm2 で、競合製品と比べて 38% の小型化を実現しています。



チップは「小型」と「低コスト」がキモ！

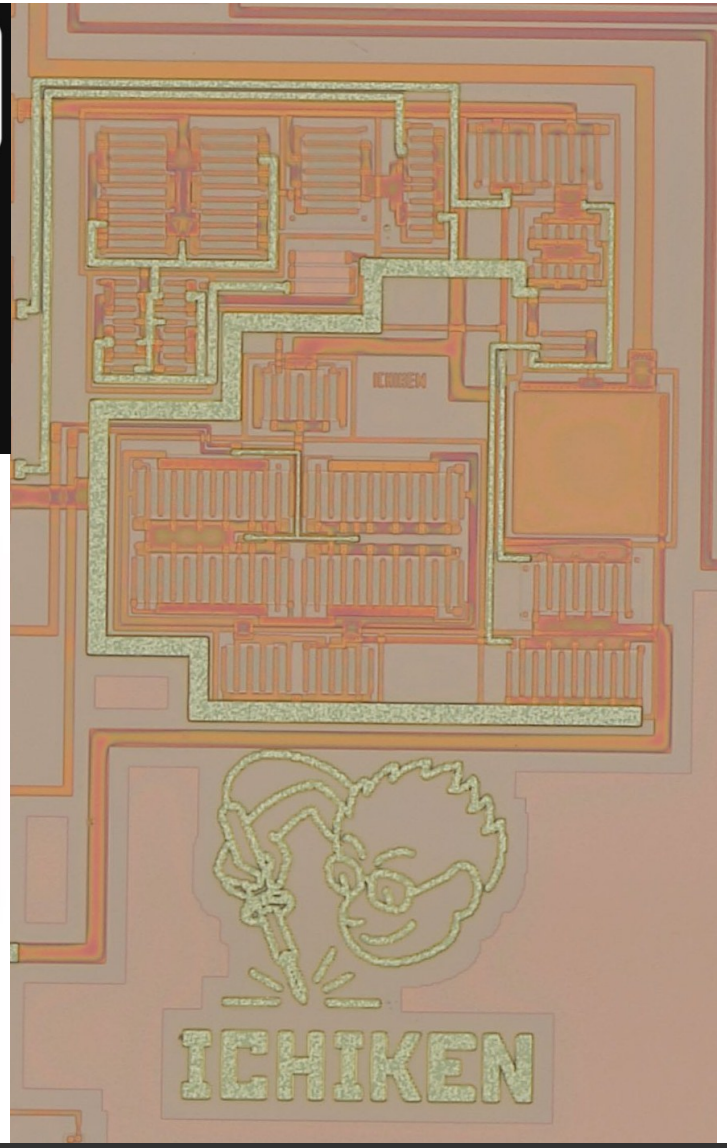
- 最終到達点は「超小型の専用チップ化」

PCB基板はチップと一体化！

- 最終到達点は、チップと基板は一体化し、センサーやアクチュエーター、筐体と統合される

次世代電子工作は
「基板設計」から「専用チップ設計」になる！

半導体を自作する世界がやってきた



こんにちは、イチケンです。

普段は YouTube で電子工作やものづくりの動画を投稿しているのですが、今回ご縁があって、この「Open Source Silicon Magazine Vol.2 と Vol.3」の冒頭の挨拶を書かせてもらうことになりました。

2025 年に、オープンソースの設計ツールを使ってオペアンプをゼロから設計し、実際にシリコンチップとして製造するということをやりました。ISHI 会の皆さんにサポートしてもらいながら、回路図を書いて、シミュレーションして、レイアウトを作って、検証して、ファウンドリに出して——そして数ヶ月後、実際にチップが届きました。届いたチップをマイクロスコープで覗いたとき、自分が設計したレイアウトがそのままシリコンの上に再現されているのを見て、素直に感動しました。そしてブレッドボード上でオペアンプとして動作したときは、「ああ、本当に動くのだ」と。正直に言えば、出力の駆動力が足りなかったり、高い周波数で波形が歪んだり、改善点はたくさんあります。しかし、自分で設計したトランジスタが、自分で描いたレイアウト通りに製造されて、ちゃんとオペアンプとして増幅している。この体験は、既製品の IC を使った電子工作とはまったく別の面白さがありました。

さて、この本を手に取ってくれた皆さんの中には、「半導体の設計って自分には関係ないだろう」と思っている方もいるかもしれません。僕も最初はそう思っていました。半導体は、巨大な企業が何百億円もかけて作るものというイメージがありますよね。しかし、今は違います。オープンソースの設計ツールがあります。無償で使える PDK（プロセスデザインキット）があります。TinyTapeout のようなシャトルサービスを使えば、1 タイルあたり数百ドルで自分の設計をシリコン化できます。国内のファウンドリでも、シャトル便を使えば数万円の負担でチップが作れる時代です。設計環境のセットアップも、有志の方々がまとめてくれていて、かなり楽になっています。つまり、電子工作で Arduino やマイコンを使ってモノを作るように、半導体チップそのものを「設計して作る」ことが、個人の手の届くところまで来ているのです。

本書 Vol.2 と次作の Vol.3 では、普段の電子工作で何気なく使っている IC チップの中身が、どのように設計されているのかを解説しています。論理回路と MOSFET の関係から始まって、スタンダードセルと配置配線、SystemVerilog での CPU 設計、TinyTapeout での IC チップ設計体験記。アナログ側では、OPAMP の増幅回路、バンドギャップリファレンス、ADC、DAC、VCO といった回路の設計原理からレイアウトまで。「何を作ったか」だけでなく「どのように作ったか」——設計の手法、苦労したポイント、工夫したポイントを、実際に手を動かした人たちが書いています。Vol.1 でツールのセットアップや基礎を学んだ方は、この Vol.2 と Vol.3 で実際に「何が作れるのか」を体感できると思います。

実際にオペアンプを設計してみて特に実感したのは、レイアウト設計の奥深さです。回路図上では線をつなぐだけですが、レイアウトではコモンセントロイドで温度特性を揃えたり、ダミーポリシリコンで製造バラツキから回路を守ったり、ダブルビアで信頼性を確保したり。回路図には載っていないノウハウが山ほどあります。そして、これがパズルのようで面白いのです。いくら時間があっても足りないくらいです。

電子工作の世界では、かつて Arduino が登場したことで「マイコンは一部の専門家だけのもの」から「誰でも使える道具」に変わりました。その結果、Maker Faire のような場で、想像もしなかったような多様な作品が生まれるようになりました。

半導体の設計も、今まさに同じ変化の入り口にいると思っています。「こんなセンサーがあったらいいのに」「この機能を 1 チップに収めたい」「既製品にはない特性のアナログ回路が欲しい」——そういったアイデアを、自分の手でシリコンにできる時代が来ています。今はまだ、L チカならぬ「最初の 1 チップを設計して動かす」段階かもしれません。しかし、電子工作も最初は L チカから始まって、そこから無数のプロジェクトが生まれていきました。半導体設計も同じ道をたどるはずです。

この本がその最初の一歩になれば嬉しいです。いつか皆さんの設計したチップの話を聞ける日を楽しみにしています。

イチケン

次世代電子工作の世界へ

- マイコンは、大雑把に分けると・・・
  - 周辺機器部（ペリフェラル部＋IO 部）
  - クロック部（内蔵発振器 or 外部発振器）
  - コンピュータ部（プロセッサ部＋メモリ部＋バス部）

今日はこれら専用チップ設計の基礎を身につける講座

マイコンを実際の半導体として自作するための知識を詰め込んだ一冊！
半導体という新しい電子工作の世界へ踏み出そう！

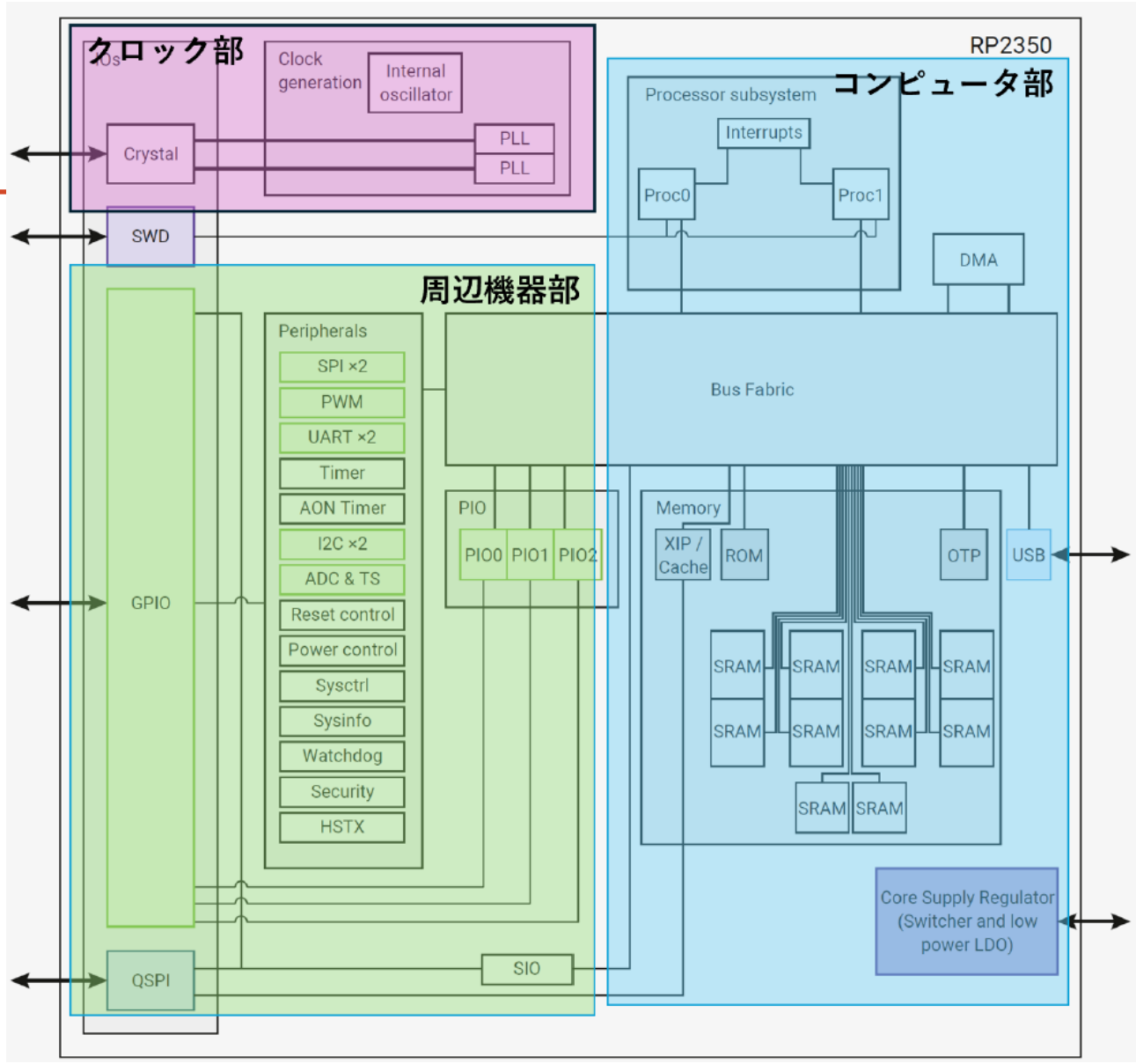
半導体設計の基礎知識を得るためのOpenSourceSiliconMagazineVol.1は無料配布中！

イチケンさんが挑戦したOPAMPも解説！

オープンソース半導体コミュニティISHI会は、Discordやリアルイベントにて活動中！

電子版：2000円
紙版：2500円
お得なセット版は2500円です‼

詳細を知りたい方はこちらをどうぞ



ラズベリーPico マイコンのブロック図

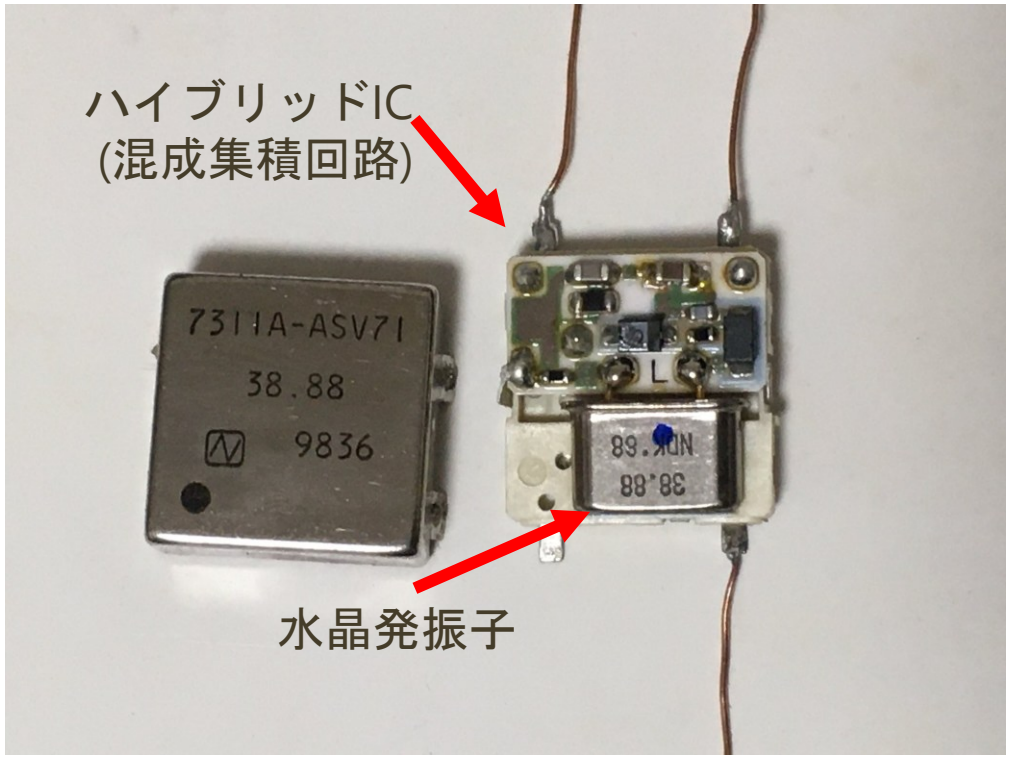
半導体の基礎知識

プレーナ型FETプロセス

ICとは？

- Integrated Circuit, 集積回路 には様々な形態がある。
  - 我々が目にするICは“**モノリシックIC**“☞
    1枚のシリコン基板に回路が構成。

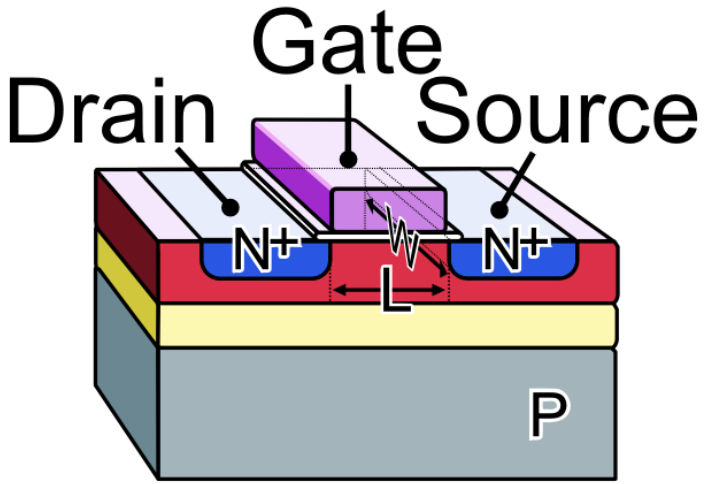
- 複数の部品を集積させたICを”**ハイブリッドIC**”という
  - 例: 水晶発振器 ＝ 水晶発振子＋発振回路
- モノリシックICの集積密度を高めたものが
  大規模集積回路、**LSI** (Large Scale Integration)



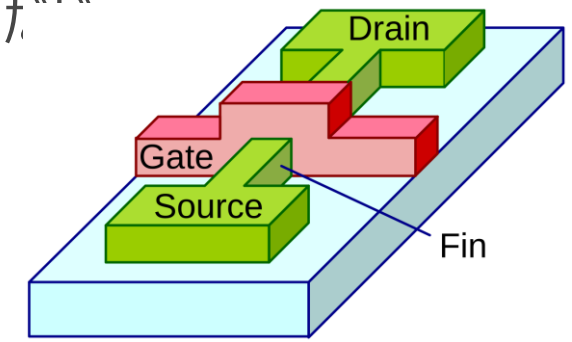
Crystal Oscillator (Photo: my own work)

プレーナ型FETプロセスとは

- 一言に半導体と言っても様々な種類があり、設計手法もそれぞれ異なる
- 東海理化(TR-1um)は**プレーナ型FETプロセス** ☞
  P型MOSFETとN型MOSFETが作成可能
- プレーナ型FETは20nm程度が限界。
  CPUのような高密度集積回路ではFinFETやGAA-FETといった
  全く別の構造をしたプロセスを現在は使用している。
- プレーナ型 45~130nmはロジック回路用としては時代遅れ (2000年前後) だ
  FinFETよりも安く製造でき、アナログLSIとしては現役。



Planar FET (~20nm)



FinFET (16nm ~ 7nm)

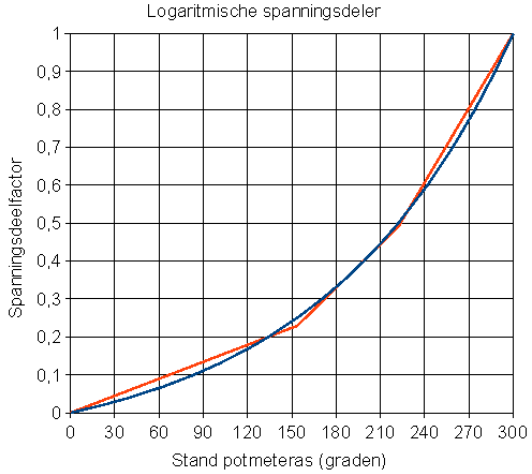
https://commons.wikimedia.org/wiki/File:Multigate_models.png
https://commons.wikimedia.org/wiki/File:MOS-FET_gate_model_(n-channel)_E.PNG

プレーナ型FETプロセスで作れる素子

- プレーナMOSFET: **PMOS**, **NMOS**
  - 耐圧により種類があることがある：1.8V, 3.3V 5.0V, 10.5V, 16V, 20V など
- ダイオード
- 抵抗: ポリ抵抗，拡散層抵抗（アクティブ抵抗），Nwell抵抗
- キャパシタ: ポリ容量，拡散層容量, MIMCAP (プロセスによっては無い), 配線層の寄生容量
- バラクタダイオード: プロセスによっては無い
- ラテラルバイポーラ (PNP, NPN)：プロセスによっては無い

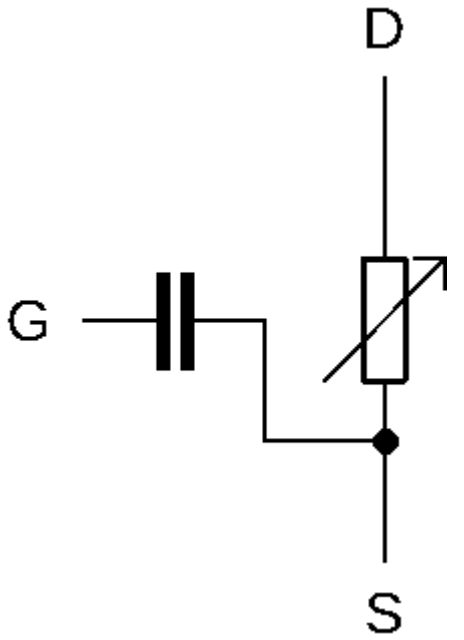
PMOS, NMOSとは

- 可変抵抗（ボリューム）に似ている
  - 0Ωから摺動子(ツマミ)を回すと、抵抗値が増加する
  - スペックでは最大抵抗値とカーブが記載
  - 抵抗値の増え方にバリエーションがある（カーブ）

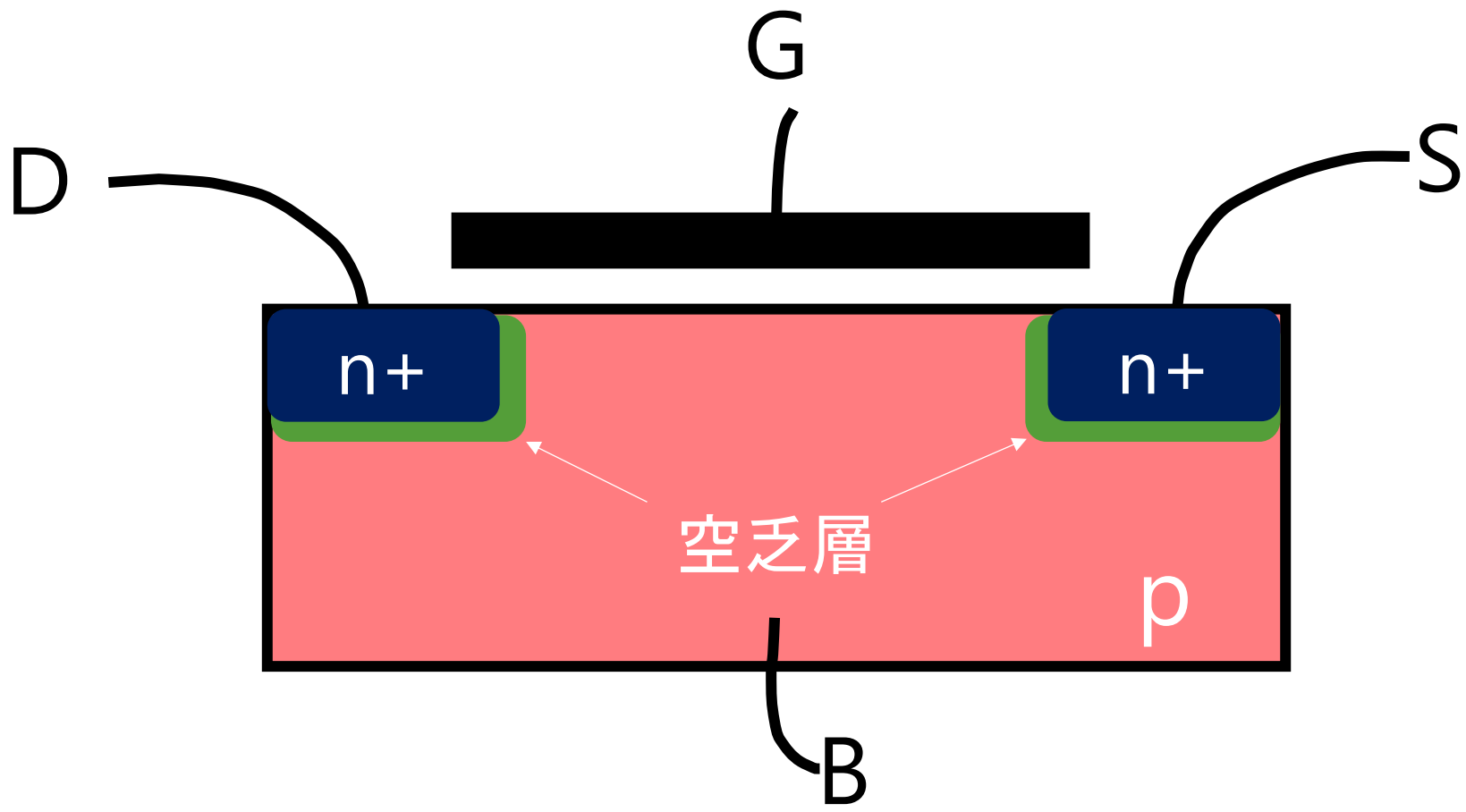


PMOS, NMOSとは

- MOSFETは電流をベースにして考える（抵抗値に置き換えて考える事は基本的にない）
- MOSFETは可変抵抗の逆
  - 摺動子（ツマミ）ではなくゲート電圧V_{GS}によってドレインソース間の電流を制御
  - ∞Ωからゲート電圧を調整すると抵抗値減少
  - スペックでは電流カーブが記載(ドレインソース間電流 I_{DS})
    - 最小抵抗値は電流カーブから計算 (オン抵抗R_{on})
  - 抵抗値の増え方はプロセス依存（閾値電圧V_{th}、トランスコンダクタンス g_m）

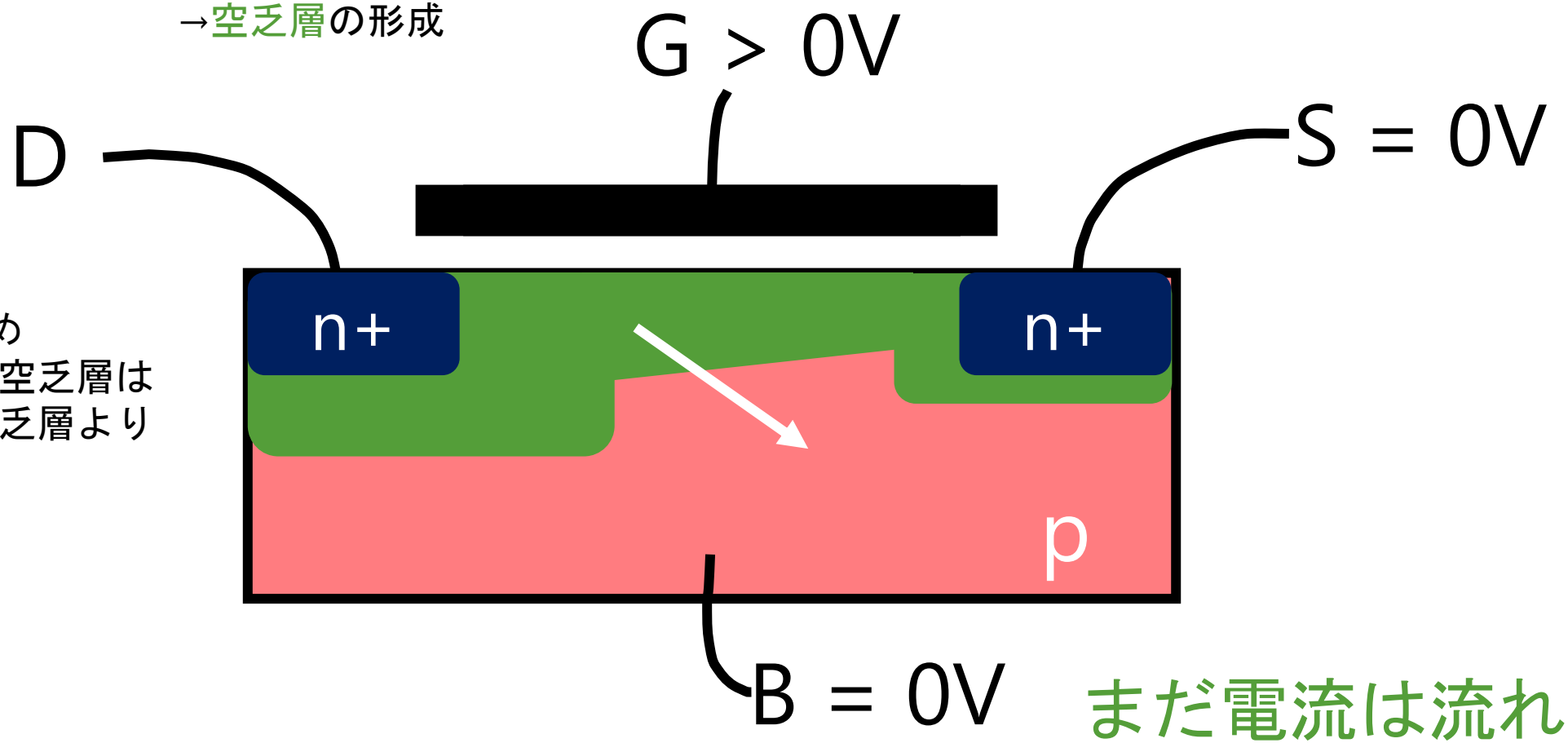


NMOS 簡単な模式図



NMOS 簡単な模式図

Vgs > 0 で基板上の正孔がゲートに移動
→空乏層の形成

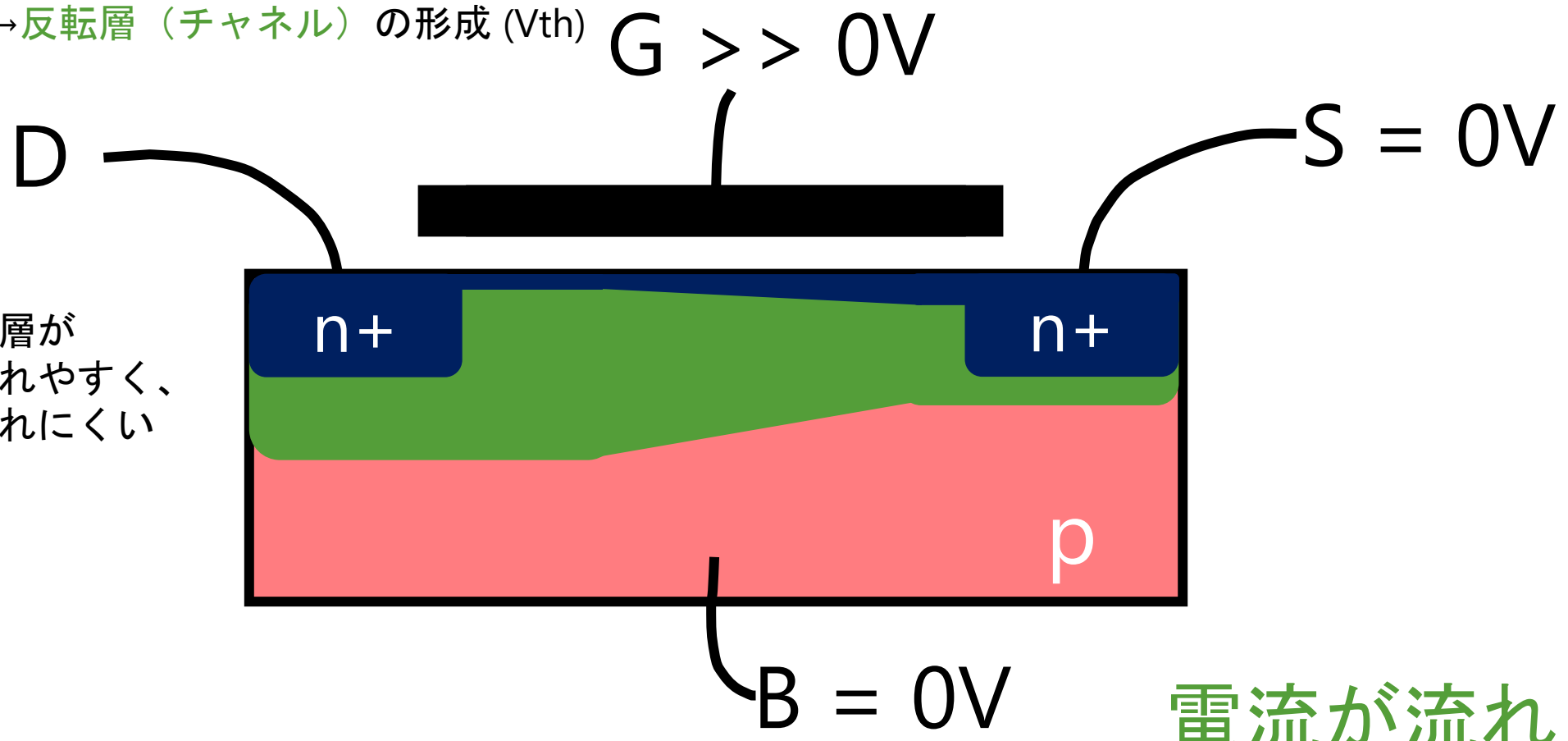


Vd > Vs のため
ドレイン側の空乏層は
ソース側の空乏層より
大きい

まだ電流は流れない

NMOS 簡単な模式図

Vgs >> 0 で伝導電子が多数キャリアとなる
→反転層（チャネル）の形成 (Vth)

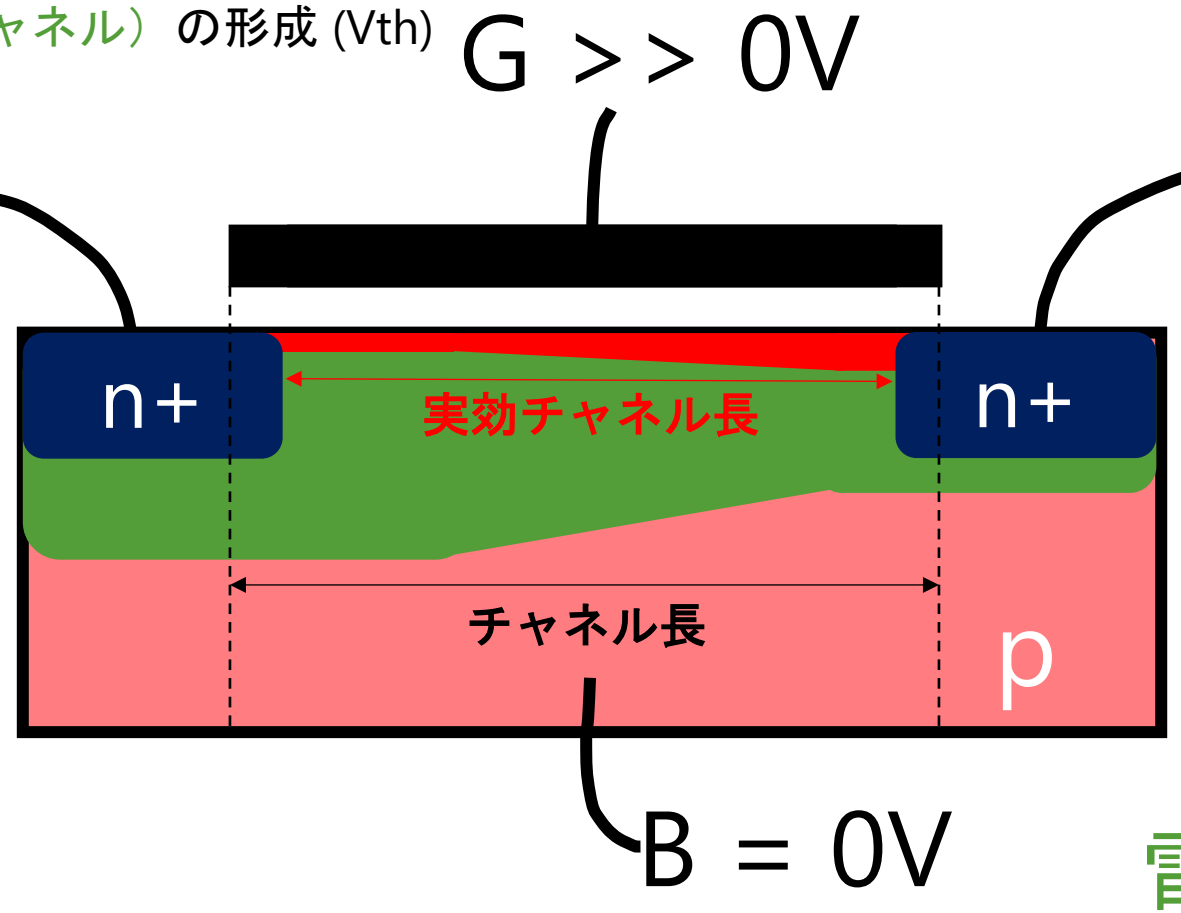


反転層は空乏層が
薄いと形成されやすく、
厚いと形成されにくい

電流が流れる

NMOS 簡単な模式図

Vgs >> 0 で伝導電子が多数キャリアとなる
→反転層（チャネル）の形成 (Vth)

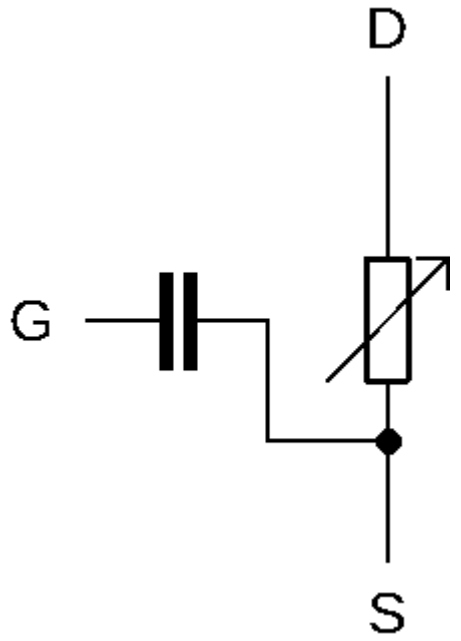


反転層は空乏層が
薄いと形成されやすく、
厚いと形成されにくい

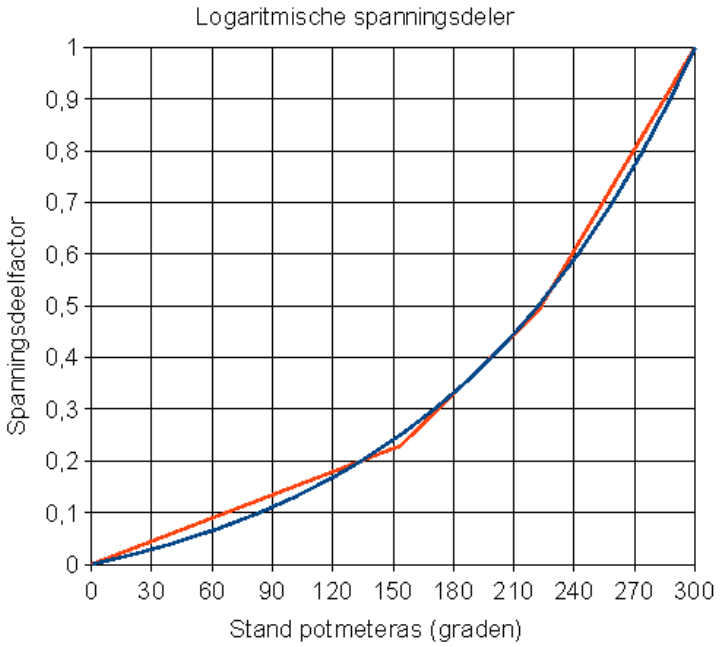
電流が流れる

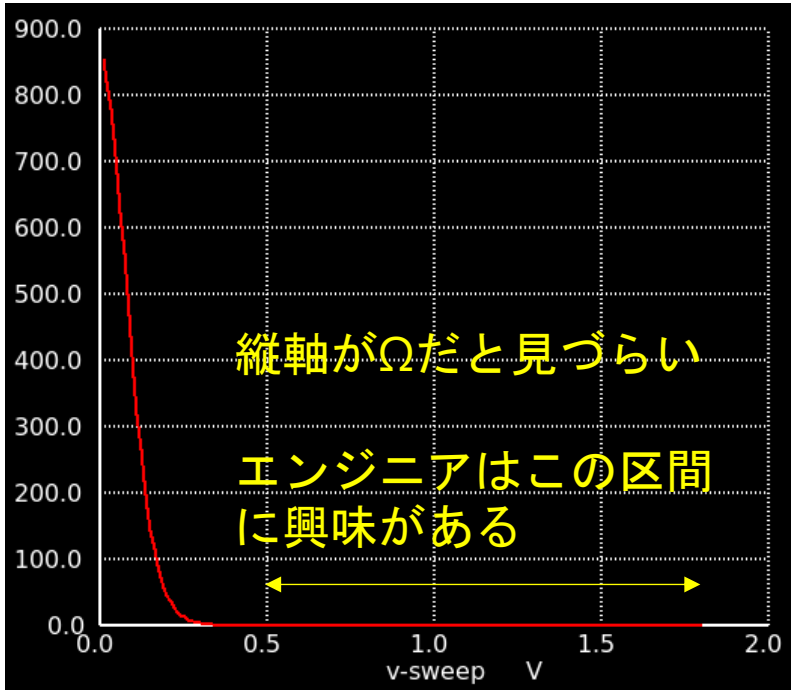
PMOS, NMOSとは

- 右図の等価回路は「MOSFETを敢えてRLCで表すならこうだろう」というもの
  - ドレイン・ソース間は「電流源」を置くのが正しいと思われるが、
    初心者にはあまり馴染みがないので可変抵抗とした
  - MOSFETの重要な特性は　トランスコンダクタンスとよばれるもの
  - また特定のドレインソース間電圧領域において、電流源として動作する
    飽和(saturation)とよばれる特性も非常に重要。
  - ゲート容量はゲート・ドレイン間とゲート・ボディ間にも存在するが、
    単純に見映えの問題でゲートソース間のみの記載とした



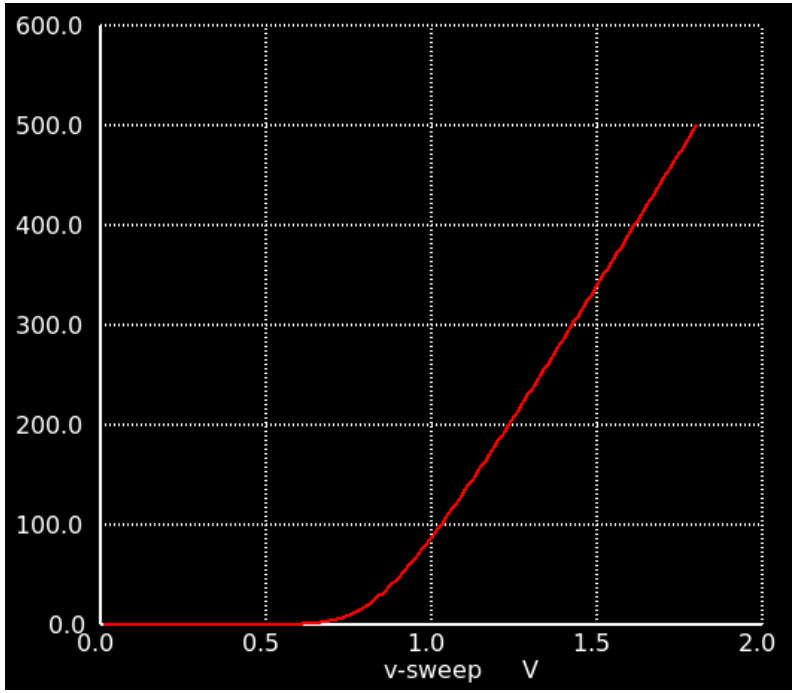
可変抵抗 vs NMOS





R_{DS}（GΩ）

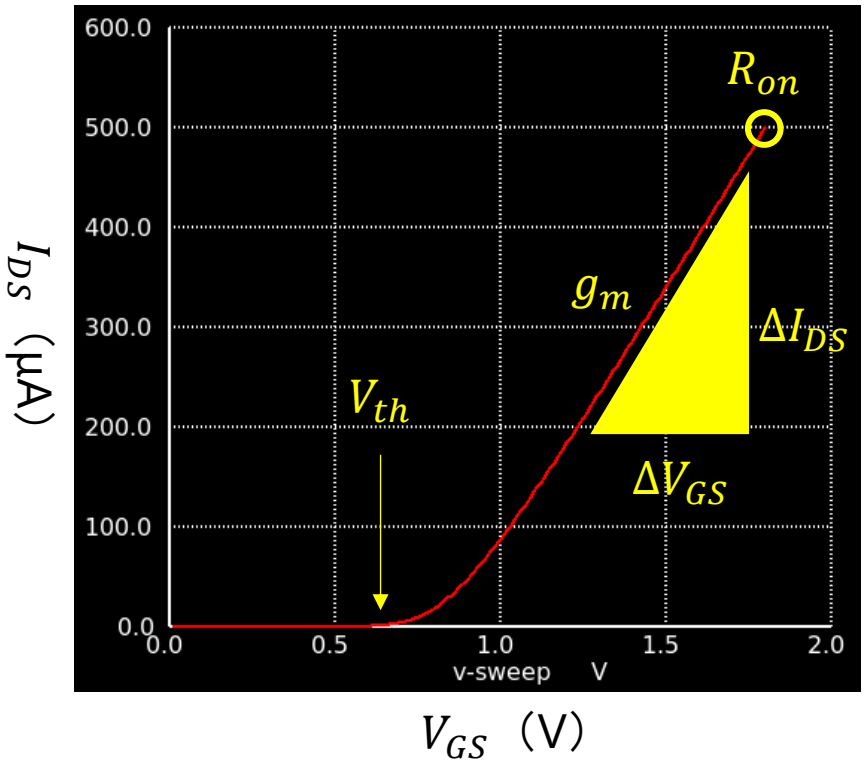
V_{GS}（V）



I_{DS}（μA）

V_{GS}（V）

NMOSの特性



- V_{th} 閾値電圧
  電流が流れ始めるゲートソース間電圧の閾値
- g_m トランスコンダクタンス
  ゲート電圧の変化分 対 ドレインソース間電流の変化分
- R_{on} オン抵抗
  オン状態での抵抗値

設計する前に…

- CMOSの特性を知りたい！
  - Vgs – Ids カーブ
  - 動作点解析 (Operational Point)

これらの詳細は、オペアンプ編にて詳しく解説します。

アナログ半導体の設計フロー

どのような設計をするか、どのような開発ツールを使用するか

アナログ半導体の設計フロー

1. 回路図（テストベンチ）を描く
2. シミュレーションをする
3. 回路図を基にレイアウトを描く
4. レイアウトを検証する (DRC, LVS)
5. レイアウトを基に寄生成分を考慮したシミュレーションをする
6. フレームに載せる

そもそもアナログ半導体回路設計に必要なものとは？

- **プロセスデザインキット　PDK**
  - **シミュレーションモデルライブラリ (SPICE)**
  - **検証ツール用ルールファイル (DRC, LVSなど)**
  - **スタンダードセルライブラリ**
    - **レイアウト (GDS)**
    - **ネットリスト(SPICE)**
    - **自動配置配線ルール(LEF)**
    - **Verilogシミュレーションライブラリ (LIB, V)**
  - **一部指定されたレイアウト (フレームなど)**

- PDKで指定された各種ツール　（EDAツール）
  - **回路図エディタ** or Verilogコンパイラ
    - **回路図エディタ：xschem,** Glade, KiCAD
  - **レイアウトエディタ** or 自動レイアウトツール
    - **レイアウトエディタ：klayout**, Glade, (KiCAD)
  - **SPICEシミュレータ** or Verilogシミュレータ
    - **SPICEシミュレータ：ngspice**, LTspice
  - **検証ツール**
    - **検証ツール：klayout**

PDKの場所

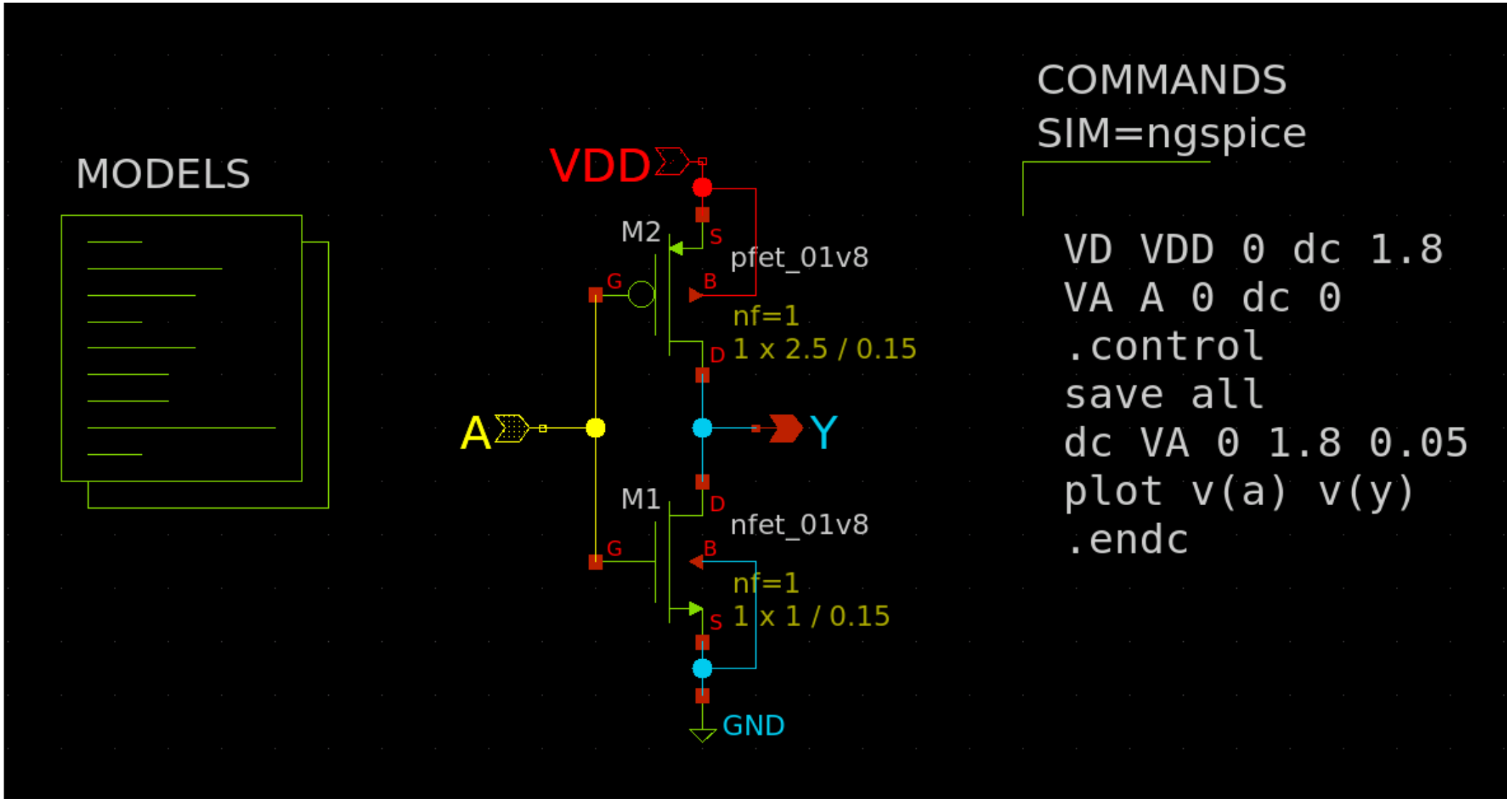
公式レポジトリ

- GF180 PDK
  https://github.com/google/gf180mcu-pdk

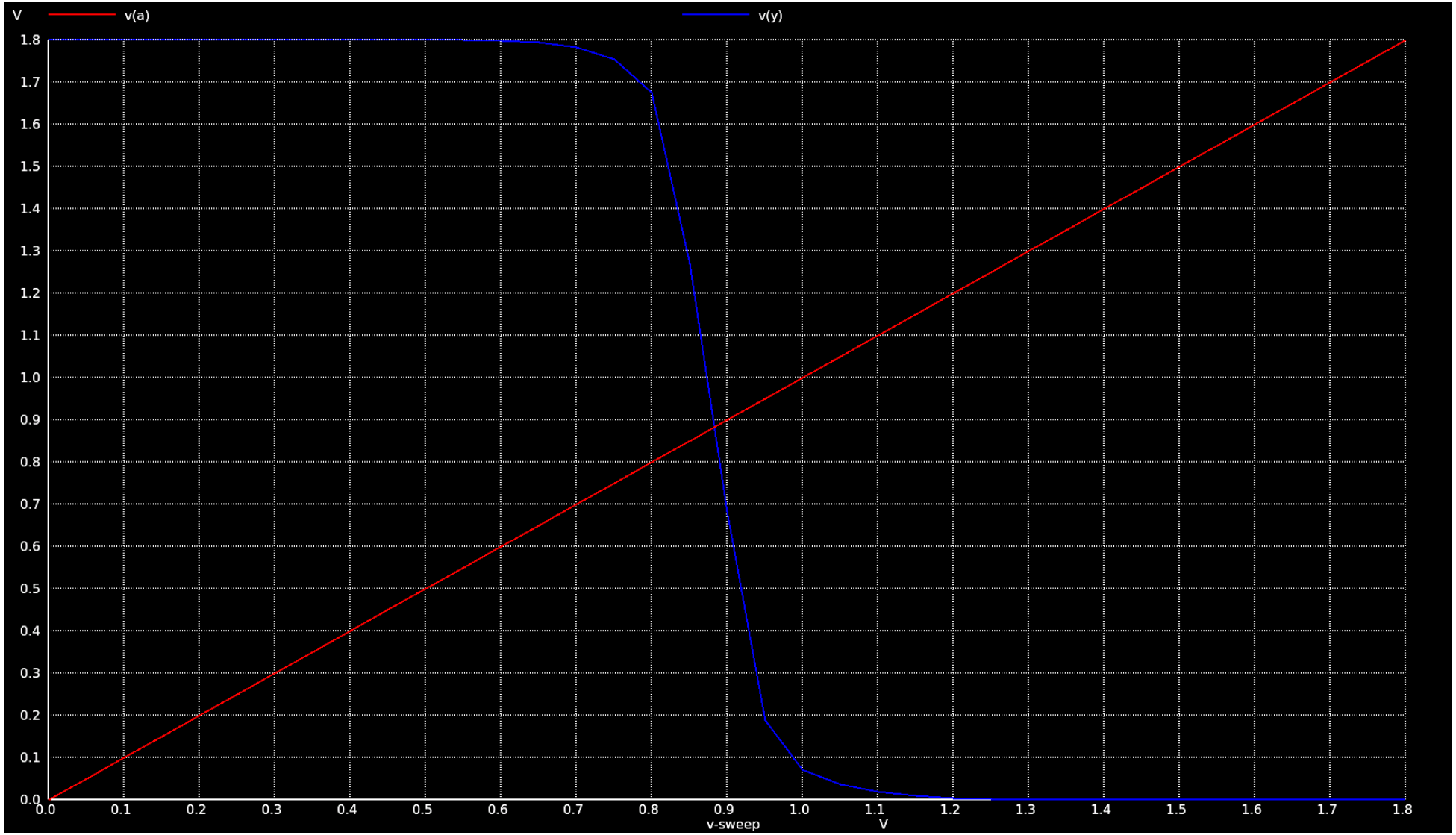
ISHI-KAIオリジナル

- EDAツールセットアップスクリプト
  https://github.com/ishi-kai/OpenEDA-PDK_SetupScript/
  - Mac用のVmwareイメージやWSLイメージなどもあります。

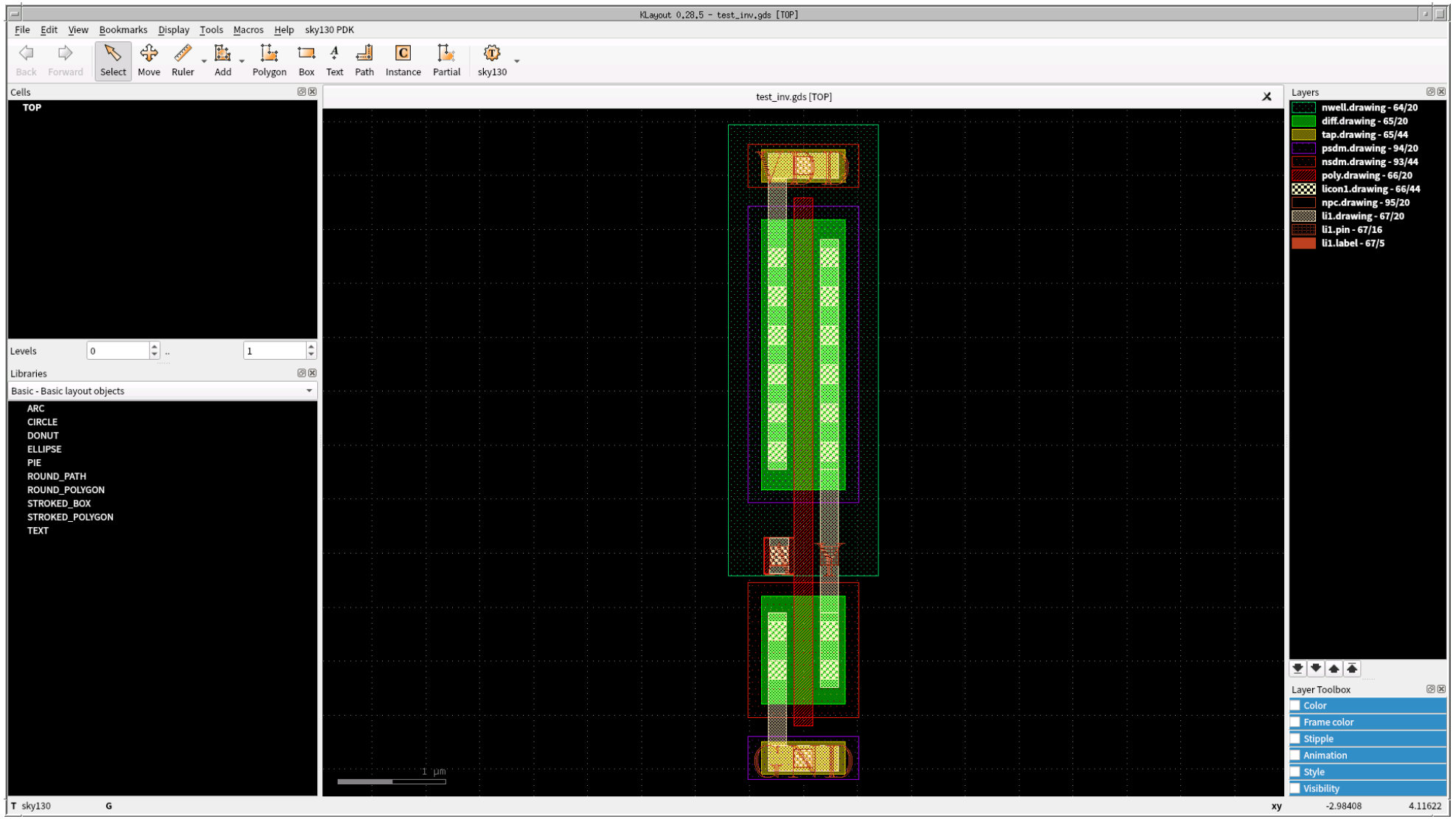
回路図(ベンチマーク)を描いて・・・



論理検証をして・・・

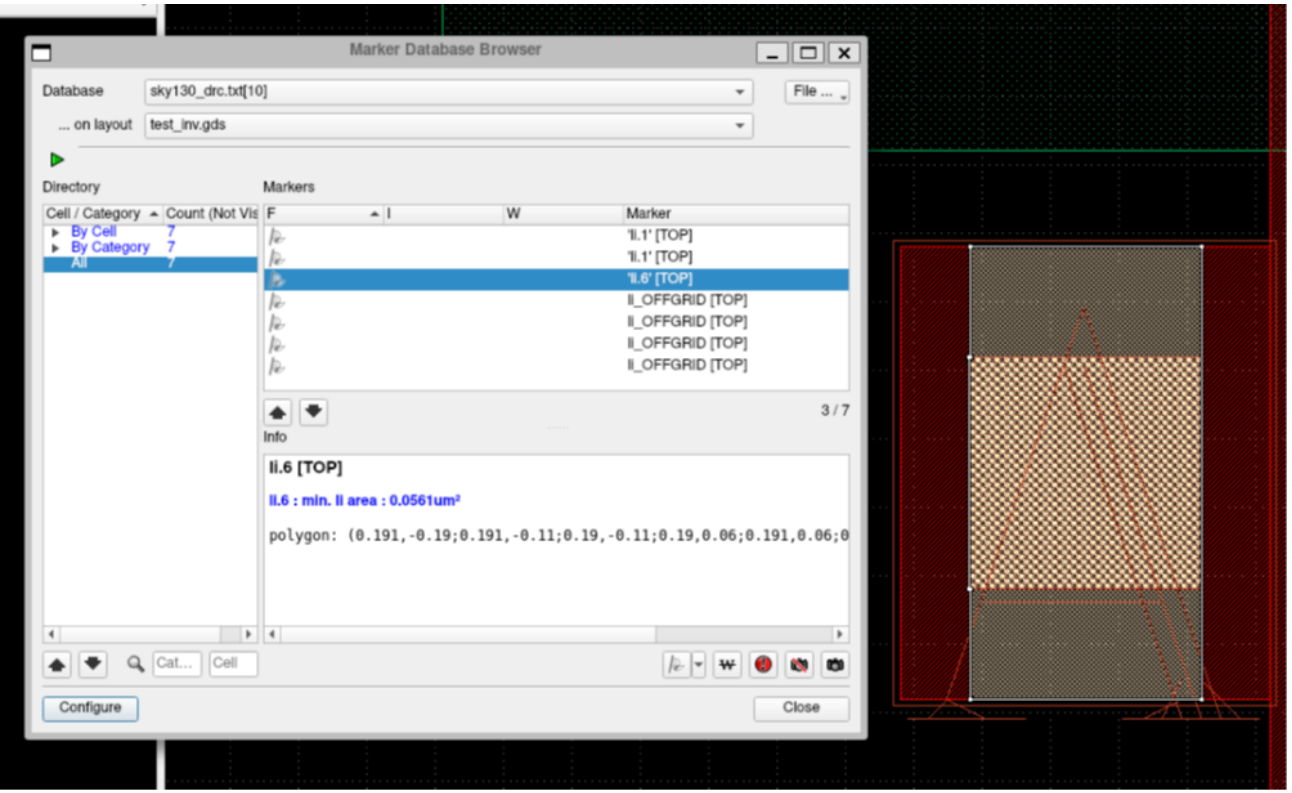

V
v(a)
v(y)
1.8
1.7
1.6
1.5
1.4
1.3
1.2
1.1
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1.0
1.1
1.2
1.3
1.4
1.5
1.6
1.7
1.8
v-sweep
V

レイアウトを描いて・・・

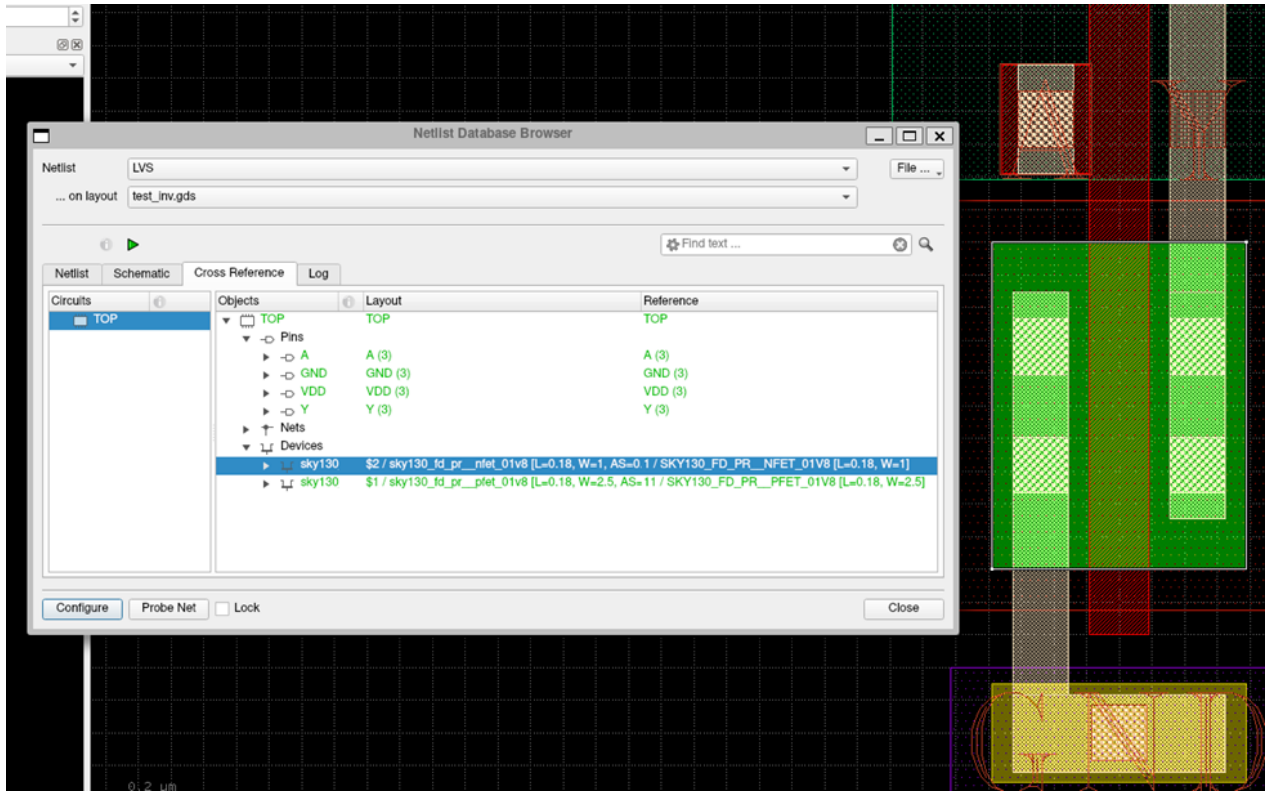

KLayout 0.28.5 - test_inv.gds [TOP]
File Edit View Bookmarks Display Tools Macros Help sky130 PDK
Back Forward Select Move Ruler Add Polygon Box Text Path Instance Partial sky130
Cells
TOP
Levels 0 .. 1
Libraries
Basic - Basic layout objects
ARC
CIRCLE
DONUT
ELLIPSE
PIE
ROUND_PATH
ROUND_POLYGON
STROKED_BOX
STROKED_POLYGON
TEXT
test_inv.gds [TOP]
1 µm
Layers
nwell.drawing - 64/20
diff.drawing - 65/20
tap.drawing - 65/44
psdm.drawing - 94/20
nsdm.drawing - 93/44
poly.drawing - 66/20
licon1.drawing - 66/44
npc.drawing - 95/20
li1.drawing - 67/20
li1.pin - 67/16
li1.label - 67/5
Layer Toolbox
Color
Frame color
Stipple
Animation
Style
Visibility
T sky130 G
xy -2.98408 4.11622

レイアウトを検証して・・・

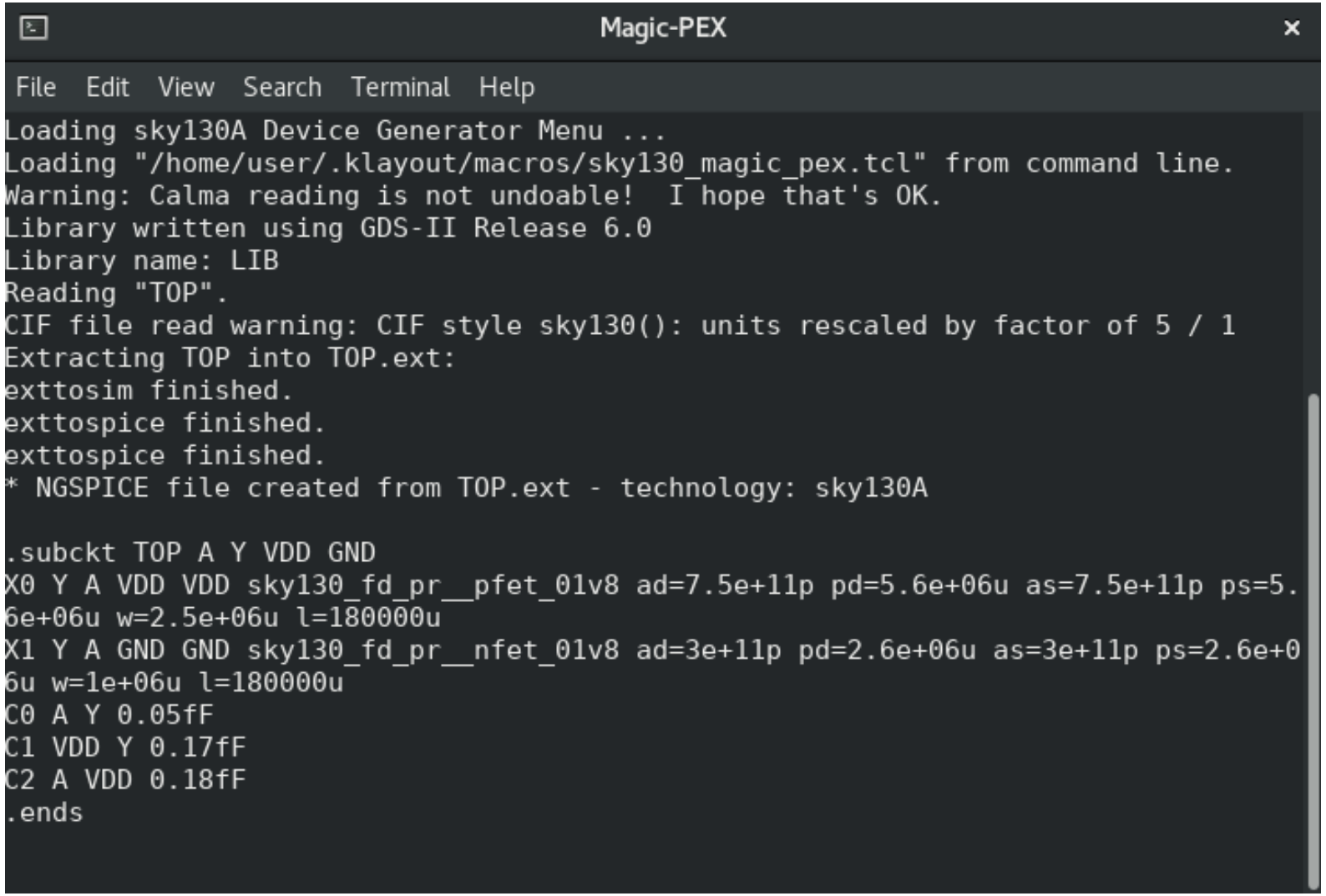
DRC



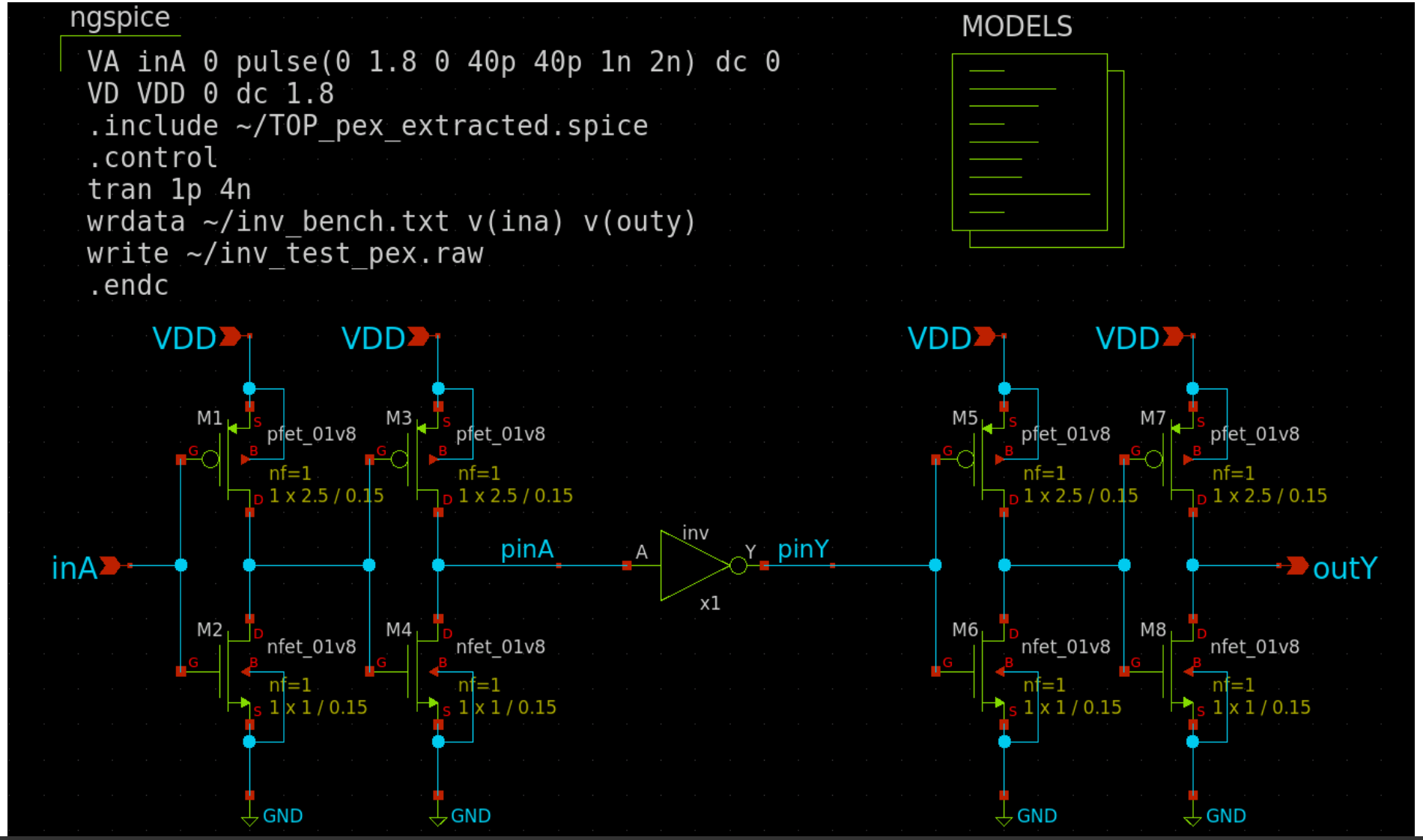
LVS



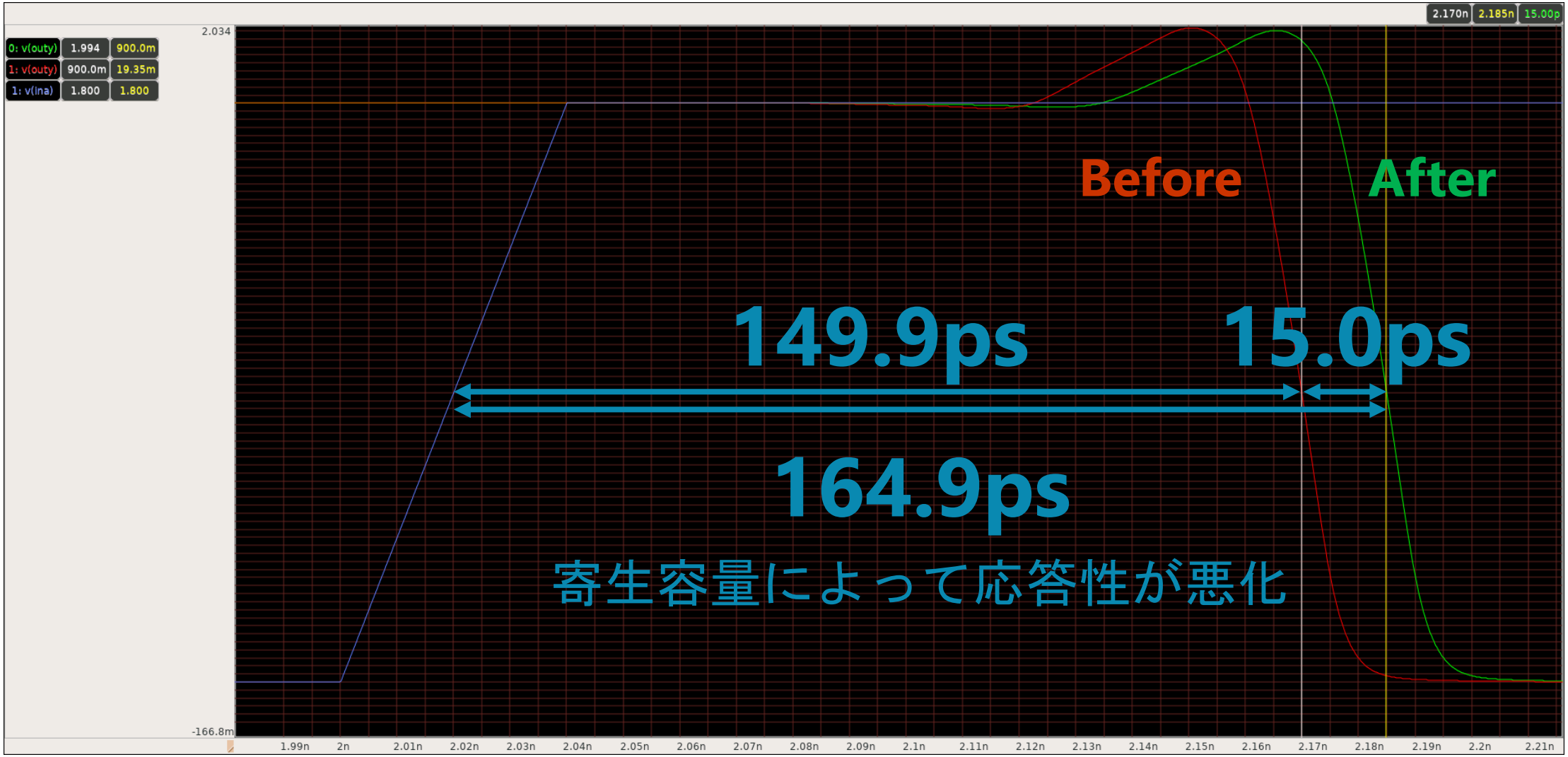
寄生成分を抽出して・・・


Magic-PEX
File Edit View Search Terminal Help
Loading sky130A Device Generator Menu ...
Loading "/home/user/.klayout/macros/sky130_magic_pex.tcl" from command line.
Warning: Calma reading is not undoable! I hope that's OK.
Library written using GDS-II Release 6.0
Library name: LIB
Reading "TOP".
CIF file read warning: CIF style sky130(): units rescaled by factor of 5 / 1
Extracting TOP into TOP.ext:
exttosim finished.
exttospice finished.
exttospice finished.
* NGSPICE file created from TOP.ext - technology: sky130A
.subckt TOP A Y VDD GND
X0 Y A VDD VDD sky130_fd_pr__pfet_01v8 ad=7.5e+11p pd=5.6e+06u as=7.5e+11p ps=5.
6e+06u w=2.5e+06u l=180000u
X1 Y A GND GND sky130_fd_pr__nfet_01v8 ad=3e+11p pd=2.6e+06u as=3e+11p ps=2.6e+0
6u w=1e+06u l=180000u
C0 A Y 0.05fF
C1 VDD Y 0.17fF
C2 A VDD 0.18fF
.ends

ベンチマークを作成して…



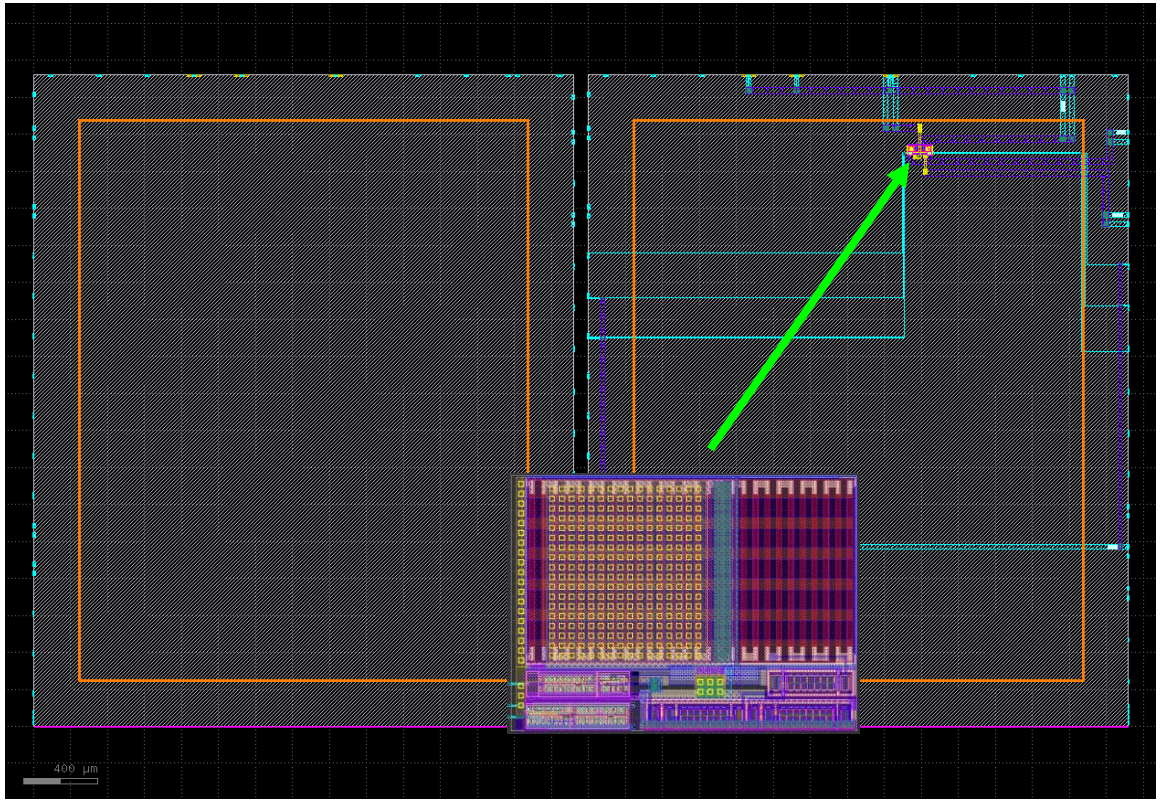
ポストレイアウトシミュレーションをする

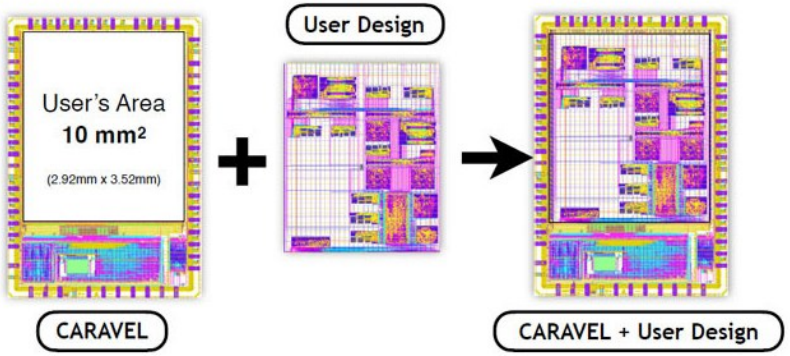


全て完成したら指定のフレームに回路を載せる

例: Caravel harness (SKY130)

- 2.92 mm × 3.52mm user's area
- 38 IO ports
- 4 Power ports
- 128 Logic analyzer probes





九州大学大学院システム情報科学府附属価値創造型半導体人材育成センター主催 2025年度実習シリーズで採用

2025_九州大学価値創造型半導体人材育成センター_実習シリーズ③

集積回路設計・評価ハンズオンセミナー

1

九州地域では、TSMC/JASMをはじめとする半導体製造メーカの集積が進んでおり、「オール九州」での半導体エコシステム構築が始まっています。しかし、その中核を担う設計人材がとても少ないことが問題となっています。そこで九州大学価値創造型半導体人材育成センターでは、設計人材育成のため、半導体の設計から評価までの技術を一気通貫で習得できるセミナーを開催します。

内容：CMOS集積回路（Inverter）をOpen Toolにより設計し、レイアウトします。各自のレイアウトは東海理化のシャトルサービスにより試作します。また、出来上がったICチップを測定し、評価します。ICチップは持ち帰ることができます。

日時場所：設計ハンズオンセミナー：　2025年9月24日～25日、九州大学　W2-325号室
　　ICチップお渡し会＆評価実習：　2026年3月（未定）、九州大学内
※　9月、3月の2回で1つのセミナーです。WindowsノートPCが必要です。

または、こちら

申込み方法：・【件名】に参加したいセミナーの名称を記入ください。
　　・お名前、お名前（ローマ時）、ご所属、メールアドレス　をお知らせください。

申し込み先：class_program-at-ecsvc.ed.kyushu-u.ac.jp　-at-　=@

締め切り：7月23日
先着順で、定員になり次第しめきりとさせていただきます

ご協力：OPEN SUSI, ISHI-KAI, AIST solutions、株式会社東海理化

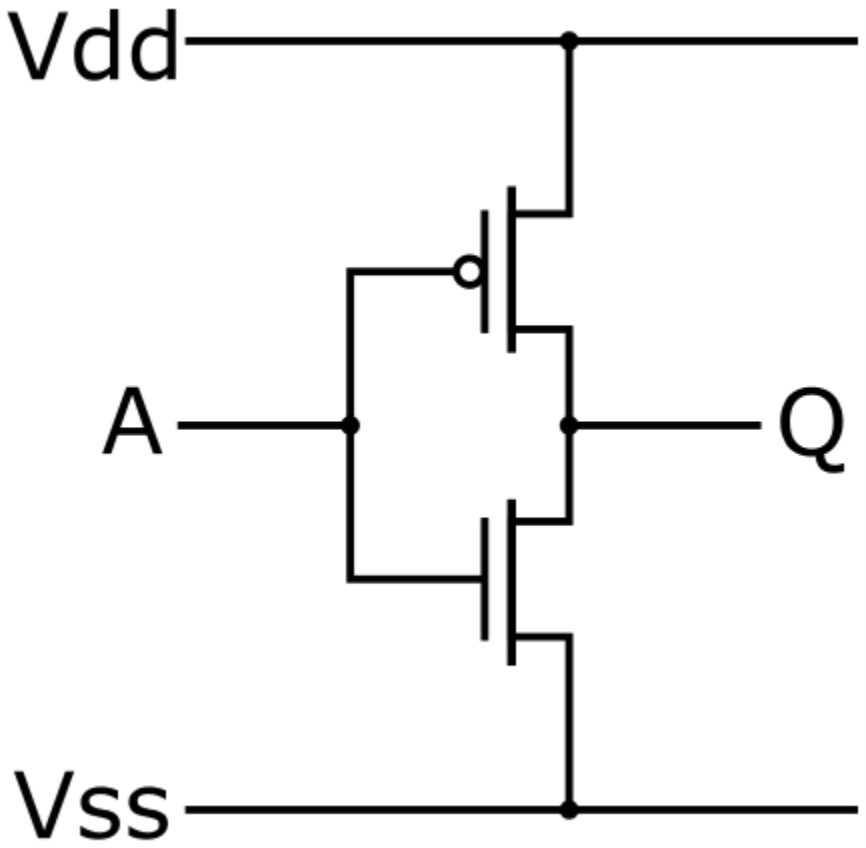
書籍紹介コーナー

- タイトル：Open Source Silicon Magazine vol.1
    - サブタイトル：はじめてのIC 設計～オリジナルのインバータを作ってみよう～
- 内容
    - 本日の解説から、さらに一歩『手前』や端折ってしまっている部分を解説した内容
    - 技術書典「第10回刺され！技術書アワード」のニュースタンダード部門のファイナリストに選出されました！
- 価格
    - 電子セット
        - 無料

アナログ半導体設計デモンストレーション

CMOSインバータ（NOT論理回路）作成

CMOSインバータ（NOT論理回路）



入力A	出力Q
L	H
H	L

入力A	出力Q
Vss	Vdd
Vdd	Vss

アナログ半導体の設計フロー

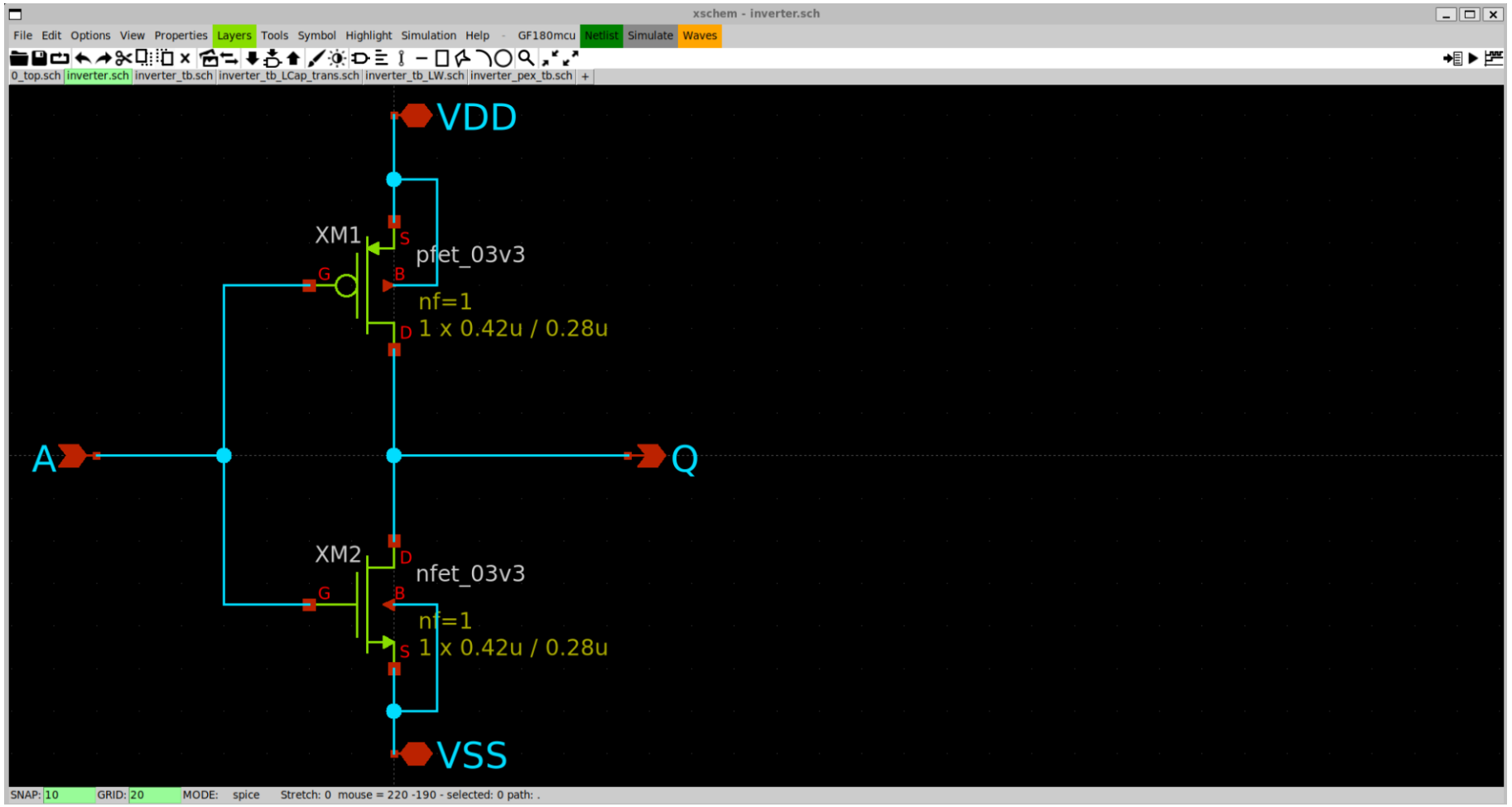
1. **回路図を描く**
2. シミュレーションをする
3. 回路図を基にレイアウトを描く
4. レイアウトを検証する
5. レイアウトを基に寄生成分を考慮したシミュレーションをする
6. フレームに載せる

論理検証

- インバータでは入力に対して反転した出力が確認できれば良い
  - 入力 0 V → 出力 Vdd V, 入力 Vdd V → 出力 0 V
  - DC解析でCMOSインバータの動作を見てみる

- DC解析　　電圧を変動させたときの定常応答
- tran解析　時間を変動させたときの過渡応答
- AC解析　　周波数を変動させたときの定常応答

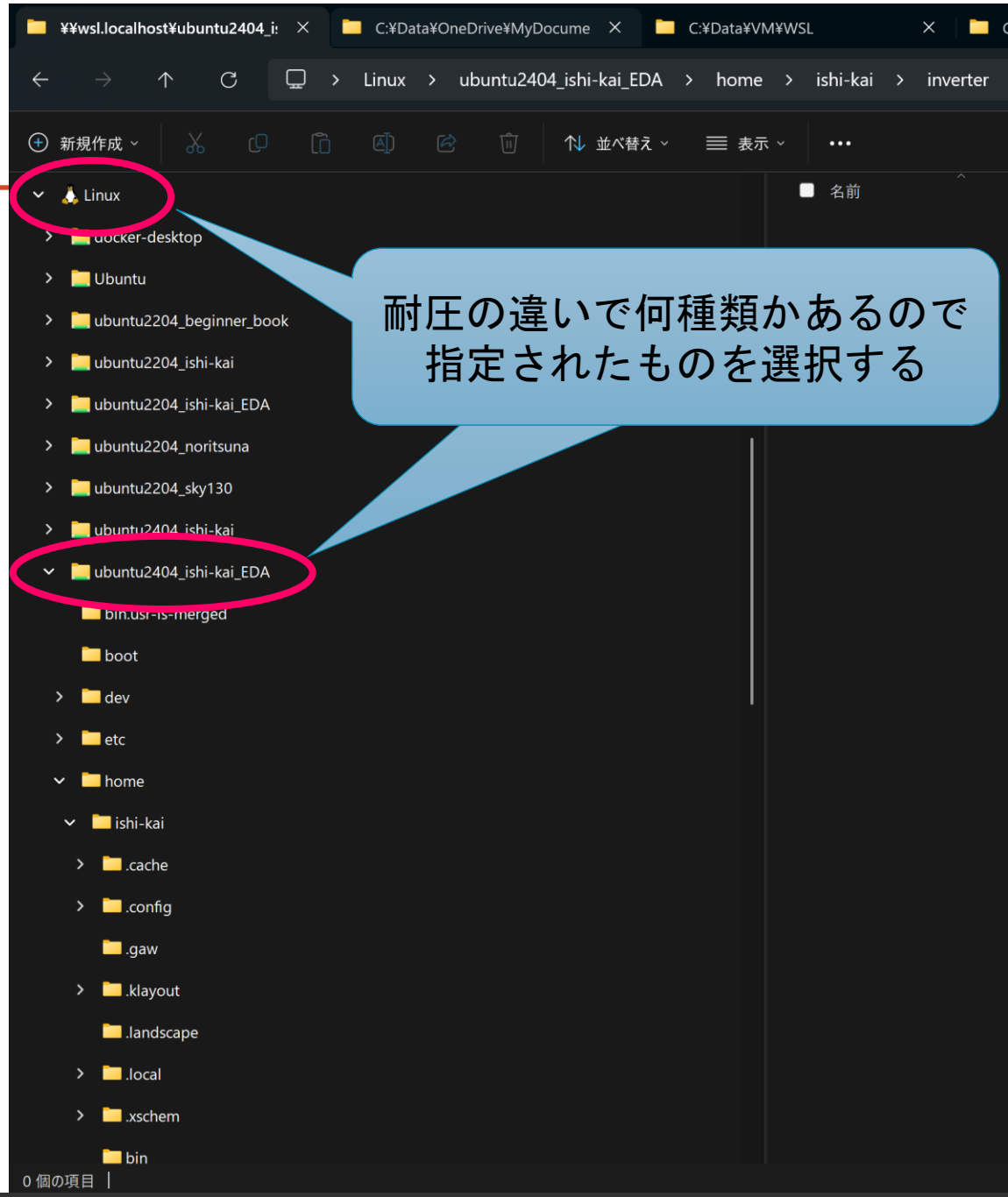
CMOSインバーター回路の例



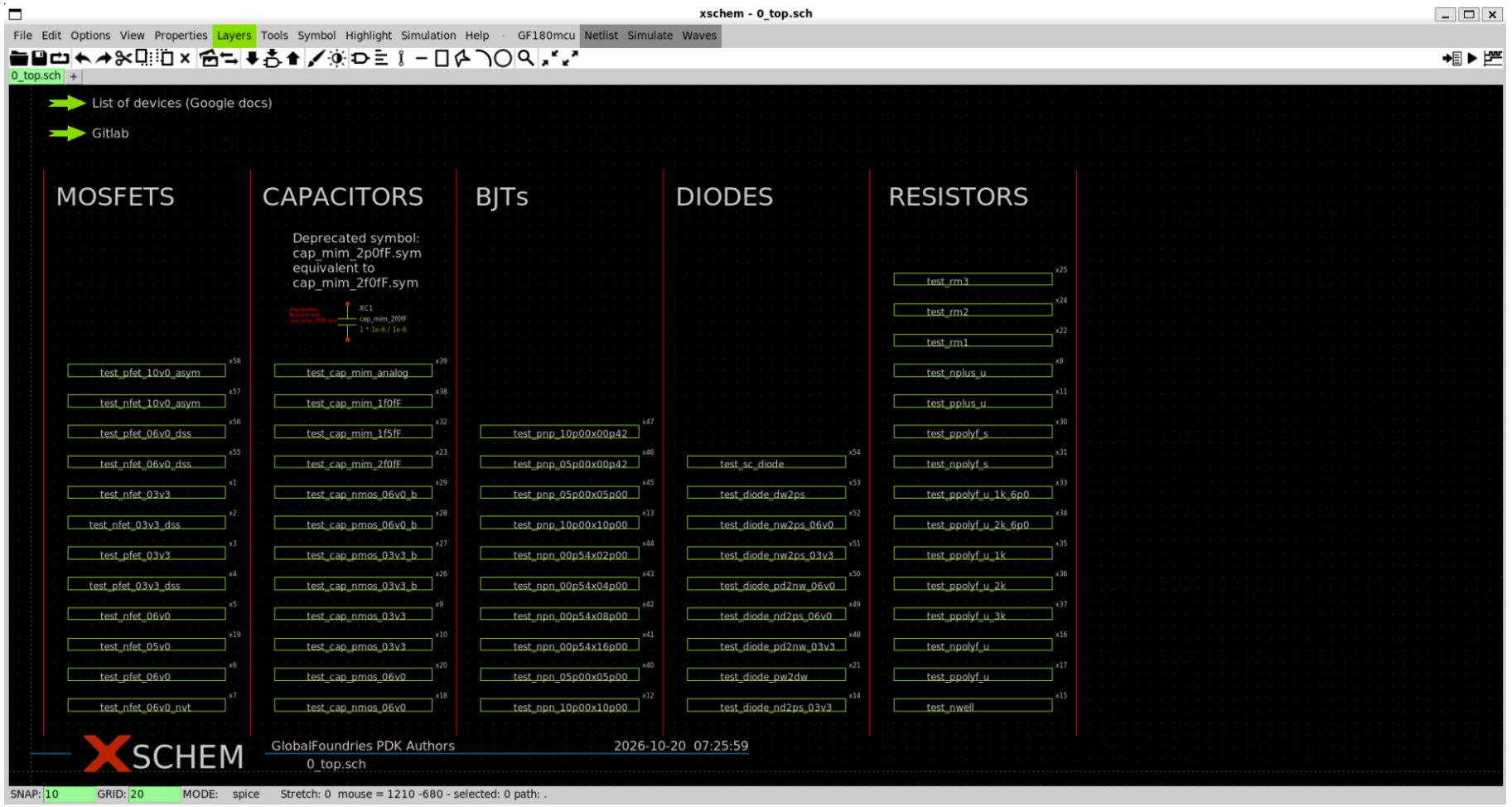
作業用ディレクトリを作成する

- 本日の作業用ディレクトリを作成し、xschemを起動します。WSL上で下記コマンドを実行してください。
  - mkdir ~/inverter/
  - cd ~/inverter/
  - xschem &

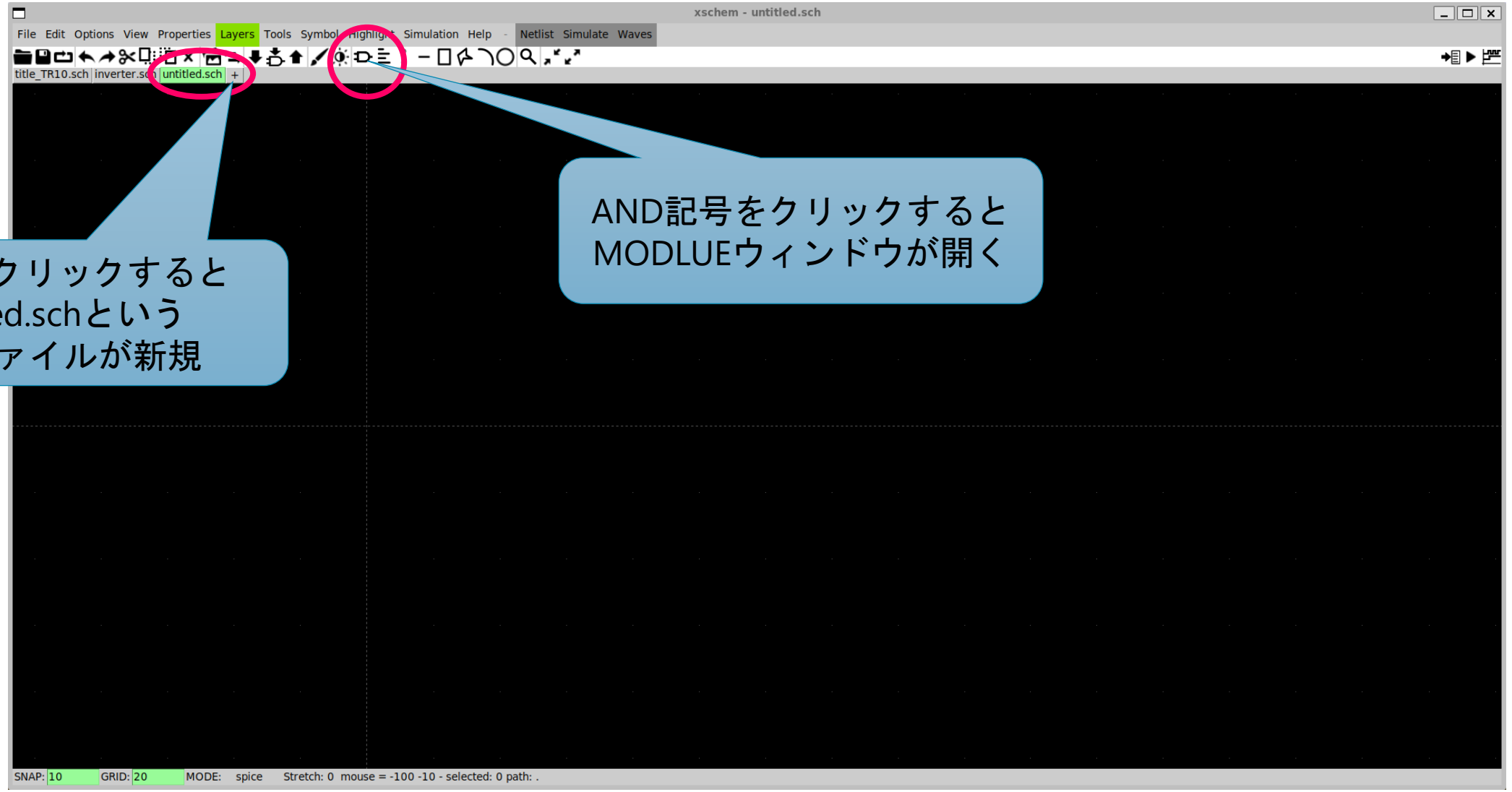
- 作成したファイルへのアクセス方法
  - Windows上のエクスプローラーからアクセス可能です。



xschemの起動画面(この画面以外はインストールに失敗しています)



xschem使い方：新規回路図とMODULEウィンドウ

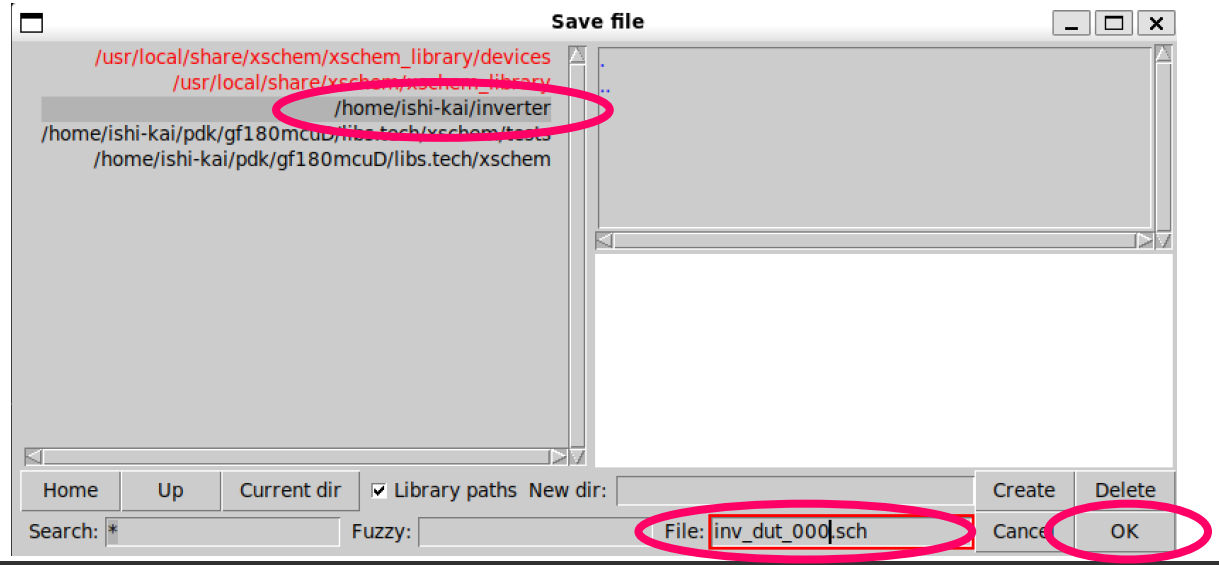


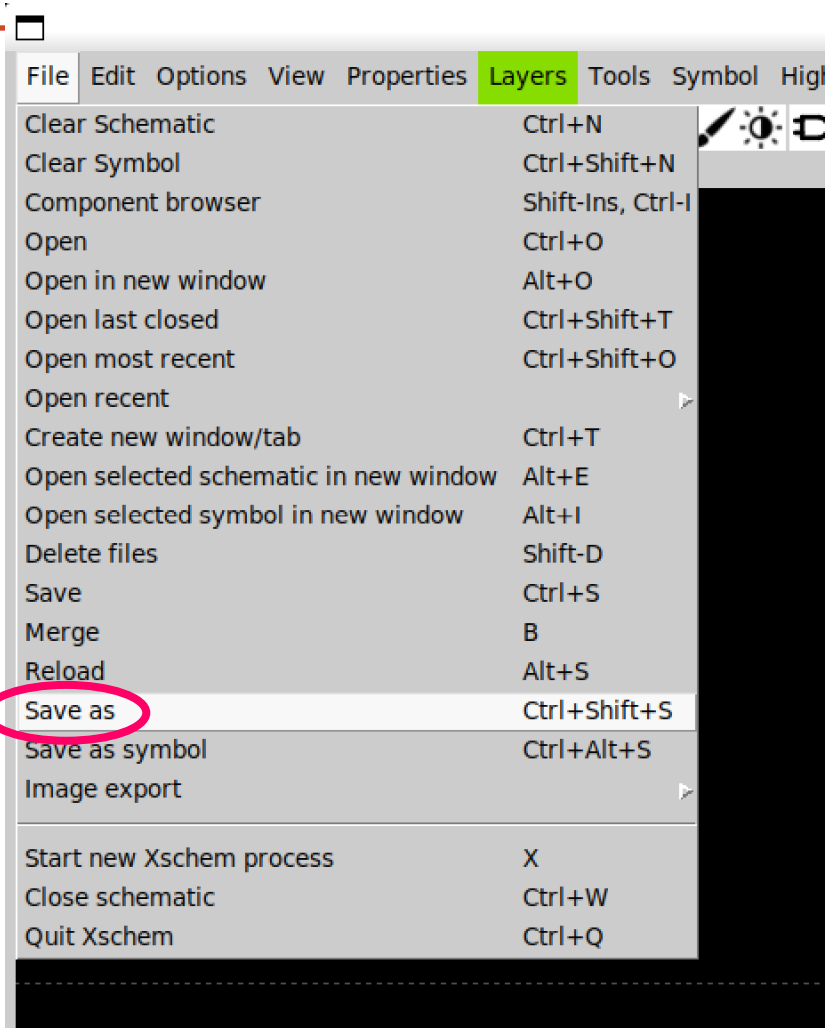
+タブをクリックすると
untitled.schという
新規ファイルが新規

AND記号をクリックすると
MODLUEウィンドウが開く

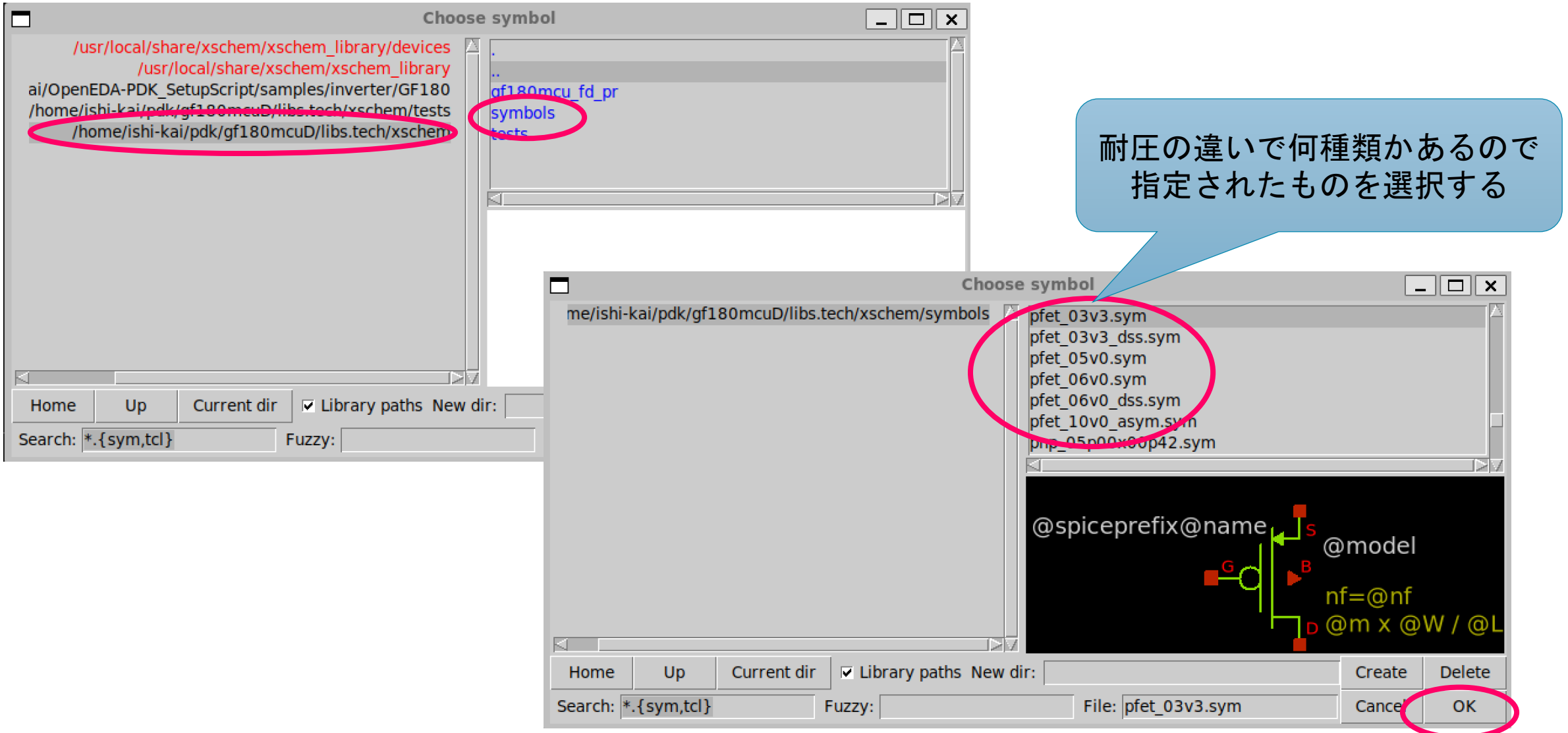
ファイルを保存する

1. File -> Save As を選択する
    1. Save Fileダイアログが表示される
2. 「/home/ishi-kai/inverter」を選択する
3. 「File:」に「inv_dut_[配布されたDUT番号].sch」を入力する
4. 「OK」をクリックする

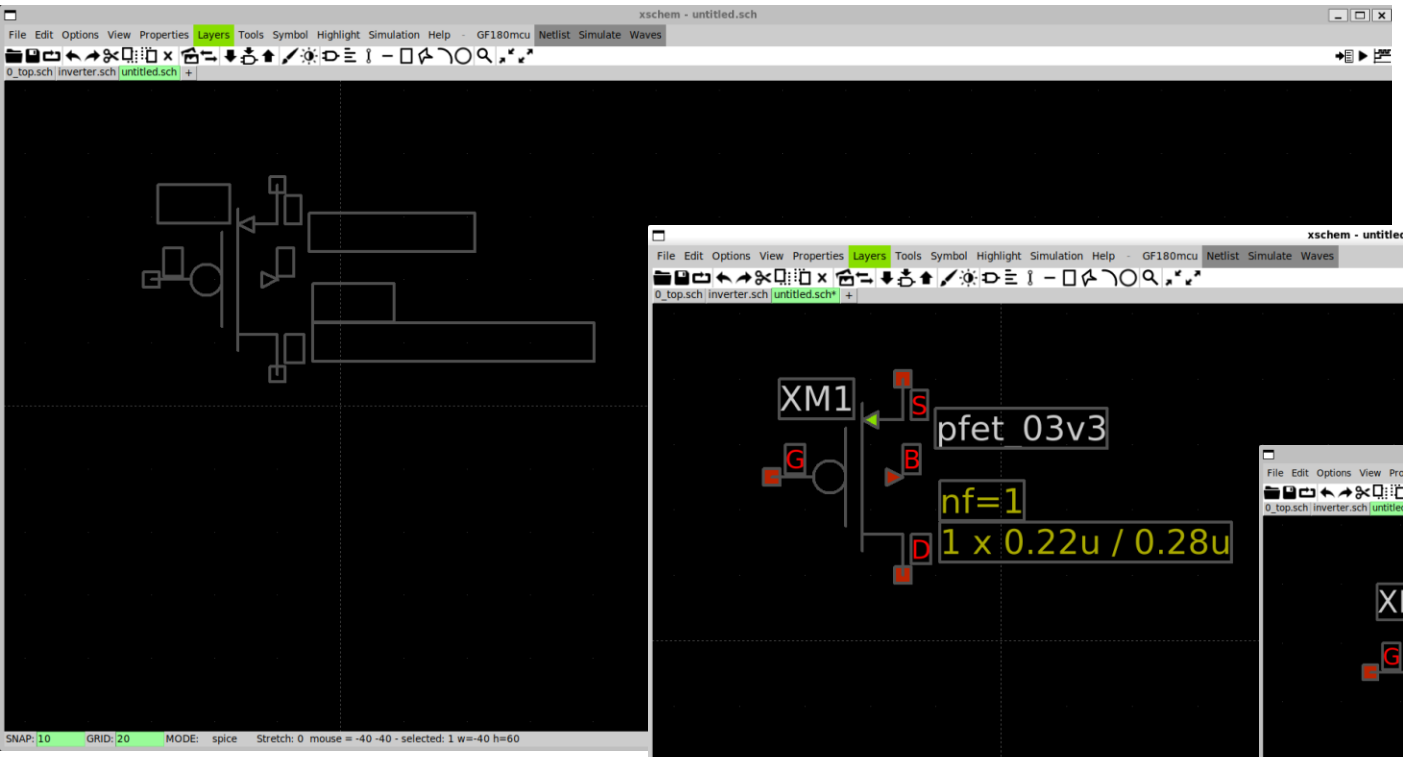


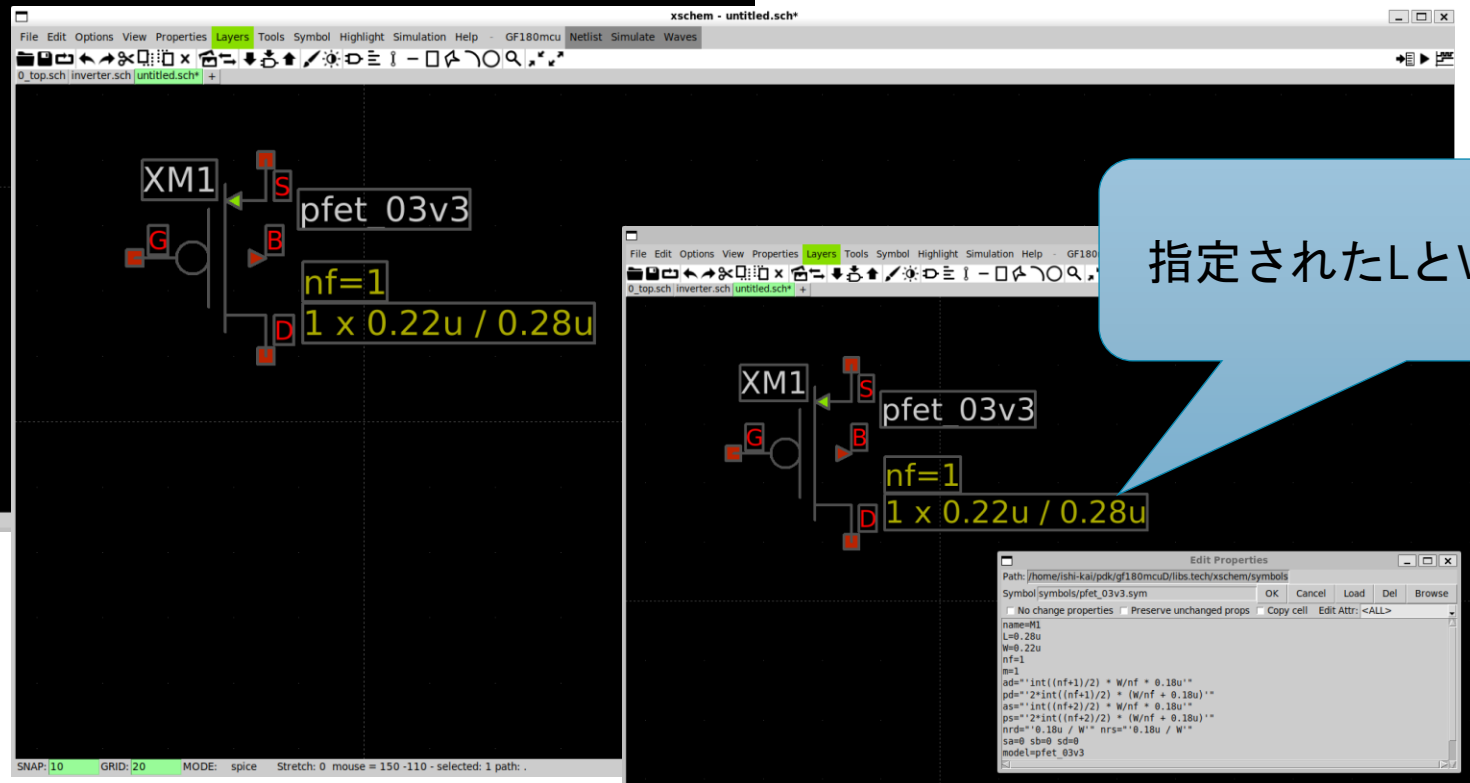


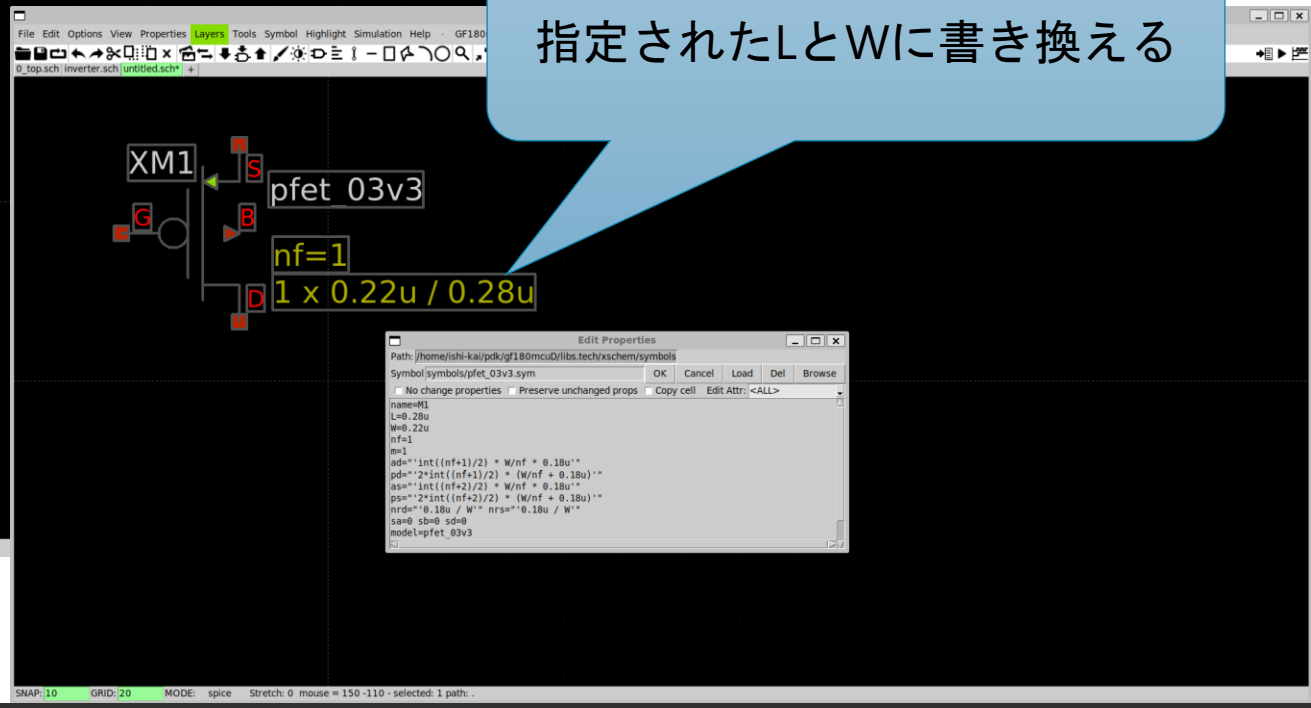
xschem使い方： gf180mcuD/libs.tech/のMODULEの選択：P-FET



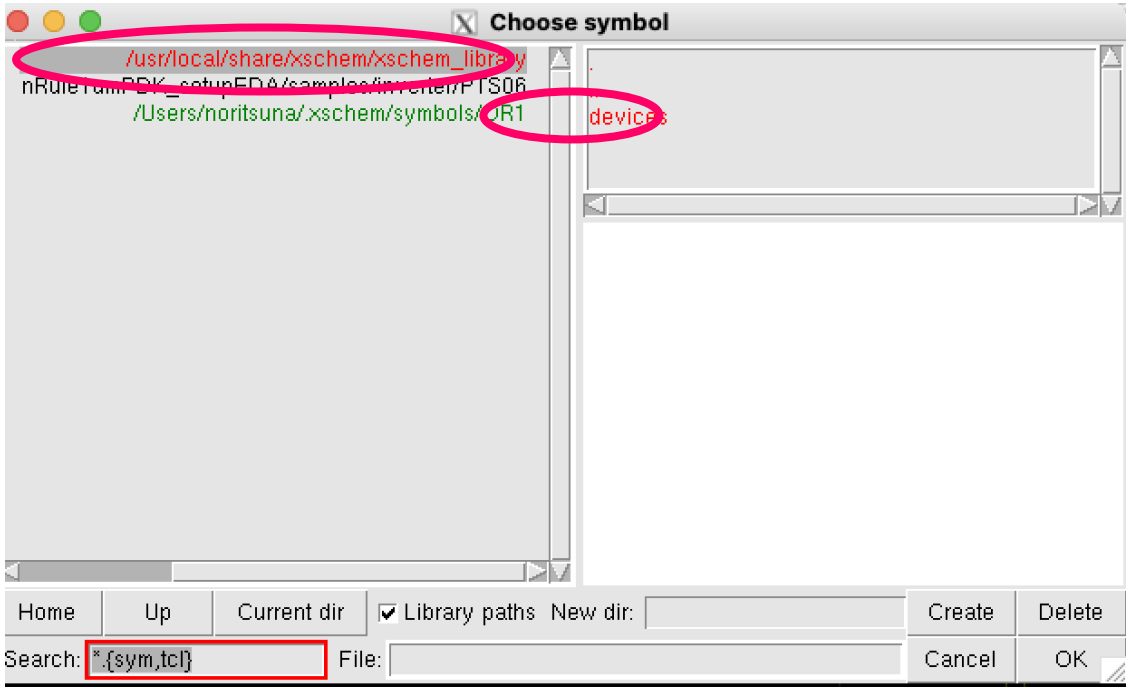
xschem使い方： MODULEの配置とプロパティ

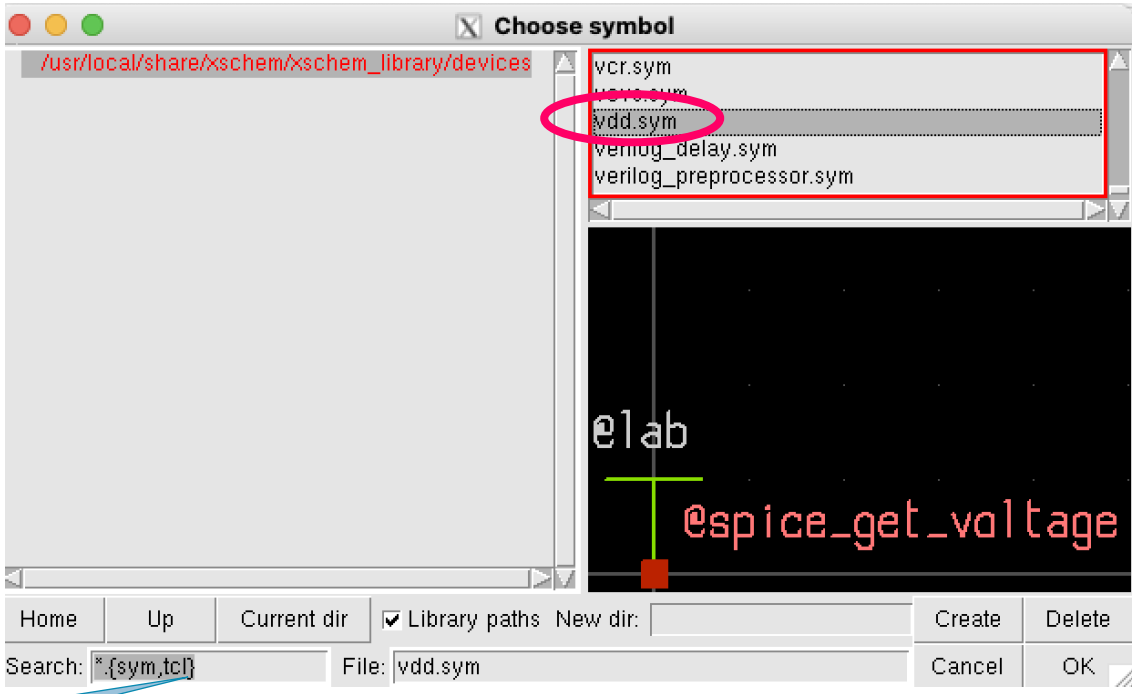






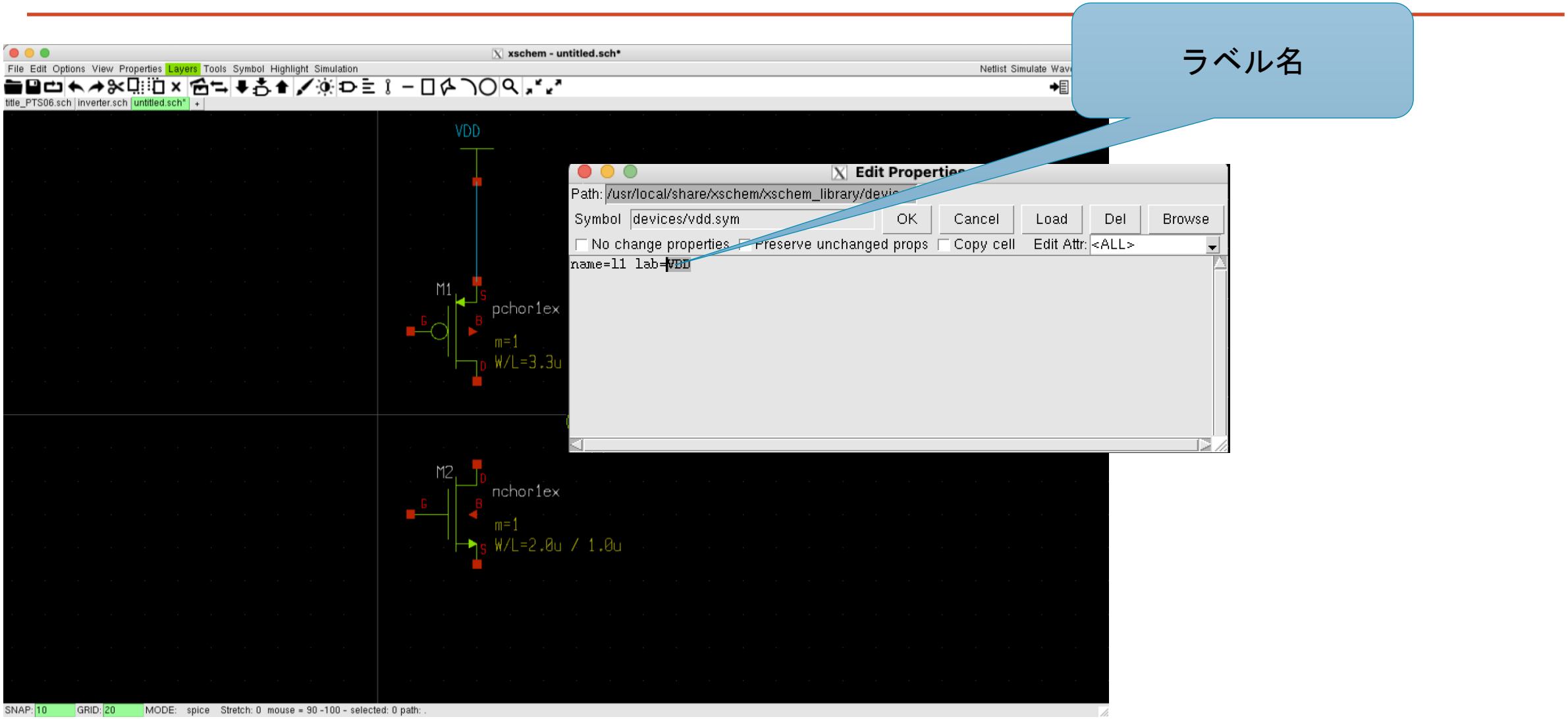
xschem使い方： 標準のMODULEの選択：VDD



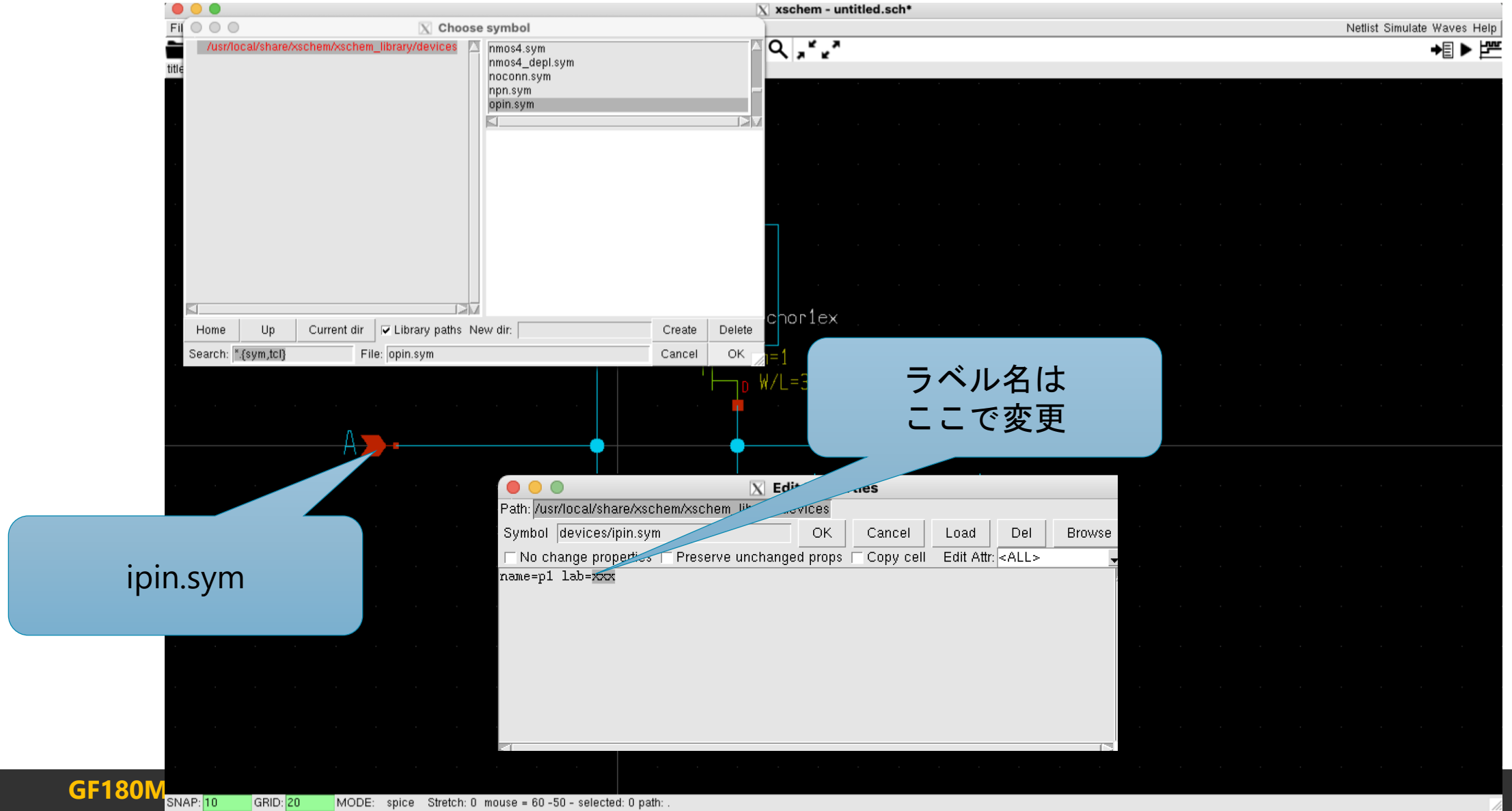


検索も可能

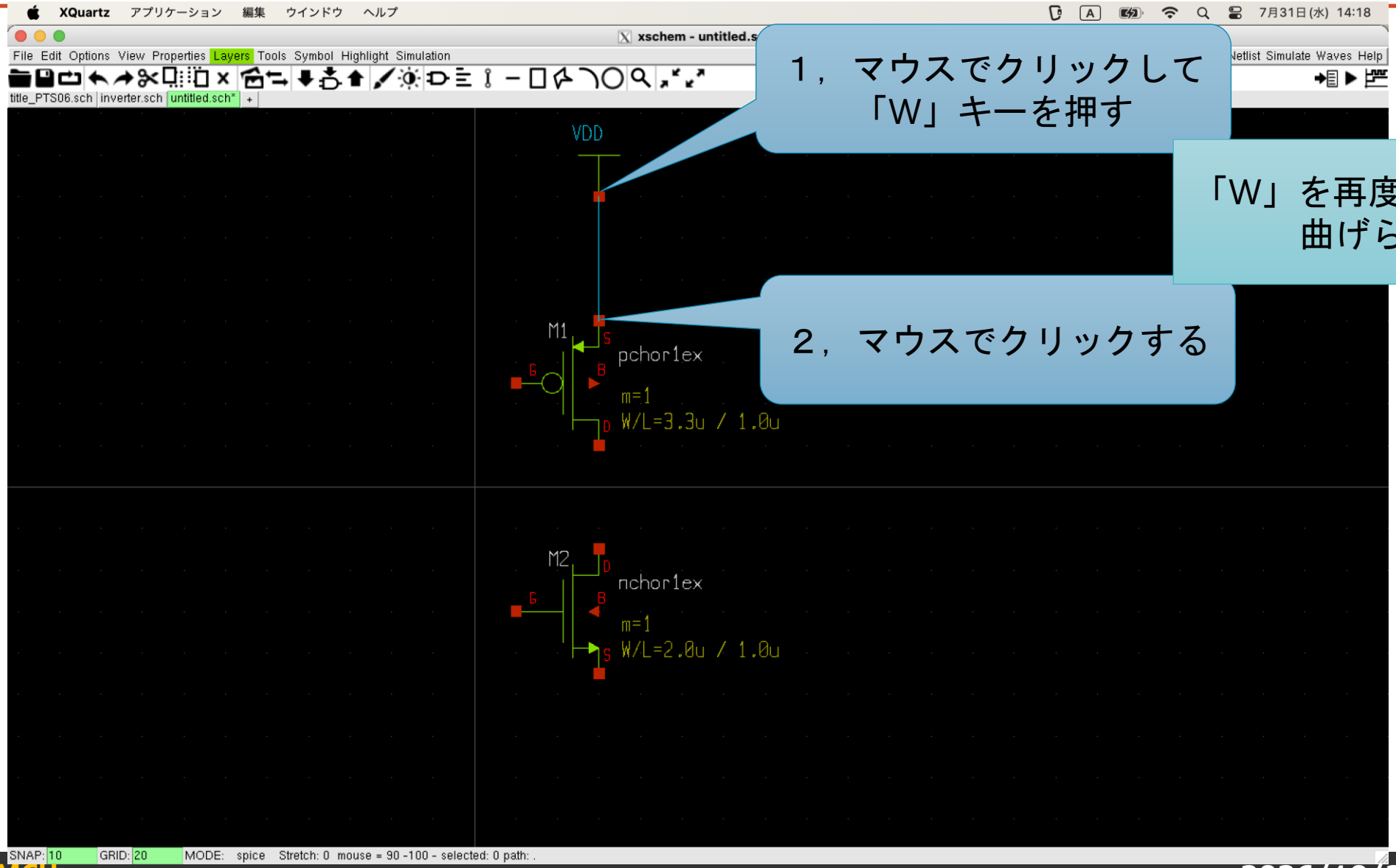
xschem使い方： 標準のMODULEの配置とプロパティ：VDD



xschem使い方： 標準のMODULEの選択：ipin, opin, iopin



xschem使い方： wireの引き方

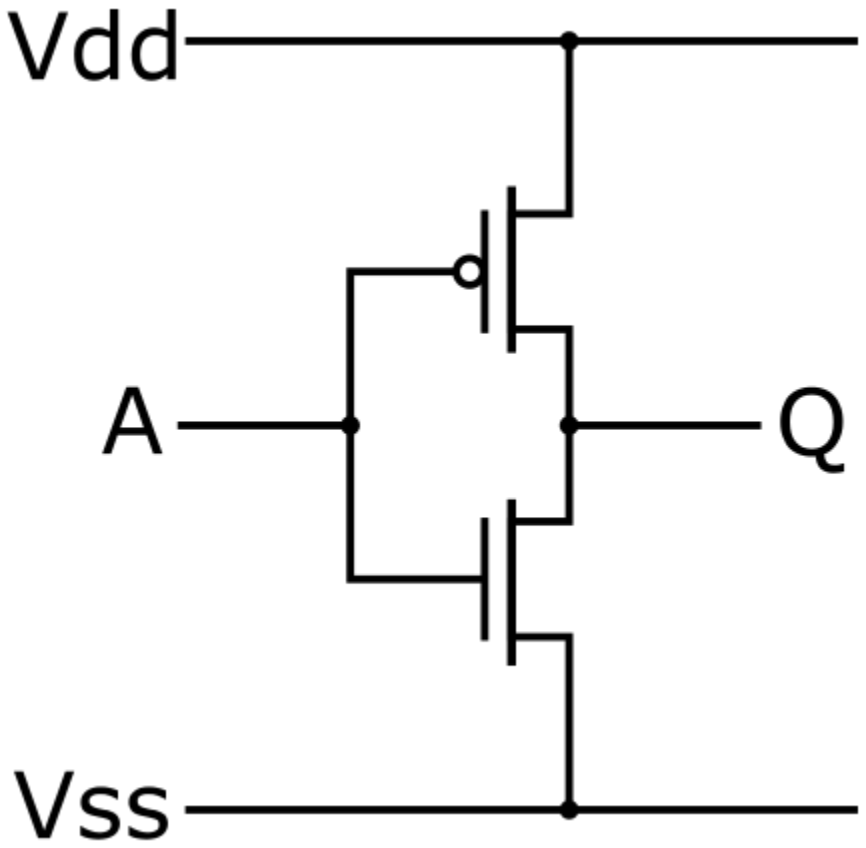


1，マウスでクリックして「W」キーを押す

「W」を再度押すと90度曲げられる

2，マウスでクリックする

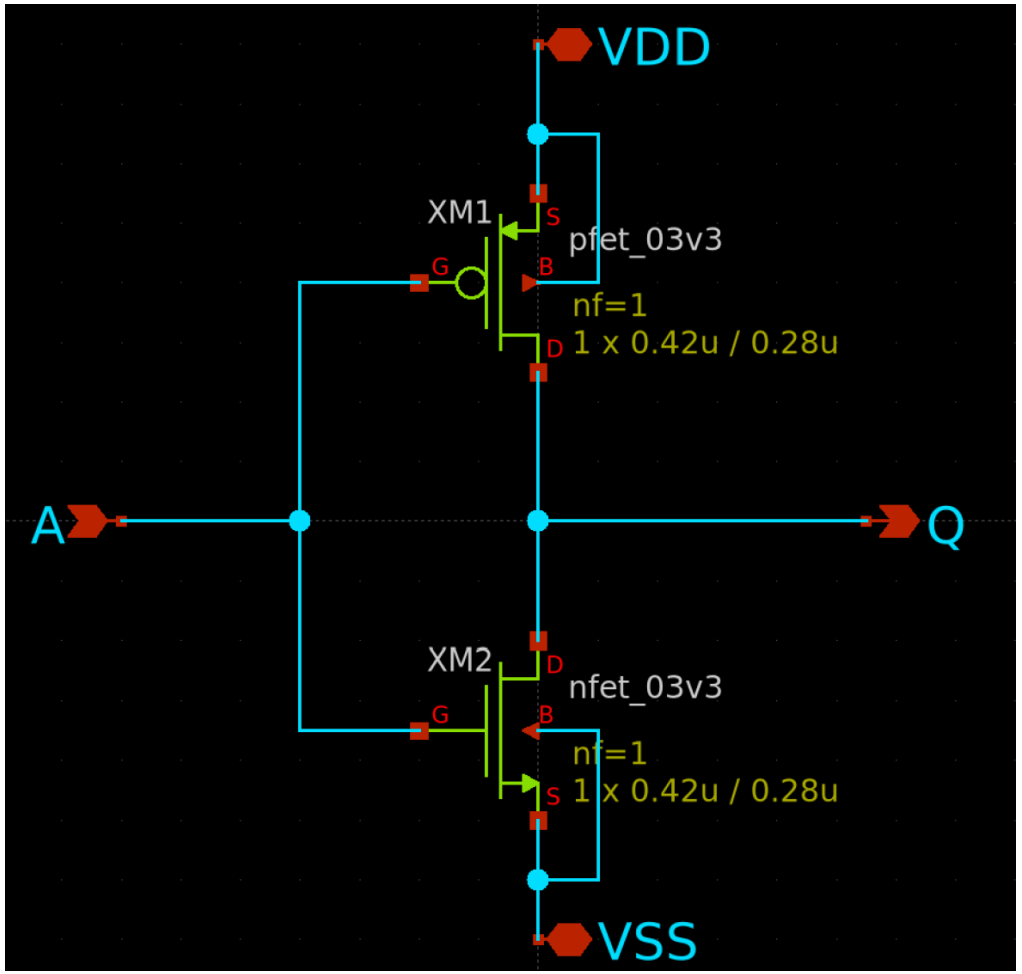
ハンズオン：CMOSインバータを作る



入力A	出力Q
L	H
H	L

入力A	出力Q
Vss	Vdd
Vdd	Vss

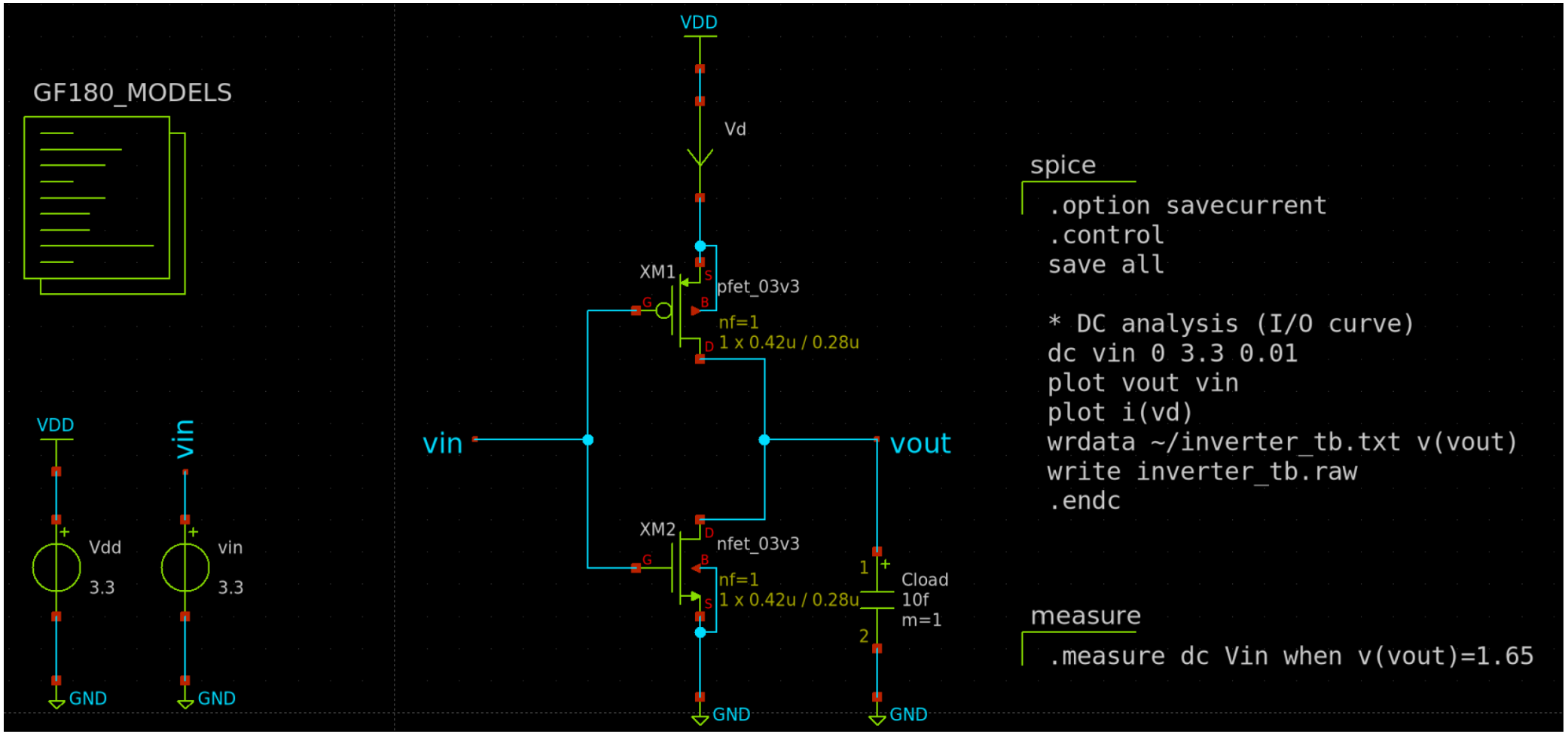
ハンズオン：CMOSインバータのサンプル



アナログ半導体の設計フロー

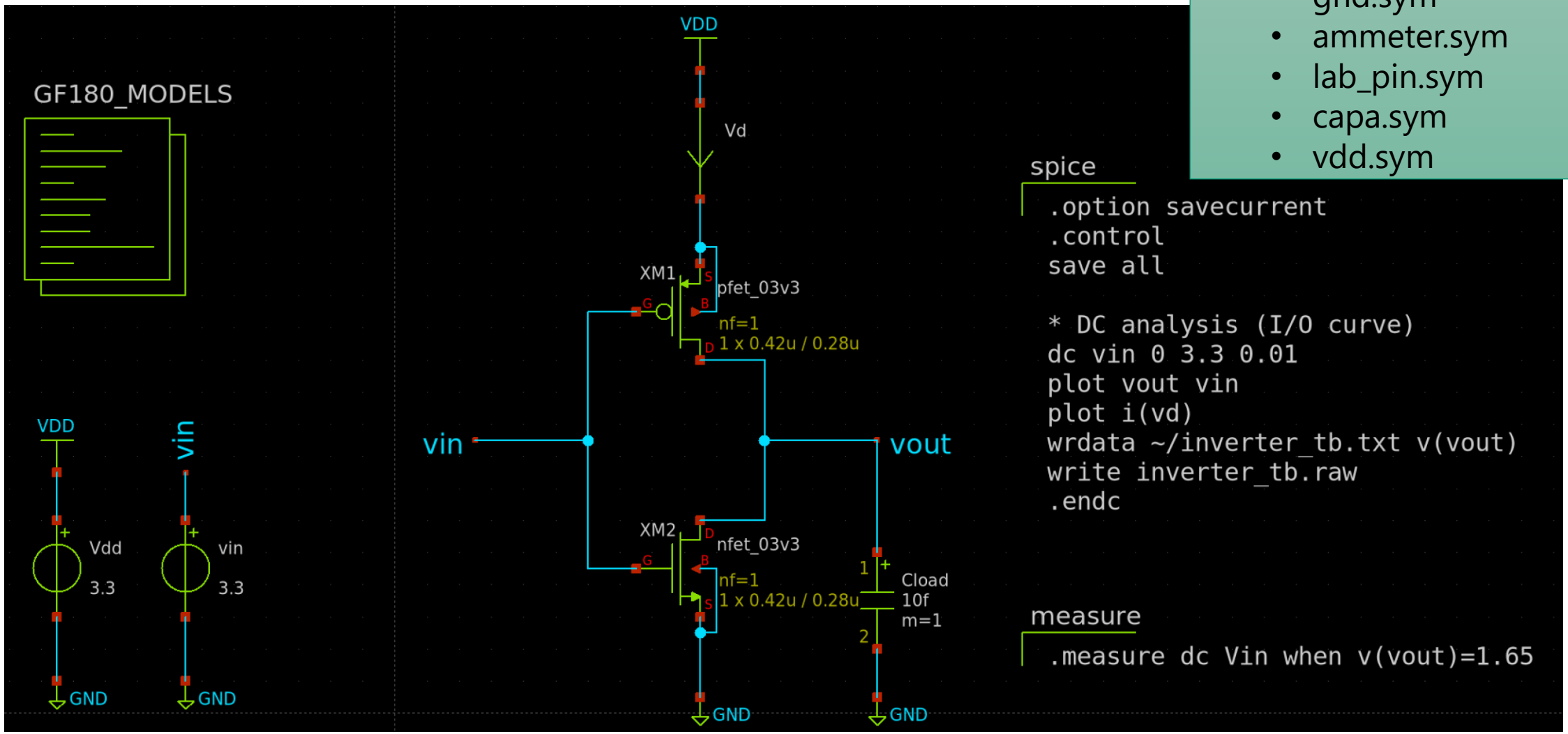
1. 回路図を描く
2. **シミュレーションをする**
3. 回路図を基にレイアウトを描く
4. レイアウトを検証する
5. レイアウトを基に寄生成分を考慮したシミュレーションをする
6. フレームに載せる

xschem：ベンチマーク例



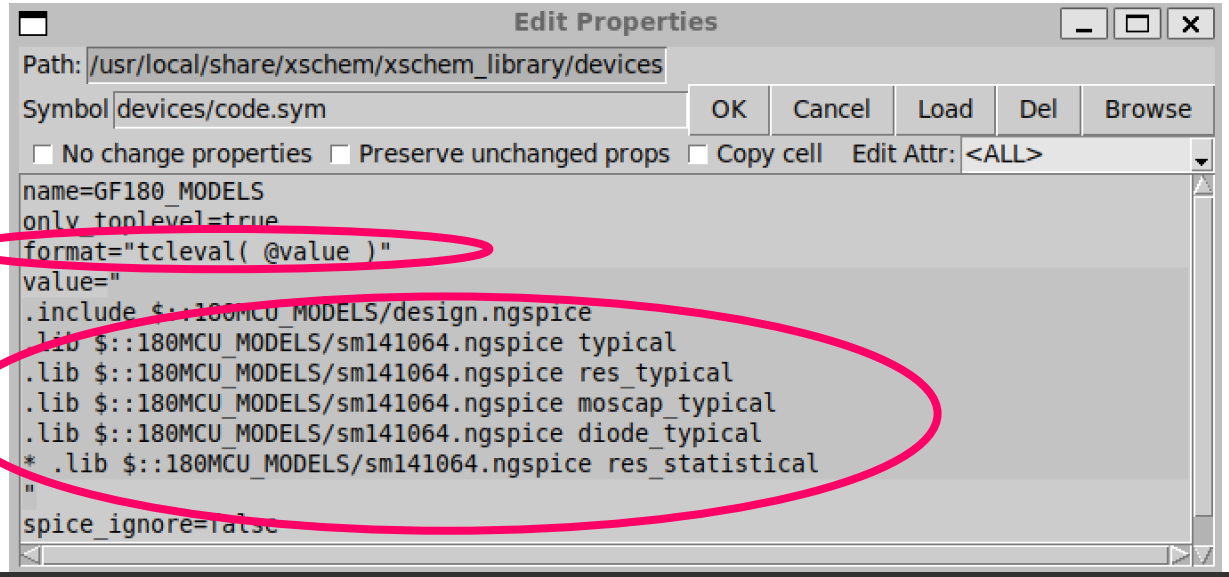
xschem：ベンチマーク

- キーワード
  - code.sym
  - code_shown.sym
  - vsource.sym
  - gnd.sym
  - ammeter.sym
  - lab_pin.sym
  - capa.sym
  - vdd.sym



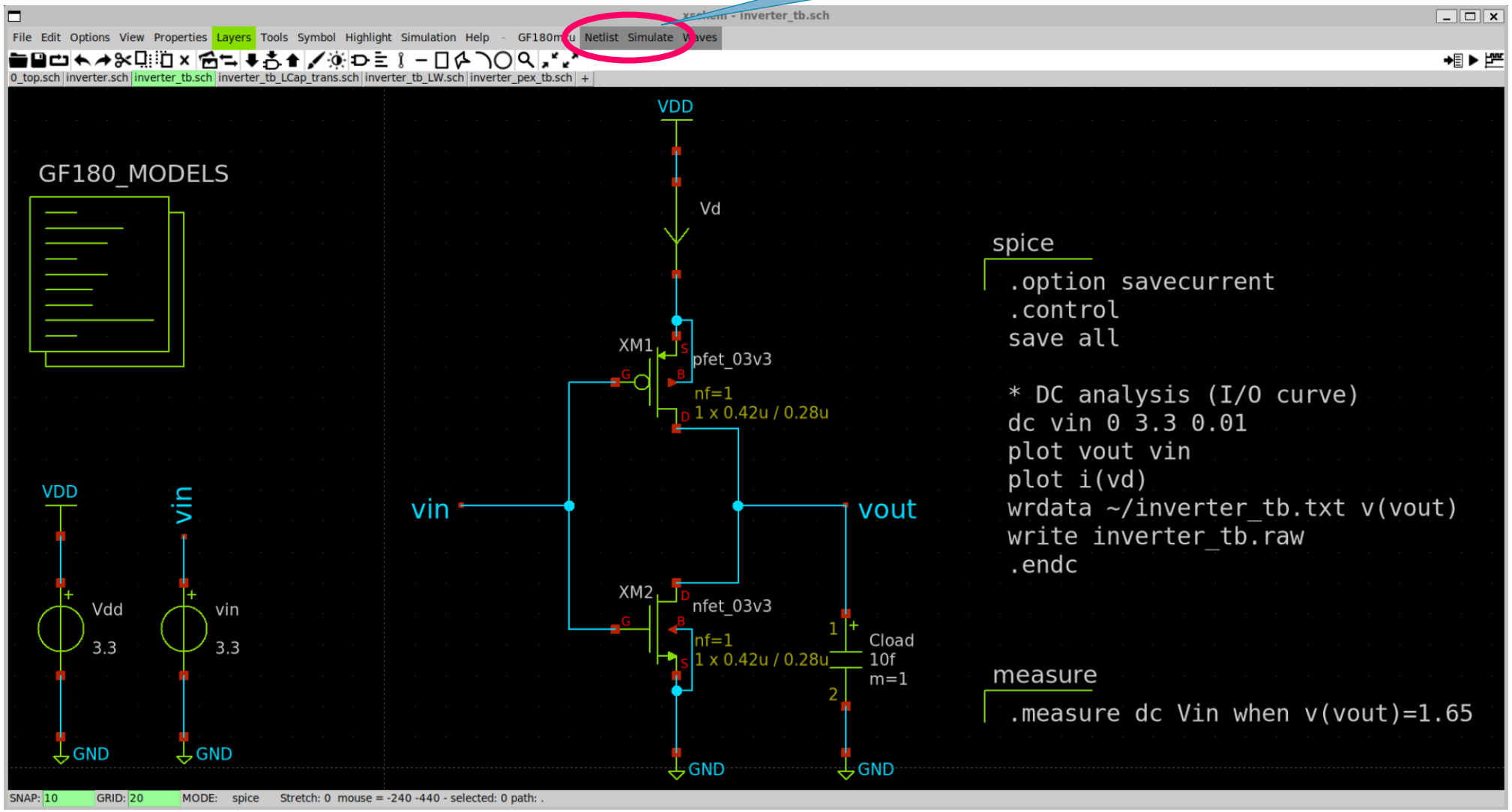
xschem：GF180_MODELSを記述する際の注意点x

- code.symを使用する際、以下の行を追記する必要がある
  - format=“tcleval(@value)”
- includeは絶対パス（フルパス）である必要がある。相対パスは使えない。
  - 「$::LIB」は特殊パスのため利用可能

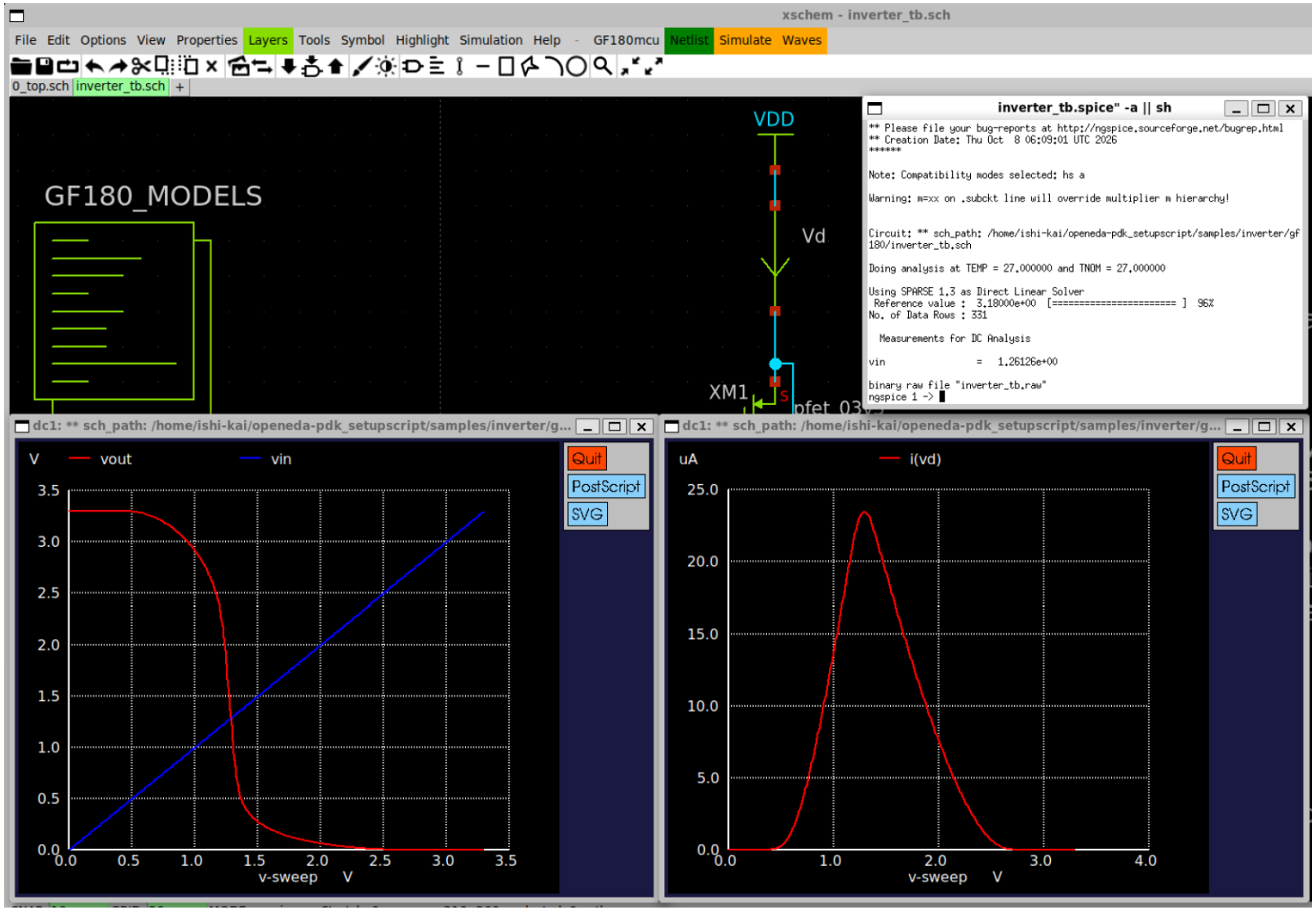


ngspice ：シミュレーションの実行

netlist→simulateの
順にクリック



ngspice：シミュレーション結果（3.3V素子の場合）



入力A = 0 V　　→出力Q = 3.3 V

入力A = 3.3 V　　→出力Q = 0 V

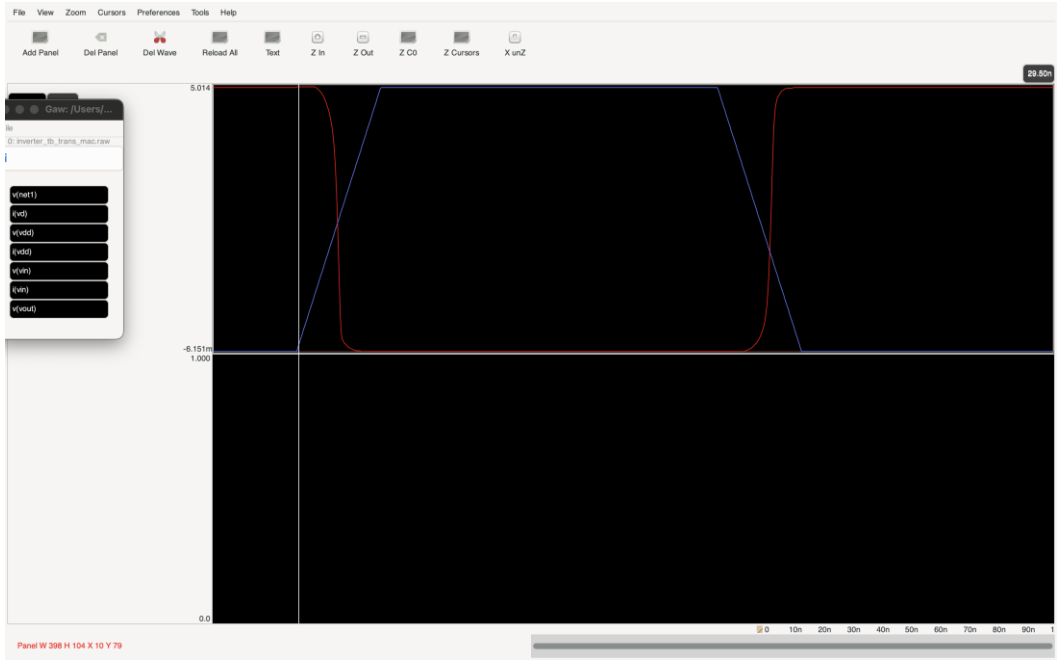
- インバータとして機能している

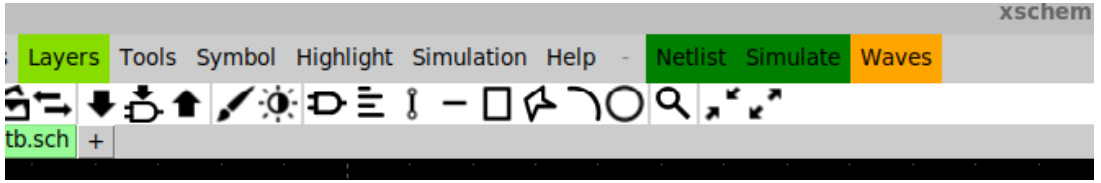
ngspice：wrdata と gawによる波形表示

wrdataを使用するとテキスト出力ができる

```
0.00000000e+00  3.29999999e+00
5.00000000e-02  3.29999999e+00
1.00000000e-01  3.29999997e+00
1.50000000e-01  3.29999990e+00
2.00000000e-01  3.29999960e+00
2.50000000e-01  3.29999842e+00
3.00000000e-01  3.29999375e+00
3.50000000e-01  3.29997564e+00
4.00000000e-01  3.29990878e+00
4.50000000e-01  3.29968137e+00
5.00000000e-01  3.29900073e+00
5.50000000e-01  3.29728006e+00
.
.
.
```

writeでrawファイルを出力すると gaw で波形表示ができる





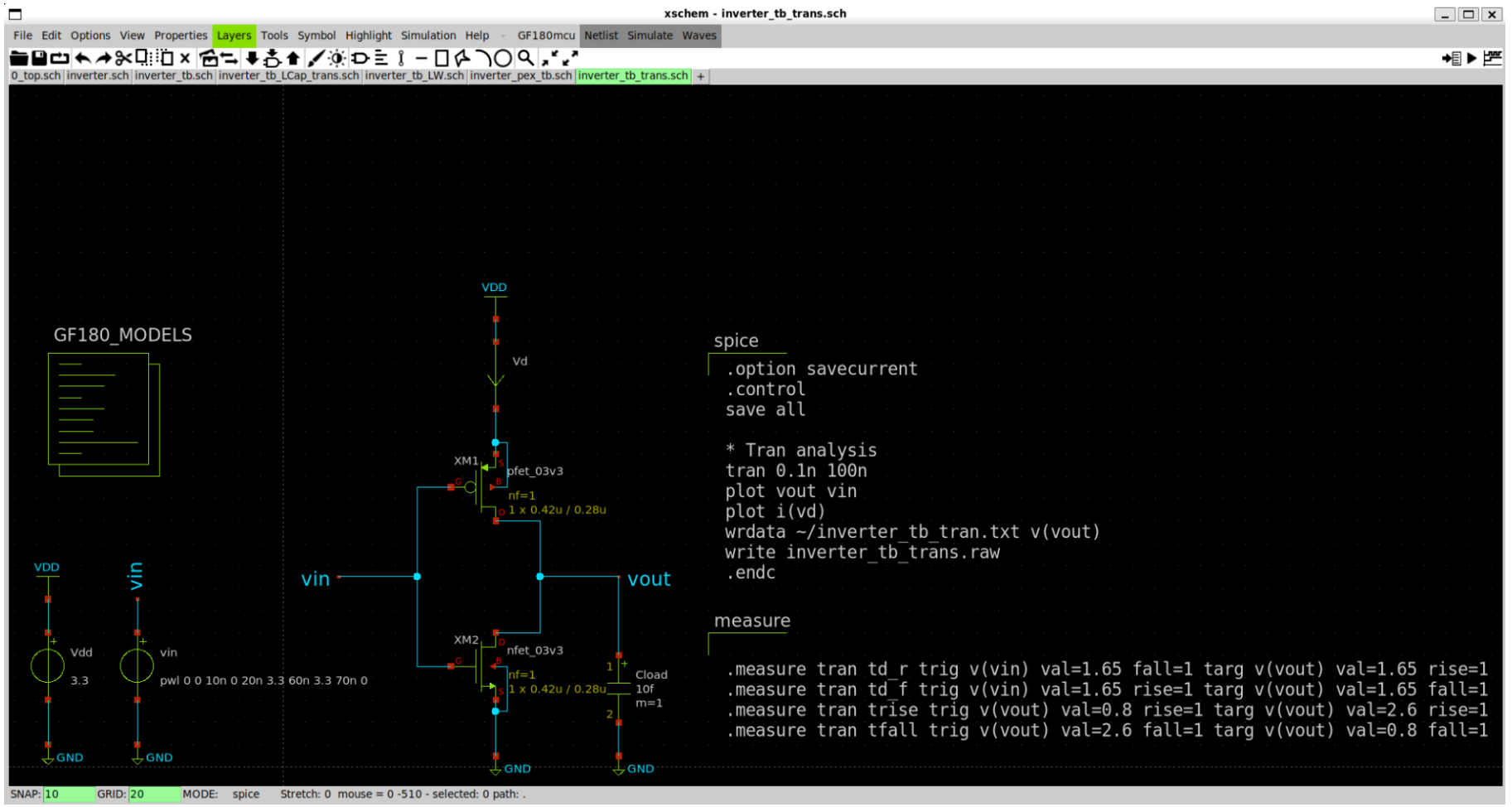
xschem 右上の
Wavesから起動できる

ngspice：過渡応答、遅延時間

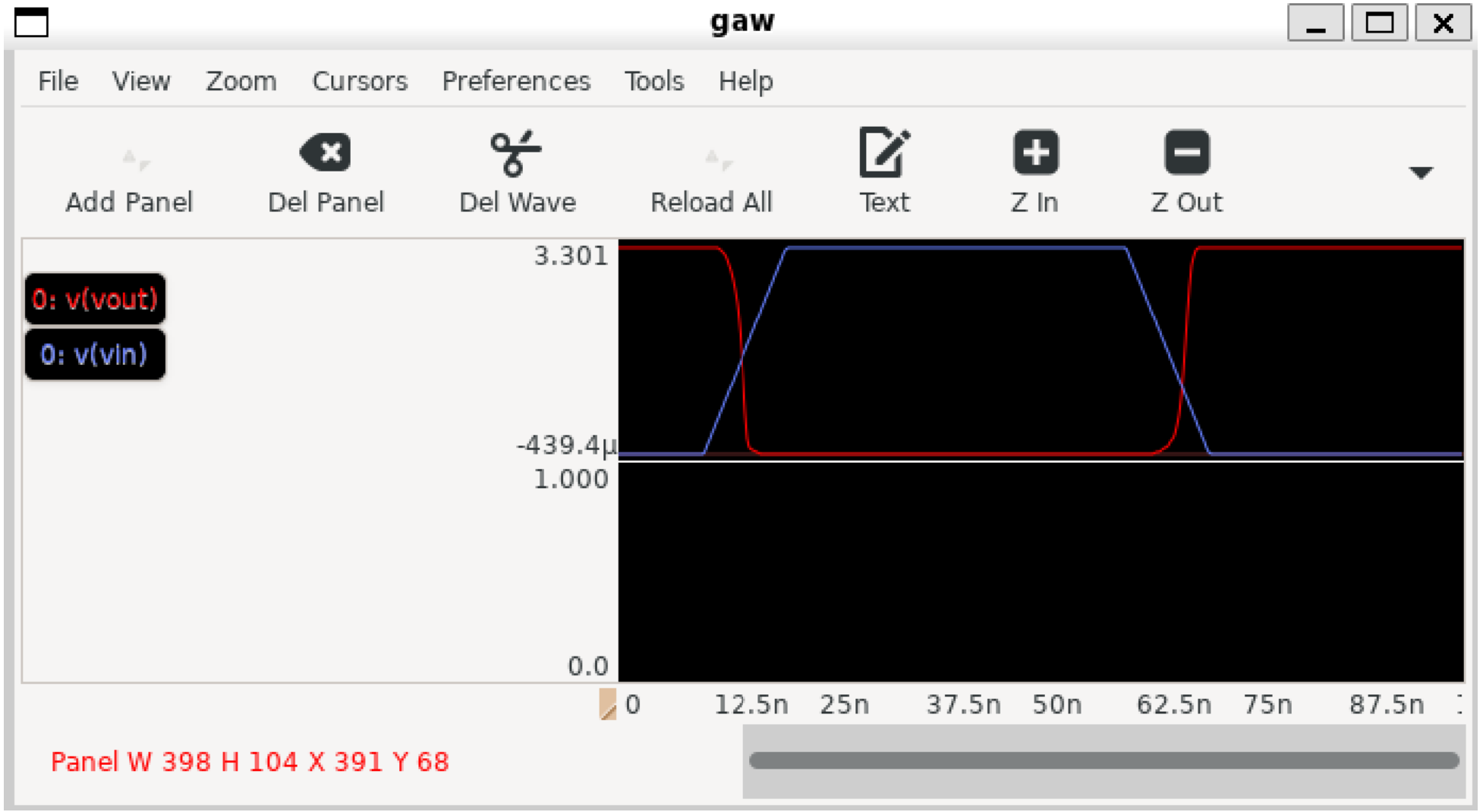
- 過渡応答から遅延時間を見たい場合はtran解析を用いる

- 出力段に負荷を設定しないと正しい過渡応答が見れない
  今回は入力段・出力段共に何も接続しない状態を見る

ngspice：過渡解析ベンチマーク例



ngspice：過渡解析シミュレーション結果

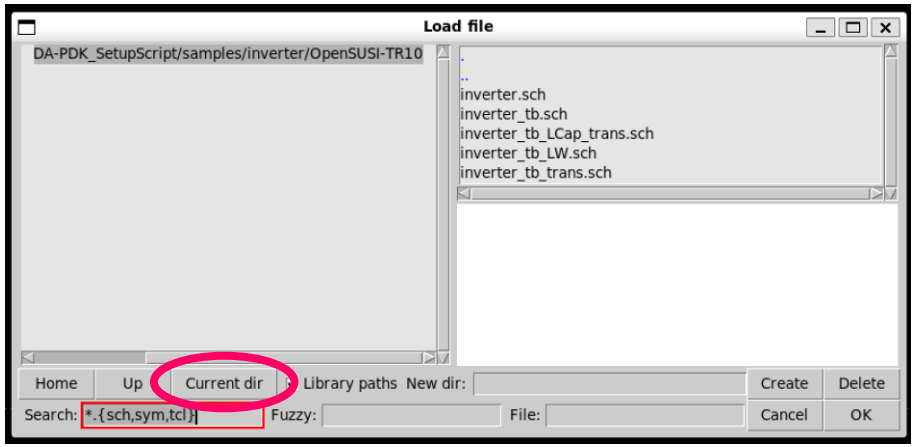


ワンポイントアドバイス：サンプル回路の読み込み

- サンプル回路：ISHI会のイメージを利用している場合
  - ~/OpenEDA-PDK_SetupScript/samples/inverter/GF180/

- 利用する場合

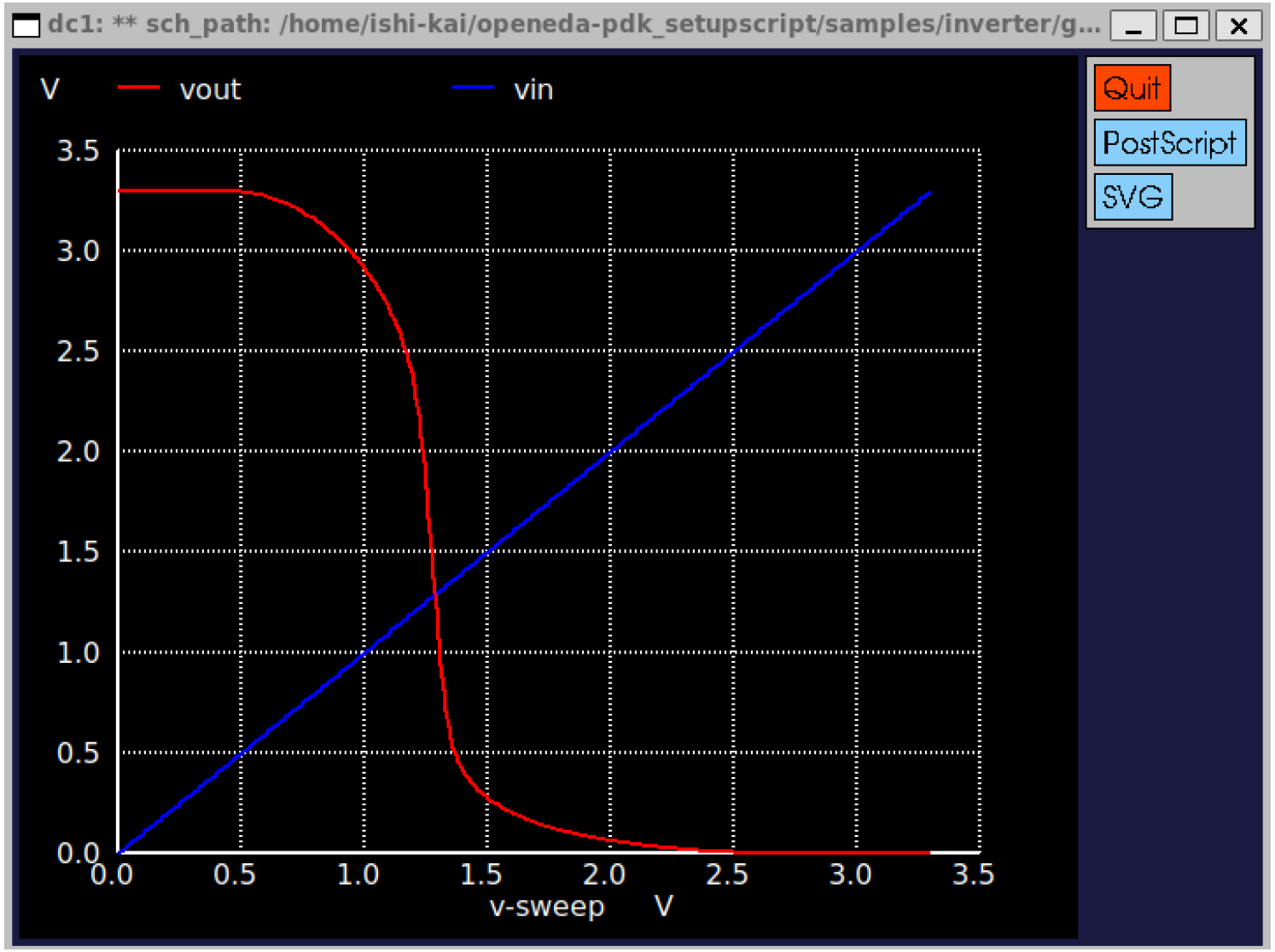
1. コマンドラインで、上記のディレクトリまで移動する
   - cd ~/OpenEDA-PDK_SetupScript/samples/inverter/GF180/
2. 上記ディレクトリでxschemを起動する
3. 新規ファイルを作成する
4. [Current dir]ボタンを押すと一覧にサンプルファイルが表示される



半導体の基礎知識

回路の設計

インバータ回路の設計ポイント

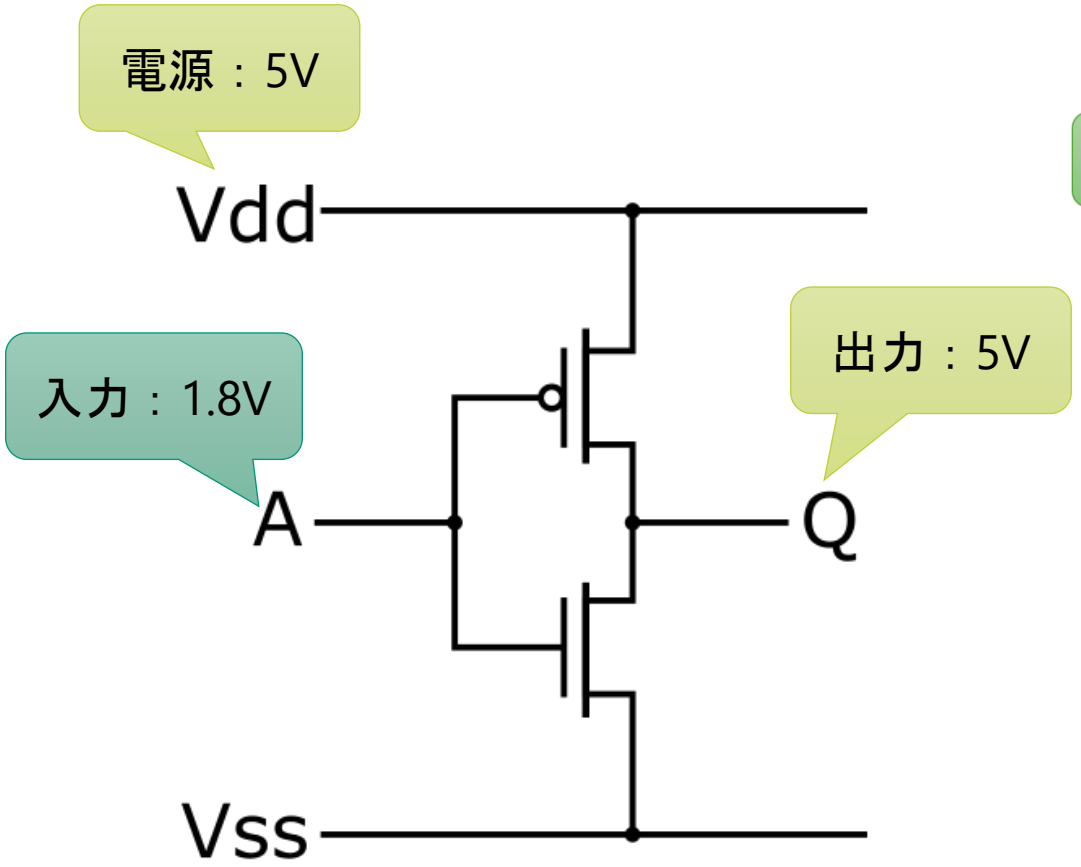


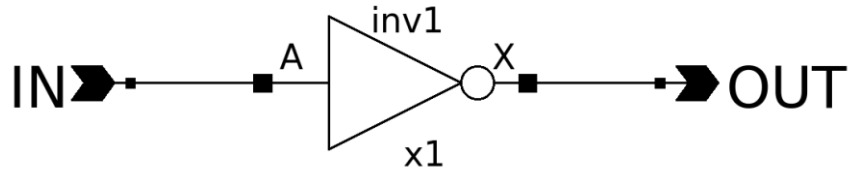
入力A = 1.2V　　→出力Q = 0 V

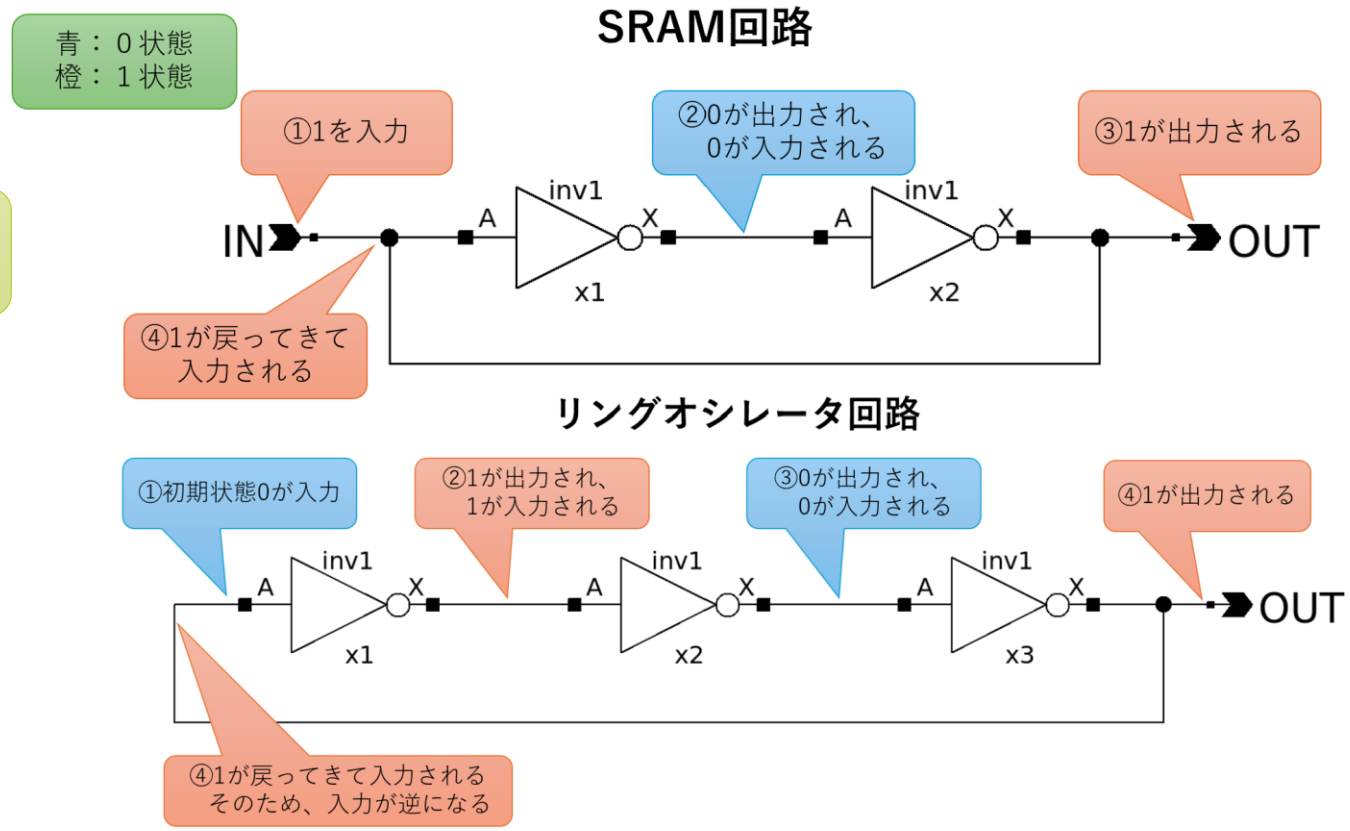
- 本来は、VDD=3.3Vの半分である
  入力A = 1.65Vで動作すべき
- 入力A = 1.65Vより低い方向のため、
  PMOSの性能が低いことが原因
  - 対策
    - PMOSの性能を上げる
    - NMOSの性能を下げる

インバータ回路ってなにもの？

- 本来は、VDD=5.0Vの半分である入力A = 2.5Vで動作すべき

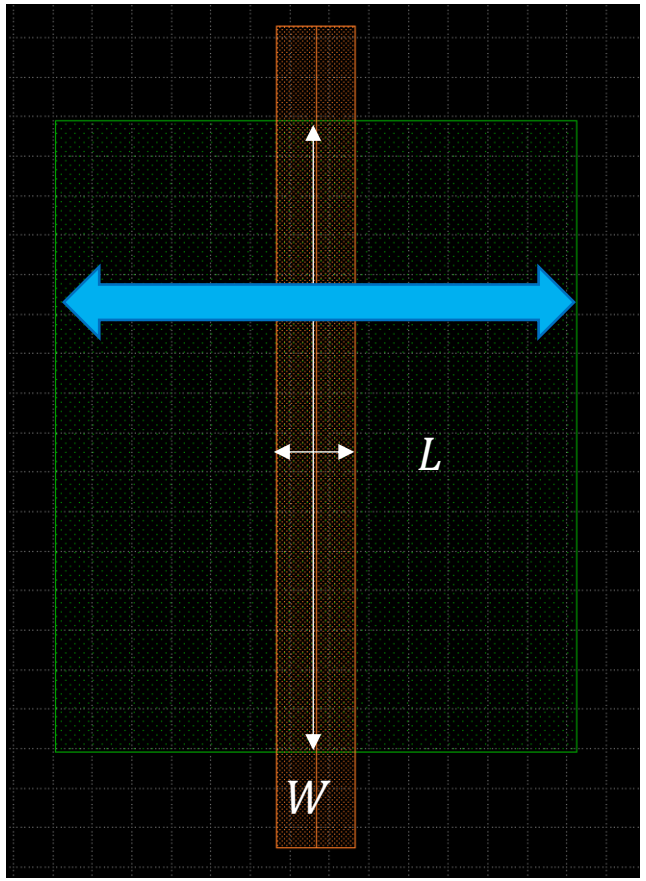


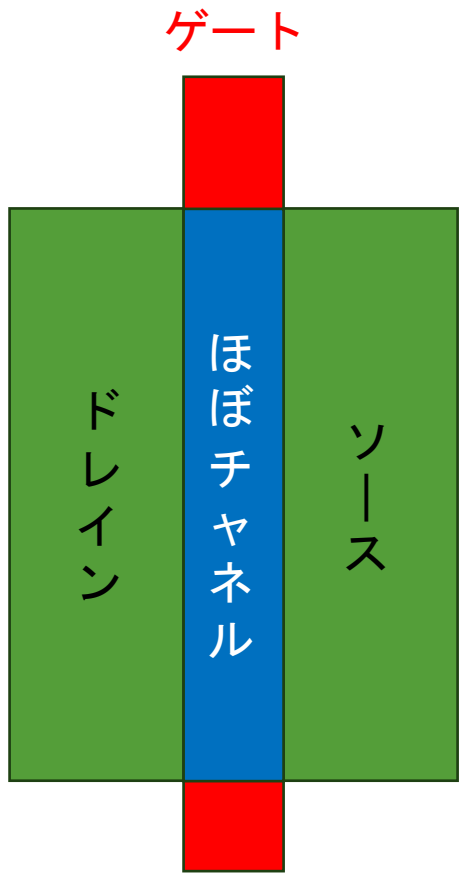




トランジスタのパラメータ

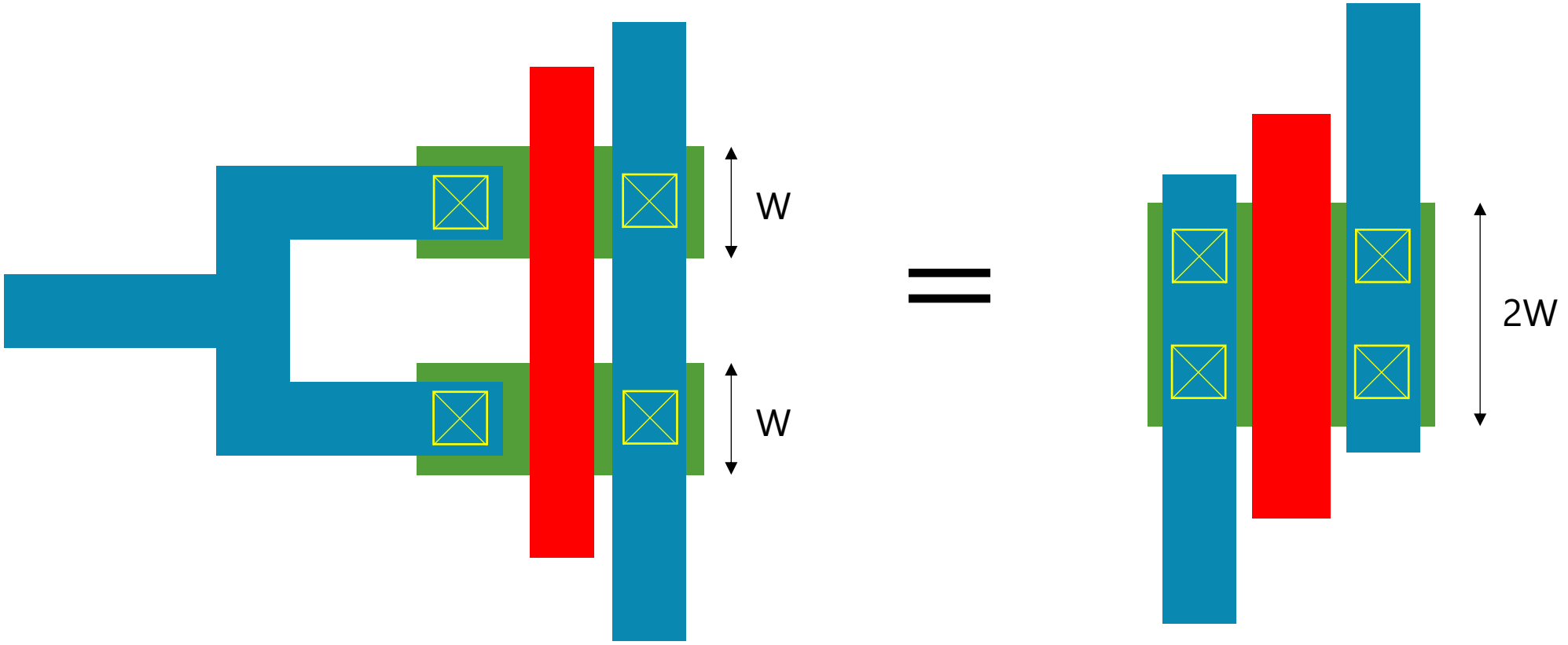
- 基本はWとLを変える
  - W　：チャネル幅
  - L　：チャネル長（※実際のチャネル長はちょっと短い）
- **ゲートと拡散層の重なった部分がトランジスタ本体**
  ゲートに充分な電位を与えると**チャネル**が形成
- m, nf, mf などはゲートの本数（マルチフィンガー）





チャネル幅　Wを変えるとどうなる？

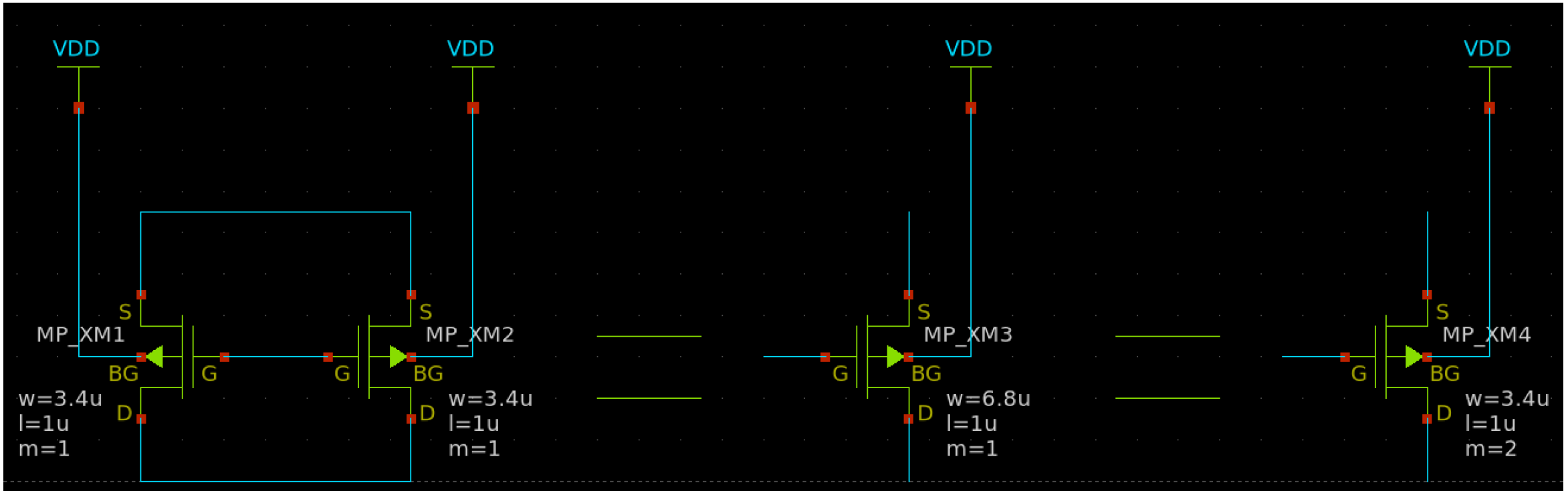
- チャネル幅を大きくとると電流の通り道が広くなるので、ドレインソース間の電流は増える
- チャネル長が同じトランジスタでは、**チャネル幅とドレインソース間電流は比例の関係**



ゲートの本数

ドレイン・ソース・ゲート・ボディが同じノードである同じサイズのトランジスタは、Wが２倍のトランジスタに等しい

これを「 m, nf, mf 」で表現し、マルチフィンガーという



チャネル長　Lを変えるとどうなる？

- **短チャネル効果**による影響で、特性が全く変化する。
  チャネル長Lが短くなると基本はIdsが増加するが、**短チャネル効果**も発生

- DIBL (Drain Induced Barrier Lowering)により、Vdsの増加に対してVthが低下
- Subthreshold swingの増加により、弱反転領域におけるパンチスルーが増加
  - オフ状態でのリーク電流が増え、Vgsの増加幅に対するIdsの増加幅が鈍くなる　→スイッチング特性劣化
- チャネル長変調効果により、Vds変動幅に対するIds変動幅が増え、**飽和状態にならない**

チャネル長　Lを変えるとどうなる？

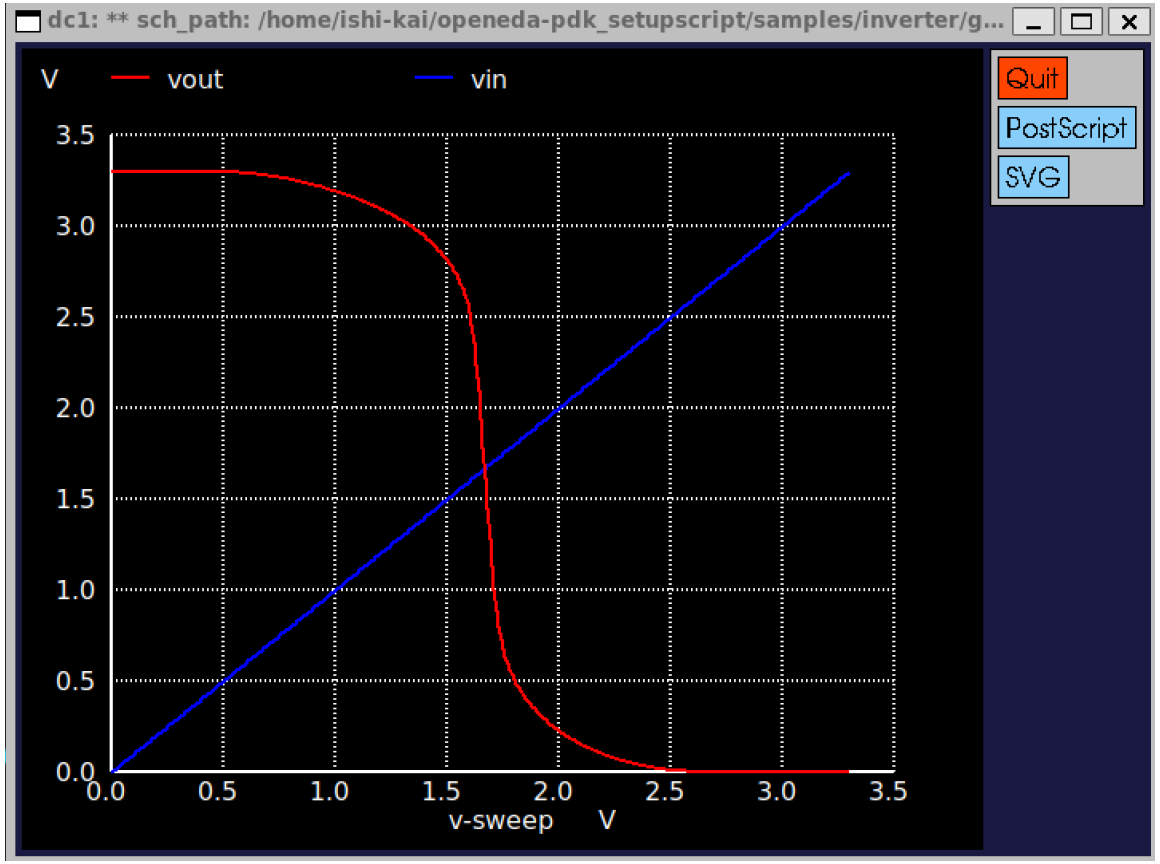
- **短チャネル効果**による影響で、特性が全く変化する。
  チャネル長Lが短くなると基本はIdsが増加するが、**短チャネル効果**も発生

ロジックゲートは通常、高集積密度、低Vth、高Idsのため、
チャネル長(L)は最短で設計される。

短チャネル効果やちゃんとした設計手法についてはオペアンプ編にて詳しく解説します。

設計方針

- PMOSの性能を上げる
  - PMOSのWを大きくする
  - PMOSのLを小さくする
- NMOSの性能を下げる
  - NMOSのWを小さくする
  - NMOSのLを大きくする



計算による設計手法についてはオペアンプ編にて詳しく解説します。

アナログ半導体の設計フロー

1. 回路図（テストベンチ）を描く
2. シミュレーションをする
3. **回路図を基にレイアウトを描く**
4. レイアウトを検証する
5. レイアウトを基に寄生成分を考慮したシミュレーションをする
6. フレームに載せる

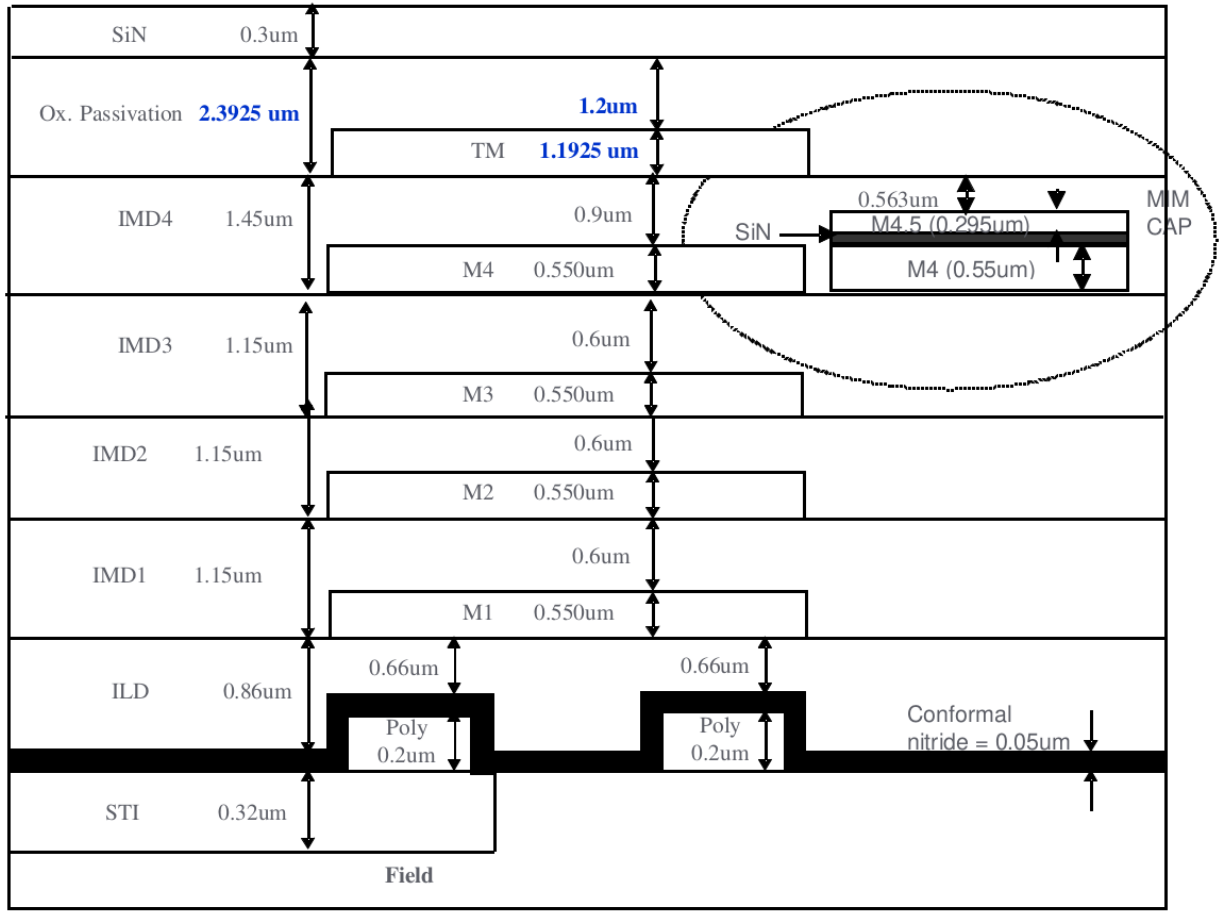
半導体の基礎知識

プレーナ型FETプロセス　レイアウト編

プレーナ（平面）型MOSトランジスタ

- substrate（基板）を最下層として積層していく。
- 基板上にMOSトランジスタ(well, diffusion, poly)
- トランジスタより上は配線層(metal*, via*)

- Wafer.Space：**gf180mcuD**
- 5-metal stack, 1P5M, 5LM (metal_level=5LM)
- **1.1um thick top metal, TM 11KA (metal_top=11K)**
- MiM B option (mim_option=B)
- 2fF/um2 MiM Capacitance (cap_mim_2f0fF)
  メモ：キャパシタの耐圧は6Vらしい
- 1kΩ high sheet rho poly (ppolyf_u_1k)



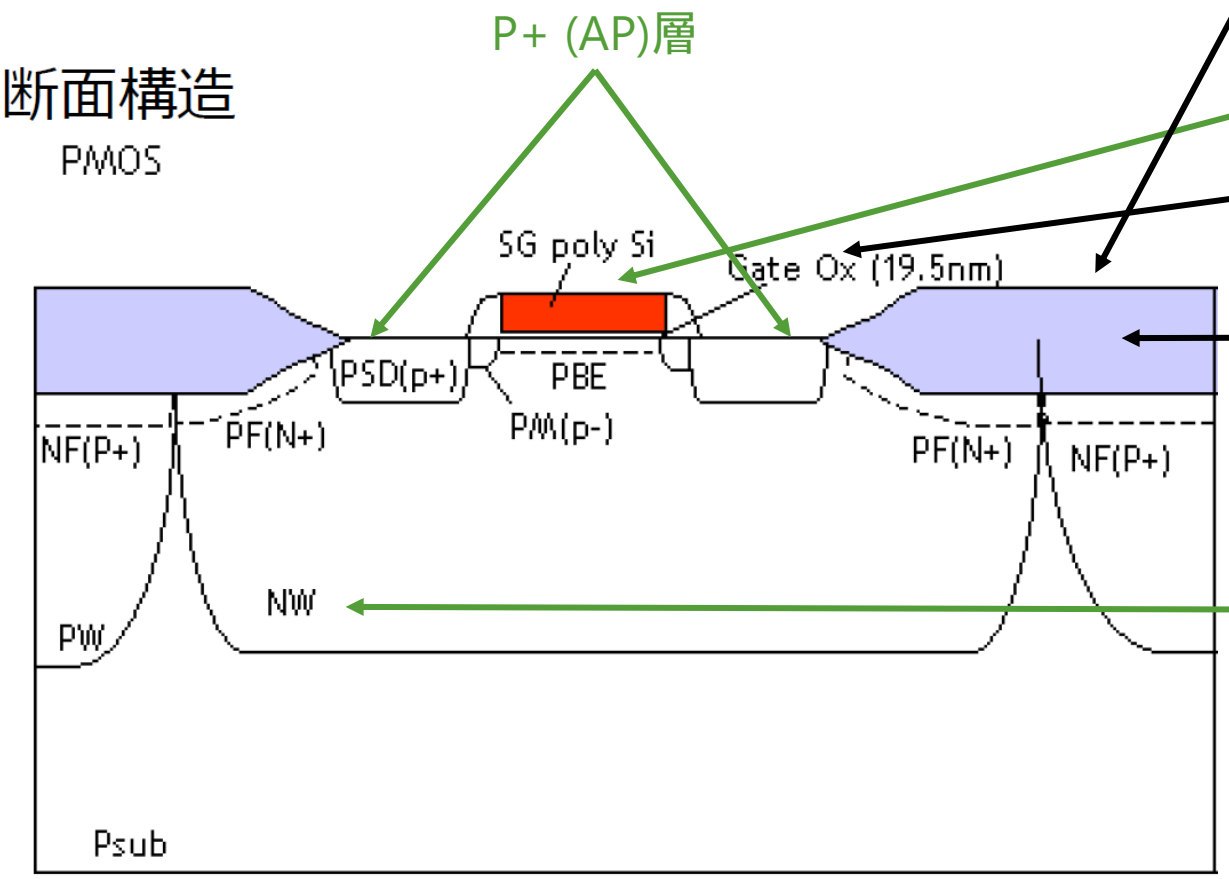
https://gf180mcu-pdk.readthedocs.io/en/latest/analog/layout/inter_specs/inter_specs_3_43.html

レイアウトを描く上での注意

- メタルは最大6層まで使えるが、今回は5層なのでVIA5, MetalTopは使用不可
- MIMキャパシタはMetal4とMetal5の間に形成する (Option=B, Level=M4-M5)
  ※MIMキャパシタはトップメタルの下に形成される
- 各検証の実行前にオプションを確認・設定する　（特にmetal_top=11K）

- **DRCチェックで検知できないルールがある**
  https://gf180mcu-pdk.readthedocs.io/en/latest/physical_verification/design_manual/drm_16.html

プレーナ型PMOS



リン-ケイ酸ガラス; レイヤ間を埋める絶縁物質

ポリ(多結晶)シリコン(GC層)

酸化膜絶層

LOCOS(local oxidation of silicon)

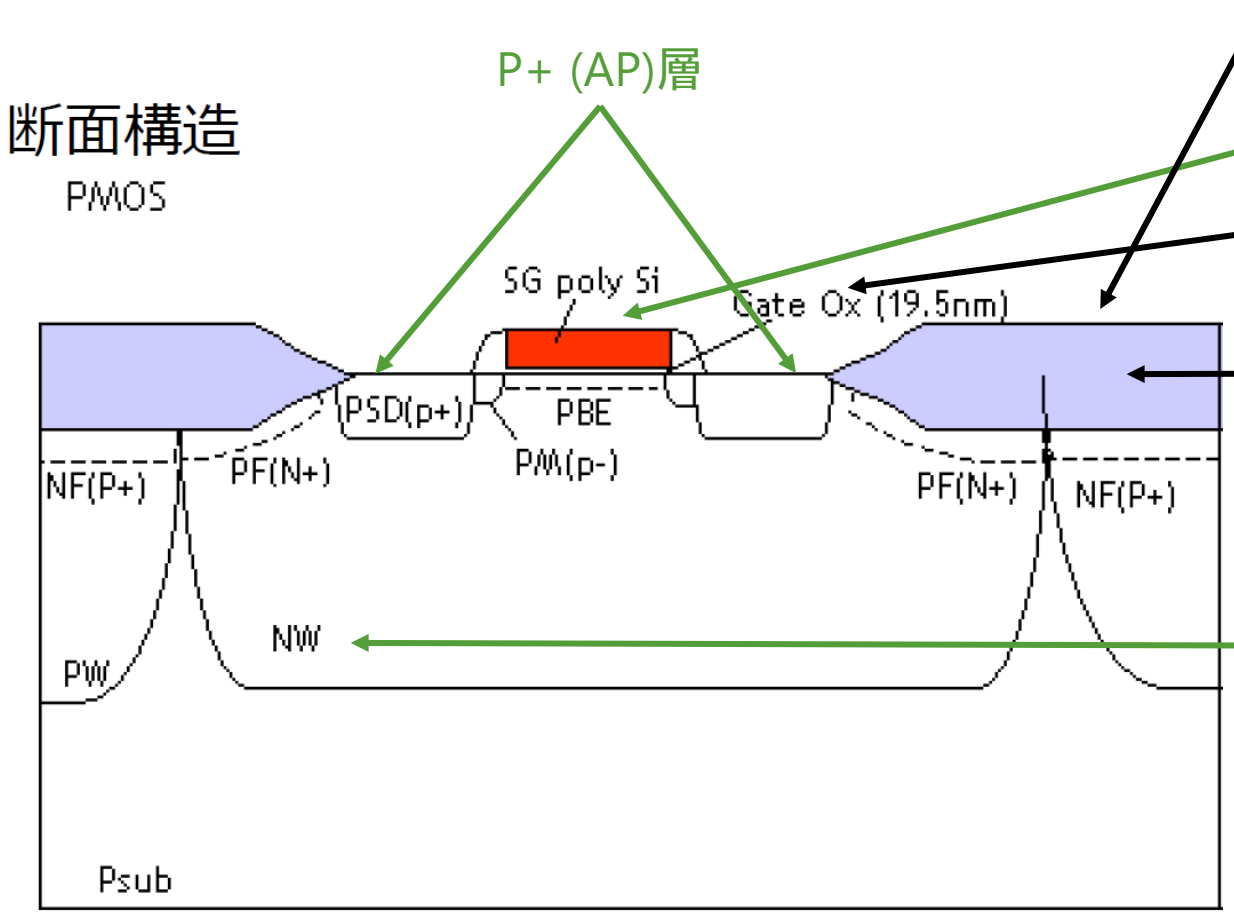
半導体デバイスを形成する際の代表的素子分離法。
窒化膜で酸化防止マスクされた部分以外のシリコン基板を酸化し、この酸化膜の層によって周囲から電気的に絶縁された領域（活性領域）を作る

NW(N-well,WN層)

Psub-サブストレート, 基板（シリコン）

緑字はレイアウトする必要がある層

プレーナ型PMOS



リン-ケイ酸ガラス; レイヤ間を埋める絶縁物質

ポリ(多結晶)シリコン(GC層)

酸化膜絶層

LOCOS(local oxidation of silicon)

半導体デバイスを形成する際の代表的素子分離法。
窒化膜で酸化防止マスクされた部分以外のシリコン基板を酸化し、この酸化膜の層によって周囲から電気的に絶縁された領域（活性領域）を作る

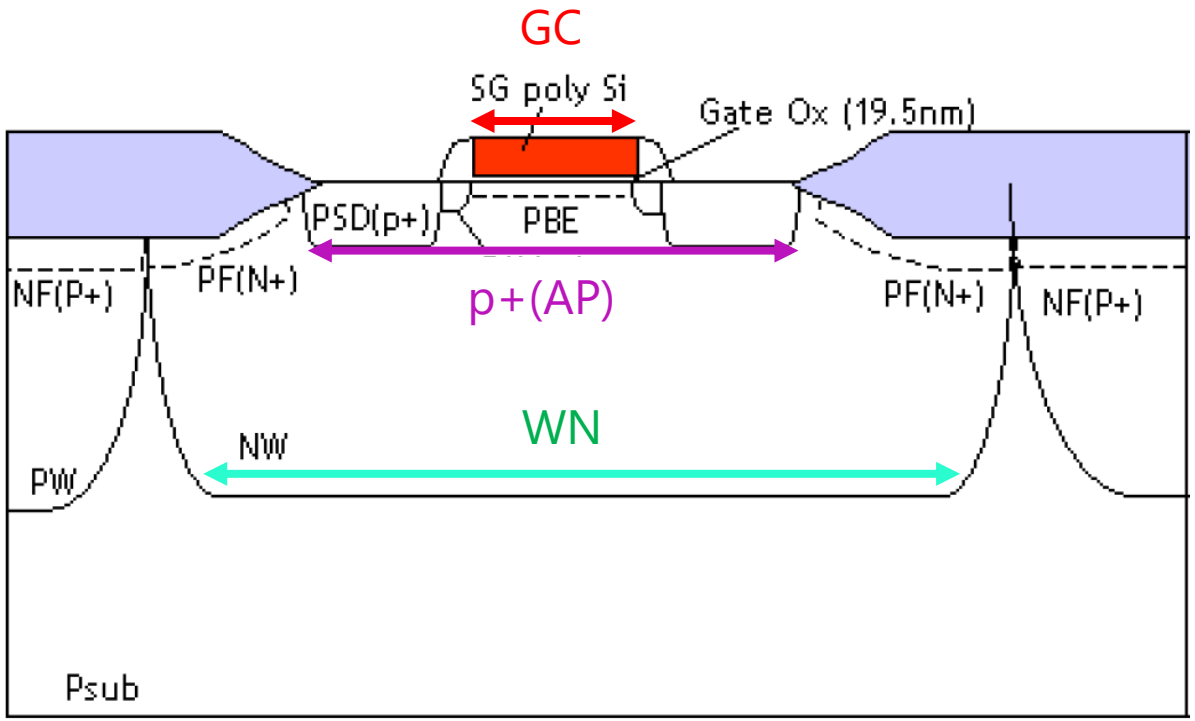
NW(N-well,WN層)

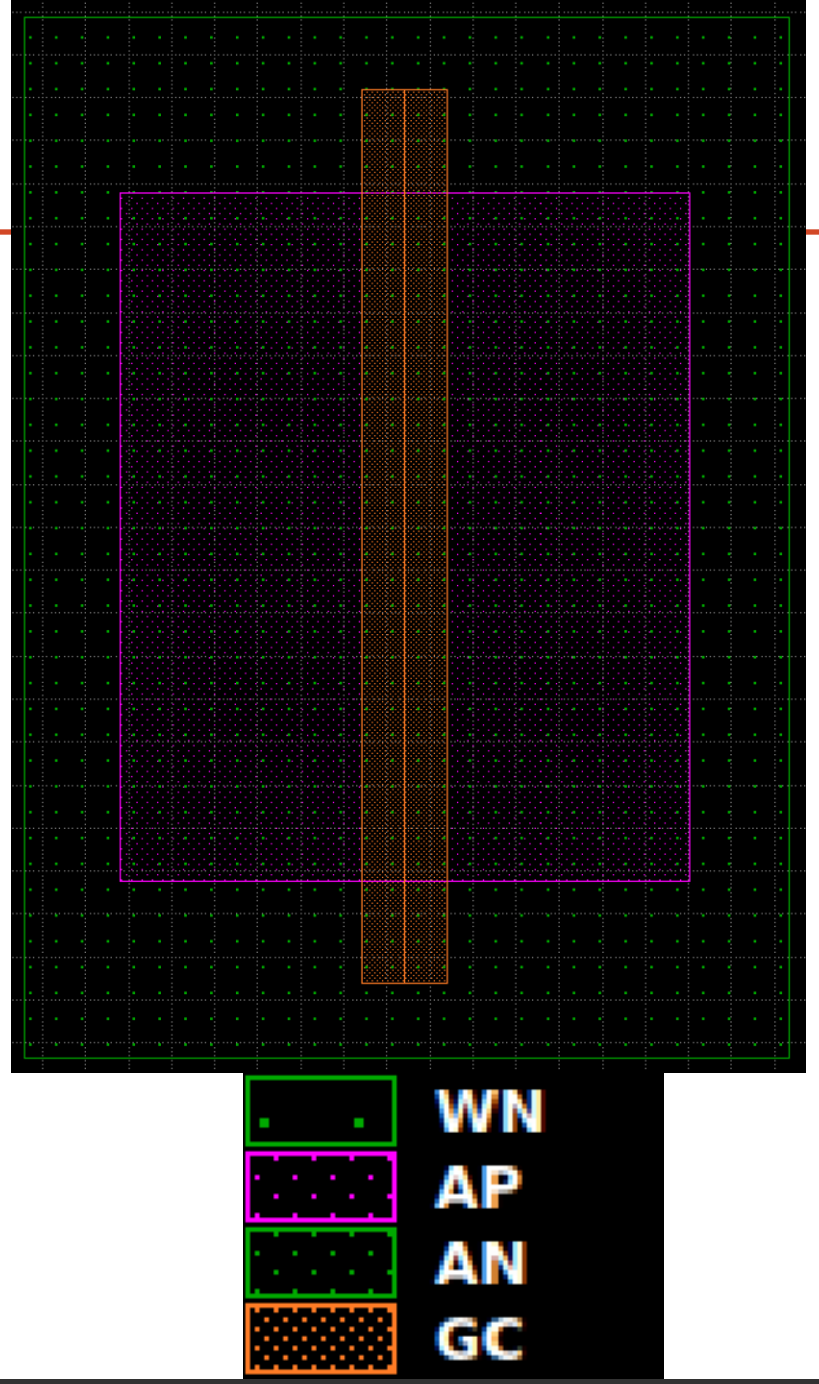
Psub-サブストレート, 基板（シリコン）

緑字はレイアウトする必要がある層

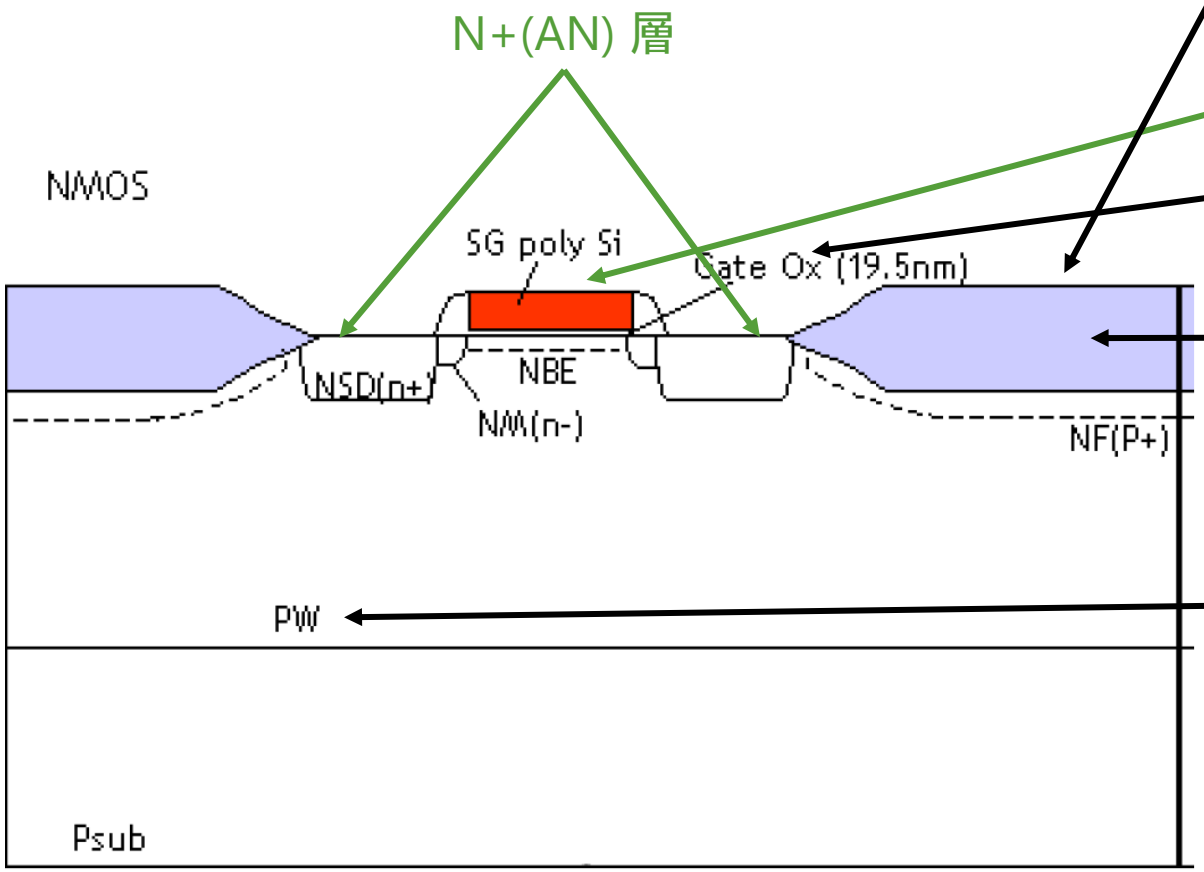
プレーナ型PMOS

WN	N-well
AP	不純物注入によるP+イオン領域と拡散層
GC	ポリシリコン





プレーナ型NMOS



リン-ケイ酸ガラス; レイヤ間を埋める絶縁物質

ポリ(多結晶)シリコン(GC層)

酸化膜絶層

LOCOS(local oxidation of silicon)

半導体デバイスを形成する際の代表的素子分離法。
窒化膜で酸化防止マスクされた部分以外のシリコン基板を酸化し、この酸化膜の層によって周囲から電気的に絶縁された領域（活性領域）を作る

PW(P-well)

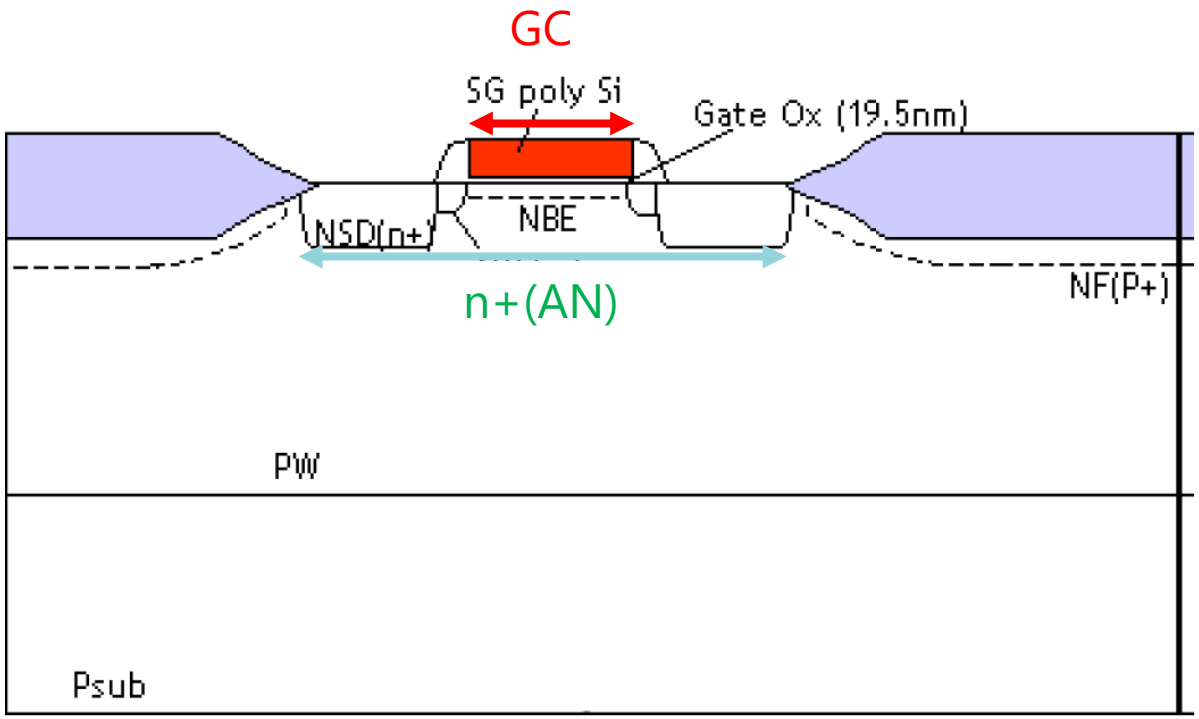
Psub-サブストレート, 基板（シリコン）

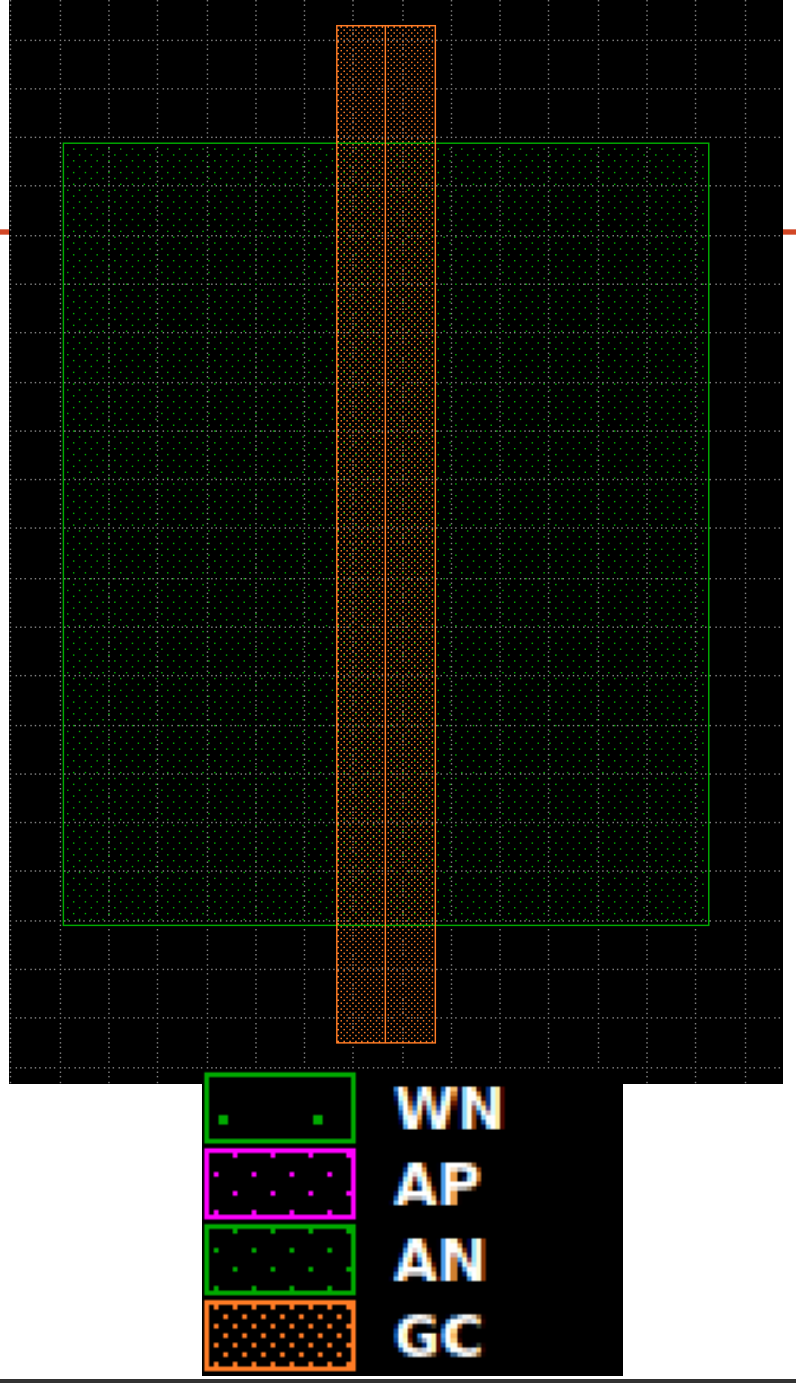
緑字はレイアウトする必要がある層

プレーナ型NMOS

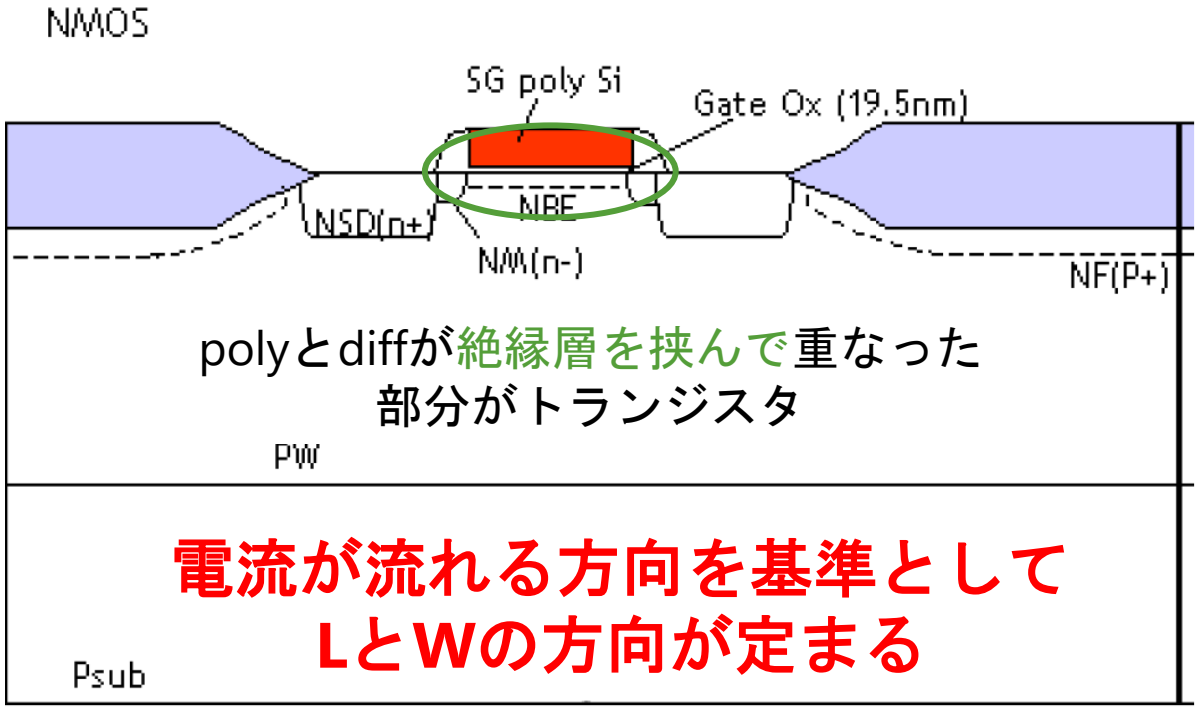
AN　　不純物注入によるN+イオン領域と拡散層
GC　　ポリシリコン

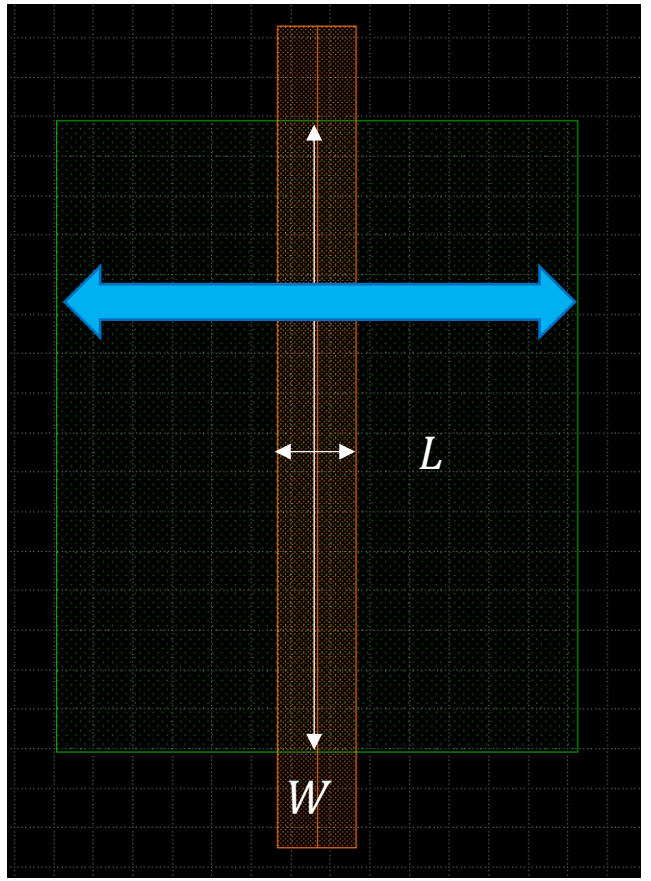
レイアウトでは、PW(P-well)は描かれない





プレーナ型NMOS

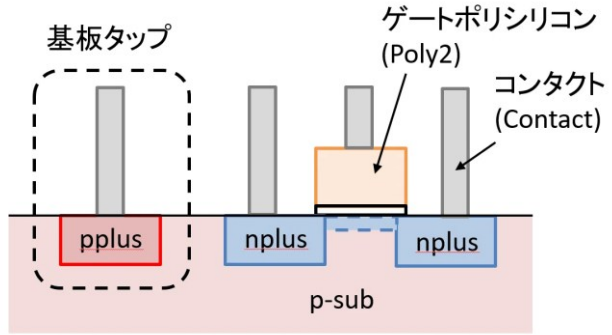

NMOS
SG poly Si
Gate Ox (19.5nm)
NSD(n+)
NBE
NW(n-)
NF(P+)
polyとdiffが絶縁層を挟んで重なった
部分がトランジスタ
PW
電流が流れる方向を基準として
LとWの方向が定まる
Psub


L
W

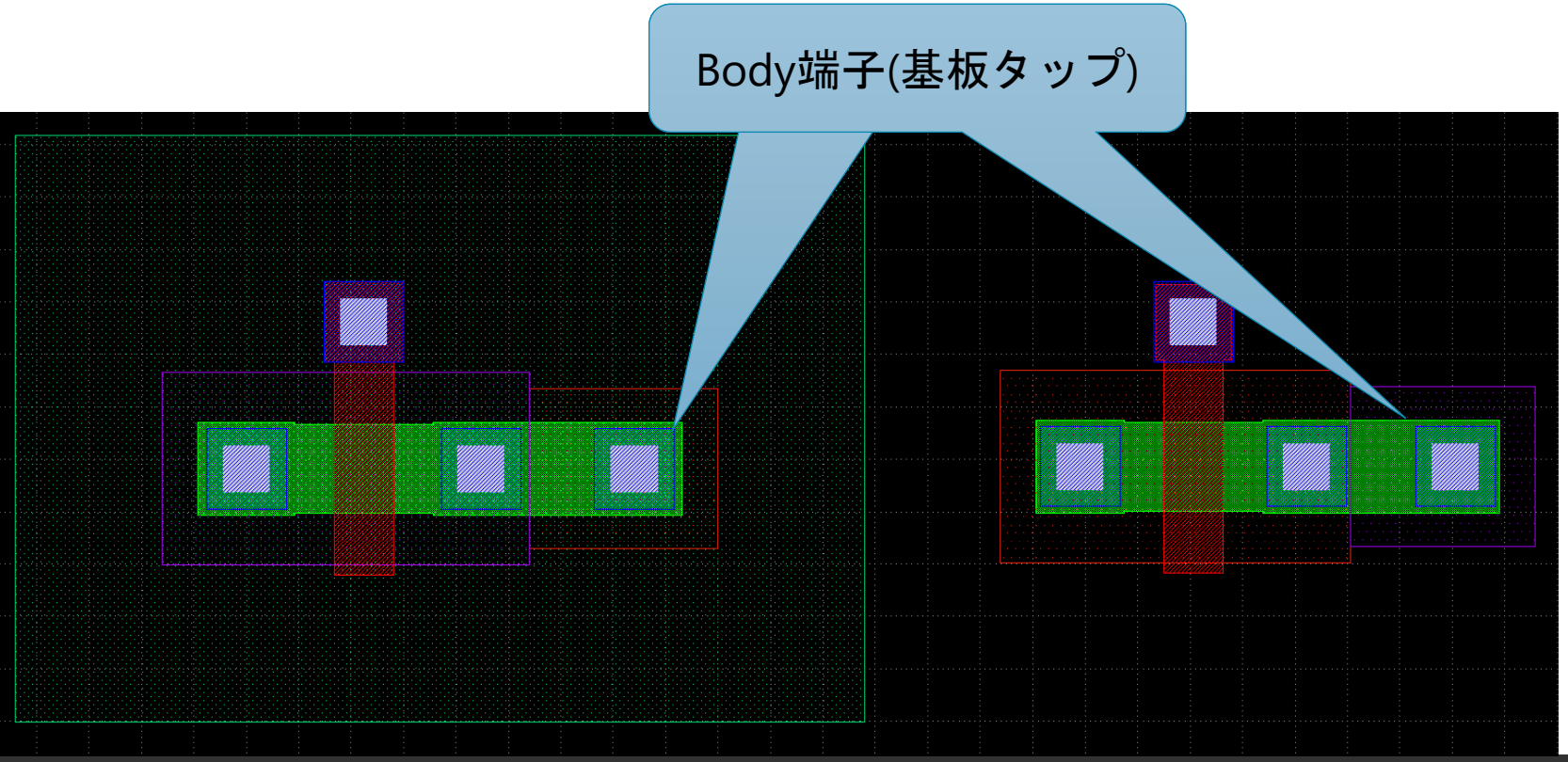
PCELLでのボディの扱い

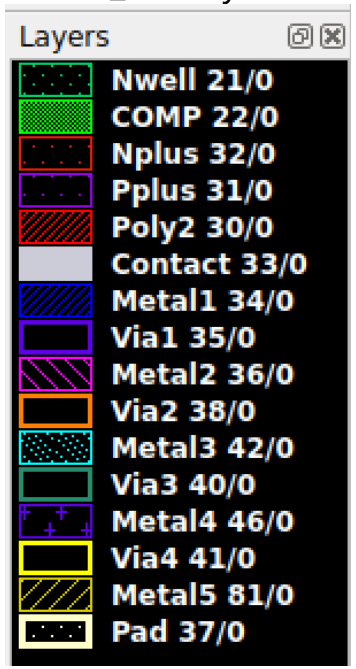
PMOSはNplusを介してN-well に繋がっている

NMOSはPplusを介してP-substrateに繋がっている



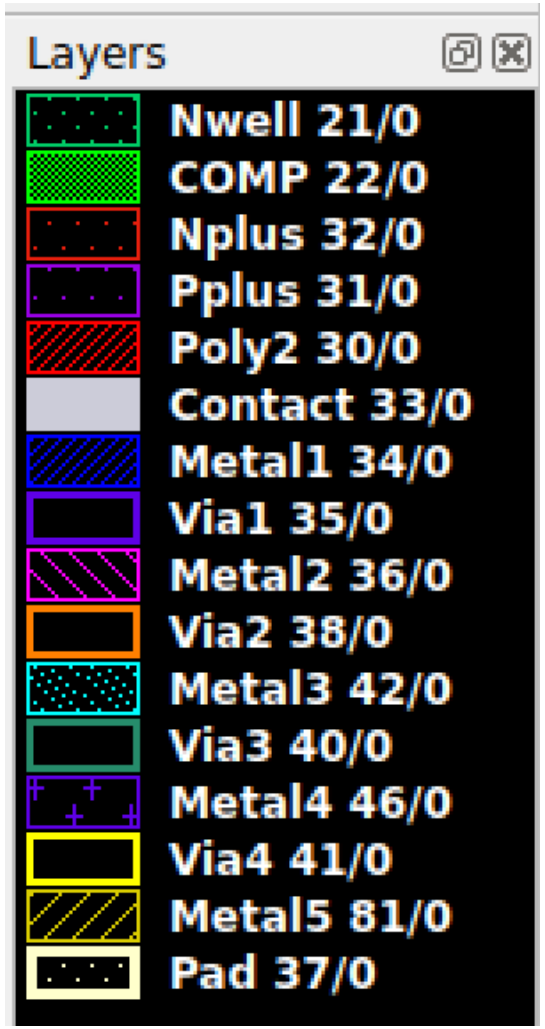
https://note.com/akira_tsuchiya/n/nd4444304f013





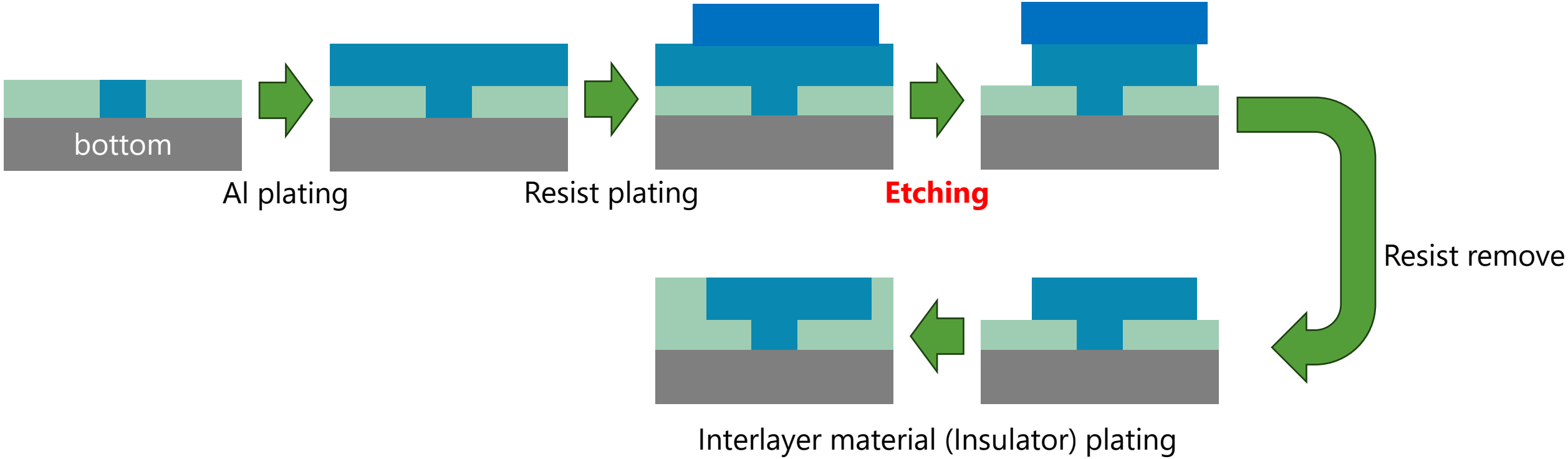
レイヤーの名前と意味（カッコ内はOpenSUSI-TR10での名前）

Nwell(WN)	N-well
Pplus(AP)	不純物注入によるP+イオン領域と拡散層の複合レイヤー
Nplus(AN)	不純物注入によるN+イオン領域と拡散層の複合レイヤー
Poly2(GC)	Polysilicon
Contact(CO)	Poly2とMetal1接続用のVIA（コンタクトと呼ぶことが多い）
Metal1～5(M1,2)	メタル1～5層目
VIA1～4(V1)	Metal接続用のVIA
Pad	パッド（むき出しのメタル）



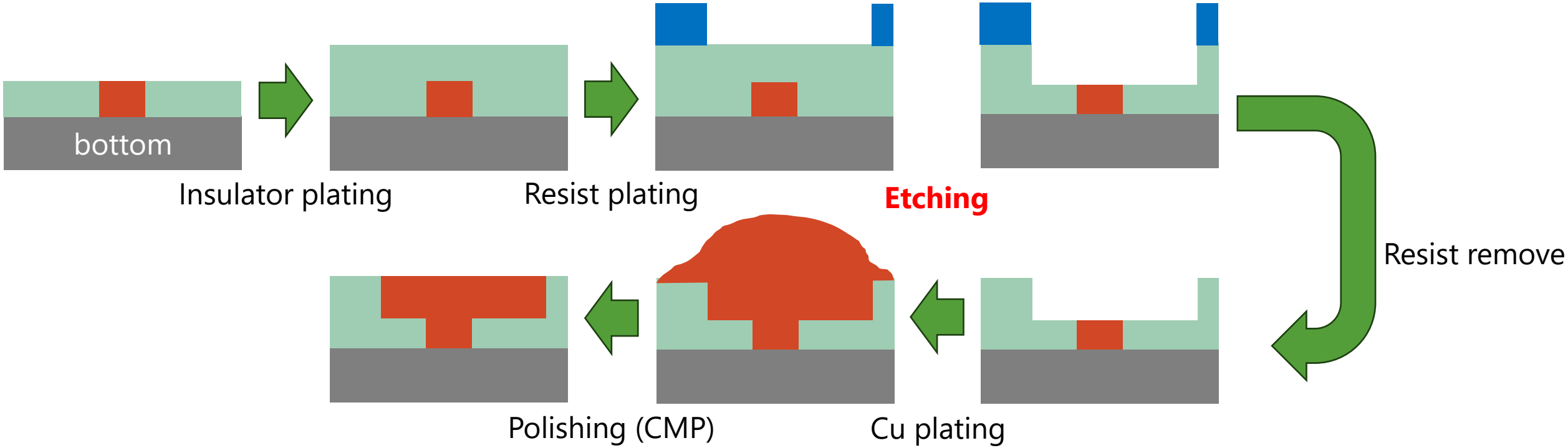
配線層ができるまで

- AlとCuでデザインルールが異なるのは、（一般論として）作り方が違うため。
- **サブトラクティブプロセス：金属めっきにレジストを堆積してエッチング　（Al配線層）**
- ダマシンプロセス：絶縁物で型を作って金属を流し込む　（Cu配線層）



配線層ができるまで

- AlとCuでデザインルールが異なるのは、（一般論として）作り方が違うため。
- サブトラクティブプロセス：金属めっきにレジストを堆積してエッチング　（Al配線層）
- **ダマシンプロセス：絶縁物で型を作って金属を流し込む　（Cu配線層）**



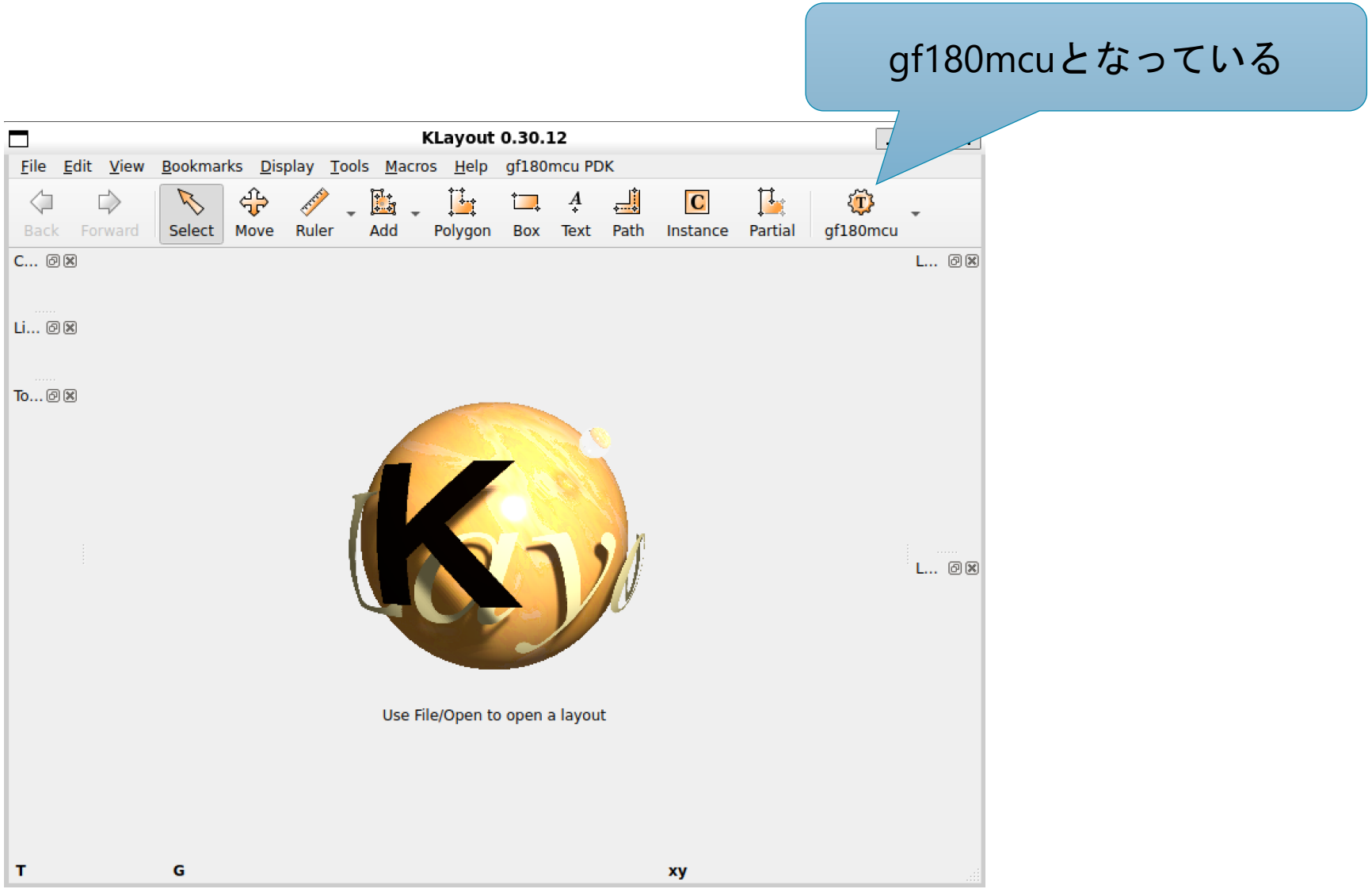
アナログ半導体の設計フロー

1. 回路図を描く
2. シミュレーションをする
3. **回路図を基にレイアウトを描く**
4. レイアウトを検証する
5. レイアウトを基に寄生成分を考慮したシミュレーションをする
6. フレームに載せる

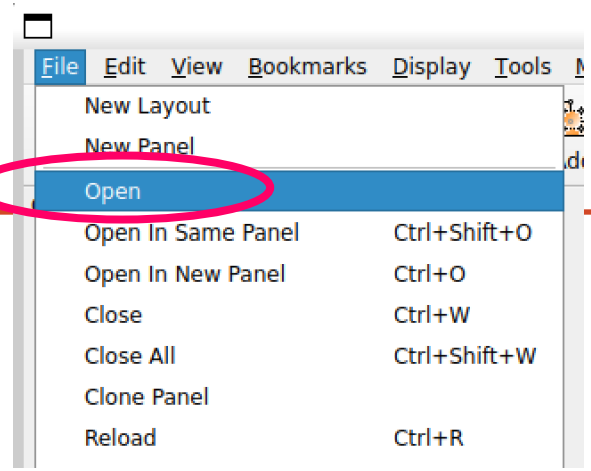
Klayout：テンプレートを開く 1/2

1. WSLのコマンドラインでテンプレートファイルを作業用ディレクトリにコピーする
    1. cp ~/sample/inv_dut_NNN.gds ~/inverter/inv_dut_[配布されたDUT番号].gds
2. klayoutを起動する
    1. cd ~/inverter/
    2. klayout &

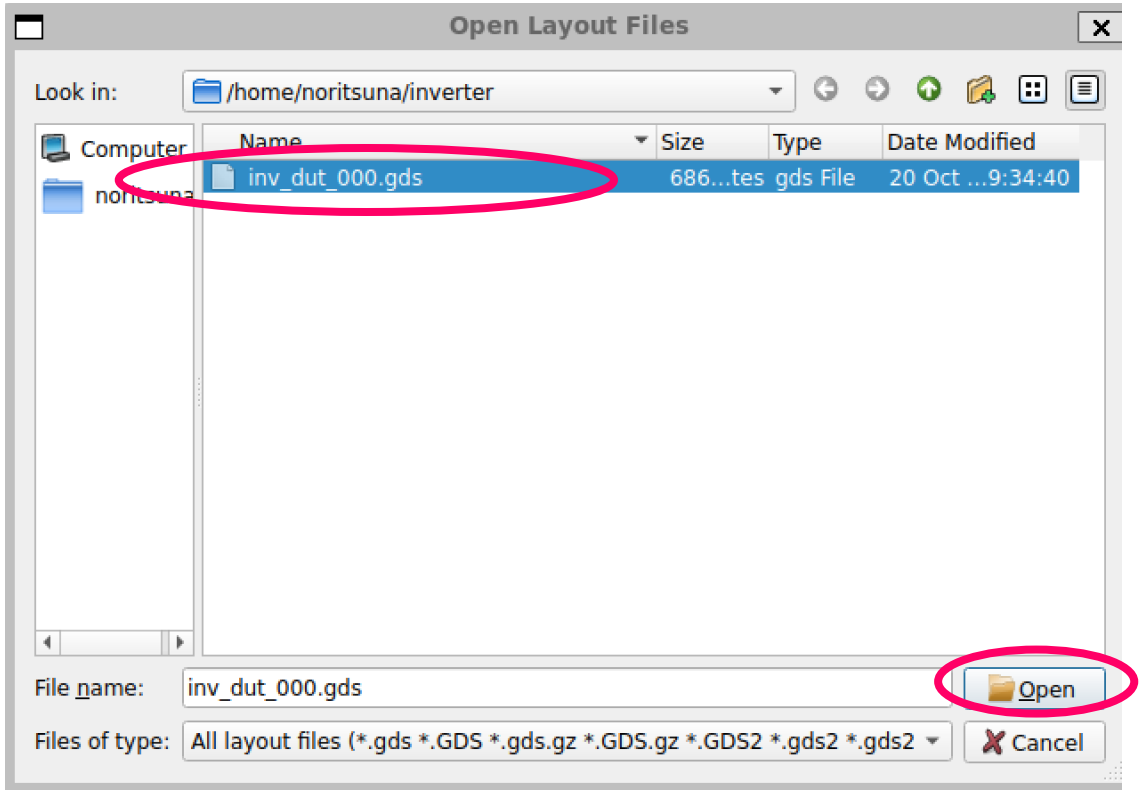
klayoutの起動画面(この画面以外はインストールに失敗しています)

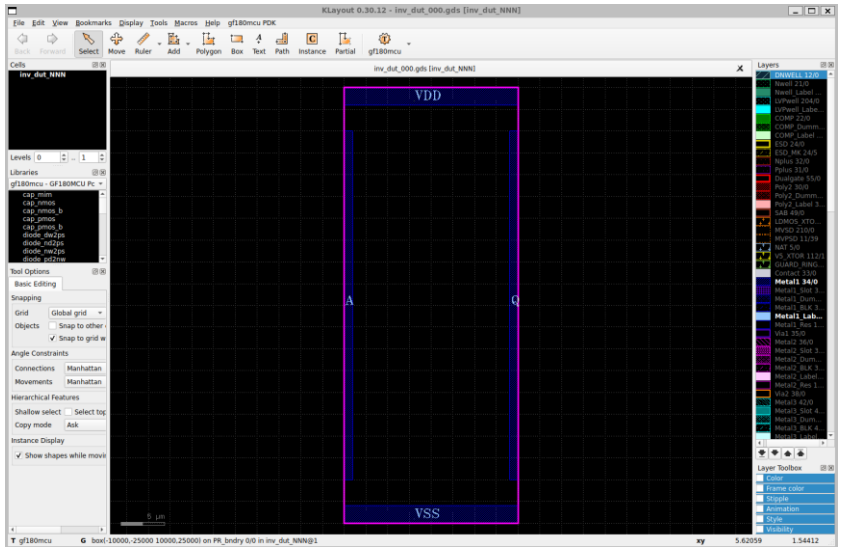


Klayout：テンプレートを開く 2/2



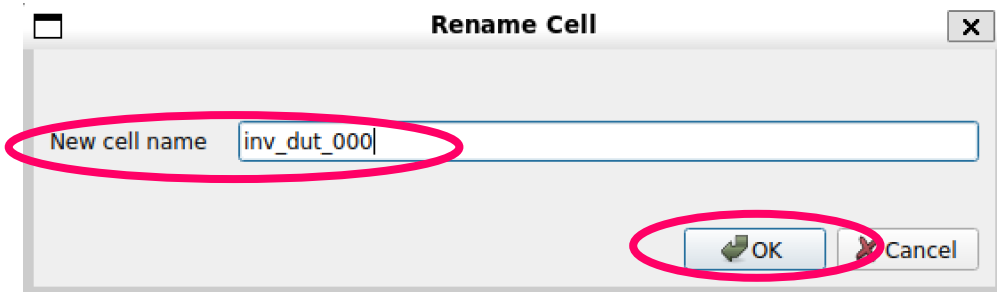
1. File -> Open を選択する
    1. Open Layout Filesダイアログが表示される。
2. inv_dut_[配布されたDUT番号].gdsを選択する
3. Openをクリックする
4. 下記の画面が出ればOKです

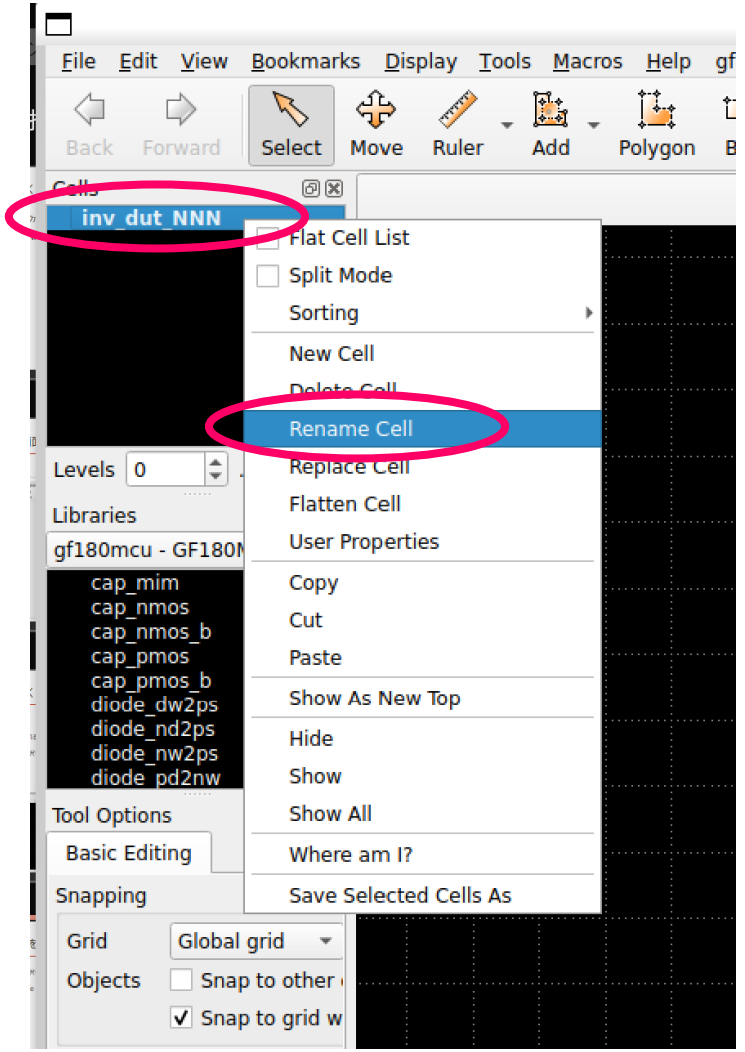




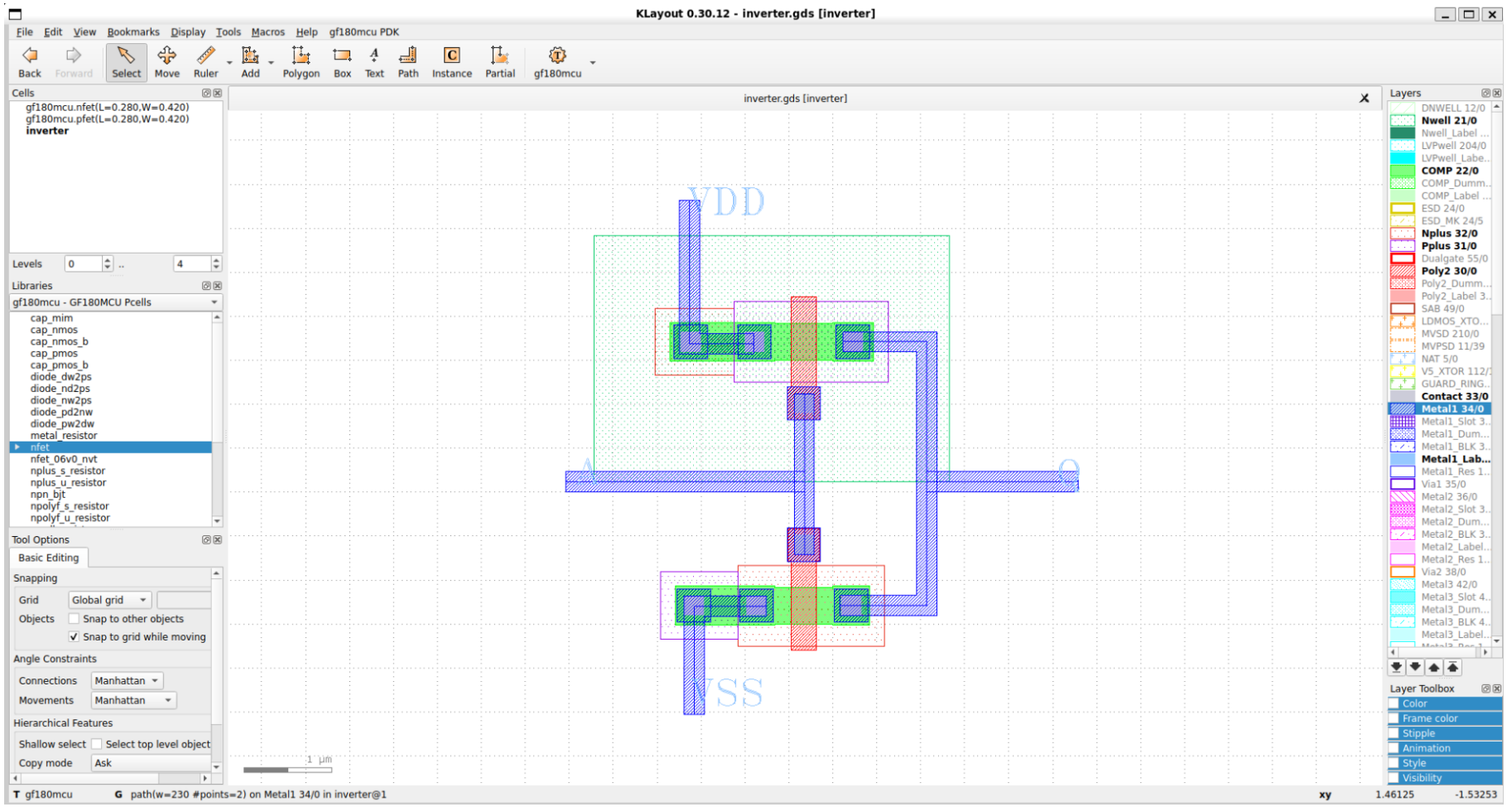
Klayout：TOPセルの名前を変更する

1. 左ペインの「Cells」の「inv_dut_NNN」を選択する
2. 右クリックすると表示されるメニューの「Rename Cell」を選択する
    1. Rename Cellダイアログが表示される
3. 「New cell name」を「inv_dut_[配布されたDUT番号]」に変更する
4. OKをクリックする

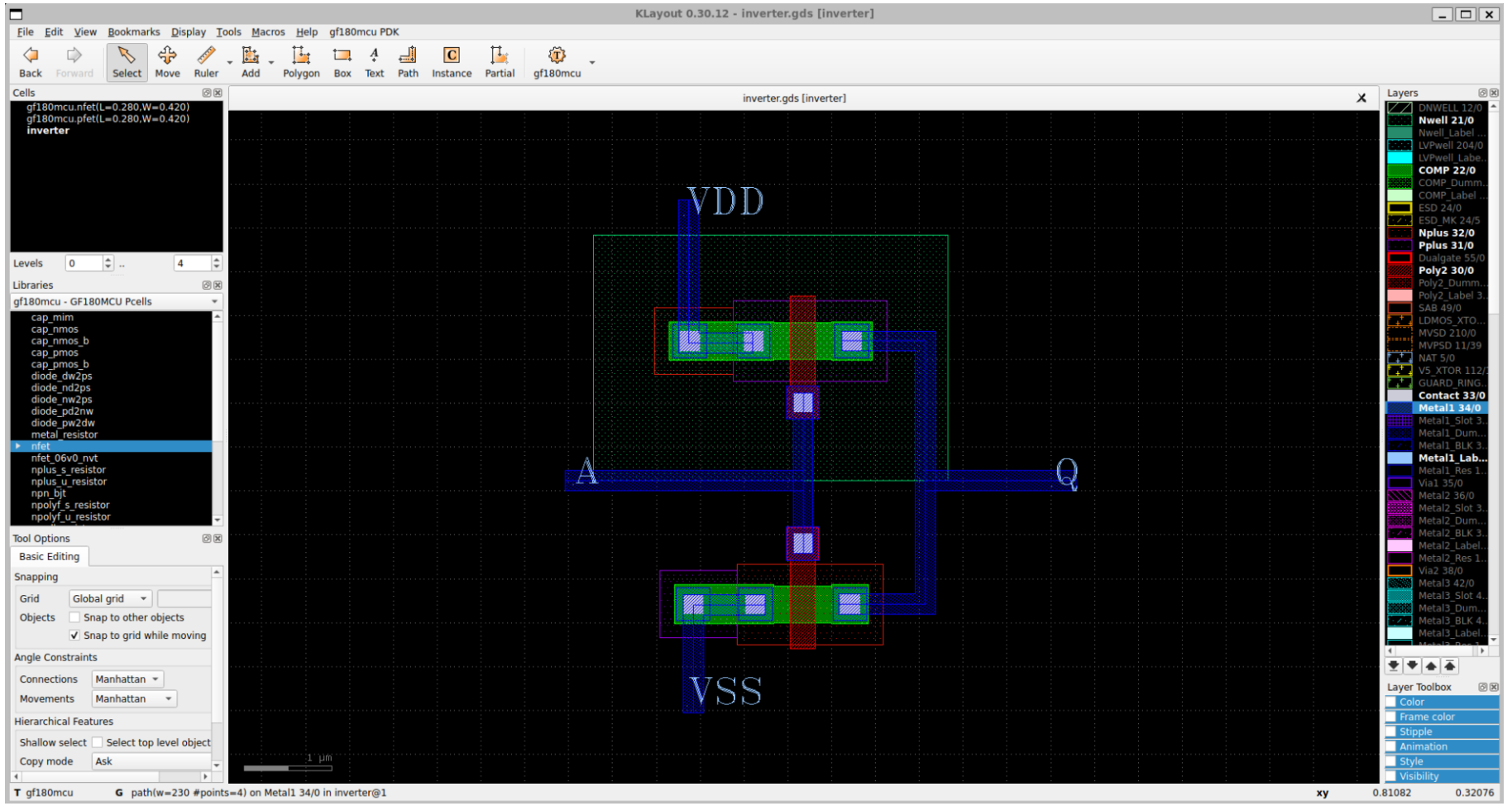




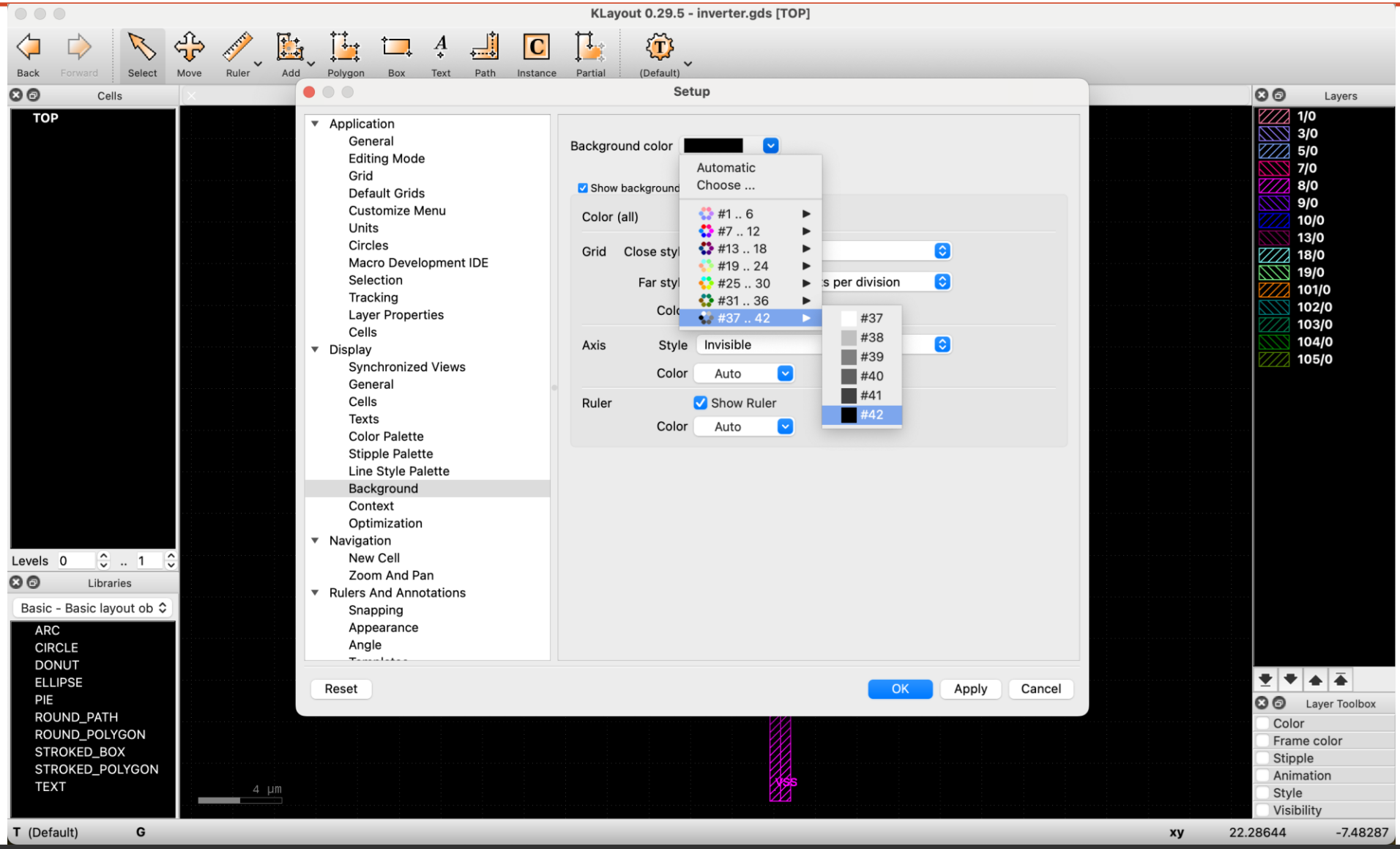
Klayout：公式PDKのレイアウト画面



Klayout：バックグラウンドが「黒」のレイアウト画面



Klayout：バックグラウンドを「黒」にする

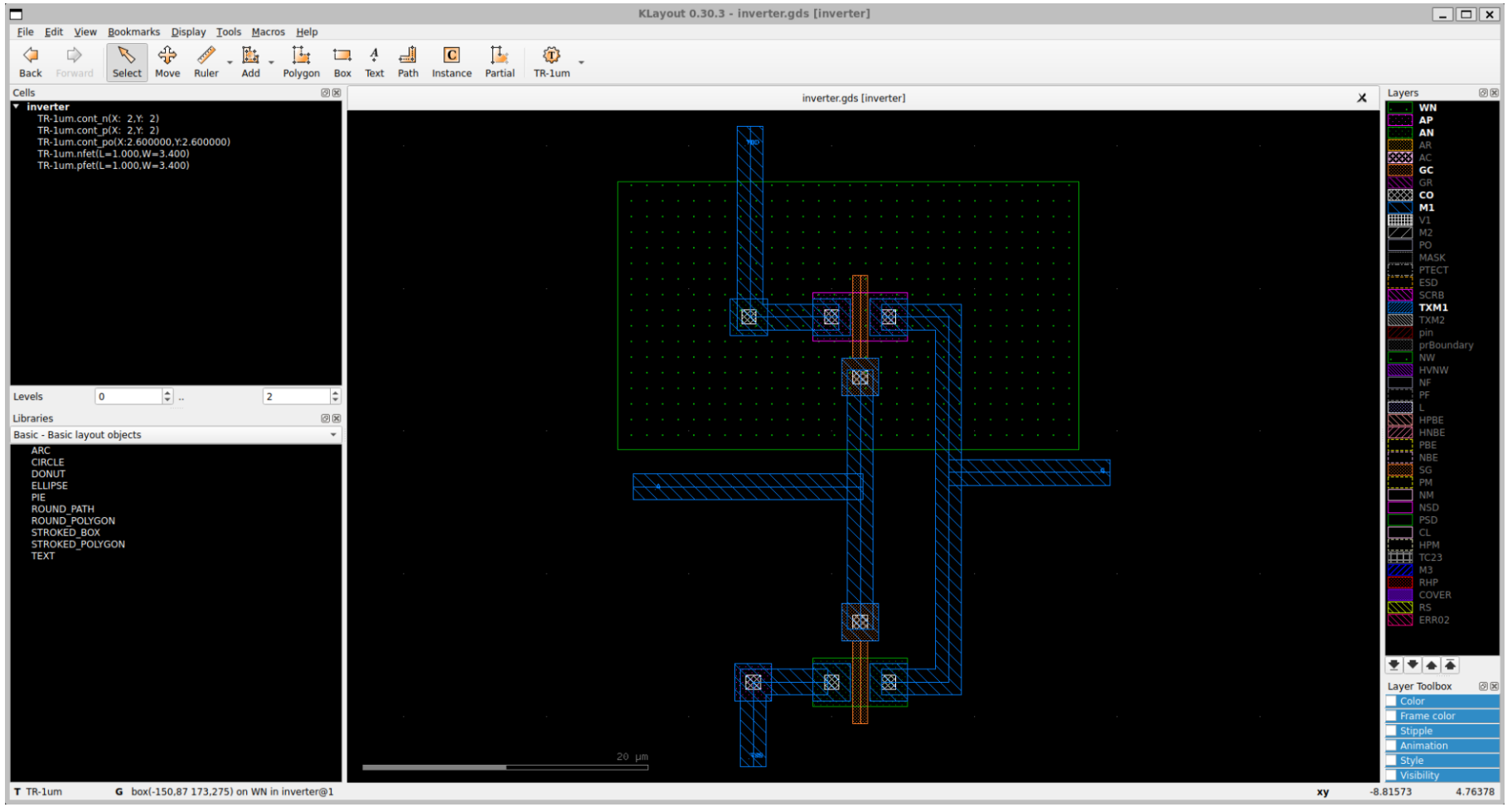


Klayout：デザインルールの場所

デザインルールマニュアル

- https://gf180mcu-pdk.readthedocs.io/en/latest/

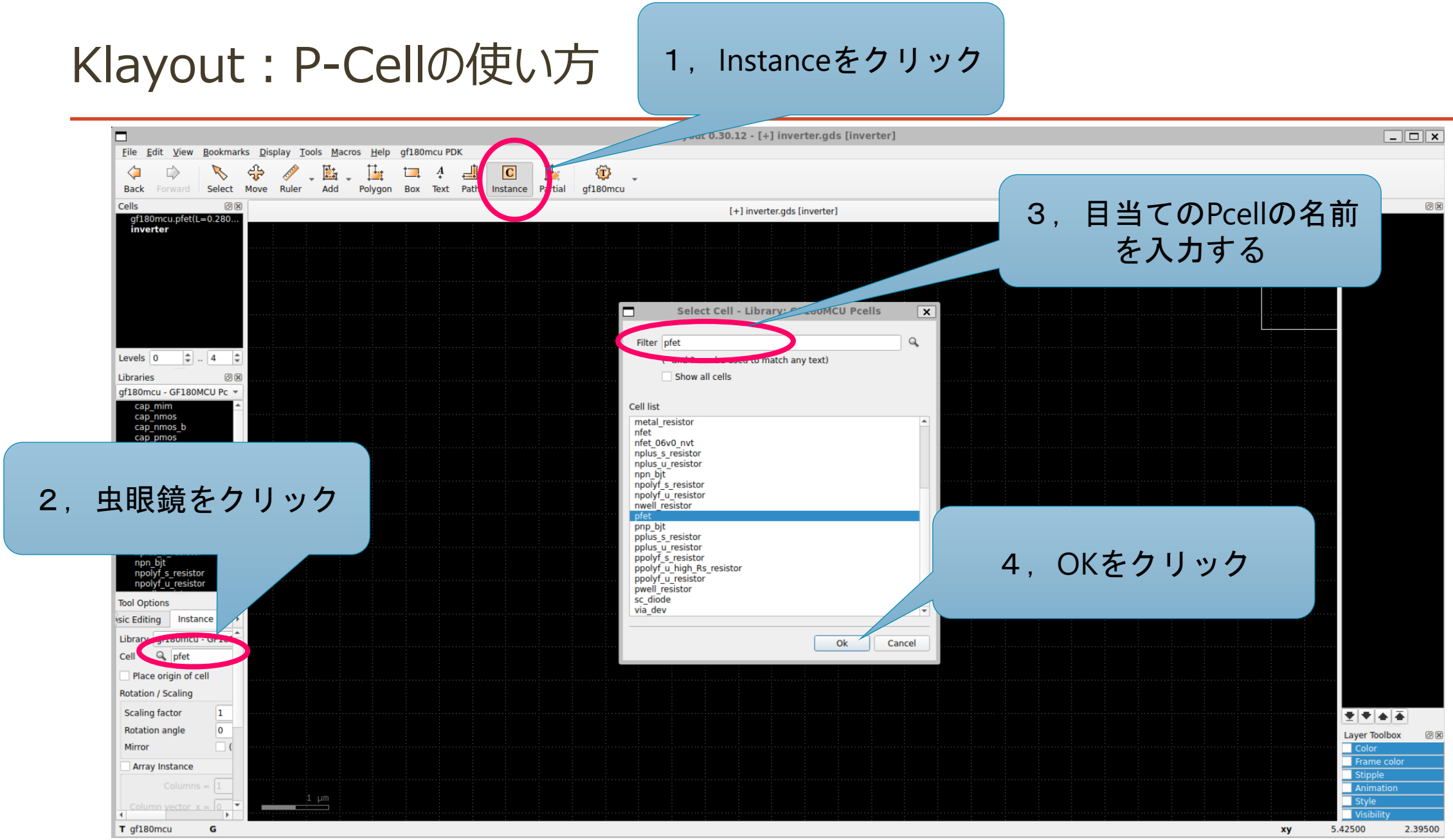
Klayout：完成サンプル



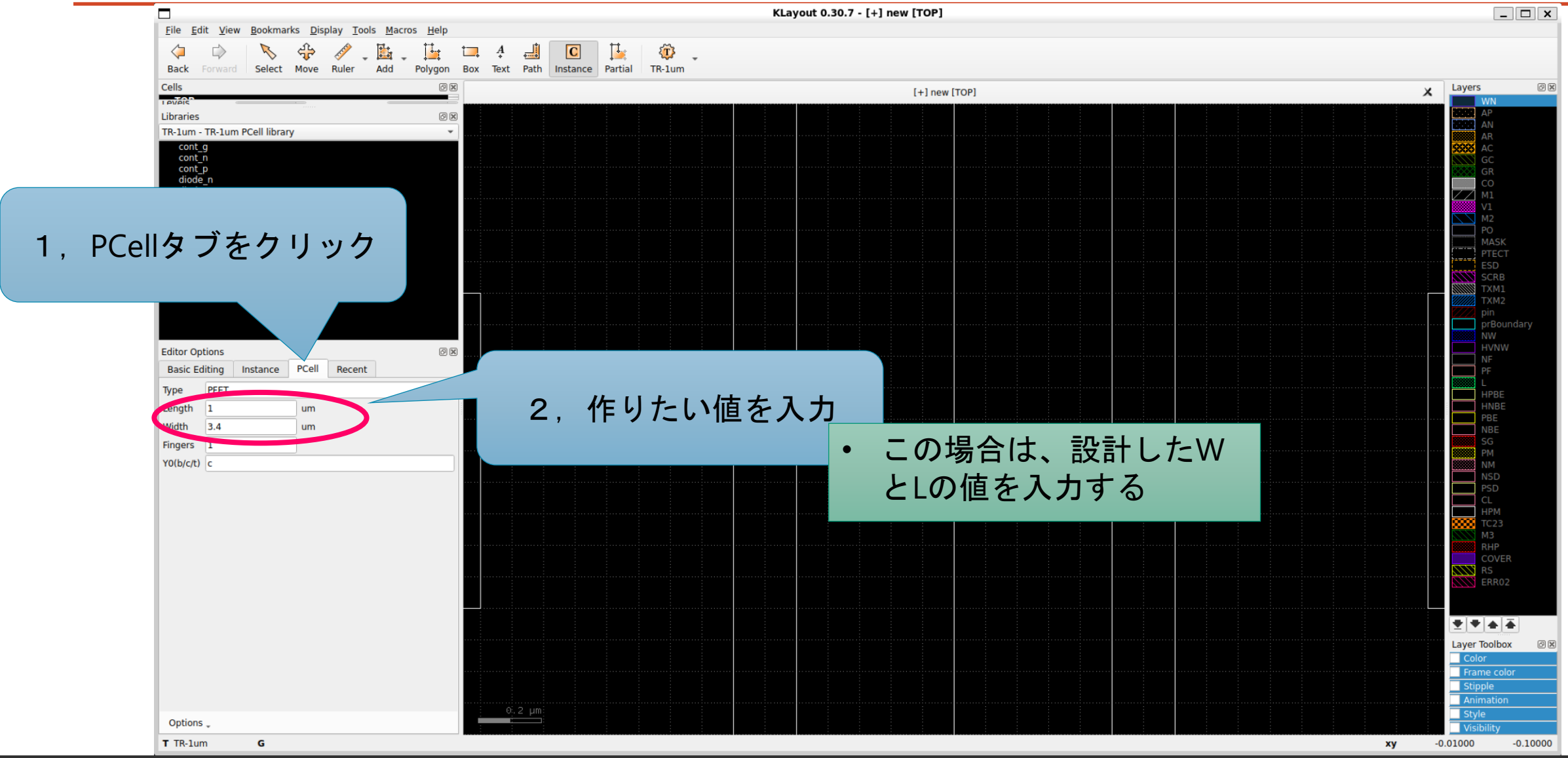
Klayout：P-Cellの使い方

- P-Cell（パラメータ化セル）とは、EDAで使用される再利用可能で構成可能なブロックのことです。P-Cellは、集積回路のレイアウトパターンを特定の設計要件に応じて調整可能なパラメータで作成することができます。これにより、各プロジェクトごとにレイアウト全体を再設計する必要がなくなり、設計プロセスの効率と柔軟性が大幅に向上します。

Klayout：P-Cellの使い方



Klayout：P-Cellの使い方

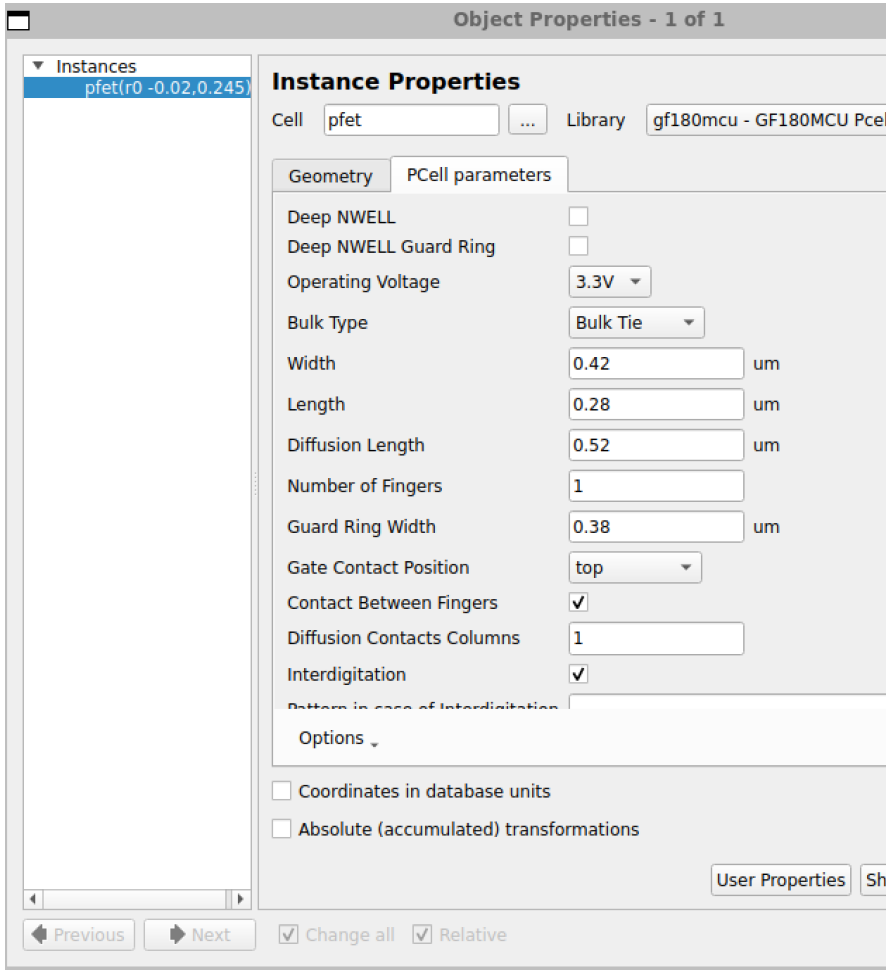


Klayout：P-Cellの使い方

Pcellのパラメータ

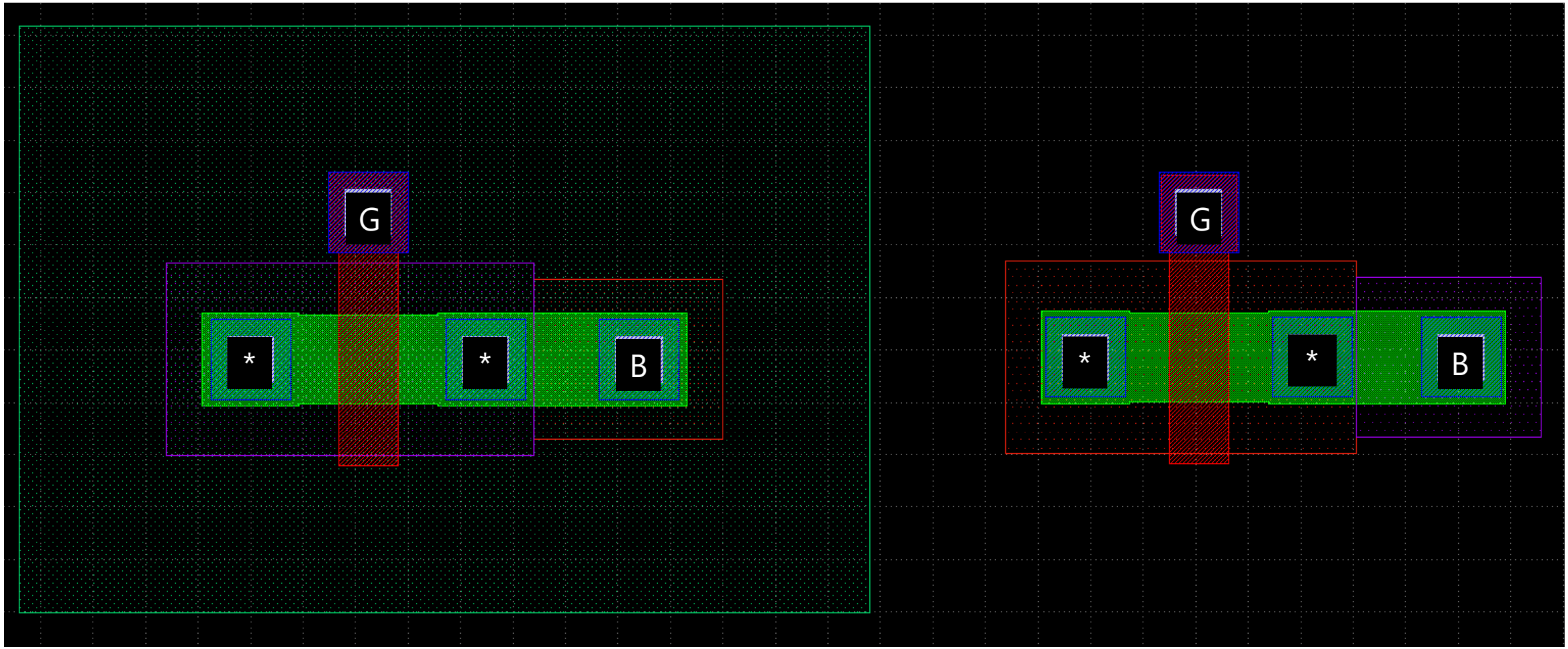
Deep NWELL	トリプルウェル用（今回は利用しない）
Operating Voltage	耐圧（設計値にする）
Bulk Type	Body端子の設置方法
Width	W値（設計値にする）
Length	L値（設計値にする）
Diffusion Length	拡散層の長さ（デフォルト値）
Number of Fingers	nf値（フィンガー数）
Gurad Ring Width	ガードリングのサイズ
Gate Contact Positon	ゲートのコンタクトの配置
Contact Between Fingers	フィンガー間の拡散層にコンタクトを設置する

これ以降はすべてデフォルトでOK

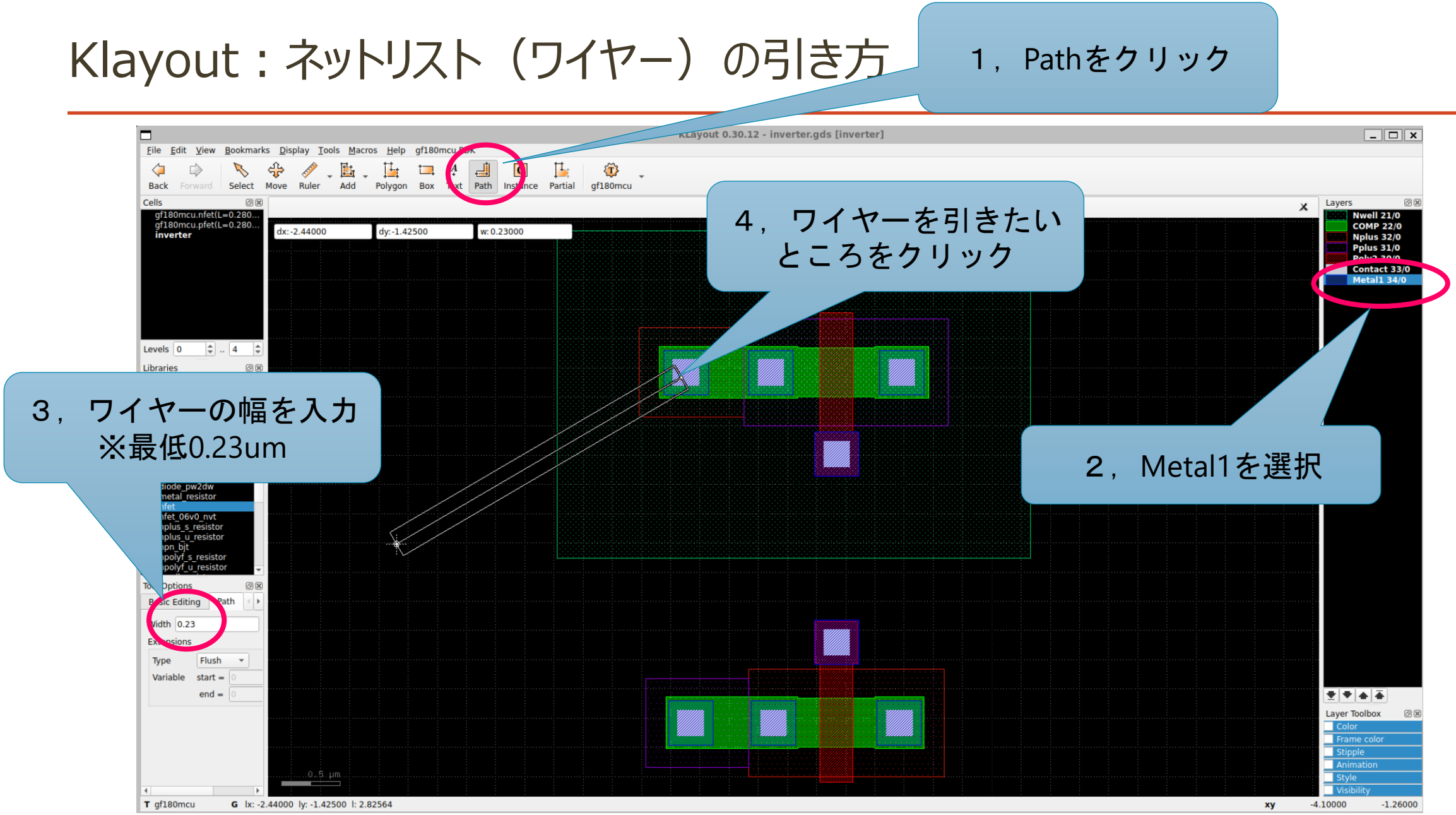


Klayout：タップ付　PFET と NFET

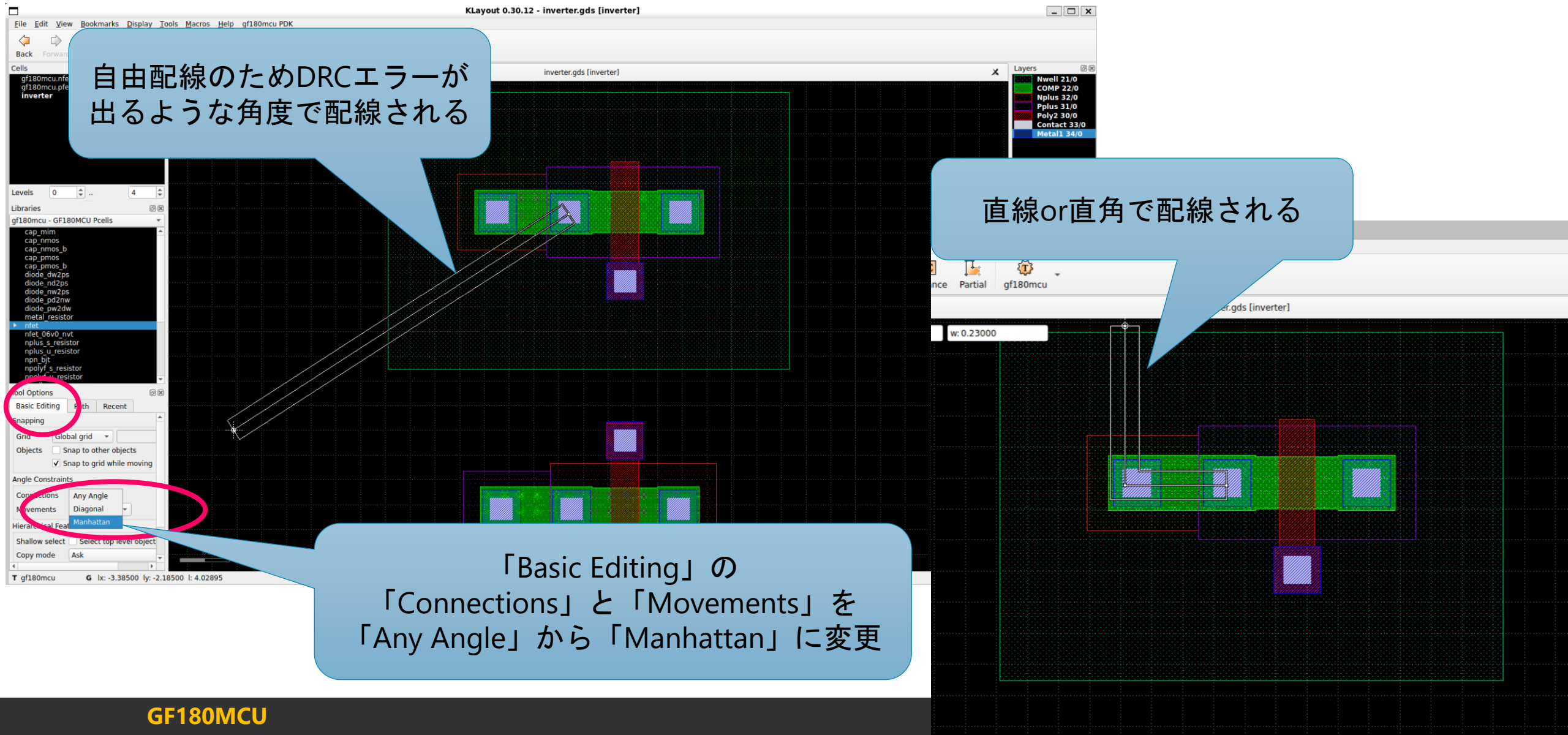
- [*] どちらをドレインにしてどちらをソースにするかは任意。



Klayout：ネットリスト（ワイヤー）の引き方



Klayout：ネットリスト（ワイヤー）の設定



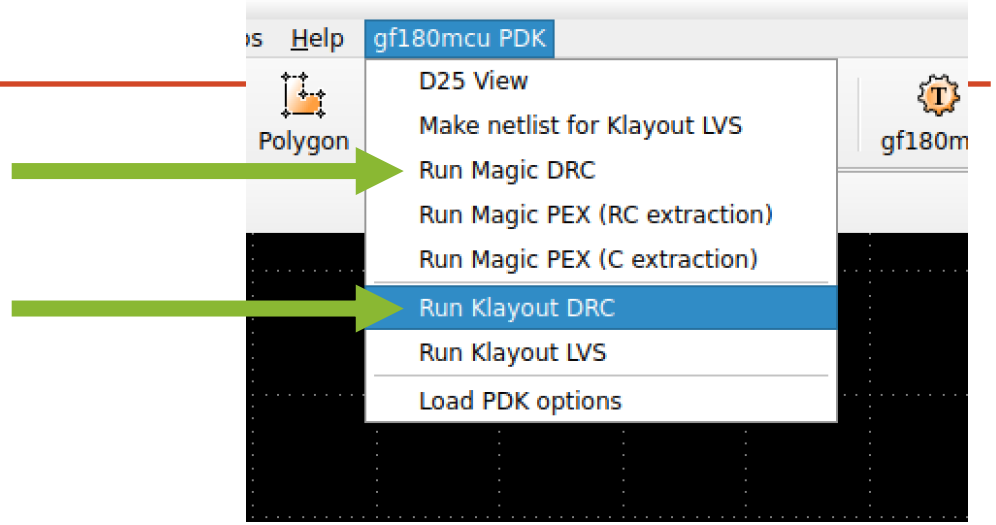
アナログ半導体の設計フロー

1. 回路図を描く
2. シミュレーションをする
3. 回路図を基にレイアウトを描く
4. **レイアウトを検証する**
5. レイアウトを基に寄生成分を考慮したシミュレーションをする
6. フレームに載せる

Klayout：レイアウト検証

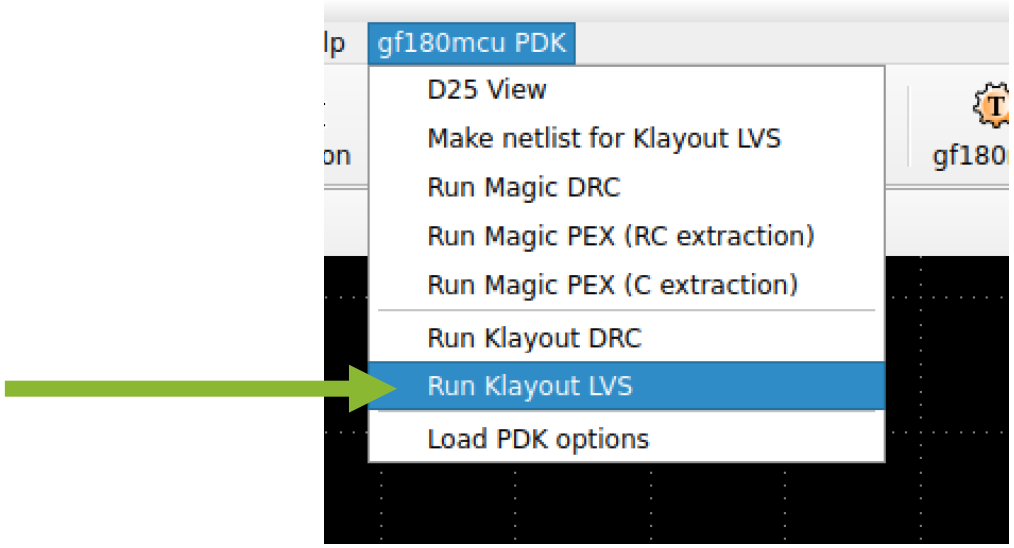
Design Rules Check (DRC)

- 指定されたデザインルールから違反していないか検証
  - Klayout DRCとMagic DRCの2種類あるので注意

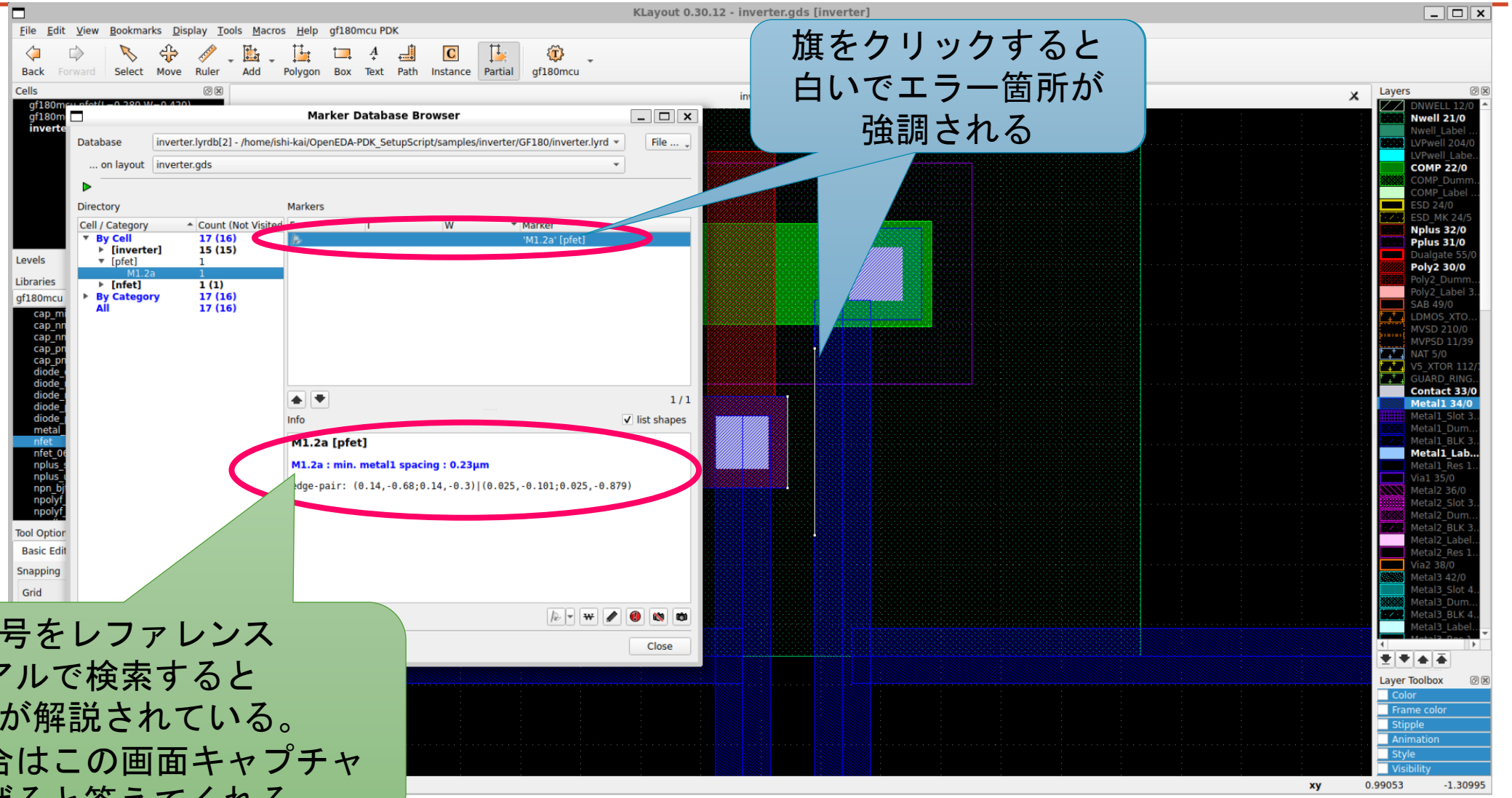


Layout Versus Schematic (LVS)

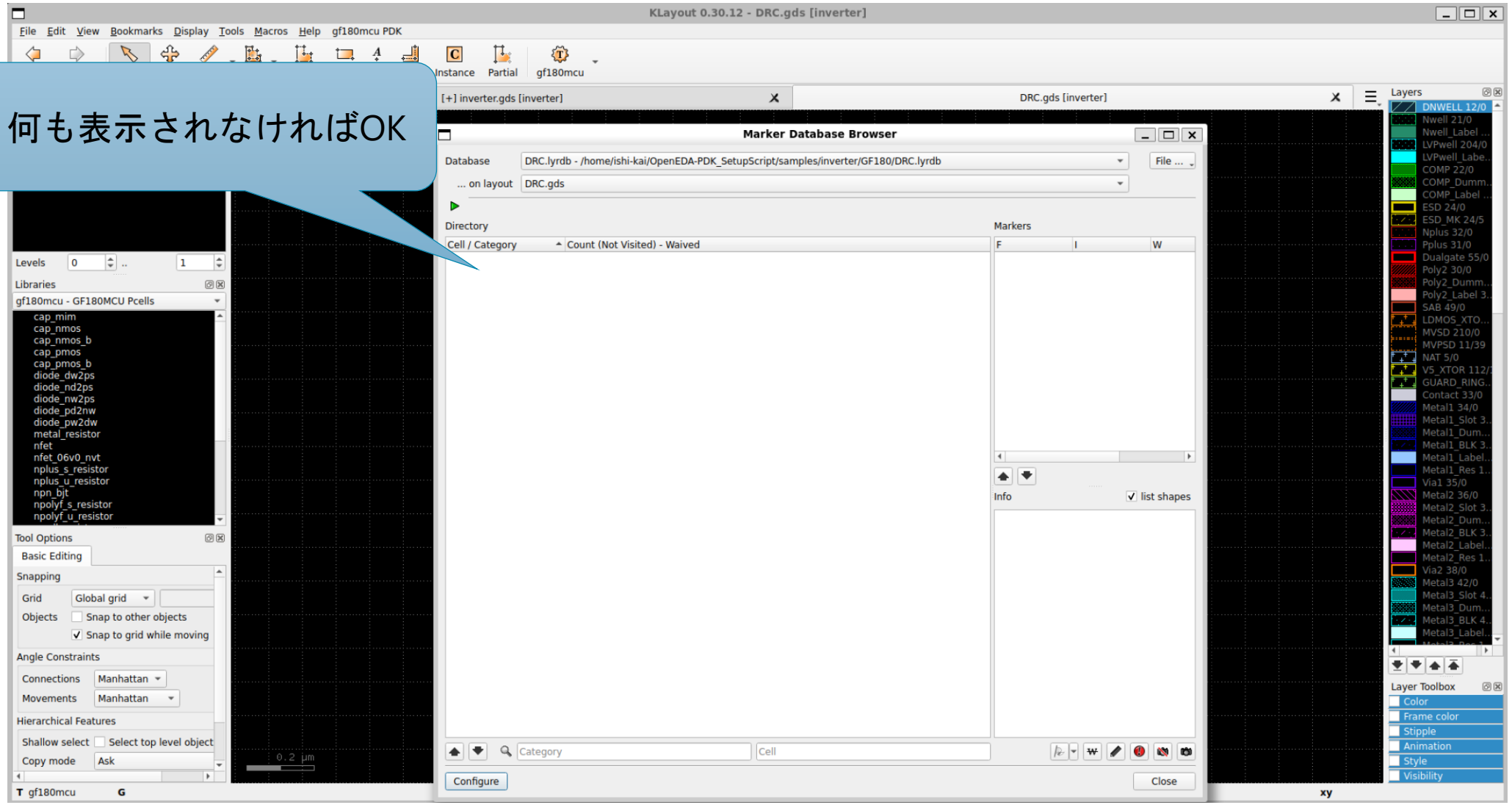
- レイアウトが回路図通りに描けているか検証



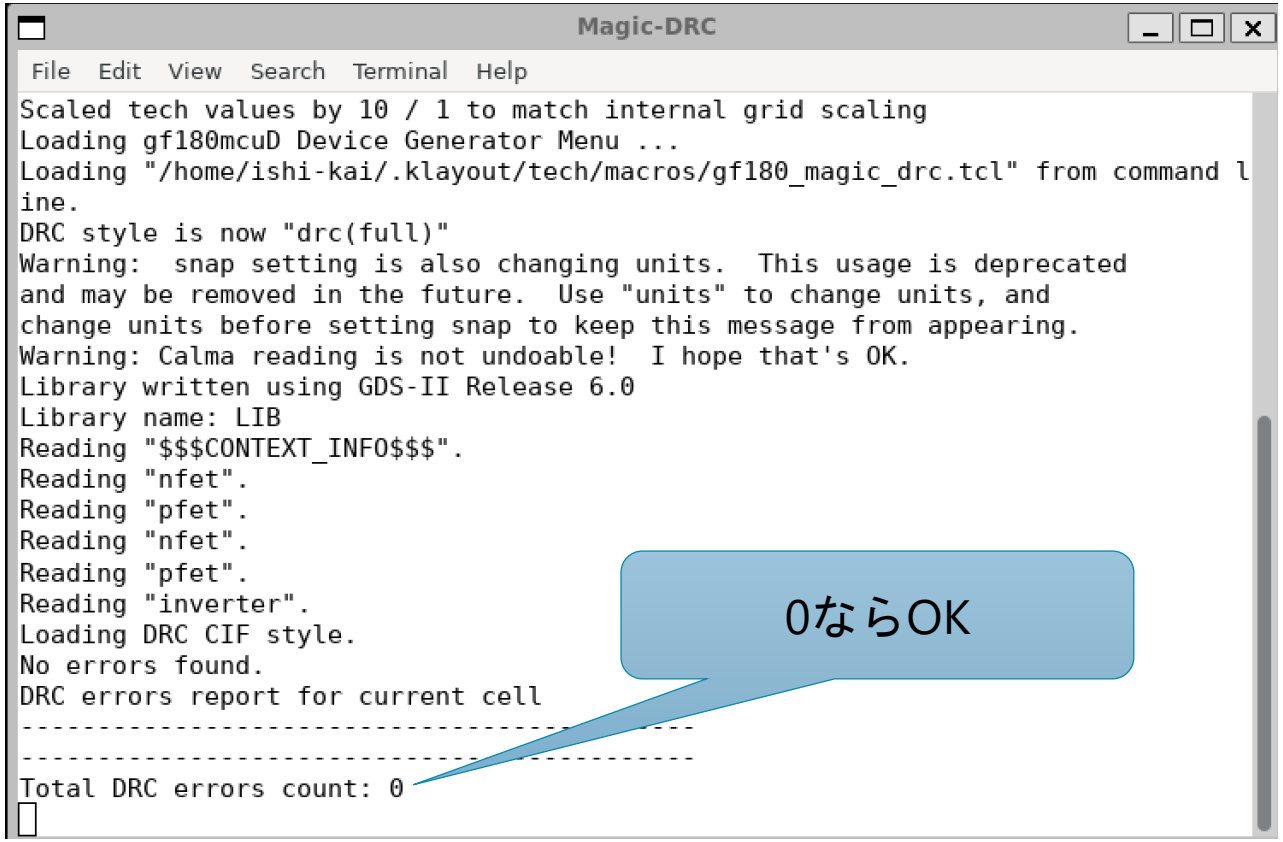
Klayout：DRCエラー



Klayout：DRC OK

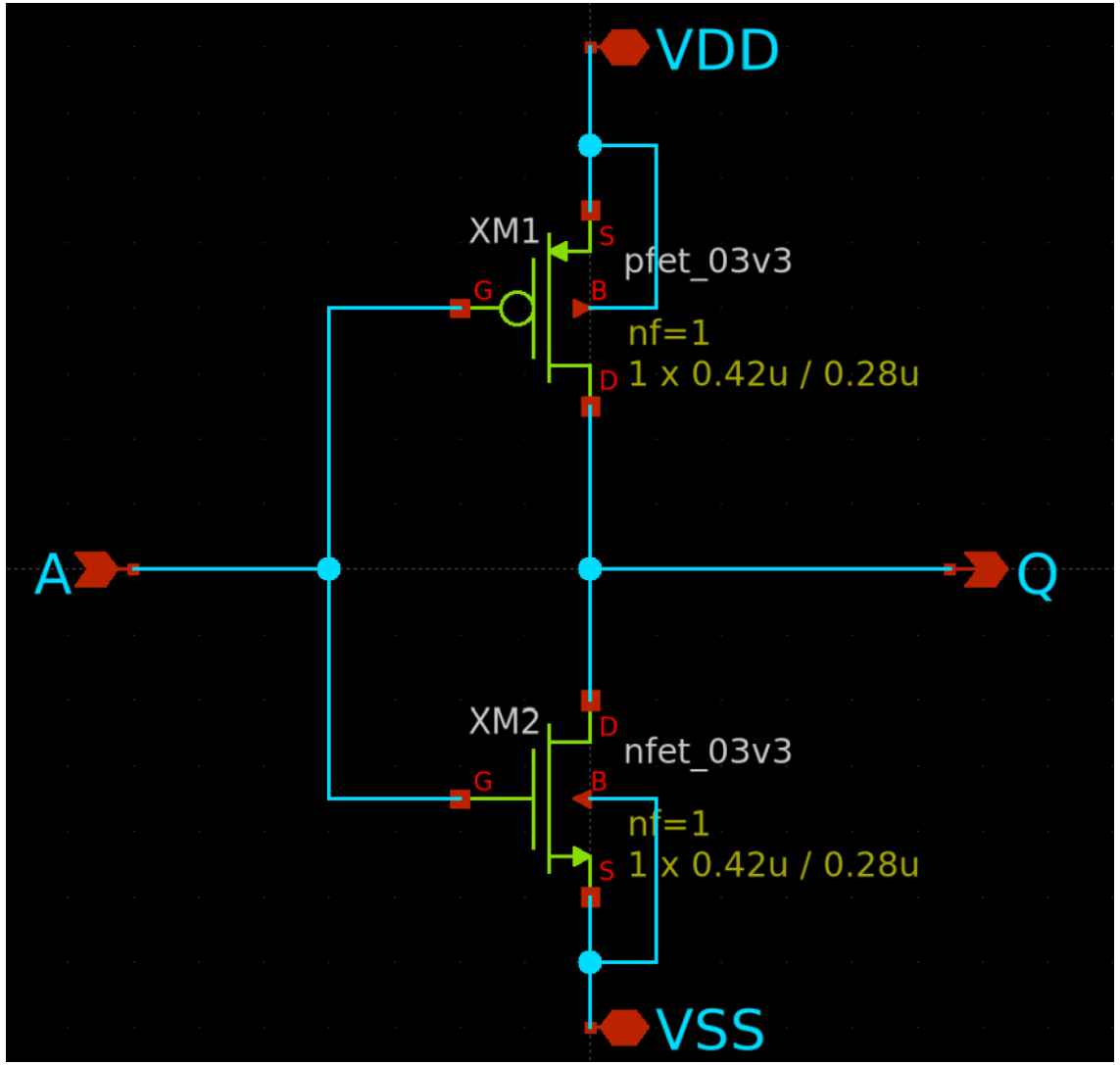


Magic : DRC OK


Magic-DRC
File Edit View Search Terminal Help
Scaled tech values by 10 / 1 to match internal grid scaling
Loading gf180mcuD Device Generator Menu ...
Loading "/home/ishi-kai/.klayout/tech/macros/gf180_magic_drc.tcl" from command l
ine.
DRC style is now "drc(full)"
Warning: snap setting is also changing units. This usage is deprecated
and may be removed in the future. Use "units" to change units, and
change units before setting snap to keep this message from appearing.
Warning: Calma reading is not undoable! I hope that's OK.
Library written using GDS-II Release 6.0
Library name: LIB
Reading "$$$CONTEXT_INFO$$$".
Reading "nfet".
Reading "pfet".
Reading "nfet".
Reading "pfet".
Reading "inverter".
Loading DRC CIF style.
No errors found.
DRC errors report for current cell
Total DRC errors count: 0
0ならOK

Klayout：LVS用ネットリストの出力方法 （1/2）

- xschem上であらかじめLVS用の回路図を作成しておく。
- ピンはipin, iopin, opin のみを使用する。
  - vdd.symやgnd.symを使用するとうまく通らない。
- ファイルを同じ名前にする 例: nand.sch, nand.gds

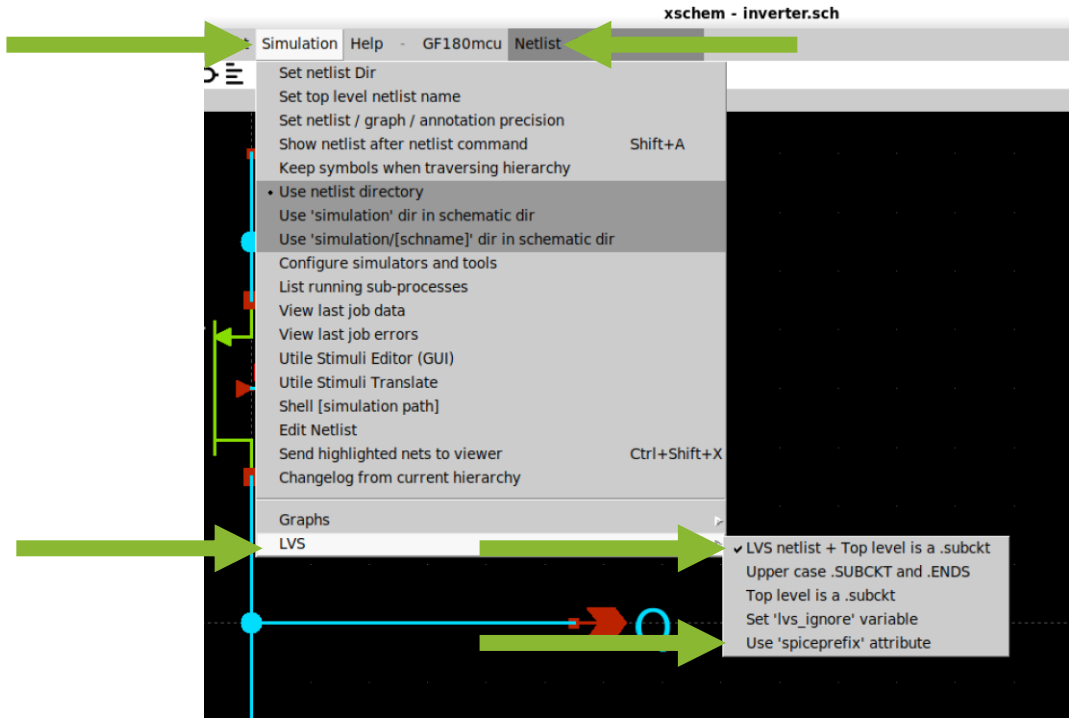


Klayout：LVS用ネットリストの出力方法 (2/2)

- xschem上で**LVS用**のネットリストを出力する。

1. メニュー Simulation > LVS > Use ‘spiceprefix’ attribute の✓を外す
2. メニュー Simulation > LVS > LVS netlist + Top level is a .subckt に✓を入れる
3. Netlistを押す

- 終わったら閉じて良い

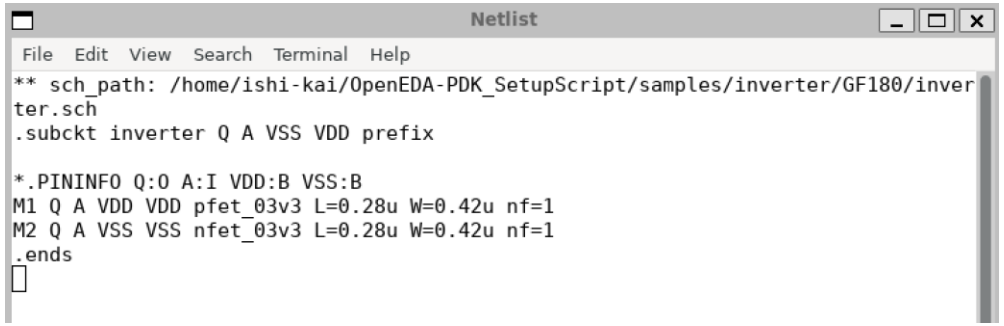


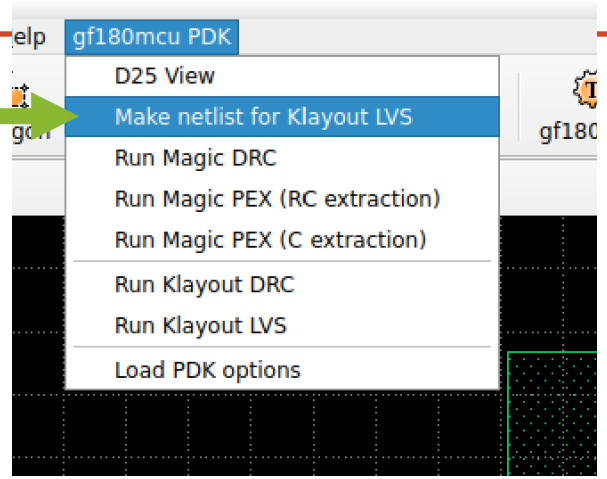
Klayout：LVSの実行

- klayout上でネットリスト比較を実行する。

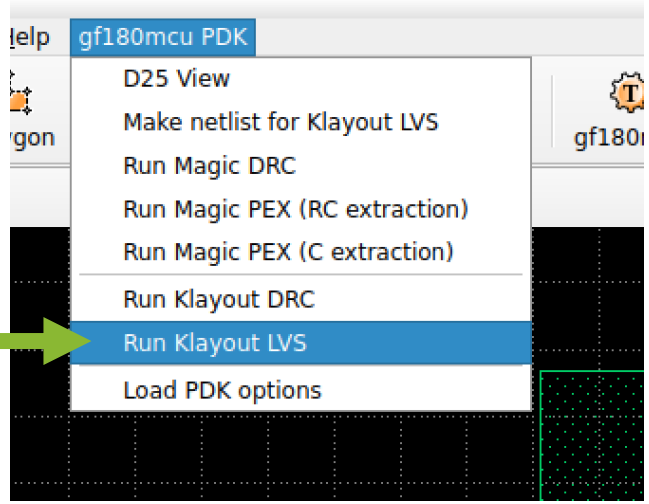
1. メニュー gf180mcu PDK > Make netlist for Klayout LVSを実行する

   1. 下記のようなウィンドウが出ればOK。閉じてもよい。

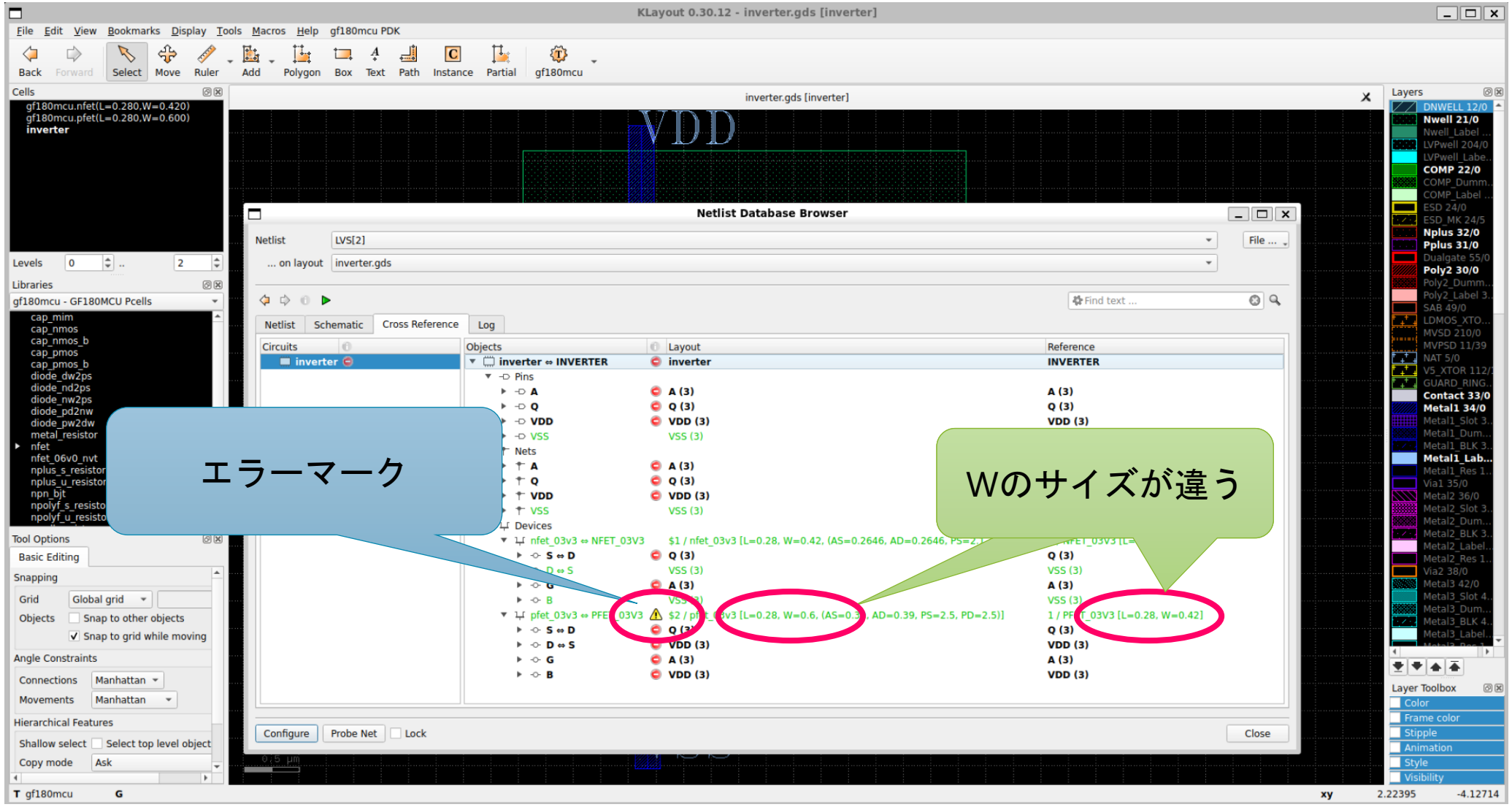




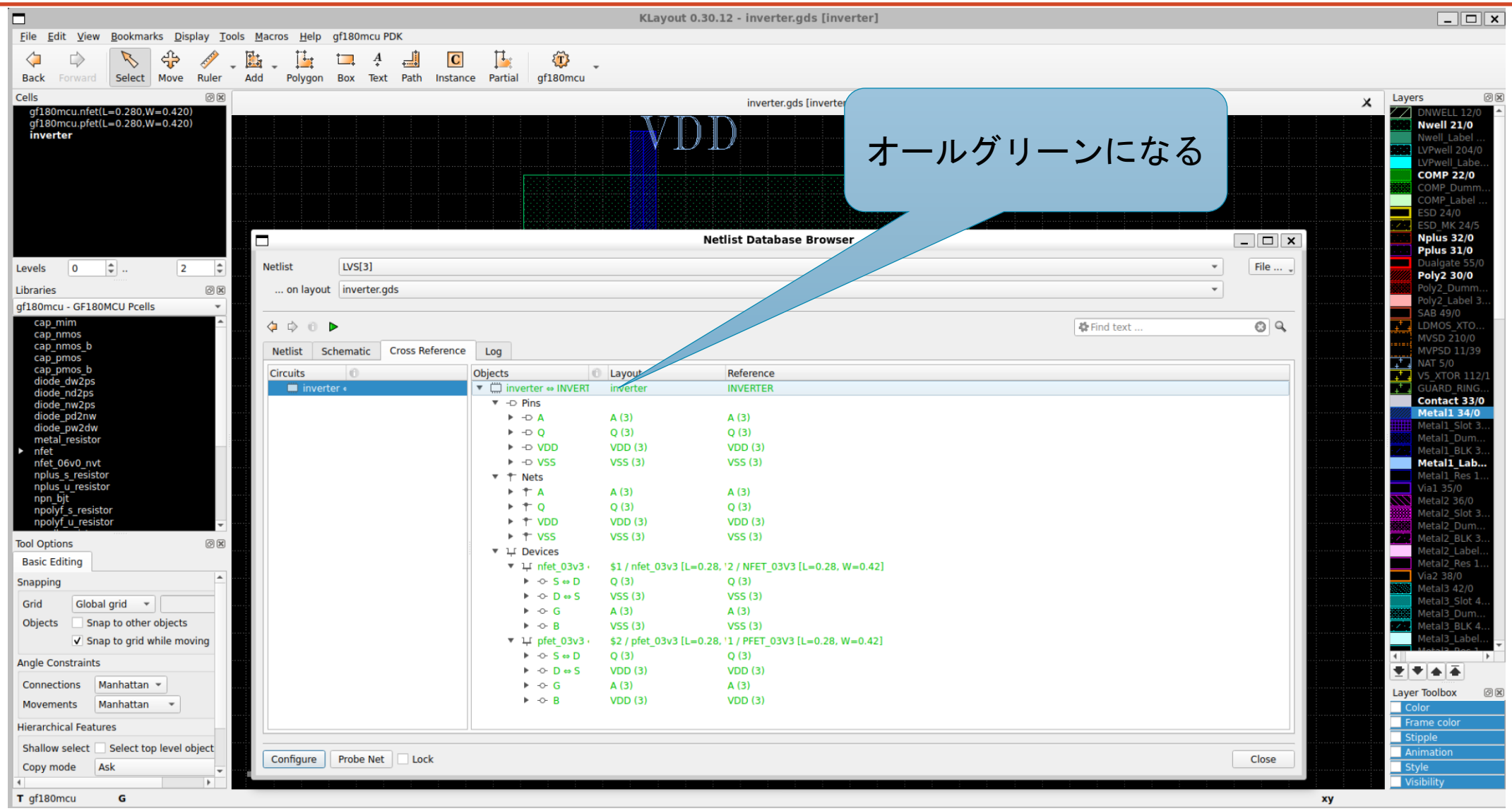
2. メニュー gf180mcu PDK > Run Klayout LVS を実行する



Klayout : LVS NG



Klayout : LVS OK



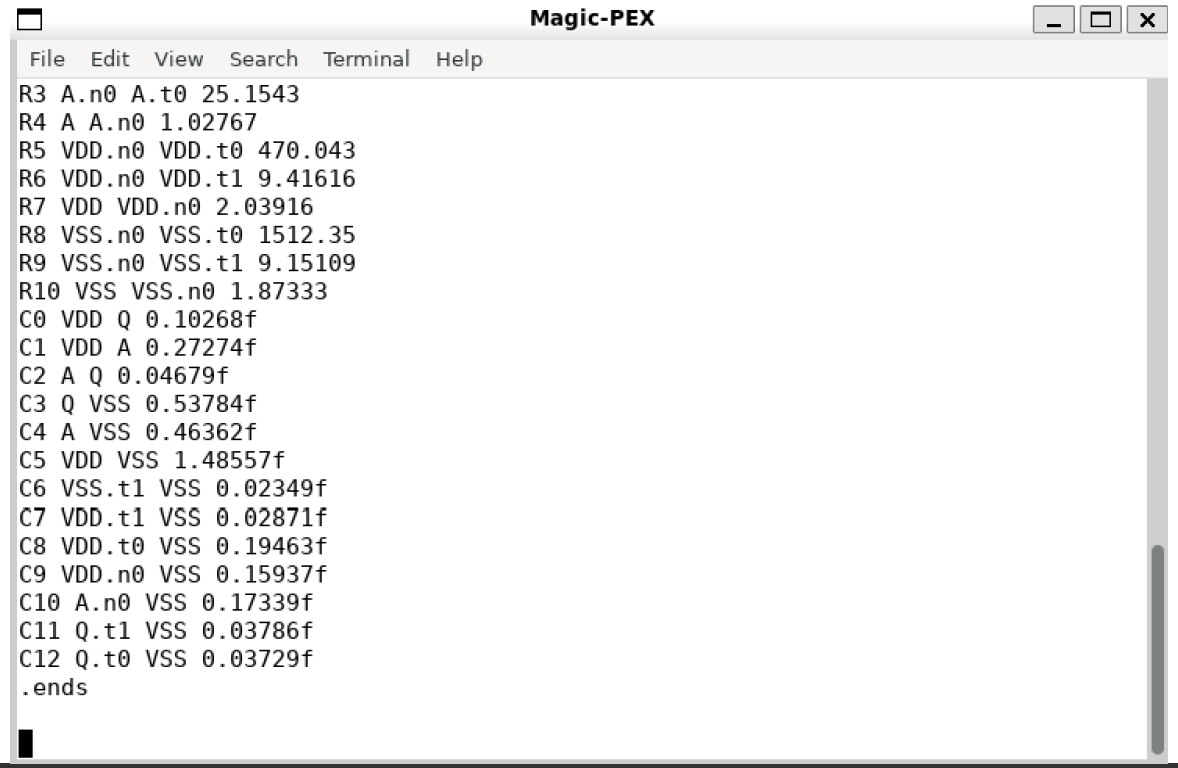
アナログLSIの設計フロー

1. 回路図（ベンチマーク）を描く
2. シミュレーションをする
3. 回路図を基にレイアウトを描く
4. レイアウトを検証する
5. **レイアウトを基に寄生成分を考慮したシミュレーションをする**
6. フレームに載せる

レイアウト検証：PEX, Parasitic Extraction

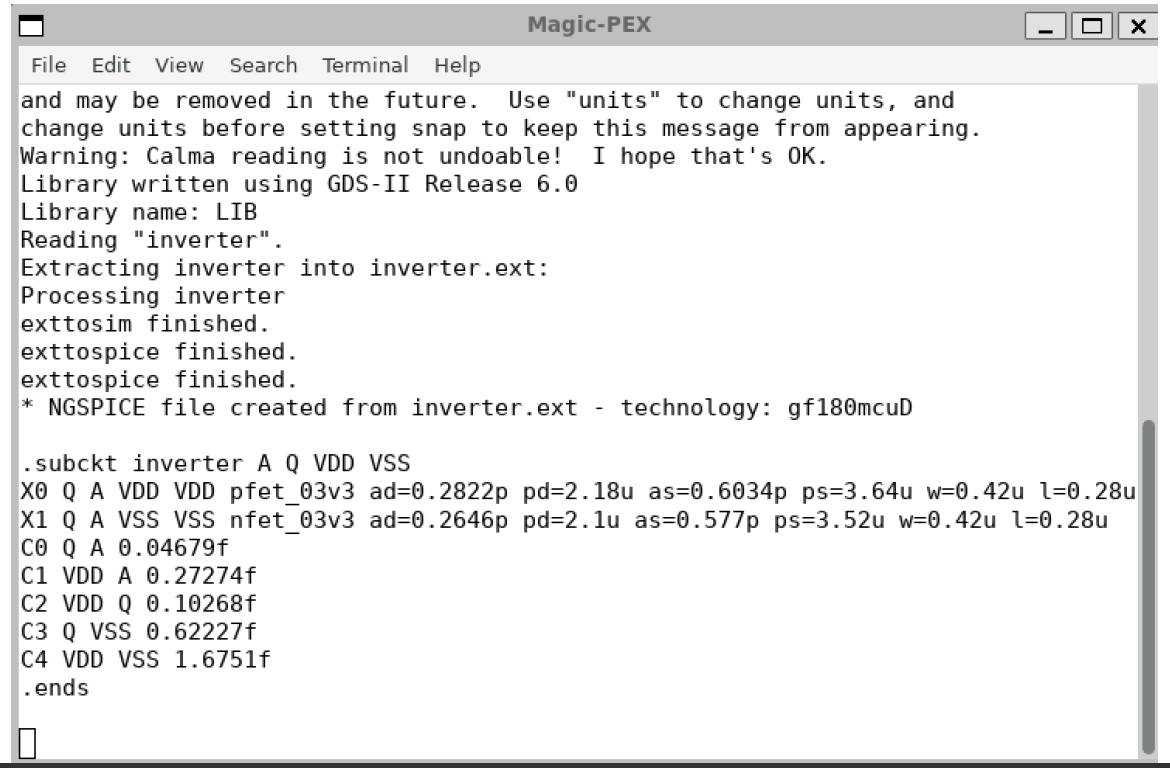
RC抽出

- 寄生抵抗と寄生容量を含むネットリストを抽出
  - Magic側のバグで一部のみしか動作



C抽出

- 寄生容量のみを含むネットリストを抽出
  - 厳密解を求めない場合はこちらを利用



PEX前の下準備：Flatten

1. コピーを保存する

- ファイル名で大文字を使うとエラーが出てしまう（不具合）
    - 拡張子がデフォルトだと.GDSなので.gdsに直す

2. ドラッグでレイアウト全体を選択する

3. Edit > Flatten Cell > Flatten Instances を All hierarchy levels で実行

4. Cells が TOP のみになっていることを確認して Save

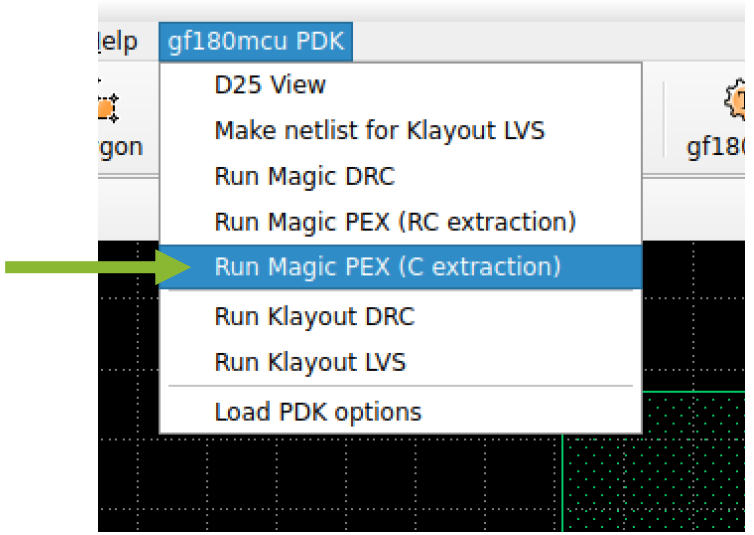
寄生抽出

Magic PEX を用いて寄生容量抽出を行う

- Run Magic PEX (C extraction) を実行

実行すると寄生容量付きネットリストが生成される

- ファイル名：[TOPセル名]_pex_extracted.spice
  - デフォルトではTOP_pex_extracted.spice



寄生容量を考慮したベンチマークに変更する

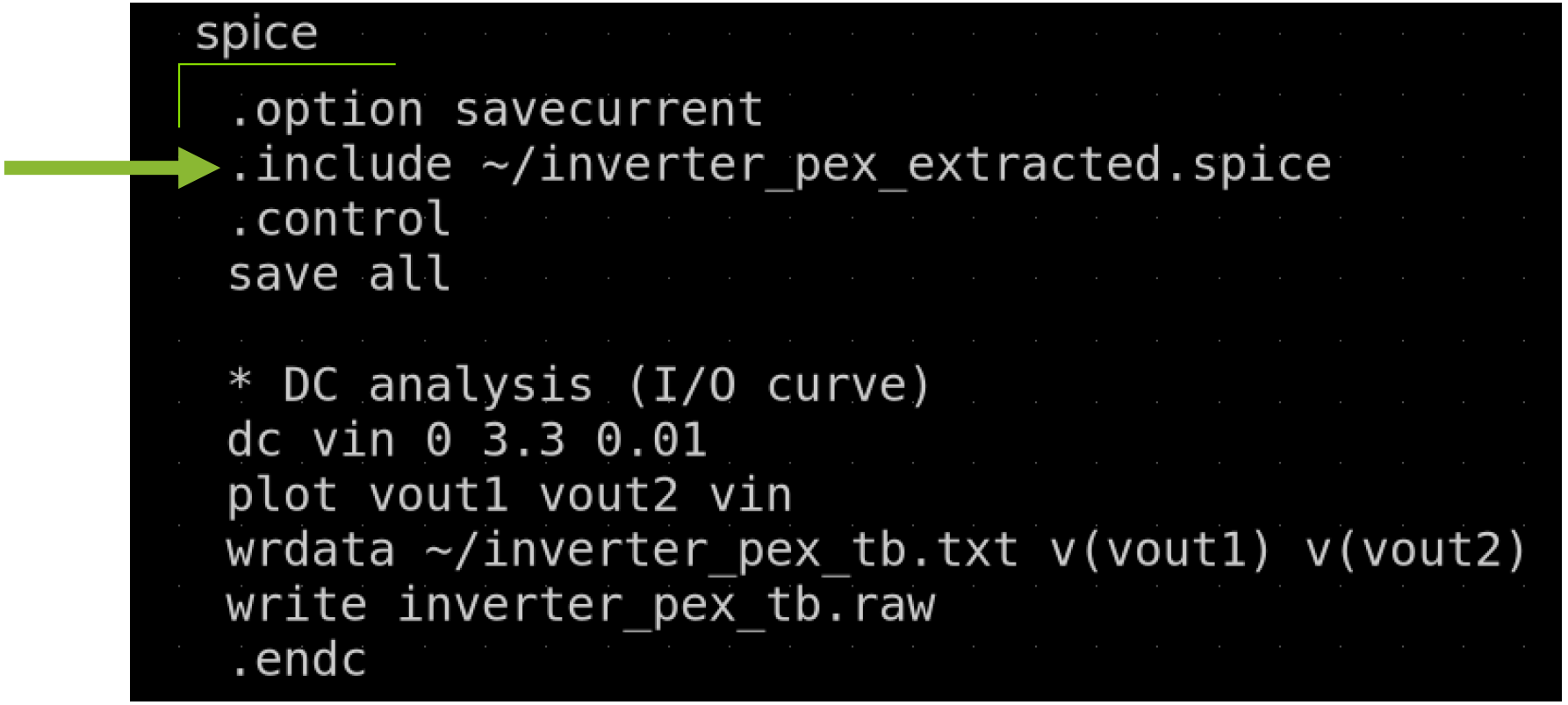
ポストシミュレーションの際は以下 2 点の変更が必要

1. 外部ネットリストへのパス
2. 外部ネットリスト用のシンボル

外部ネットリストへのパスを通す

シミュレーションスクリプトに.includeでパスを通す

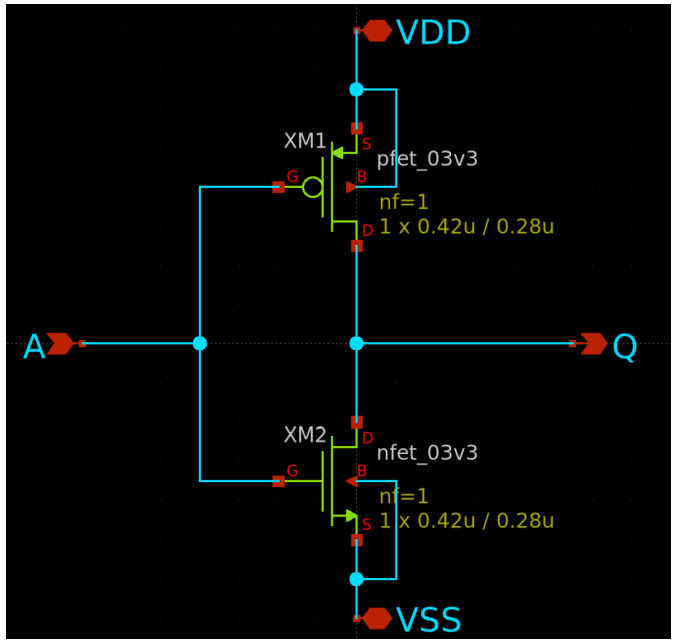
- レイアウトデータと同じ階層に [TOPセル名]_pex_extracted.spice が生成される

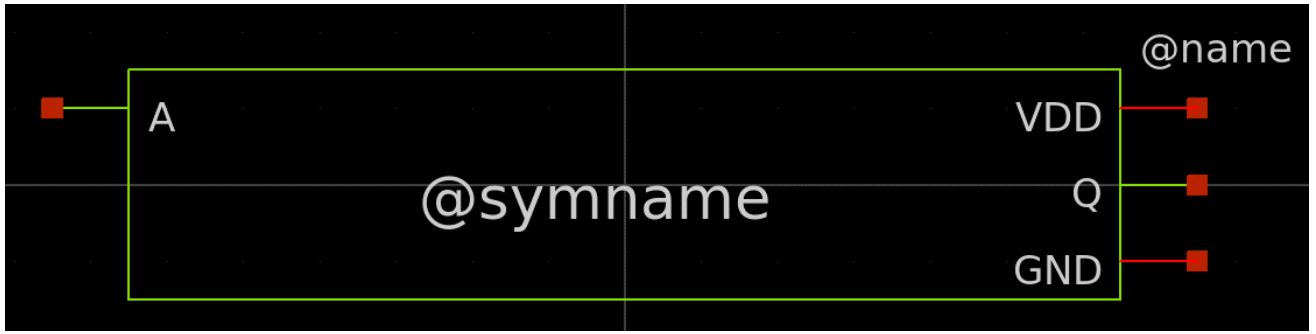


外部ネットリスト用のシンボル作成

LVSで使用した回路図からシンボルを作成する

- メニュー Symbol > Make symbol from schematic





このままだとシンボルはLVSで使用した回路図を参照するため
ネットリストを参照するように改変する

外部ネットリスト用のシンボル作成　（基本形）

シンボルのプロパティを開いてこんな感じの中身にする　（Qキーでプロパティを開く）

type=primitive

format="@name [ネットリスト内のピン] @prefix"

template="name=x1 prefix=[subcktの名前]"

extra="prefix"

highlight=true

詳しくは　http://repo.hu/projects/xschem/xschem_man/symbol_property_syntax.html　を参照

外部ネットリスト用のシンボル作成例　（インバータ）

type=primitive

format="@name @@A @@Q @@VDD @@GND @prefix“

template="name=x1 prefix=inverter“

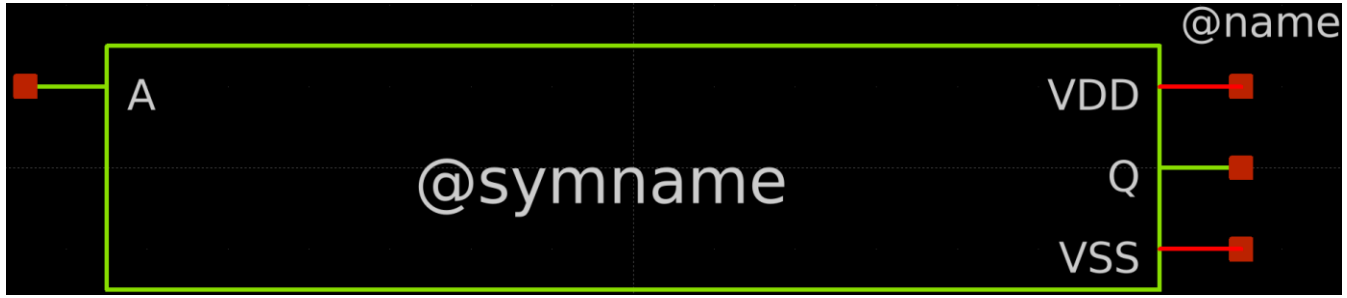
extra="prefix“

highlight=true

```
.subckt inverter A Q VDD VSS
X0 Q A VDD VDD pfet_03v3 ad=0.2822p pd=2.18u as=0.6034p ps=3.64u w=0.42u l=0.28u
X1 Q A VSS VSS nfet_03v3 ad=0.2646p pd=2.1u as=0.577p ps=3.52u w=0.42u l=0.28u
C0 Q A 0.04679f
C1 VDD A 0.27274f
C2 VDD Q 0.10268f
C3 Q VSS 0.62227f
C4 VDD VSS 1.6751f
.ends
```

抽出したネットリストのピンと同じ並びで記述する

外部ネットリスト用のシンボル作成例　（インバータ）



type=primitive

format="@name @@A @@Q @@VDD @@GND @prefix“

template="name=x1 prefix=inverter“

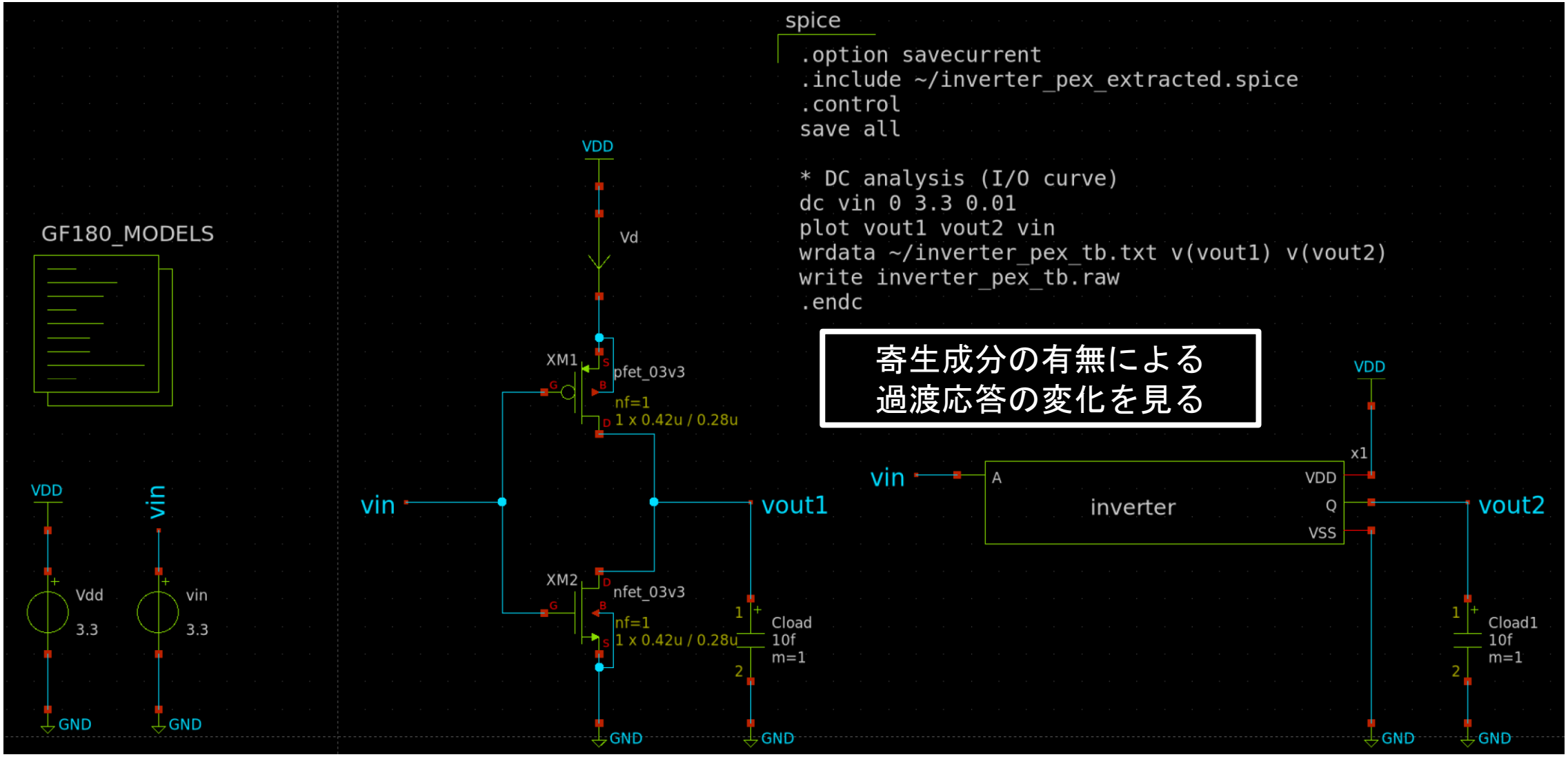
extra="prefix“

highlight=true

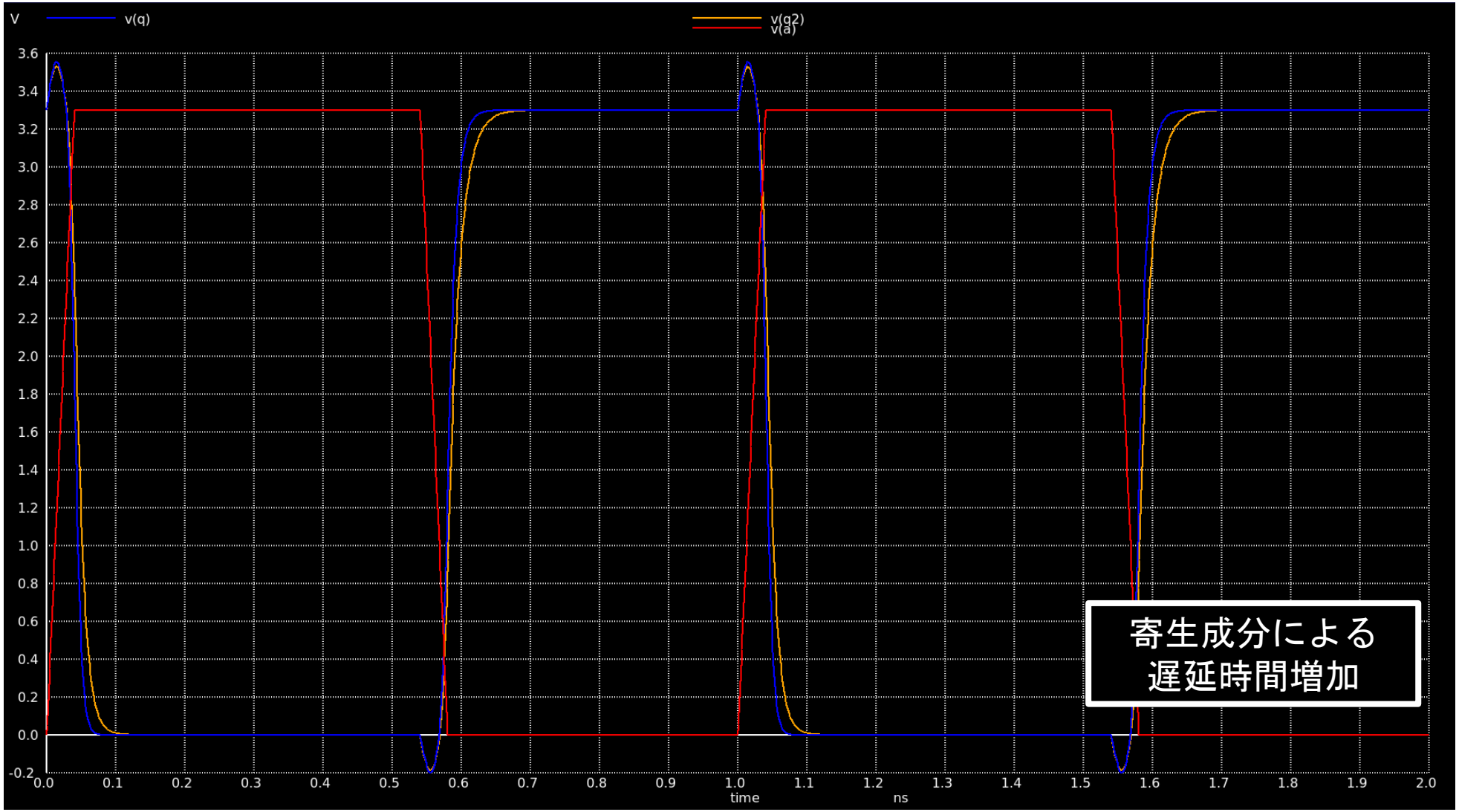
シンボルから出ているピンには”@@”のprefixをつける

これで一応完成

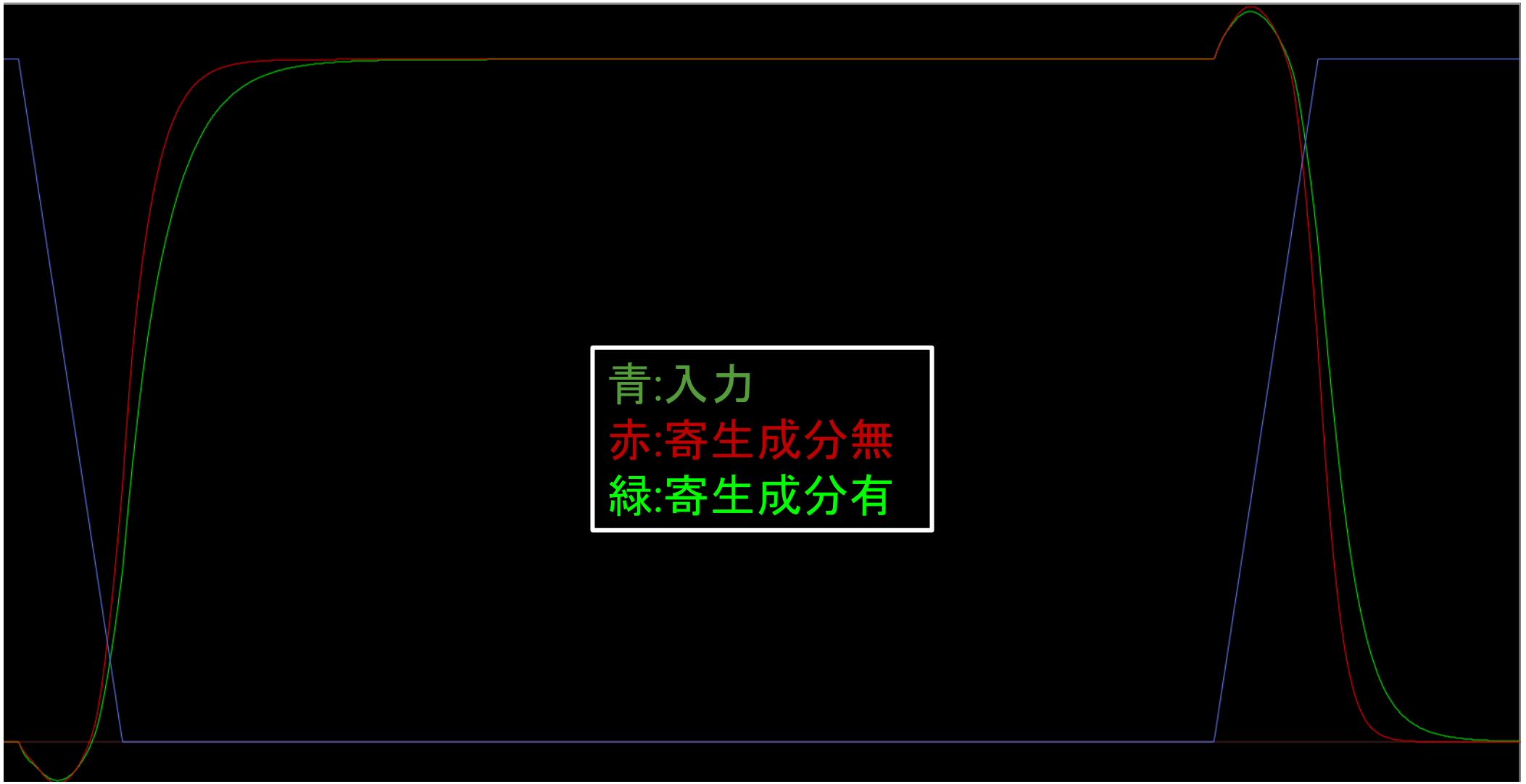
ベンチマーク例


spice
.option savecurrent
.include ~/inverter_pex_extracted.spice
.control
save all
* DC analysis (I/O curve)
dc vin 0 3.3 0.01
plot vout1 vout2 vin
wrdata ~/inverter_pex_tb.txt v(vout1) v(vout2)
write inverter_pex_tb.raw
.endc
寄生成分の有無による
過渡応答の変化を見る
GF180_MODELS
VDD
Vd
XM1
pfet_03v3
nf=1
1 x 0.42u / 0.28u
vin
vout1
XM2
nfet_03v3
nf=1
1 x 0.42u / 0.28u
Cload
10f
m=1
GND
VDD
vin
Vdd
3.3
vin
3.3
GND
GND
x1
A
VDD
Q
VSS
inverter
vin
vout2
VDD
Cload1
10f
m=1
GND
GND

シミュレーション結果


V
v(q)
v(q2)
v(a)
3.6
3.4
3.2
3.0
2.8
2.6
2.4
2.2
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
-0.2
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1.0
1.1
1.2
1.3
1.4
1.5
1.6
1.7
1.8
1.9
2.0
time
ns
寄生成分による
遅延時間増加

シミュレーション結果

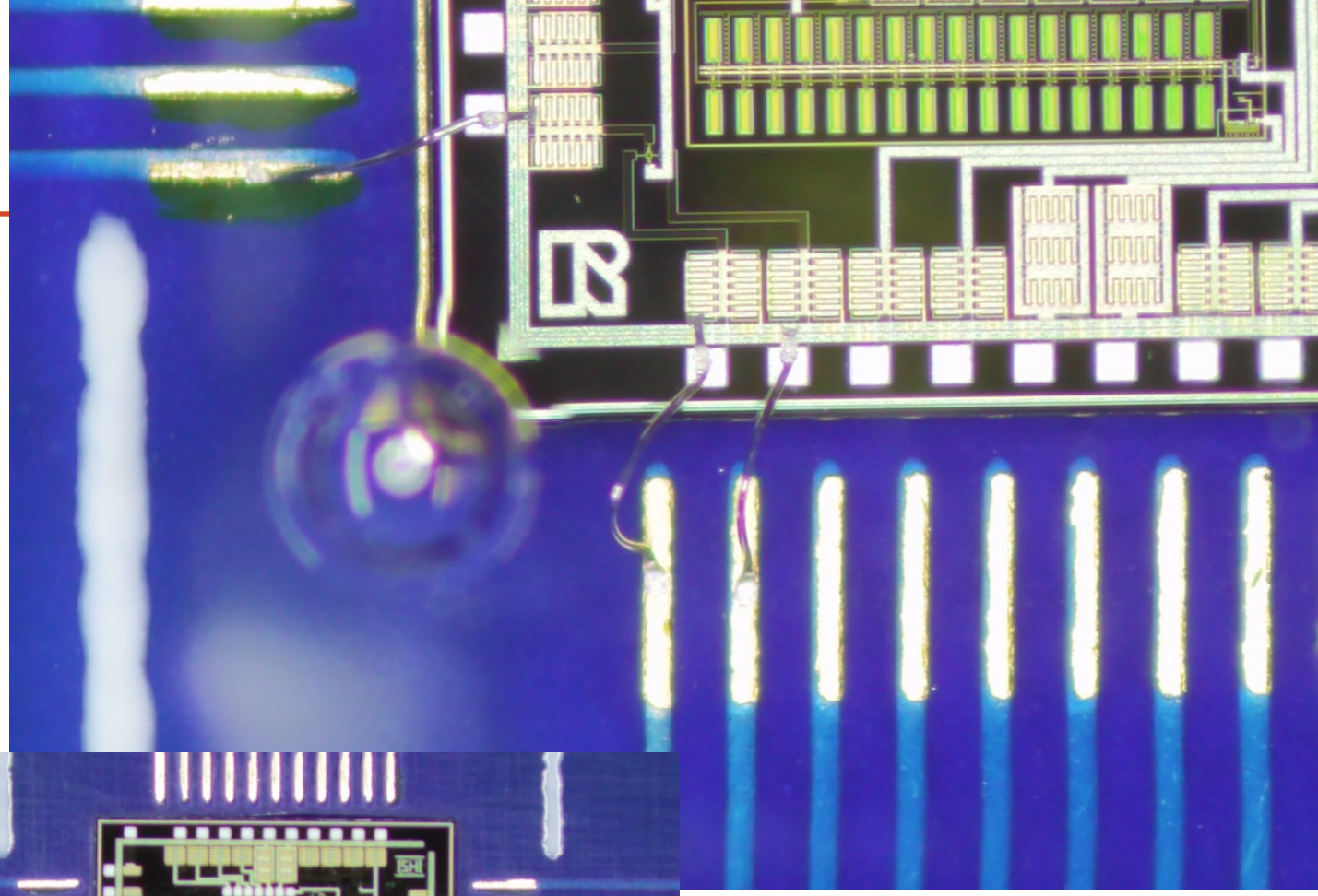


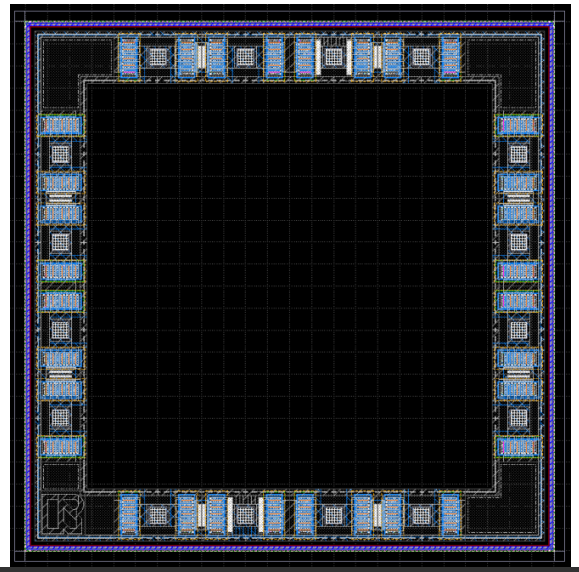
アナログ半導体の設計フロー

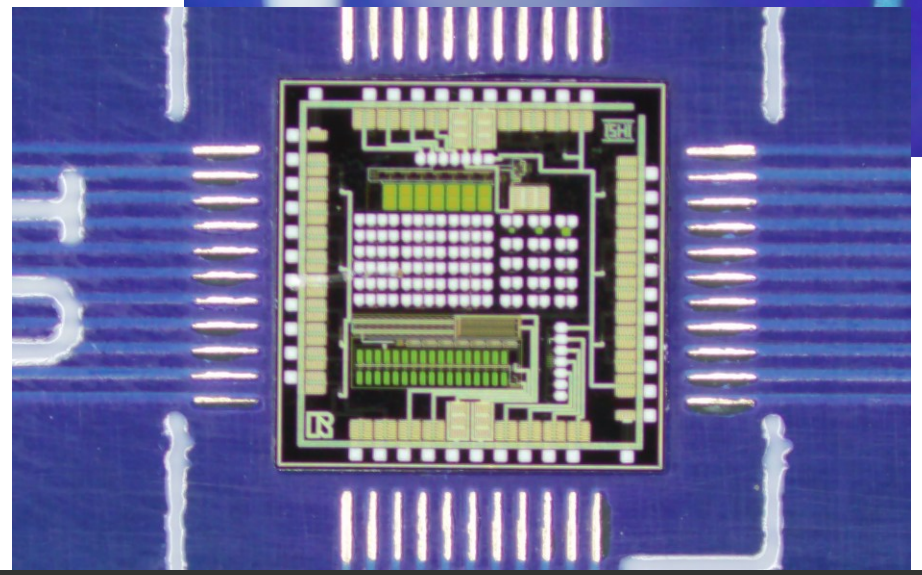
1. 回路図を描く
2. シミュレーションをする
3. 回路図を基にレイアウトを描く
4. レイアウトを検証する
5. レイアウトを基に寄生成分を考慮したシミュレーションをする
6. **フレームに載せる**

フレームとパッド

- パッド
  - フレーム上に配置される配線用の電極
  - ボンディングでパッケージ基板と接続する
- フレーム
  - カットされる単位







ウェハーとレチクル（マスク）とフレームの関係

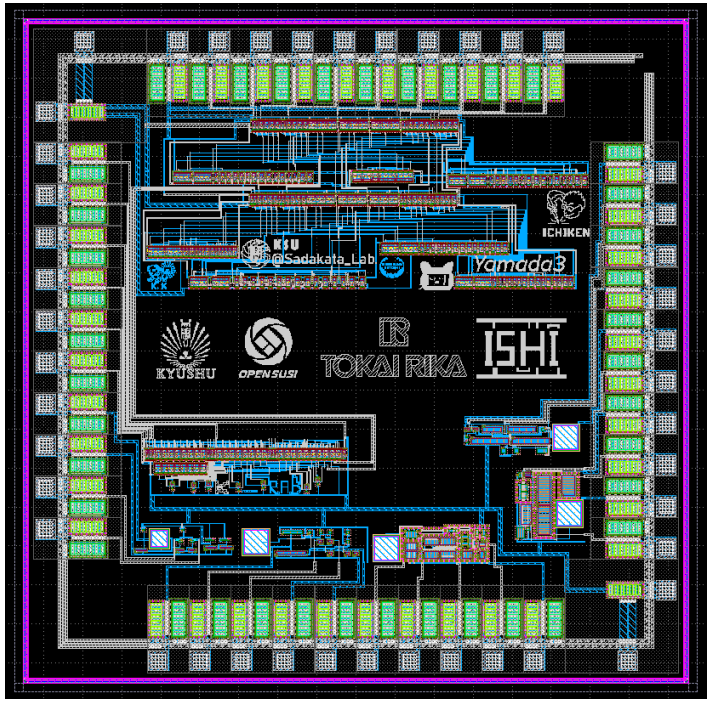
- ウェハー
  - 一番大きな単位
  - シリコンウェハーそのもの
- レチクル（マスク）
  - ウェハー上に転写する単位
  - この転写数が1ウェハーから取れるチップ数となる
  - 1チップとしてはこれ以上大きくできない最大単位となる
- フレーム
  - 1チップとなるカットされる単位
  - 今回は、1レチクルに8x8の64フレームが入っている
    - 複数の形状を1レチクル内に入れることも可能

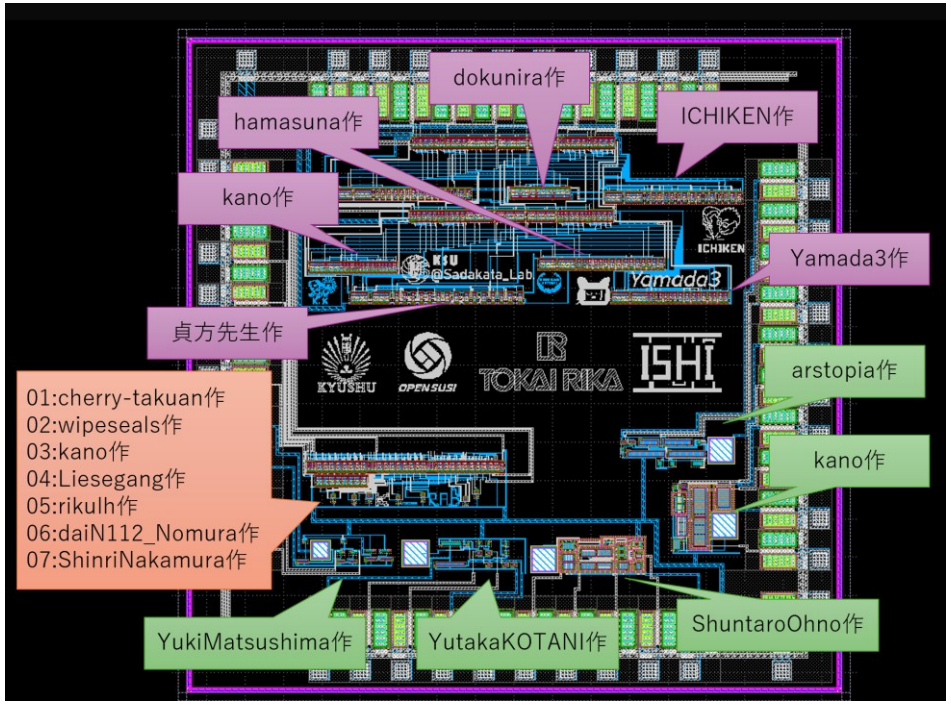
OpenSUSI MPW Users Layout

	0	1	2	3	4	5	6	7
0	**TEG** tr_1um_000_system_teg Run: - Tile: (0,0)	**jun1okamura** tr_1um_jun1okamura_tr-1u Run: 2389411632/5 Tile: (1,0)	**jun1okamura** tr_1um_jun1okamura_tk-1u Run: 2388382264/0 Tile: (2,0)	**EMPTY** tr_1um_000_system Run: - Tile: (3,0)	**EMPTY** tr_1um_000_system Run: - Tile: (4,0)	**EMPTY** tr_1um_000_system Run: - Tile: (5,0)	**EMPTY** tr_1um_000_system Run: - Tile: (6,0)	**EMPTY** tr_1um_000_system Run: - Tile: (7,0)
1	**EMPTY** tr_1um_000_system Run: - Tile: (0,1)	**EMPTY** tr_1um_000_system Run: - Tile: (1,1)	**EMPTY** tr_1um_000_system Run: - Tile: (2,1)	**EMPTY** tr_1um_000_system Run: - Tile: (3,1)	**EMPTY** tr_1um_000_system Run: - Tile: (4,1)	**EMPTY** tr_1um_000_system Run: - Tile: (5,1)	**EMPTY** tr_1um_000_system Run: - Tile: (6,1)	**EMPTY** tr_1um_000_system Run: - Tile: (7,1)
2	**EMPTY** tr_1um_000_system Run: - Tile: (0,2)	**EMPTY** tr_1um_000_system Run: - Tile: (1,2)	**EMPTY** tr_1um_000_system Run: - Tile: (2,2)	**EMPTY** tr_1um_000_system Run: - Tile: (3,2)	**EMPTY** tr_1um_000_system Run: - Tile: (4,2)	**EMPTY** tr_1um_000_system Run: - Tile: (5,2)	**EMPTY** tr_1um_000_system Run: - Tile: (6,2)	**EMPTY** tr_1um_000_system Run: - Tile: (7,2)
3	**EMPTY** tr_1um_000_system Run: - Tile: (0,3)	**EMPTY** tr_1um_000_system Run: - Tile: (1,3)	**EMPTY** tr_1um_000_system Run: - Tile: (2,3)	**EMPTY** tr_1um_000_system Run: - Tile: (3,3)	**EMPTY** tr_1um_000_system Run: - Tile: (4,3)	**EMPTY** tr_1um_000_system Run: - Tile: (5,3)	**EMPTY** tr_1um_000_system Run: - Tile: (6,3)	**EMPTY** tr_1um_000_system Run: - Tile: (7,3)
4	**EMPTY** tr_1um_000_system Run: - Tile: (0,4)	**EMPTY** tr_1um_000_system Run: - Tile: (1,4)	**EMPTY** tr_1um_000_system Run: - Tile: (2,4)	**EMPTY** tr_1um_000_system Run: - Tile: (3,4)	**EMPTY** tr_1um_000_system Run: - Tile: (4,4)	**EMPTY** tr_1um_000_system Run: - Tile: (5,4)	**EMPTY** tr_1um_000_system Run: - Tile: (6,4)	**EMPTY** tr_1um_000_system Run: - Tile: (7,4)
5	**EMPTY** tr_1um_000_system Run: - Tile: (0,5)	**EMPTY** tr_1um_000_system Run: - Tile: (1,5)	**EMPTY** tr_1um_000_system Run: - Tile: (2,5)	**EMPTY** tr_1um_000_system Run: - Tile: (3,5)	**EMPTY** tr_1um_000_system Run: - Tile: (4,5)	**EMPTY** tr_1um_000_system Run: - Tile: (5,5)	**EMPTY** tr_1um_000_system Run: - Tile: (6,5)	**EMPTY** tr_1um_000_system Run: - Tile: (7,5)
6	**EMPTY** tr_1um_000_system Run: - Tile: (0,6)	**EMPTY** tr_1um_000_system Run: - Tile: (1,6)	**EMPTY** tr_1um_000_system Run: - Tile: (2,6)	**EMPTY** tr_1um_000_system Run: - Tile: (3,6)	**EMPTY** tr_1um_000_system Run: - Tile: (4,6)	**EMPTY** tr_1um_000_system Run: - Tile: (5,6)	**EMPTY** tr_1um_000_system Run: - Tile: (6,6)	**EMPTY** tr_1um_000_system Run: - Tile: (7,6)
7	**EMPTY** tr_1um_000_system Run: - Tile: (0,7)	**EMPTY** tr_1um_000_system Run: - Tile: (1,7)	**EMPTY** tr_1um_000_system Run: - Tile: (2,7)	**EMPTY** tr_1um_000_system Run: - Tile: (3,7)	**EMPTY** tr_1um_000_system Run: - Tile: (4,7)	**EMPTY** tr_1um_000_system Run: - Tile: (5,7)	**EMPTY** tr_1um_000_system Run: - Tile: (6,7)	**EMPTY** tr_1um_000_system Run: - Tile: (7,7)

MPW：今回の形態

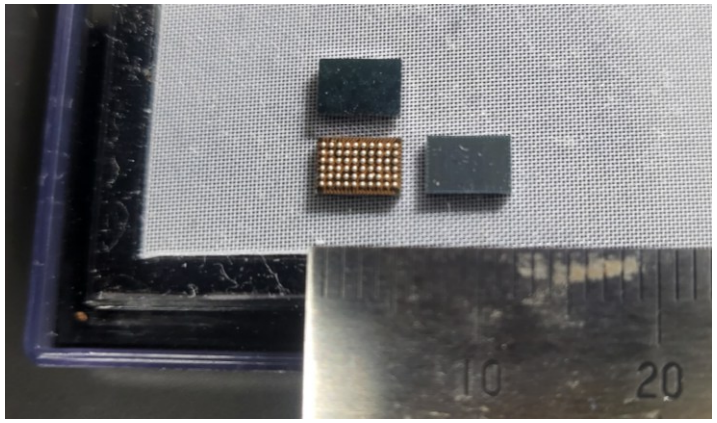
- MPW（Multi Person/Project Wafer）という概念もある
  - 1つのフレームを複数人やプロジェクトでシェアする形態
- メリット：さらに安く作れる
- デメリット：パッド（ピン）やエリア（面積）、チップをシェアすることになるため、取り合いになることがある

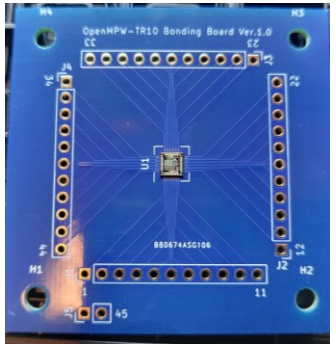


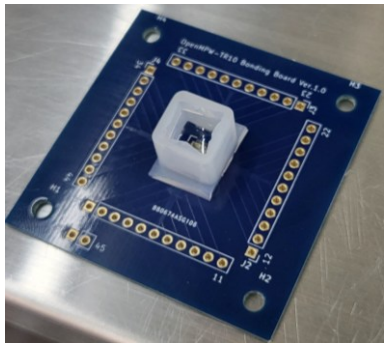


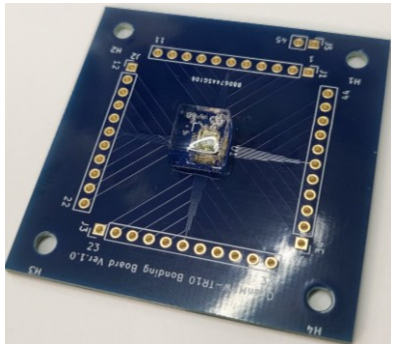
パッケージ

- パッケージの方法
  - チップ＋基板＋ワイヤリング＋封入で構成される
- パッケージの分類
  - QFN, BGAなど
    - 一般的な方式。側面や背面などにピン（足）を出す方式
  - Chiplet, SiP(System in Package)など
    - 複数のチップを1つのパッケージにまとめる方式。封入法がQFN, BGAなどと同じであることが多い。
  - WLP (Wafer Level Package)
    - ウェハーのまま配線し、封入する方式
  - CoB(Chip on Board)
    - PCB基板に直接パッケージする方式
- 基板：PCB基板やセラミック基板などを利用
- ワイヤリング：ワイヤボンディングやフリップチップを利用
- 封入：樹脂やセラミックを利用







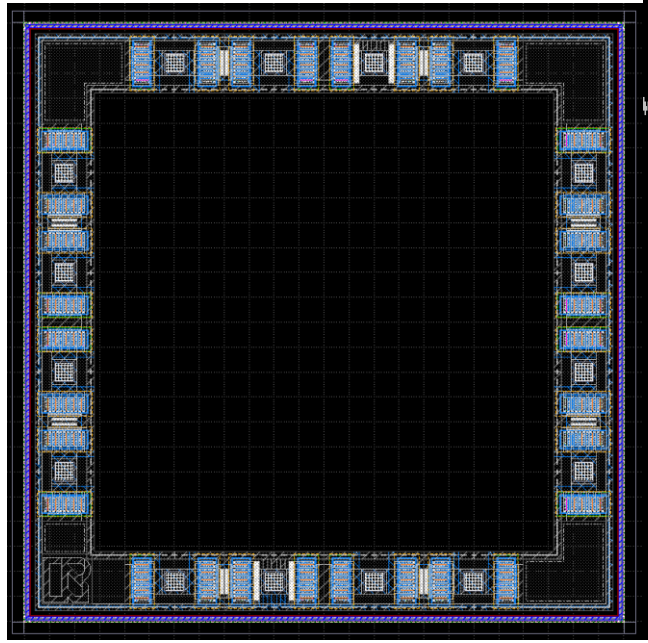


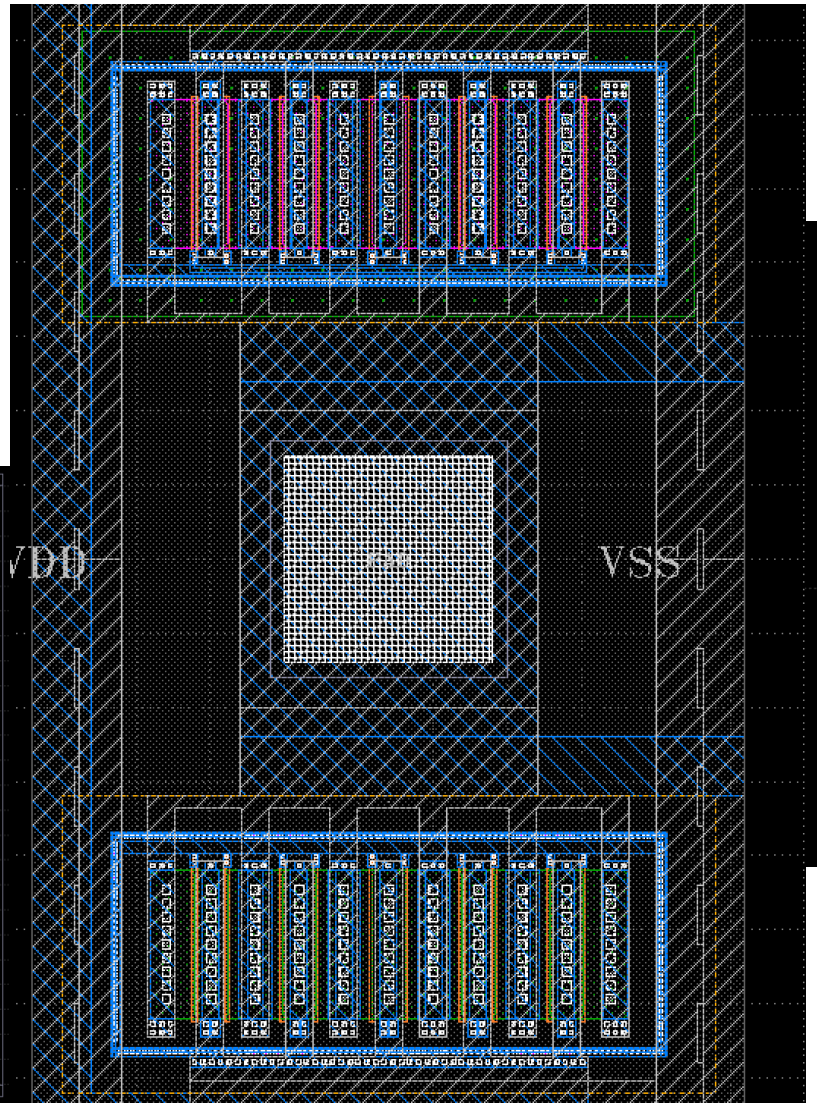
ファブの関係性：PCB基板との対比

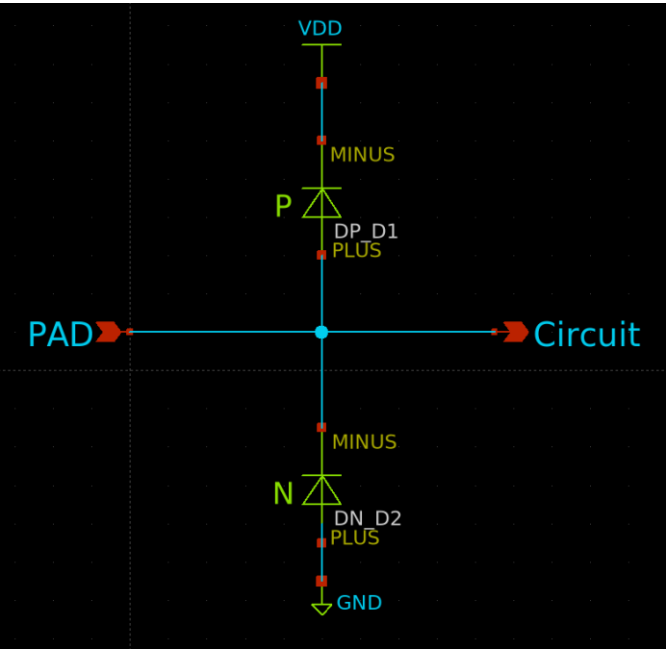
- EDAツール（設計ツール）
  - 回路図とレイアウト
    - KiCAD -> XSchem & Klayout
  - シミュレーター
    - LTSpice -> ngspice
- 製造会社：代理店（ファブからウェハーを購入する）
  - ユーザサポートを行う
  - レチクルの中身やフレームを提供する
    - P版.com -> Wafer.Space
- 製造会社：ファブ（前工程工場）
  - ウェハーやレチクルのサイズなどの製造仕様の提供する
  - テープアウト日を指定する（シャトルの場合）
    - PCB基板と違い、年に数回しか製造チャンスがない
  - 実際の製造を行う
    - PCB基板製造工場 -> GF
- 製造会社：アセンブリ（後工程工場）
  - パッケージングを行う
    - PCB基板の部品実装工場 -> ODT

パッドとESD

- パッドからゲートにつなげる場合は要注意
  - ゲート酸化膜の厚さ＝19.5nm
  - 酸化膜の耐圧：6～10MV/cm
    - 19.5Vで破壊される







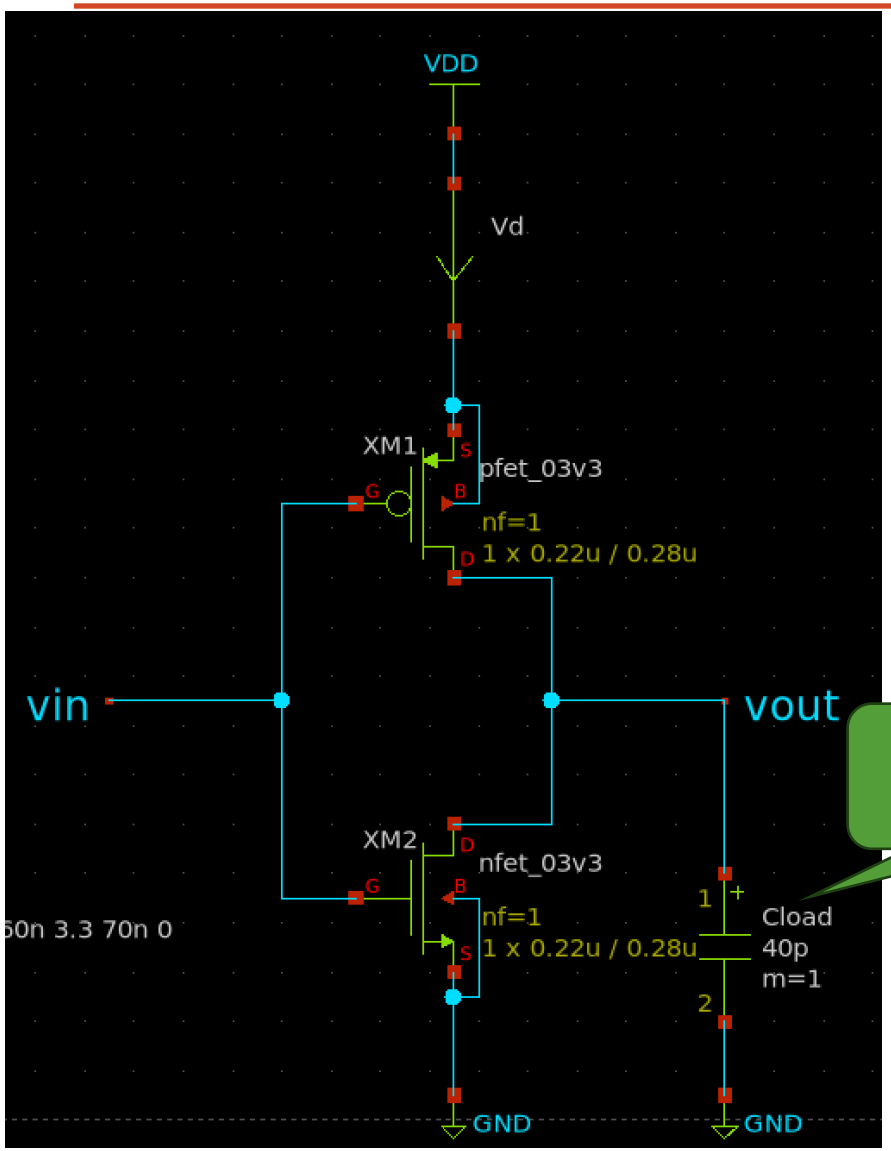
ドライブ能力

- **出力先がピンの外の場合は**IC内のロジックセル等の容量負荷よりも**膨大な容量負荷がかかる**
  - どれぐらいの負荷容量がかかるかはICの利用者に依存
  - 要因：ICパッケージ、（ソケット）、基板配線パターン、ケーブル　など

- ターゲットの出力周波数を、想定した負荷容量で駆動できるサイズで設計する必要がある
  - 今回は外界の負荷容量を 40 pF に設定してシミュレーションを行う

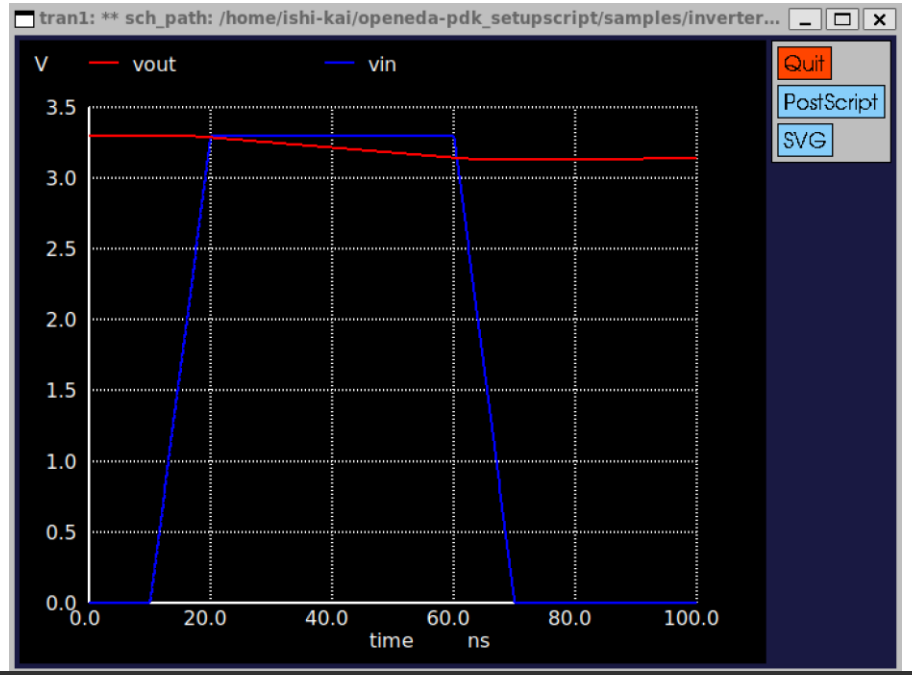
参考：一般的なIC + ソケット + 30cm ケーブル + パッシブプローブの容量を測った時は28pFでした

ngspice：過渡解析ベンチマークのCload=40pFにした例



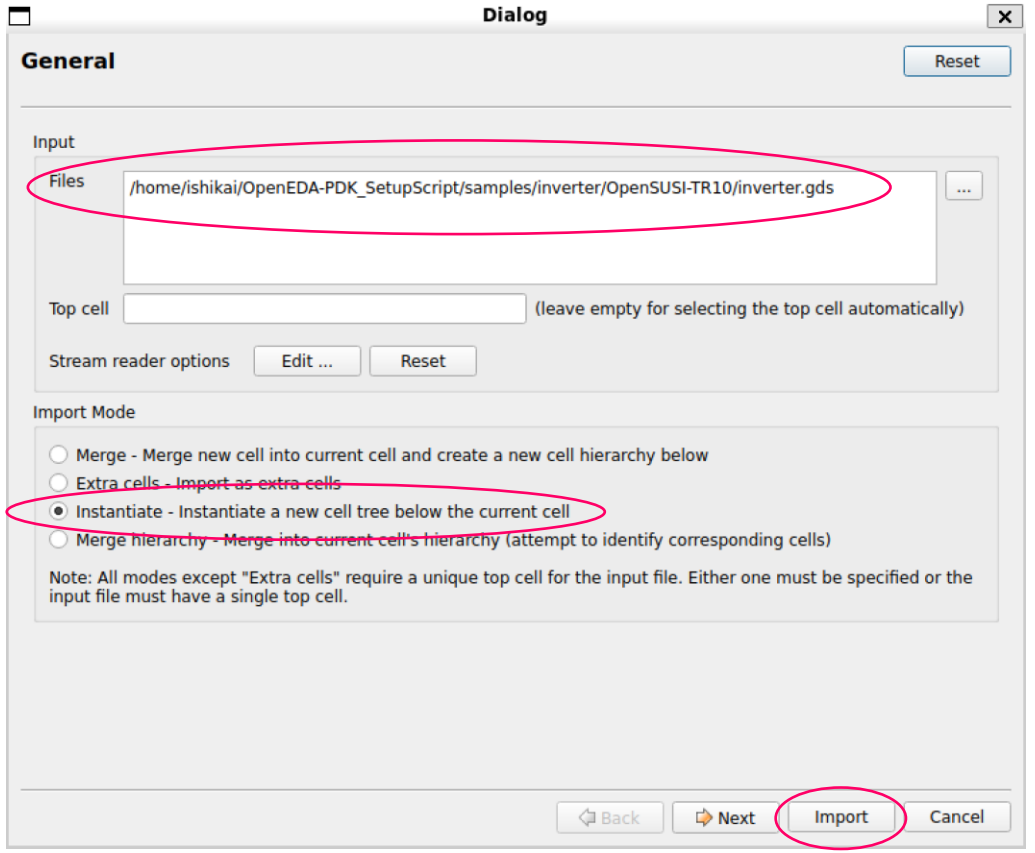
- PMOSとNMOSの性能を上げる
  - PMOSとNMOSのWを大きくする
  - PMOSとNMOSのLを小さくする
    - W/Lの比が同じ場合、PMOSとNMOSの性能バランスは保たれる

これを変える



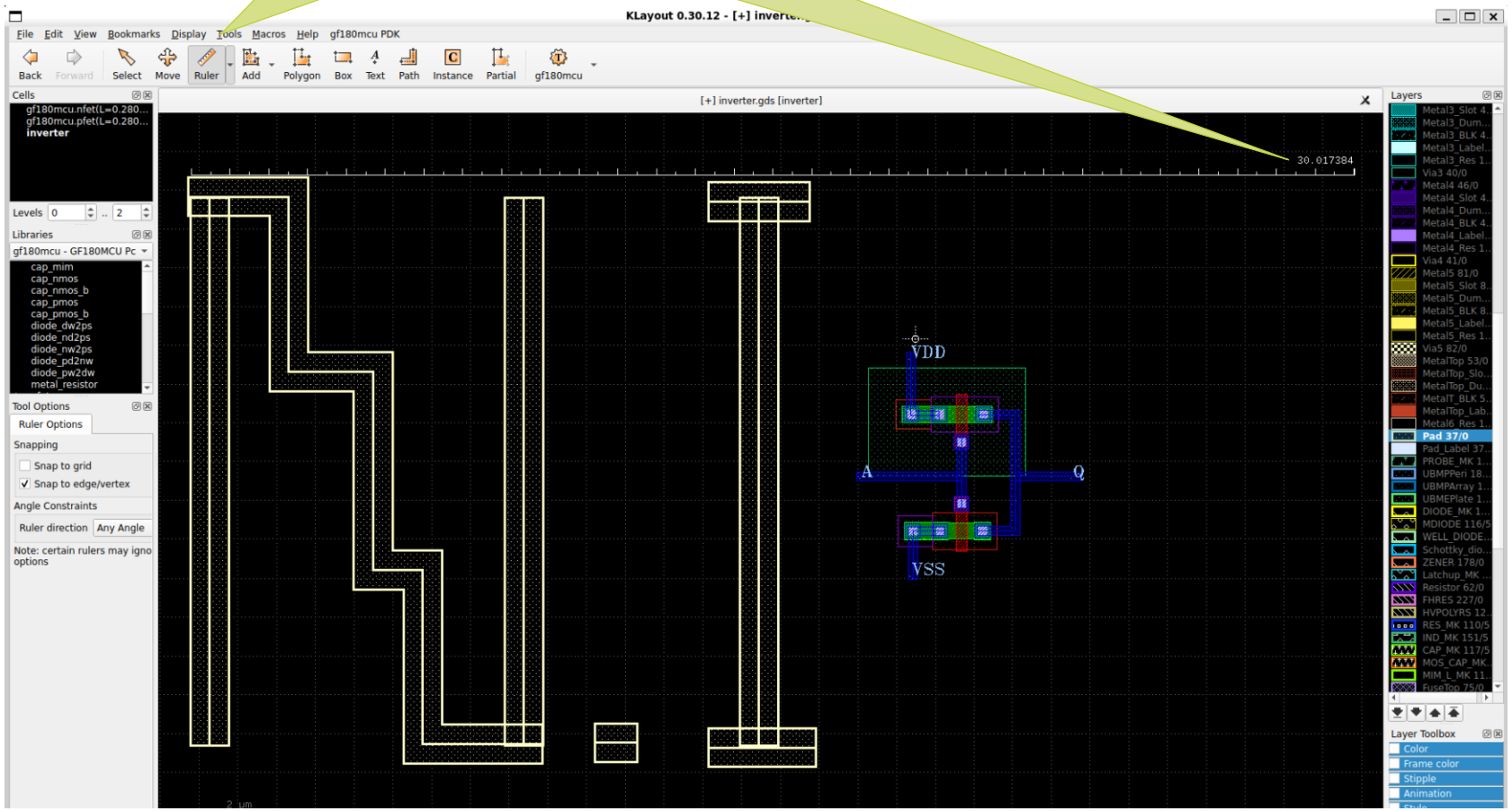
自分でフレームに載せる場合

1. フレームをKlayoutで読み込む
    1. ~/OpenEDA-PDK_SetupScript/GDS/GF180/top_frame.gds
2. インバータ回路のGDSファイルをインポートする
    1. File -> Import -> Other Files Into Current
    2. 右図に合わせて自作のインバータ回路のGDSファイルを読み込む
3. 指定の場所にインバータ回路のレイアウトを配置する
    1. こちらで指定します
4. 指定のパッドにVDD,VSS,OUTピンを接続する
    1. こちらで指定します



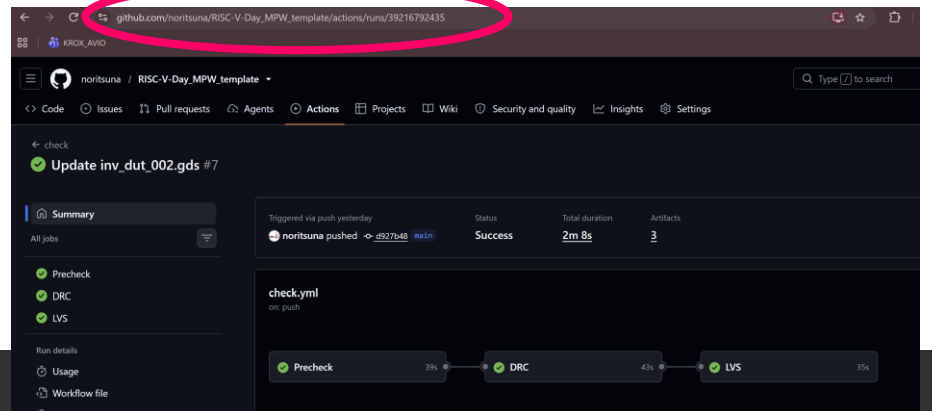
シリコンアート

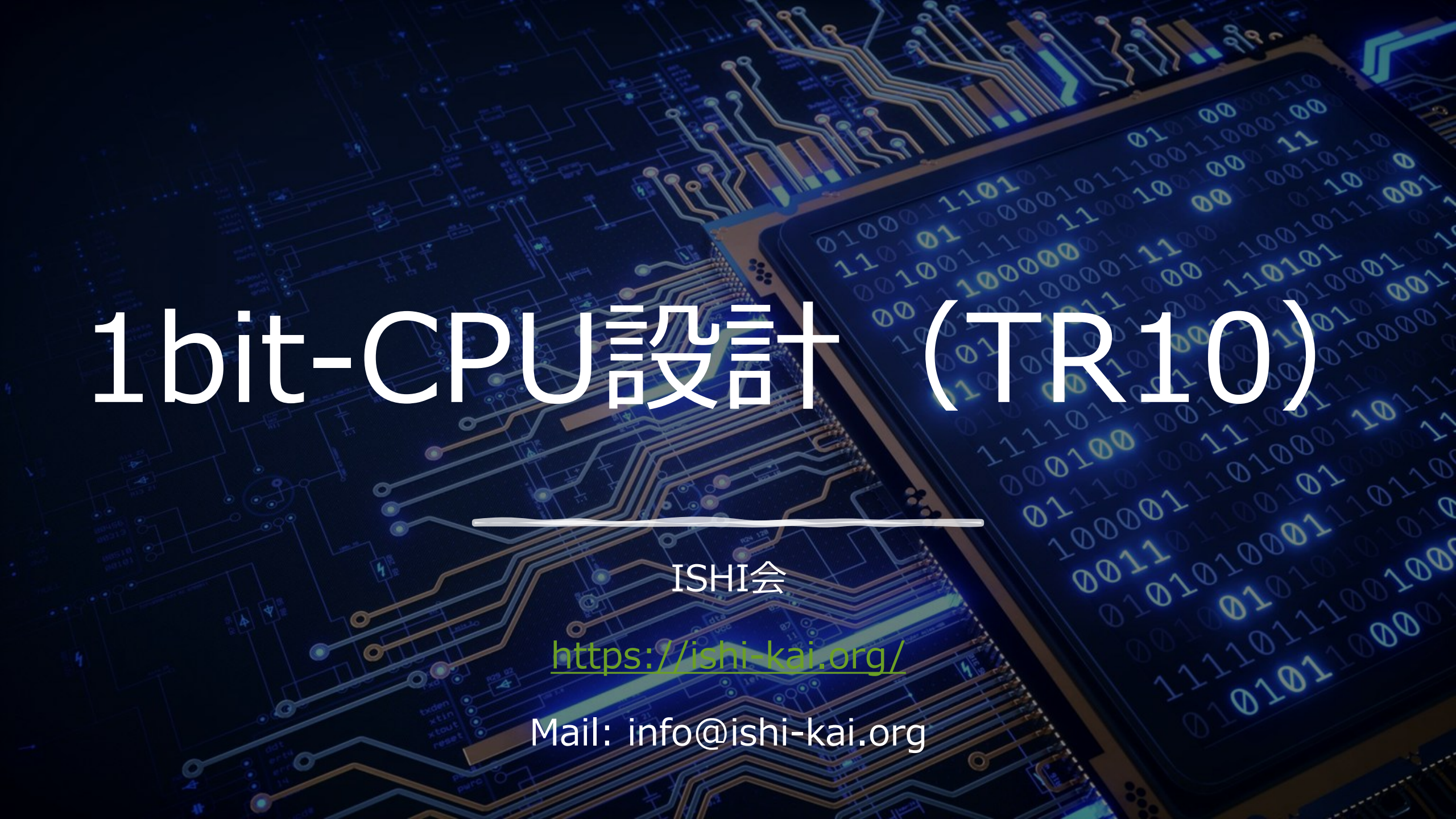
- サイズ制限
  - 指定エリア内
- 利用可能レイヤー
  - Metal1～Metal5
  - Pad
- 備考
  - DRCチェックをしてください
  - 状況により削除する可能性ありますRulerで長さチェック可能



提出方法

1. https://github.com/ishi-kai/RISC-V-Day_MPW_template.git を自分のgithubへフォークする
2. 上記リポジトリの「src」に作成した「inv_dut_[配布されたDUT番号].gds」をコミットする
3. Github Actionsがオールグリーンになっていることを確認する
    1. エラーがある場合は、GDSファイルを修正してください。
4. Github ActionsのURLを「登録先サイトのURL」にアクセスして、登録する
5. 1時間後に、https://github.com/ishi-kai/RISC-V-Day_2611_inverter_workshop.git に登録されていることを確認する
    1. reportディレクトリに「[配布されたDUT番号]_[作成した日時].md」という名前でレポートがあるので確認すること
    2. エラーの場合は、修正して再提出すること



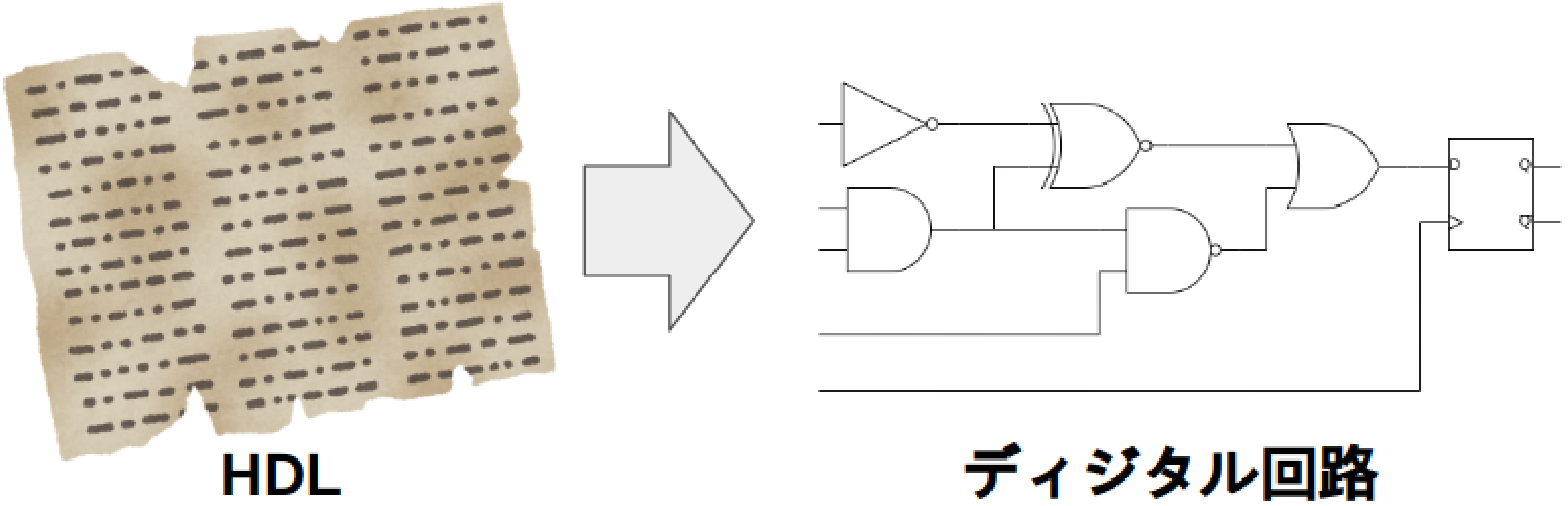


1bit-CPU設計（TR10）

ISHI会

https://ishi-kai.org/

Mail: info@ishi-kai.org



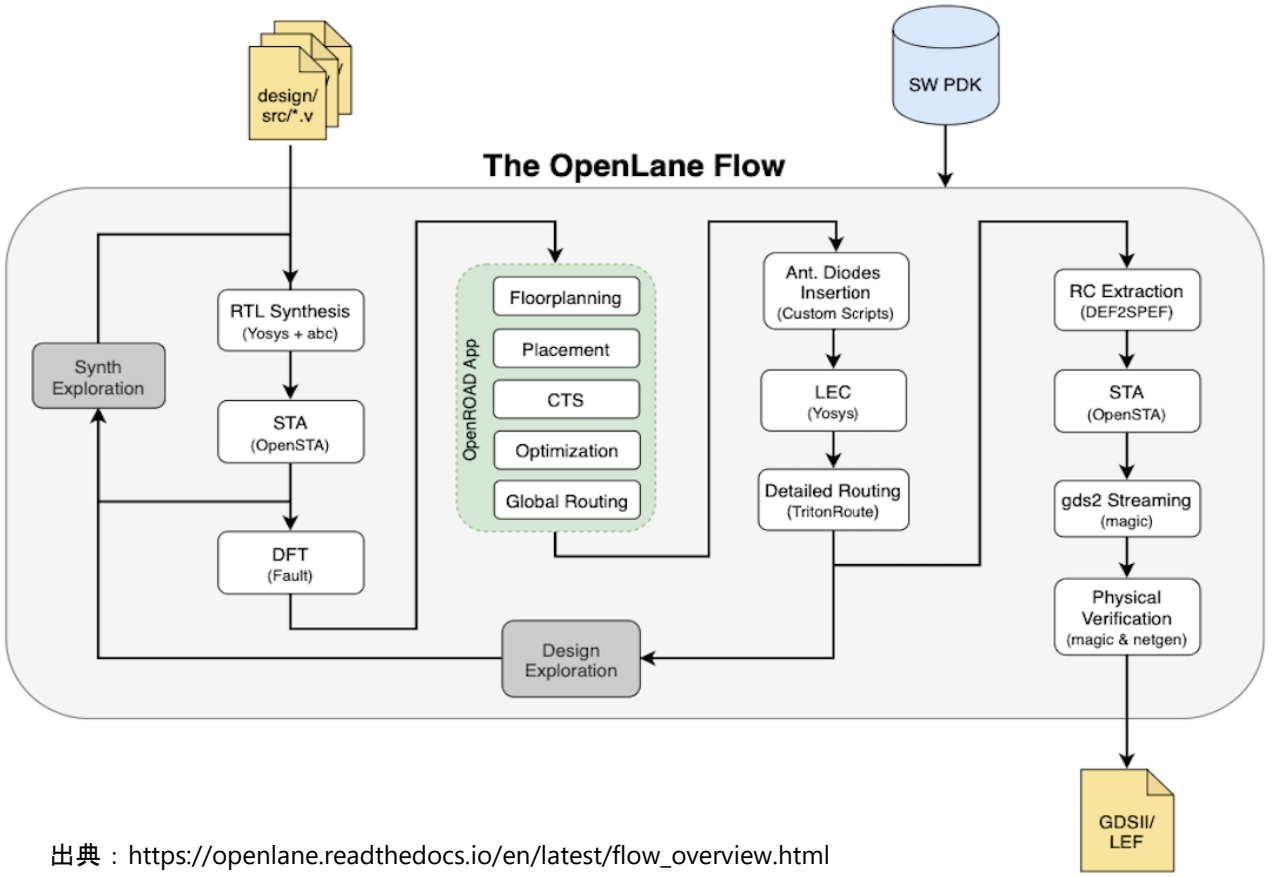
論理合成のイメージ

論理合成

HDL（ハードウェア記述言語）で

記述して、論理回路（RTL）にする

OpenLANEによる生成フロー

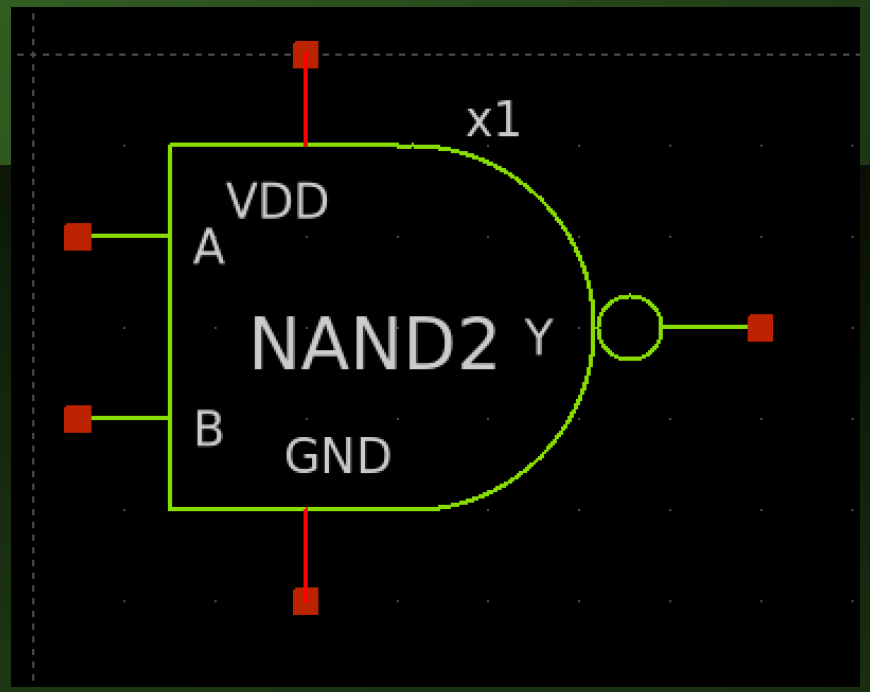


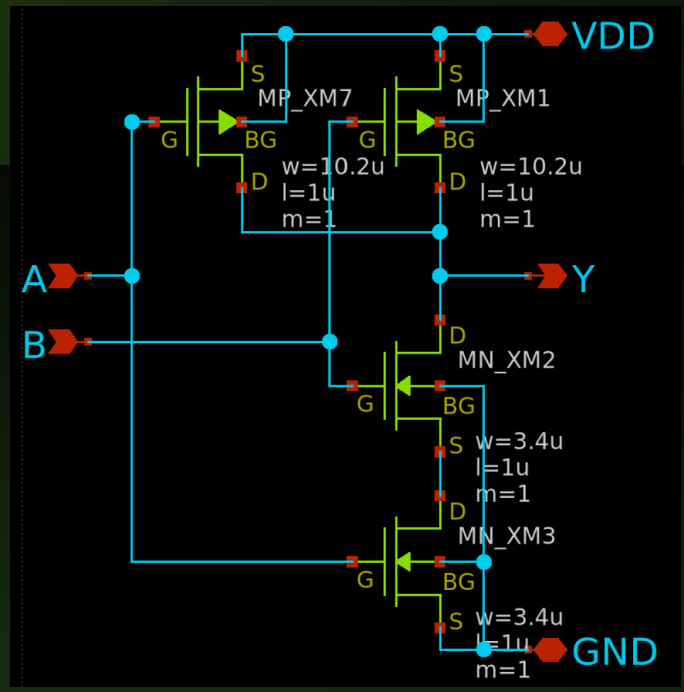
出典：https://openlane.readthedocs.io/en/latest/flow_overview.html

生成フロー詳細

合成		フロアプランと電源供給		配置		クロックツリー合成と配線		GDS生成とチェック	
ツール名	機能内容	ツール名	機能内容	ツール名	機能内容	ツール名	機能内容	ツール名	機能内容
yosys	RTLを論理合成	init_fp	コア領域の定義	RePLace	グローバル配置	TritonCTS	クロックツリーの合成	Magic	GDSIIファイル生成
abc	PDKマッピング	ioplacer	入出力ポートの設置	Resizer	最適化	FastRouteとCU-GR	グローバル配線	Magic	DRC（Design Rule Check）とアンテナチェック
OpenSTA	静的タイミング解析	pdn	給電ネットワークの生成	OpenDP	ローカル配置	TritonRoute	ローカル配線	Netgen	LVS（Layout vs Schematic）チェック
		tapcell	タップとデキャップセルの挿入			SPEF_Extractor	寄生フォーマットの摘出	CVC	回路妥当性検証

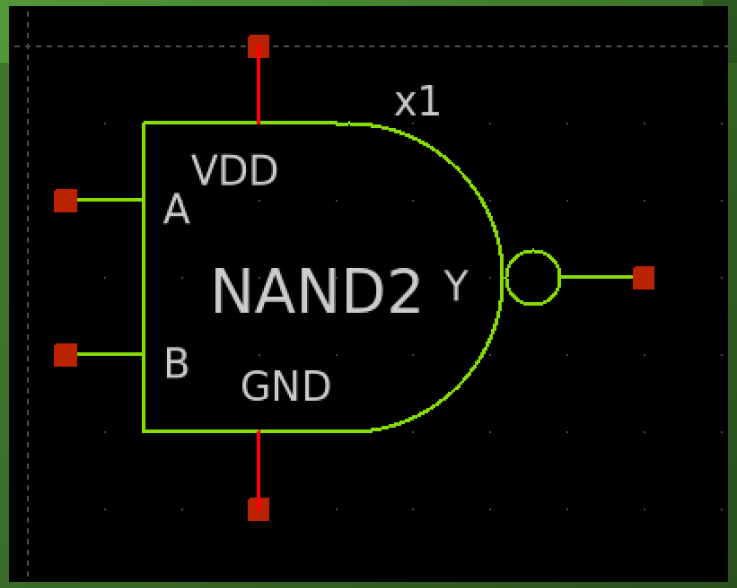
論理回路の正体

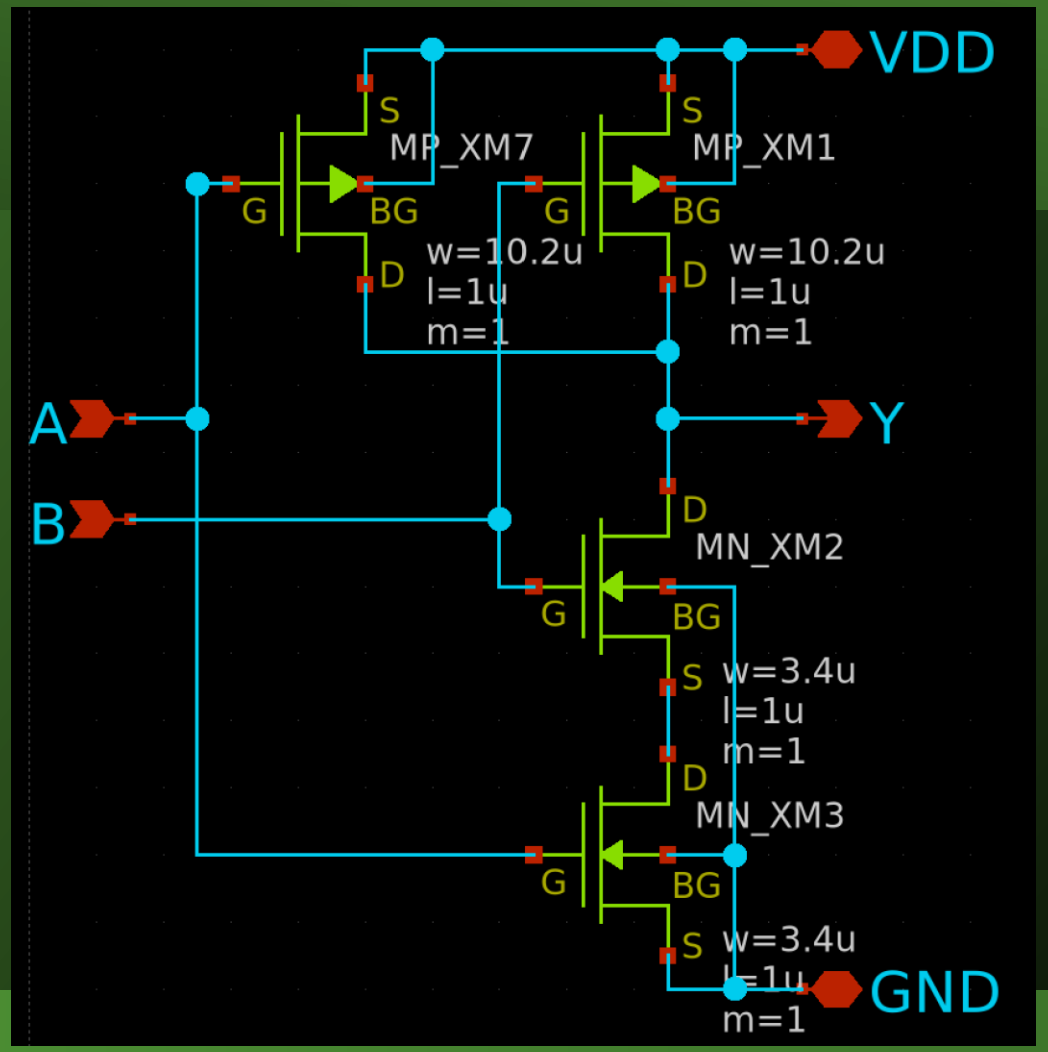


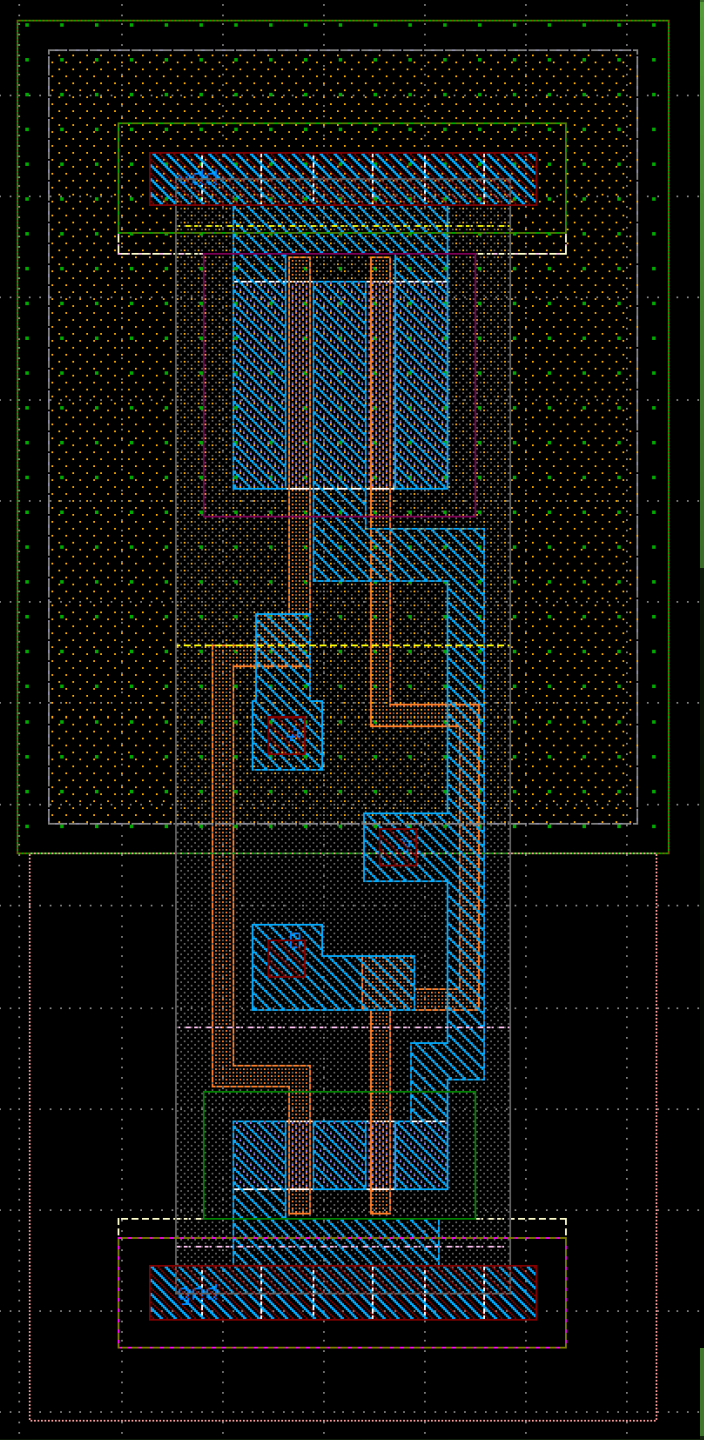


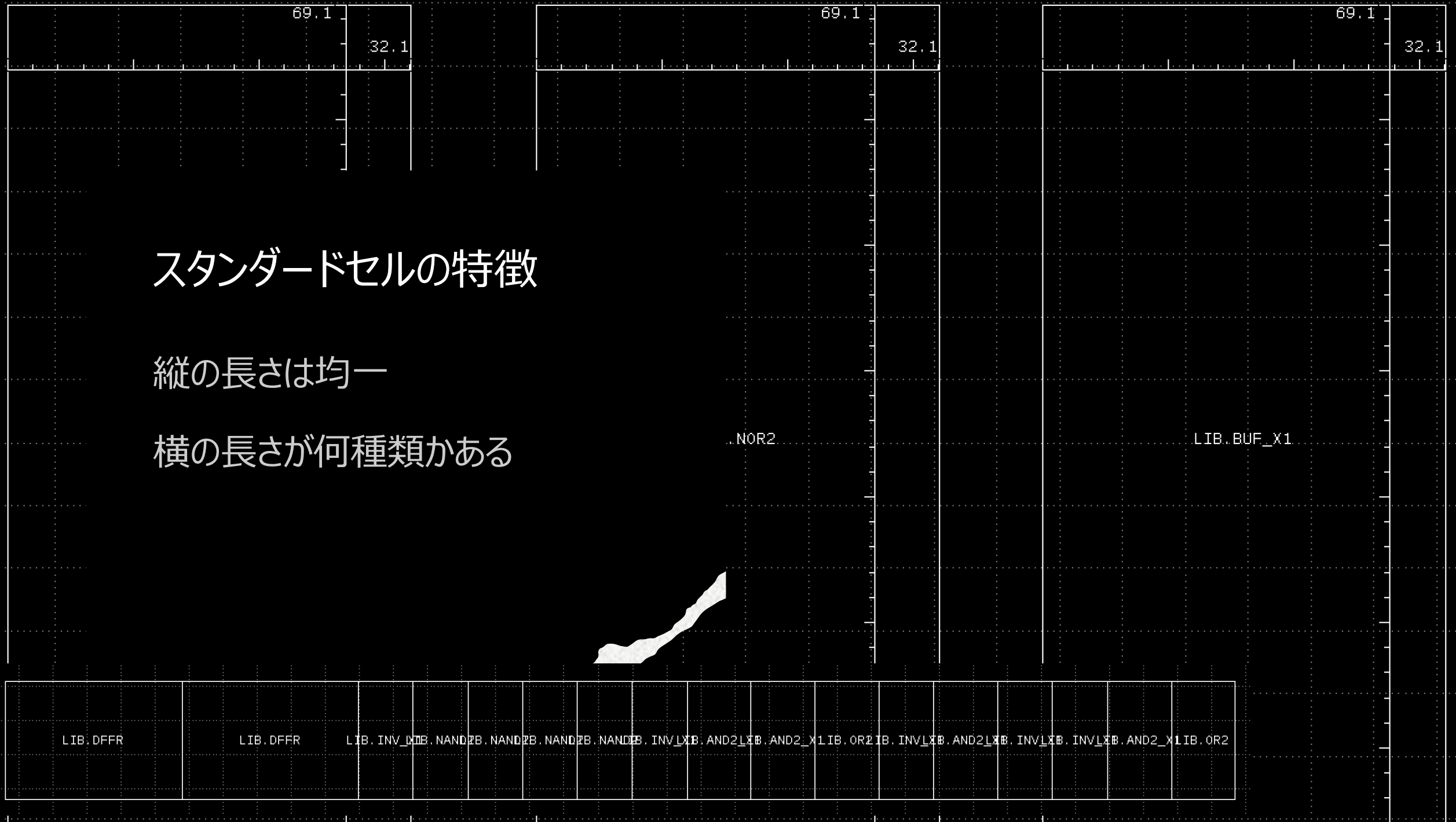
中身はPMOSとNMOS

スタンダードセル


x1
VDD
A
NAND2
Y
B
GND


VDD
S
MP_XM7
G
BG
D
w=10.2u
l=1u
m=1
S
MP_XM1
G
BG
D
w=10.2u
l=1u
m=1
A
B
Y
D
MN_XM2
G
BG
S
w=3.4u
l=1u
m=1
D
MN_XM3
G
BG
S
w=3.4u
l=1u
m=1
GND





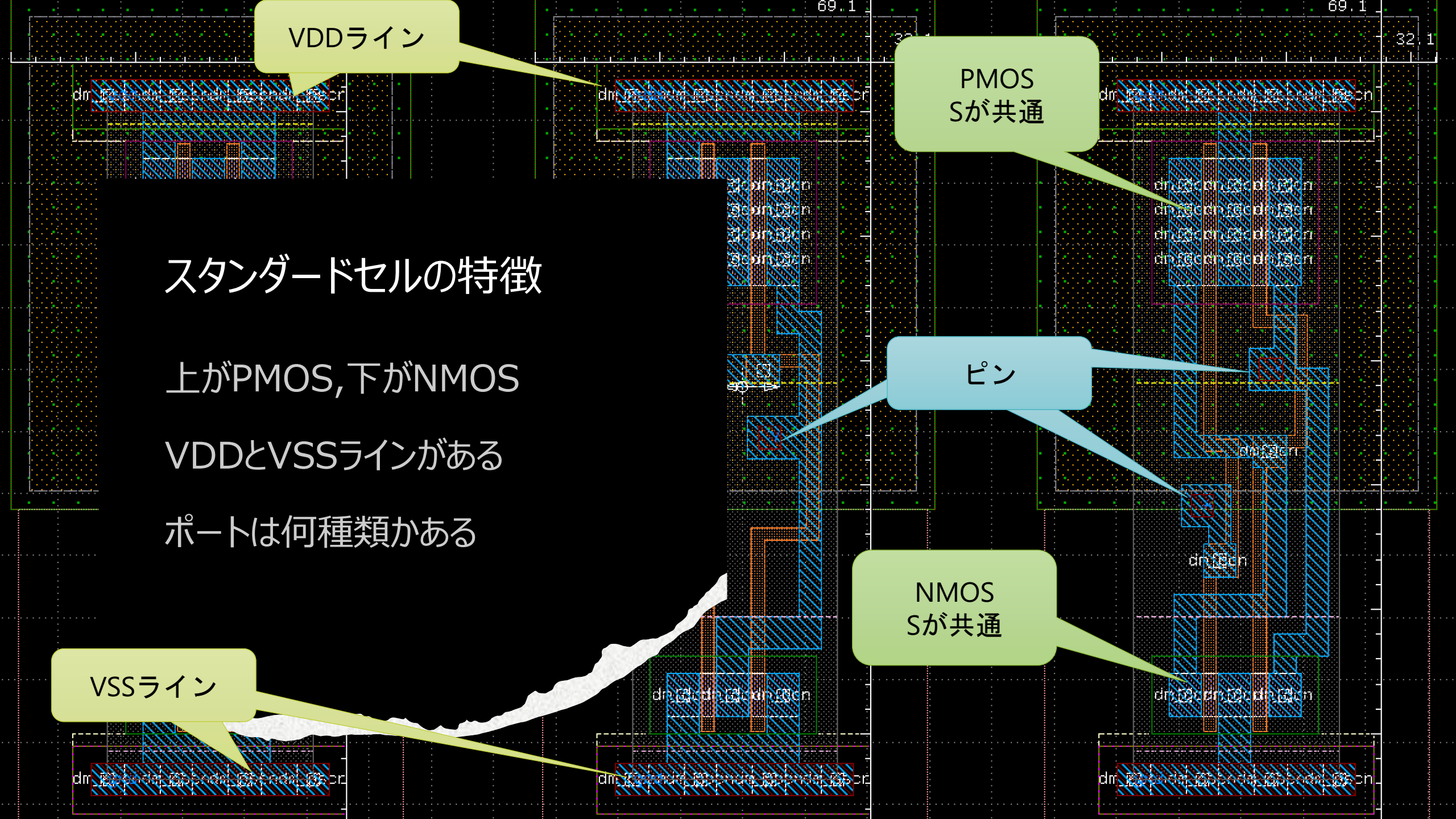
スタンダードセルの特徴

縦の長さは均一

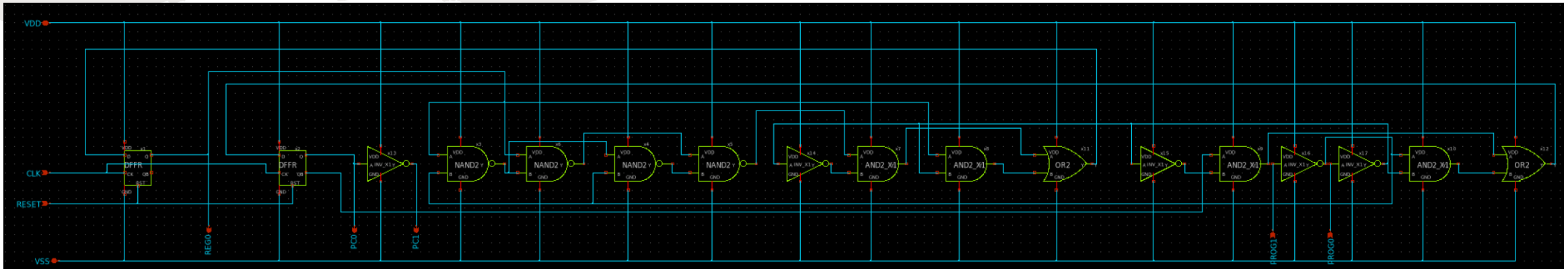
横の長さが何種類かある

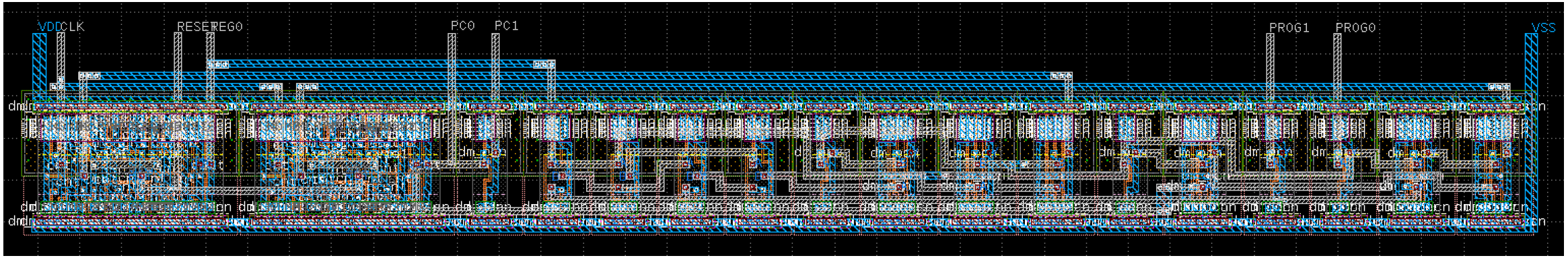
Library: IP62_5_stdcell

No	Cell	種類	入力ピン数	出力ピン数	セル高さ[um]	セル幅[um]	MP W[um]	MP L[um]	MN W[um]	MN L[um]
1	AND2_X1	ANDゲート	2	1	55.0	22.0	10.2	1	3.4	1
2	AND3_X1	ANDゲート	3	1	55.0	27.5	10.2	1	3.4	1
3	AND4_X1	ANDゲート	4	1	55.0	33.0	10.2	1	3.4	1
4	BUF_X1	バッファ	1	1	55.0	16.5	10.2	1	3.4	1
5	BUF_X2	バッファ	1	1	55.0	22.0	10.2 x 2	1	3.4 x 2	1
6	BUF_X4	バッファ	1	1	55.0	33.0	10.2 x 4	1	3.4 x 4	1
7	BUF_X8	バッファ	1	1	55.0	49.5	10.2 x 8	1	3.4 x 8	1
8	BUF_X12	バッファ	1	1	55.0	66.0	10.2 x 12	1	3.4 x 12	1
9	BUF_X16	バッファ	1	1	55.0	93.5	10.2 x 16	1	3.4 x 16	1
10	CLKBUF_X1	クロックバッファ	1	1	55.0	38.5	10.2 x 5	1	3.4 x 5	1
11	CLKBUF_X2	クロックバッファ	1	1	55.0	66.0	10.2 x 10	1	3.4 x 10	1
12	CLKBUF_X4	クロックバッファ	1	1	55.0	115.5	10.2 x 20	1	3.4 x 20	1
13	CLKBUF_X8	クロックバッファ	1	1	55.0	203.5	10.2 x 40	1	3.4 x 40	1
14	CLKBUF_X12	クロックバッファ	1	1	55.0	291.5	10.2 x 60	1	3.4 x 60	1
15	CLKBUF_X16	クロックバッファ	1	1	55.0	379.5	10.2 x 80	1	3.4 x 80	1
16	DEL1	ディレイセル	1	1	55.0	38.5	10.2 x 2	1	3.4 x 2	1
17	DEL2	ディレイセル	1	1	55.0	60.5	10.2	1	3.4	1
18	DEL4	ディレイセル	1	1	55.0	104.5	10.2	1	3.4	1
19	DFFR	Dフリップフロップ	3	2	55.0	88.0	10.2	1	3.4	1
20	DFFS	Dフリップフロップ	3	2	55.0	88.0	10.2	1	3.4	1
21	INV_X1	インバータ	1	1	55.0	16.5	10.2	1	3.4	1
22	INV_X2	インバータ	1	1	55.0	16.5	10.2 x 2	1	3.4 x 2	1
23	INV_X4	インバータ	1	1	55.0	27.5	10.2 x 4	1	3.4 x 4	1
24	INV_X8	インバータ	1	1	55.0	44.0	10.2 x 8	1	3.4 x 8	1
25	INV_X12	インバータ	1	1	55.0	60.5	10.2 x 12	1	3.4 x 12	1
26	INV_X16	インバータ	1	1	55.0	77.0	10.2 x 16	1	3.4 x 16	1
27	MUX2	マルチプレクサ	3	1	55.0	49.5	10.2	1	3.4	1
28	NAND2	NANDゲート	2	1	55.0	16.5	10.2	1	3.4	1
29	NAND3	NANDゲート	3	1	55.0	22.0	10.2	1	3.4	1
30	NAND4	NANDゲート	4	1	55.0	27.5	10.2	1	3.4	1
31	NOR2	NORゲート	2	1	55.0	16.5	10.2	1	3.4	1
32	NOR3	NORゲート	3	1	55.0	22.0	10.2	1	3.4	1
33	NOR4	NORゲート	4	1	55.0	27.5	10.2	1	3.4	1
34	OR2	ORゲート	2	1	55.0	22.0	10.2	1	3.4	1
35	OR3	ORゲート	3	1	55.0	27.5	10.2	1	3.4	1
36	OR4	ORゲート	4	1	55.0	33.0	10.2	1	3.4	1
37	XNOR2	XNORゲート	2	1	55.0	33.0	10.2	1	3.4	1
38	XOR2	XORゲート	2	1	55.0	33.0	10.2	1	3.4	1


VDDライン
PMOS
Sが共通
スタンダードセルの特徴
上がPMOS,下がNMOS
VDDとVSSラインがある
ポートは何種類かある
ピン
NMOS
Sが共通
VSSライン

回路をレイアウトにする工程



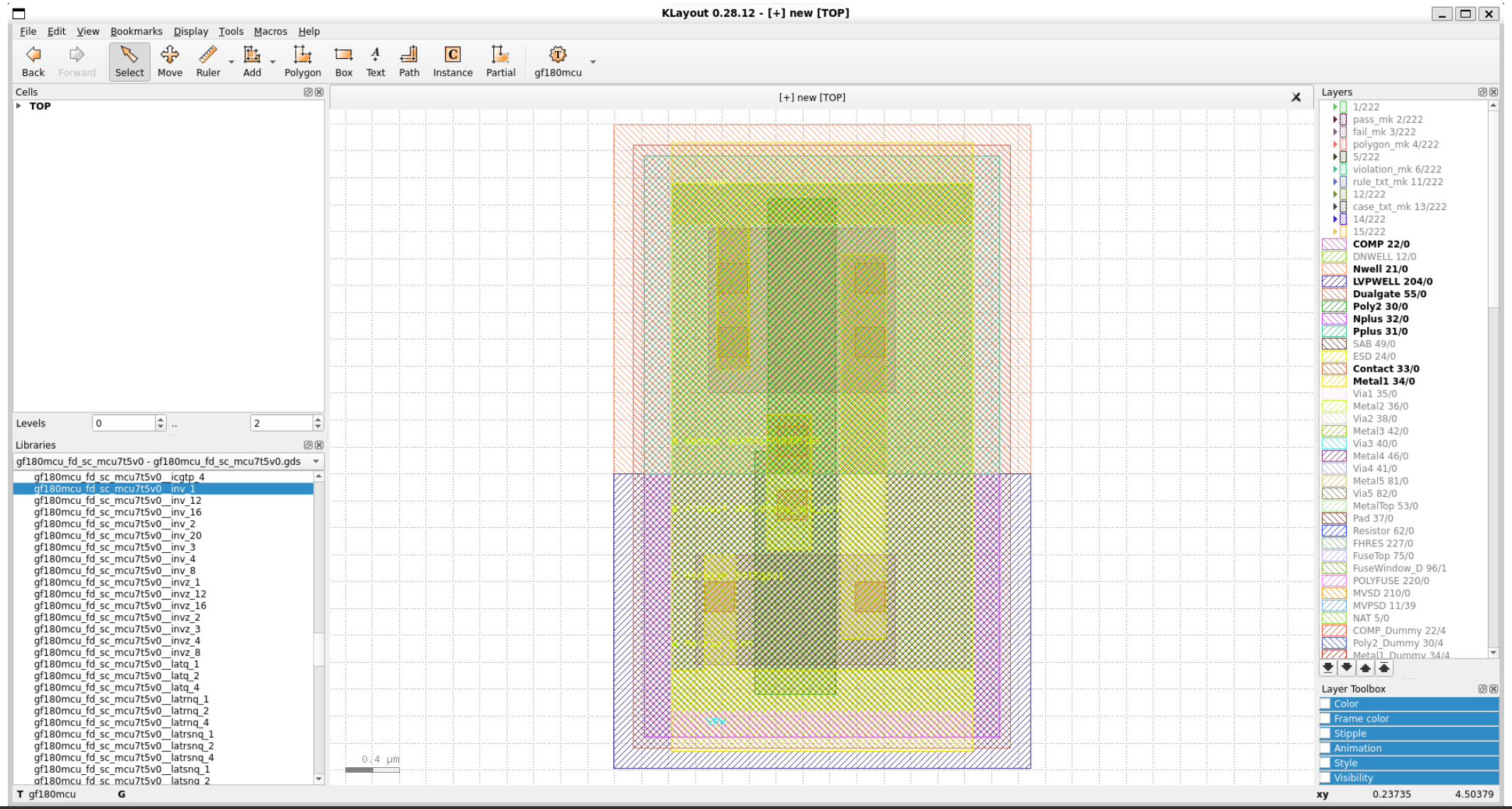


アナログLSIの設計フロー

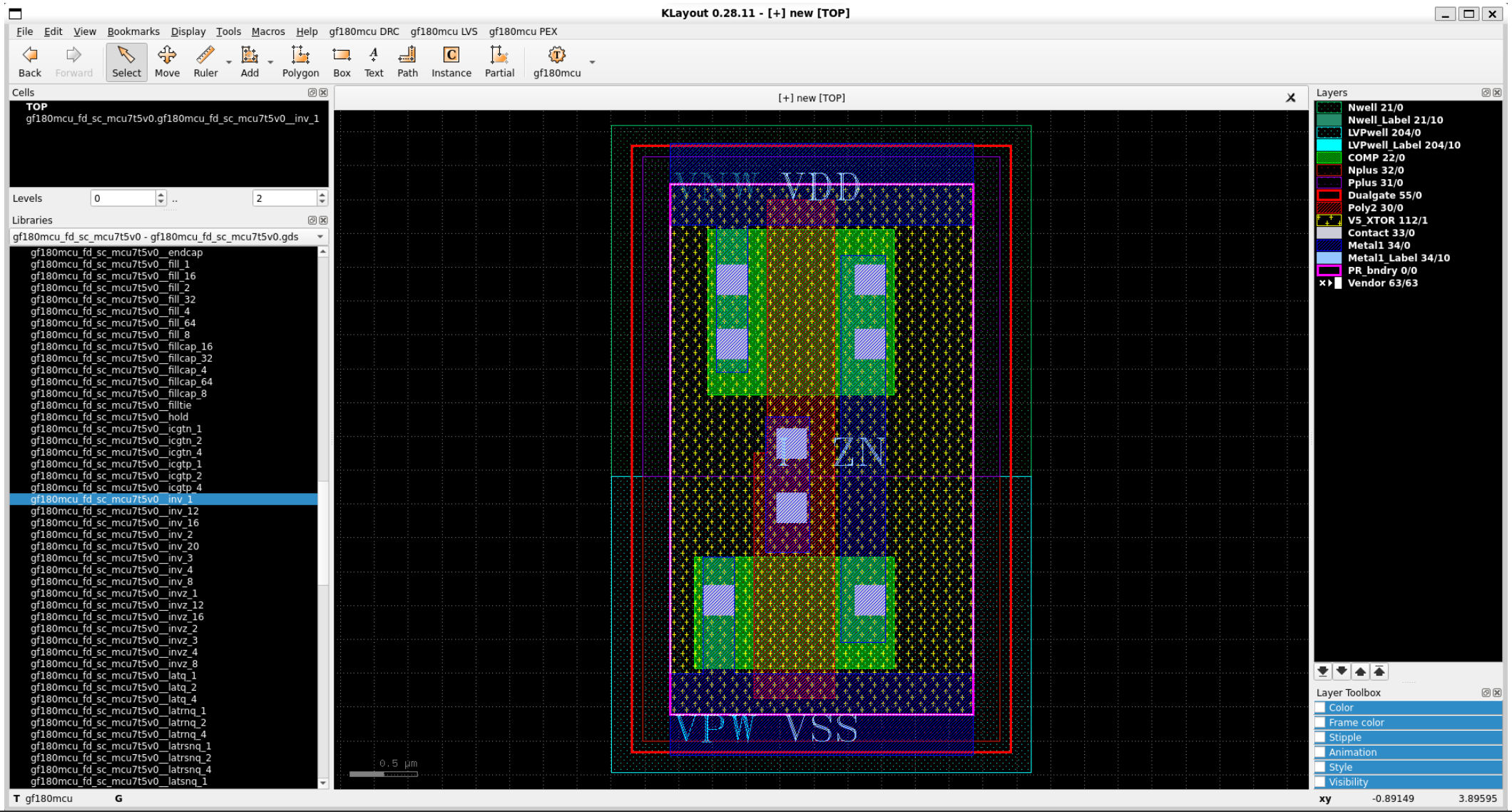
1. 回路図（ベンチマーク）を描く
2. シミュレーションをする
3. **回路図を基にレイアウトを描く**
4. レイアウトを検証する
5. レイアウトを基に寄生成分を考慮したシミュレーションをする

6. （フレームに載せる）

公式PDKのレイアウト画面

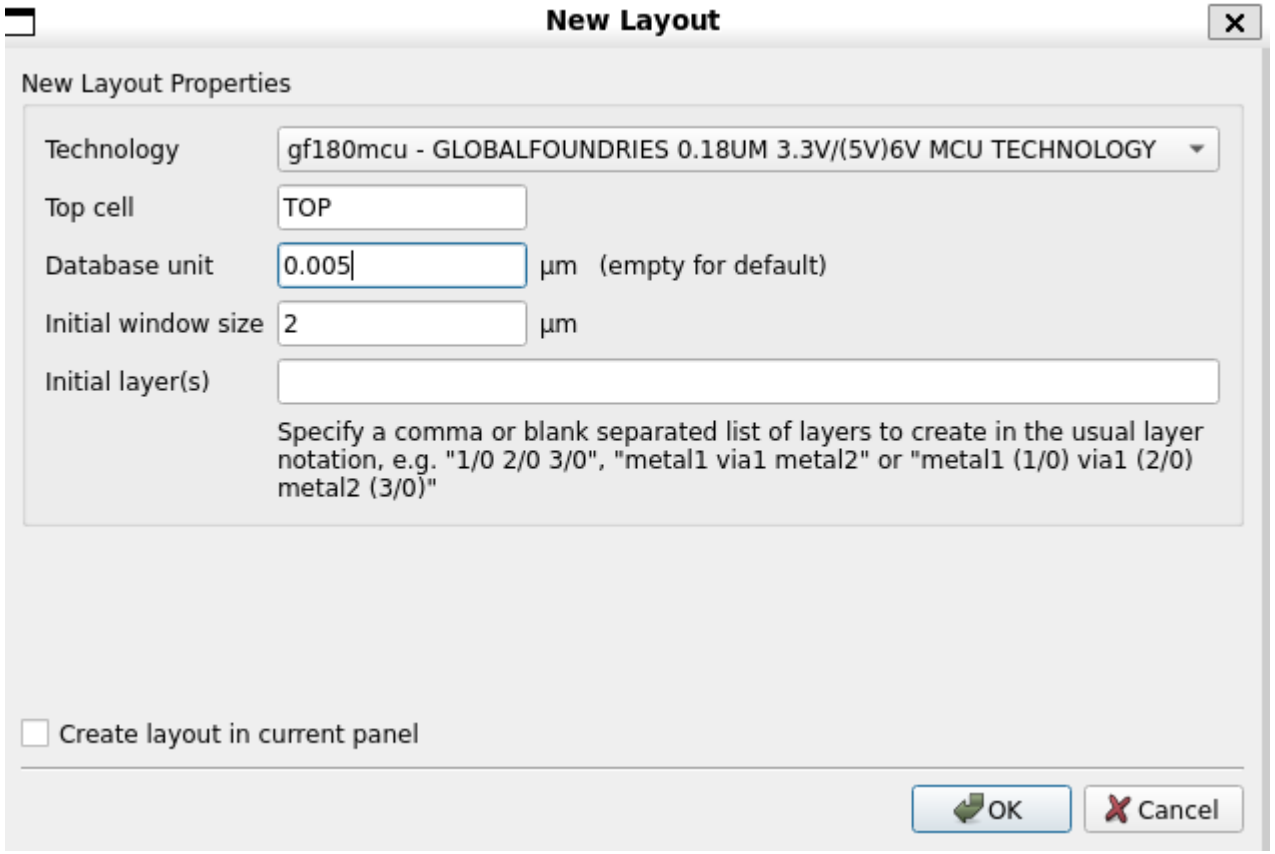


3zki / wsl_gf180mcu のレイアウト画面



レイアウトを描く上での注意

- **Database unit を 0.005にする**



デザインルールの場所

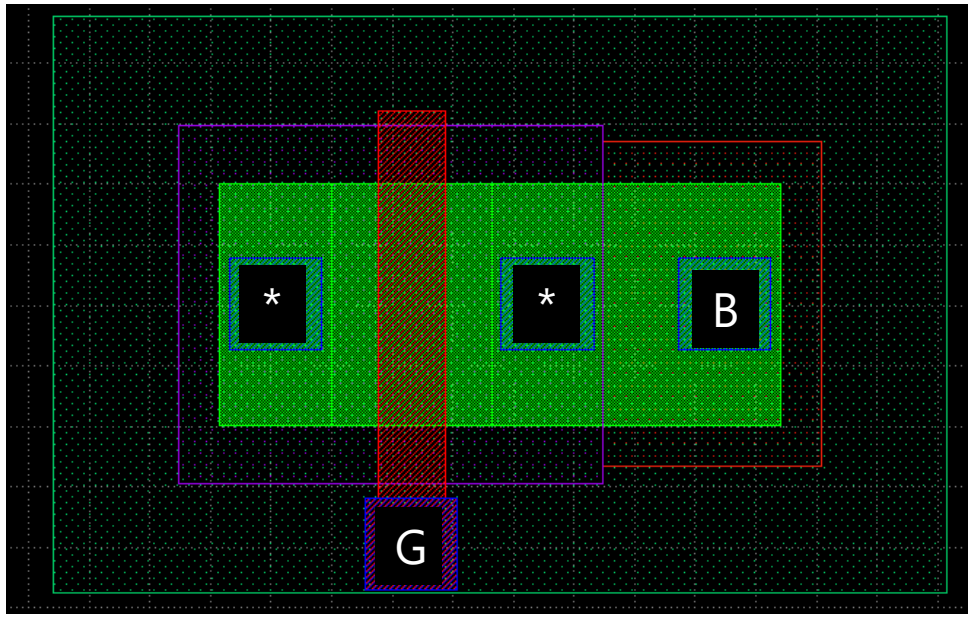
- 必ずデザインルールを守る

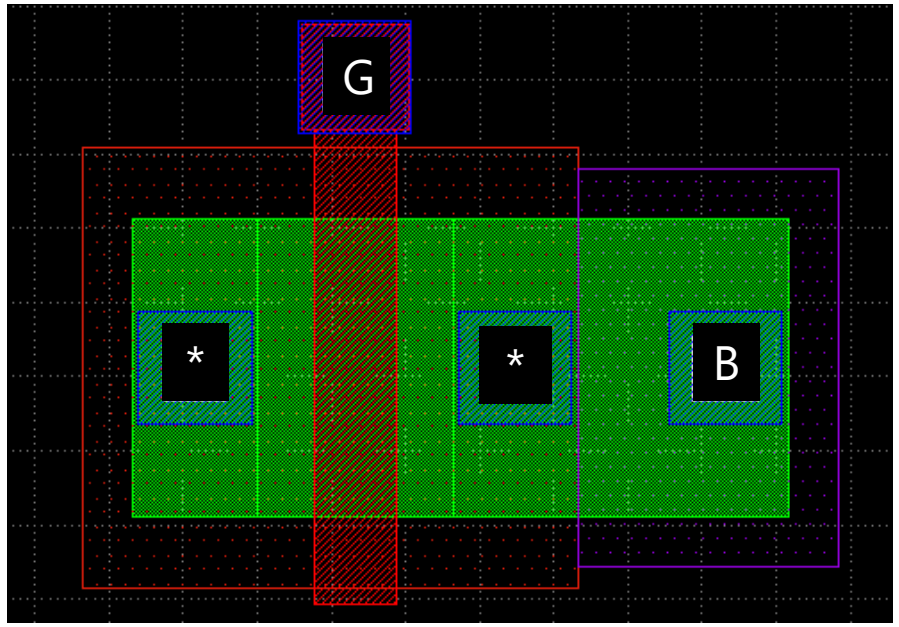
https://gf180mcu-pdk.readthedocs.io/en/latest/physical_verification/design_manual/Design_Manual.html

- レイヤー名を検索したほうがはやいかも・・・

タップ付　PFET と NFET

- [*] どちらをドレインにしてどちらをソースにするかは任意。





アナログLSIの設計フロー

1. 回路図（ベンチマーク）を描く
2. シミュレーションをする
3. 回路図を基にレイアウトを描く
4. **レイアウトを検証する**
5. レイアウトを基に寄生成分を考慮したシミュレーションをする

6. （フレームに載せる）

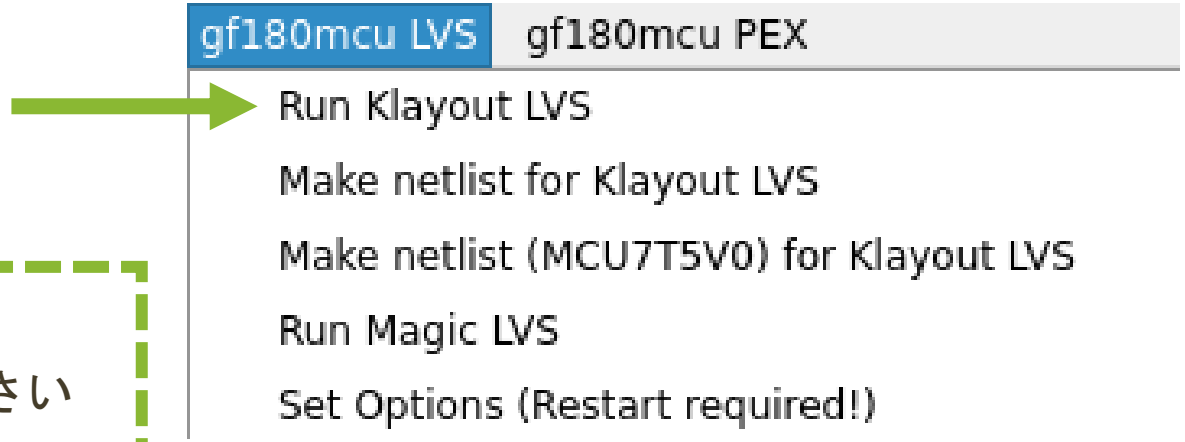
3zki / wsl_gf180mcu で利用できる レイアウト検証

Design Rules Check (DRC)

- 指定されたデザインルールから違反していないか検証

Layout Versus Schematic (LVS)

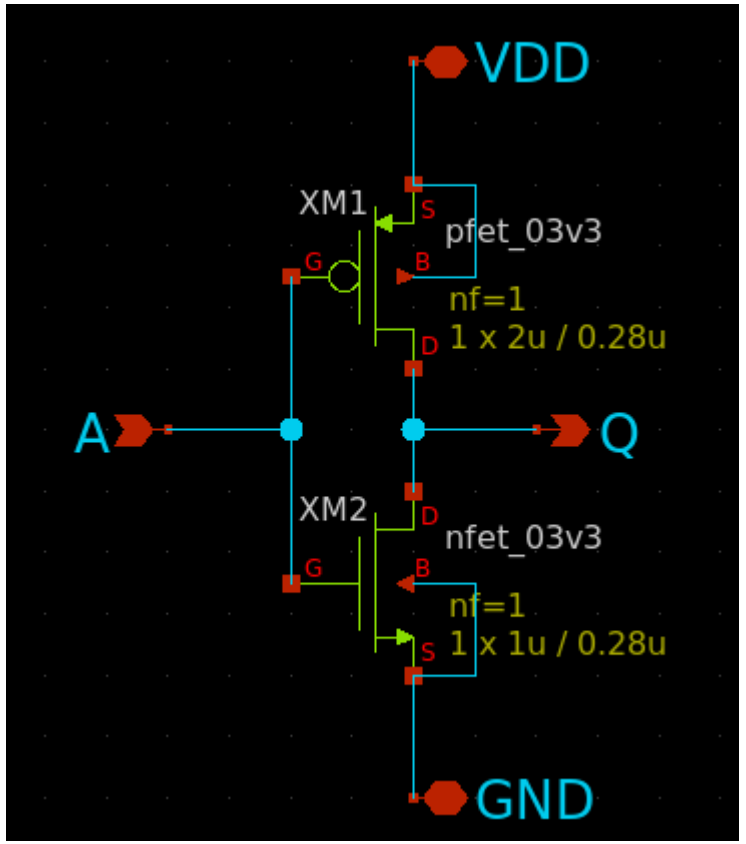
- レイアウトが回路図通りに描けているか検証



Magic DRC, Magic LVS は未整備です
KLayout DRC, KLayout LVSを使用してください

LVS用ネットリストの出力方法 (1/2)

- xschem上であらかじめLVS用の回路図を作成しておく。
- ピンはipin, iopin, opin のみを使用する。
  - vdd.symやgnd.symを使用するとうまく通らない。
- ファイルを同じ名前にする 例: nand.sch, nand.gds



LVS用ネットリストの出力方法 (2/2)

- xschem上で**LVS用**のネットリストを出力する。

1. メニュー Simulation > LVS netlist に✓を入れる
2. Netlistを押す

- 終わったら閉じて良い
- デフォルトの生成場所は ~/.xschem/simulations/